

AI科普系列

拆解AI大模型

人人都看得懂AI大脑

猜下一个词

词向量

N-gram

RNN

N-gram

注意力

Transformer

刘老师 著

目录

序言：为什么你要了解 AI?	1
5 分钟，先玩一把	1
这本书想回答什么	1
两条路，两种走法	2
这本书讲什么	2
作者的话	3
启蒙篇	1
第 1 章：AI 是什么——从图灵到 ChatGPT	2
第 2 章：三块基石——数据、模型、学习	6
第 3 章：监督学习——有标准答案的学习	10
第 4 章：无监督学习——自己找规律	14
第 5 章：强化学习——在试错中成长	19
第 6 章：概率的语言——AI 的“不确定哲学”	25
深度学习基础篇	31
第 7 章 感知机：一个会投票的神经元	32
第 8 章 多层神经网络：把投票器叠起来	36
第 9 章 反向传播：错误追责的艺术	40
第 10 章 Softmax 与交叉熵：让网络学会“打分”	46
第 11 章 梯度下降：蒙着眼怎么下山	51
第 12 章 激活函数：那条让网络“活过来”的曲线	56
第 13 章 过拟合与正则化：背答案还是学原理	61
第 14 章 Dropout：随机点名——让学生不敢偷懒	66
核心架构篇	71
第 15 章 CNN 卷积神经网络	72
第 16 章 卷积与池化	77
第 17 章 ResNet 与残差连接	82
第 18 章 GNN 图神经网络	87
第 19 章 统计语言模型与 N-gram：让计算机学会猜词	93
第 20 章 Embedding 与表征学习	98
第 21 章 前馈神经网络做语言模型：用语义猜词	106
第 22 章 RNN 循环神经网络	110
第 23 章 LSTM 与 GRU	116
第 24 章 自注意力机制：Q、K、V，你在问自己?	122
第 25 章 多头注意力：八个角度看同一句话	128
第 26 章 位置编码：给没有座位号的自习室编号	134

第 27 章 Transformer 架构：一张改变世界的图纸	139
第 28 章 分词器与 BPE：模型的“识字课本”	146
生成模型篇	152
第 29 章 GAN：生成对抗网络	153
第 30 章 VAE：变分自编码器	159
第 31 章 Diffusion：扩散模型	165
第 32 章 Stable Diffusion 大揭秘	172
第 33 章 文生视频：让画面流动起来	178
大模型核心篇	185
第 34 章 自监督学习：当完形填空遇上 AI	186
第 35 章 预训练：把一个人关进国家图书馆一年	错误！未定义书签。
第 36 章 指令微调 SFT：让接龙机器学会对话	错误！未定义书签。
第 37 章 Prompt Engineering：给超级实习生分配任务的艺术	错误！未定义书签。
第 38 章 LLM 的推理革命：从“脱口而出”到“深思熟虑”	错误！未定义书签。
第 39 章 RLHF 与 DPO：给 AI 上“职业道德课”	错误！未定义书签。
大模型应用篇	错误！未定义书签。
第 40 章 RAG 检索增强生成	错误！未定义书签。
第 41 章 AI Agent 智能体	错误！未定义书签。
第 42 章 MCP 协议	错误！未定义书签。
第 43 章 多模态 AI 实战	错误！未定义书签。
第 44 章 大模型落地指南	错误！未定义书签。
系统工程篇	错误！未定义书签。
第 45 章 混合精度训练：用计算器记账，还是用专业财务软件？	错误！未定义书签。
第 46 章 算子融合：别让快递员跑十趟	错误！未定义书签。
第 47 章 数据并行与模型并行：搬砖可以分工，搬发动机不行	错误！未定义书签。
第 48 章 流水线并行：餐厅后厨的分工哲学	错误！未定义书签。
第 49 章 MoE 混合专家：大医院的会诊制度	错误！未定义书签。
第 50 章 量化与剪枝：给大模型减肥的两把手术刀	错误！未定义书签。
第 51 章 推理引擎：造一辆车和开一辆车是两码事	错误！未定义书签。
第 52 章 DeepSeek 工程实践解密：用更少的积木，搭出同样的城堡	错误！未定义书签。
安全与对齐篇	错误！未定义书签。
第 53 章 AI 安全：给大模型装上护栏	错误！未定义书签。
第 54 章 评估体系：AI 的“高考”长什么样	错误！未定义书签。
第 55 章 联邦学习：数据不出门，模型照样训	错误！未定义书签。
第 56 章 可解释 AI：打开密封的黑盒	错误！未定义书签。
前沿与未来篇	错误！未定义书签。
第 57 章 迁移学习：站在别人的肩膀上修自己的路	错误！未定义书签。

第 58 章 对比学习：没有标签，就给 AI 看 100 个人的照片	错误！未定义书签。
第 59 章 小样本学习与元学习：不是学会打球，是学会怎么练球	错误！未定义书签。
第 60 章 知识蒸馏：把大学的百科全书消化成小学生的讲义	错误！未定义书签。
第 61 章 中国大模型生态全景	错误！未定义书签。
第 62 章 Transformer 之后的新架构	错误！未定义书签。
第 63 章 具身智能与因果推断	错误！未定义书签。
第 64 章 世界模型与 AI 终极形态	错误！未定义书签。
第 65 章 学习路线图	错误！未定义书签。
附录	错误！未定义书签。
附录 A：动手实战——从零搭建 RAG 系统	错误！未定义书签。
附录 B：自然启发算法选读	错误！未定义书签。
附录 C：面试题精选	错误！未定义书签。
附录 D：热点技术专栏	错误！未定义书签。

公余易AI赛道

序言：为什么你要了解 AI？

2026 年的某一天。你掏出手机随手拍了张照片，AI 在 0.3 秒内帮你调好了光、修掉了路人、配了一段让你自己都觉得“写得真好”的文案。你点了发送，朋友秒赞。一切自然得像呼吸。

你大概不会想：这背后到底发生了什么？

几十年前，一群人问了一个听起来有点奇怪的问题：机器能思考吗？这个问题引发了一场跨越半个多世纪的智力探险。从几个数学家在纸上的演算，到今天你手机里的智能助手，这条路走得跌跌撞撞，充满了狂妄、失败、寒冬和奇迹般的重生。而 2020 年代——就是那个“奇迹般重生”终于开花结果的时代。

这本书，就是想把这段旅程讲清楚。

5 分钟，先玩一把

在翻下一页之前，我想请你做一件事。

打开任何一个 AI 对话工具——ChatGPT、Claude、DeepSeek、豆包，随便哪个。然后输入这句话：

“请用 200 字给我解释一下黑洞，语气要像一个喝醉了的天文学家在酒吧吹牛。”

看看它给你的回复。笑完之后，再试试这个：

“帮我写一段 Python 代码，计算斐波那契数列的前 50 项，并画出折线图。”

你会看到一个程序凭空出现，附带解释。然后你把代码复制下来，扔进电脑里，它真的能跑。

这 5 分钟的体验，比你花 5 个小时读任何关于 AI 的文章都更有说服力。因为只有亲身体会过那种“居然可以这样”的瞬间，你才能真正理解为什么整个科技界在 2022 年底被 ChatGPT 炸得人仰马翻。

玩完了？回来坐下。我们正式开始。

这本书想回答什么

三个问题。

第一，AI 到底是什么？ 别笑。我见过太多人张口闭口“大模型”“深度学习”，但你问他“AI 和普通程序到底哪里不一样”，就卡住了。第一章会把这事掰开揉碎讲清楚——不是用学术语言，是用你写过的 Excel 公式就能懂的方式。

第二，它为什么在 2020 年代突然爆发？ AI 的很多核心技术诞生于上世纪 50 年代到 80 年代。神经网络的基本原理，反向传播算法，甚至 Transformer 的雏形“注意力机制”，都不是什么新鲜玩意。那为什么偏偏是最近几年？答案藏在三个东西里面：数据、算力、还有人们不信邪坚持了 40 年。

第三，大模型凭什么“大”？ “大模型”这个词已经烂大街了，但你可能没意识到一件事：ChatGPT 并没有被“教会”聊天。它只是被训练去“预测下一个字”。一个只会预测下一个字的程序，怎么就突然会写诗、编程、翻译、推理了？这事到目前为止，所有 AI 专家都只能说“我们也不太确定”。是的，创造它的人也不知道它为什么这么聪明。这不酷吗？

两条路，两种走法

这本书的每一章开头，你会看到两个图标：

- **观光路线：**一条轻松的路。你只需要读正文，跳过所有代码块。就像坐缆车上山，风景尽收眼底，不费什么力气。适合——“我就想搞明白 AI 是什么，有啥用，背后的原理大概怎么回事”的读者。
- **登山路线：**一条硬核的路。你不仅要读正文，还要跟着代码一步步跑。可以边读边在电脑上敲，改参数，看结果变化。适合——“我不光想知道是什么，我还想亲手做出来”的读者。登上山顶之后，你看到的风景跟坐缆车上来的完全不一样。

两种路线不分高低，只是目标不同。我自己两种都走过。有时候只想快速理解一个概念，有时候非要自己把代码跑通才舒服。这本书的设计就是让你可以随时切换——读到感兴趣的代码，停下来敲一敲；觉得这段太深了，跳到下一章，完全没问题。

这本书不对标任何教科书。你可以跳着读，挑感兴趣的章节先读，反复读某一章直到把概念内化。不用有“必须从第一页读到最后一页”的负担。AI 的学习曲线本身就不是线性的，你的阅读也不必是。

这本书讲什么

这本书分十一个部分，外加四个附录。主线是一条清晰的“猜词”故事——从让计算机猜下一个词开始，一路走到大模型。

第一部分：启蒙篇（01-06）。从“AI 到底是什么”聊起，认识数据、模型、学习三大基石，走进监督、无监督、强化三种学习范式，最后用概率串起一切——AI 的核心哲学是“优雅地接受不确定性”。

第二部分：深度学习基础篇（07-14）。神经网络的骨架。感知机、多层网络、反向传播、梯度下降、激活函数、过拟合与正则化。这部分让你真正理解“深度”两字的分量。

第三部分：核心架构篇（15-18）。视觉与结构这条线。CNN 如何让计算机拥有“眼睛”、ResNet 的残差“高速公路”、GNN 如何看懂图。读完可自由跳去语言建模篇或生成模型篇。

第四部分：语言建模演进篇（19-28）。全书的”心脏”线。从 N-gram 一路到 Transformer：用”猜下一个词”这同一道题，把统计语言模型、词向量、RNN、LSTM、注意力、位置编码串成一条连续的、每一章都回答”为什么需要它”的叙事。

第五部分：生成模型篇（29-33）。图像与视频生成线。GAN、VAE、扩散模型，到 Stable Diffusion 和文生视频——AI 是怎么”画”和”拍”的。

第六部分：大模型核心篇（34-39）。从预训练到 RLHF，模型是怎么被”炼”出来的，以及令所有人意外又着迷的”涌现能力”。

第七部分：大模型应用篇（40-44）。真正把模型用起来：RAG、AI Agent、MCP、多模态、落地指南。

第八部分：系统工程篇（45-52）。训练与推理背后的硬功夫：混合精度、并行、量化、推理引擎，以及 DeepSeek 的工程实践解密。

第九部分：安全与对齐篇（53-56）。AI 的安全、评估、联邦学习与可解释性。

第十部分：进阶技法篇（57-60）。迁移学习、对比学习、小样本学习、知识蒸馏——高手的分界线。

第十一部分：前沿与未来篇（61-65）。中国大模型生态、后 Transformer 新架构、具身智能、世界模型，最后用一张学习路线图收尾。

附录（A-D）。从零搭 RAG 的实战、自然启发算法选读、面试题精选、热点技术专栏。

作者的话

写这本书的时候，我反复提醒自己一件事：别写成那种让你读了三页就想睡觉的东西。

你知道那种感觉——拿起一本技术书，第一页就出现”机器学习是一门研究如何利用计算机从数据中自动分析获得规律，并利用规律对未知数据进行预测的学科”。你把书放下了。不是因为写得不好，是因为它把你当成了一个需要被”教育”的人，而不是一个想”了解”的朋友。

我的目标是写一本你在咖啡馆能读得进去的技术书。旁边放杯拿铁，翻几页，看到一个比喻会心一笑，看到一个代码示例掏出电脑试试。读到不懂的地方不焦虑——因为这章本来是登山路线，而你走的是观光路线嘛。

如果这本书有一个人设，它不是教授站在讲台上，是朋友坐在你旁边，拿笔画了张草图说：“你看，其实就这么回事。”



扫码添加作者微信



微信扫码加入星球

扫码订阅知识库



好，咖啡倒好了吗？我们开始第一章。

启蒙篇

易经入门之道

第 1 章：AI 是什么——从图灵到 ChatGPT

难度：★☆☆☆☆

观光路线：直接阅读正文，跳过代码块。20 分钟了解 AI 的全貌。

登山路线：读完正文后，用代码体验 AI 和传统程序的区别。30 分钟。

2012 年深秋的一天夜里，多伦多大学计算机系一个头发灰白的教授拿起了电话。他拨给远在英国妻子，声音有点抖：“我们做到了。我们真的做到了。”

他的名字叫 Geoffrey Hinton。几个小时前，他带着两个学生设计的神经网络模型 AlexNet，在 ImageNet 图像识别比赛中把第二名甩开了 10 个百分点——这在计算机视觉的历史上从未发生过。要知道，在此之前，大家觉得用神经网络做图像识别这事基本属于“学术自嗨”。

但 Hinton 知道，这次不一样。

仅仅十年后，这项他从 80 年代就在坚持的研究，催生出了 ChatGPT，人类历史上用户增长最快的产品。又过了两年，Sora 让全世界看到 AI 生成视频的可能。再过一年，DeepSeek-R1 用纯强化学习的方式教会了 AI“思考”。

从一个深夜电话开始，到现在。AI 走进了每个人的生活。但这一切到底是怎么回事？

你写的程序和 AI 程序，本质上不一样

在聊 AI 之前，我们得先搞清楚一件事：你平时写的程序和 AI 程序，到底哪里不一样？

你用过 Excel 吧。你在 A 列输入 1 到 12，在 B 列写公式 $=A1*2$ ，往下一拖，B 列就变成了 2 到 24。这背后是一段程序，它的逻辑是：你给我一个数，我乘以 2 还给你。规则是你定的，程序只负责忠实地执行。

但想象另一个场景：你给程序看了 100 张猫的照片和 100 张狗的照片，然后问它——喏，这是张新照片，你说是猫还是狗？

这事用传统编程方式做，你会疯掉。你没办法写一个“规则”来判断猫和狗的区别——耳朵尖不尖？脸圆不圆？毛色？体型？这些规则加到一起，不是漏了就是错了，而且换个品种就全崩了。90 年代有一群工程师花了几年时间想用纯规则做图像识别，结果连“区分男人和女人”都没搞定。

AI 程序本质上走的是另一条路。它不看规则，看例子。

你给它看 100 张猫的照片，它自己从像素里总结“猫大概长这样”。你再给它看 100 张狗的照片，它自己调整理解——“哦，原来不是所有毛茸茸的都是猫”。等它看完了，你把新照片给它，它说：“这只耳朵尖、脸圆、体型小……我判断它是猫，但我只有 95% 的把握。”

注意两个关键词：自己总结和只有 95% 的把握。

传统程序是人告诉机器规则，机器执行。AI 程序是机器从数据中自己发现规则。传统程序永远给出确定的结果，AI 程序永远在说“我觉得”。

这不是 AI 的”缺陷”。这是它最强大的地方——因为真实世界本来就没有 100%确定的事。你怎么定义猫？你怎么定义美？你怎么定义”对用户友好的设计”？当你没办法精确描述规则的时候，AI 就是你唯一的出路。

七十年，一场接力

AI 不是一夜之间冒出来的。这条路上站满了人，每个人都往前递了一棒。

1950 年，图灵扔出第一块石头。他在论文《计算机与智能》里问了一个问题：机器能思考吗？但他没有直接回答，而是设计了一个巧妙的实验——图灵测试。简单说，就是让人跟机器对话，如果人分不出对面是机器还是人，那机器就算”能思考”了。图灵聪明地把一个哲学问题变成了一个可操作的工程问题。这个思维方式的转换，影响了之后所有的 AI 研究。

1956 年，达特茅斯会议。一群年轻学者（包括”人工智能之父”麦卡锡、“神经网络先驱”明斯基）聚在新罕布什尔的一个小镇上，提出了一个野心勃勃的目标：在机器上模拟人类智能的方方面面。他们把”人工智能”这个词正式定了下来。会上大家信心爆棚，觉得一个暑假就能搞定关键问题。后来证明，他们低估了这事有多难——低估了好几个数量级。

1950-1970 年代，规则时代。早期 AI 走的是”专家系统”路线。思路很直白：找到领域的专家，把他们的决策逻辑写成一条条”If-then”规则。这个方案在某些领域（比如医疗诊断、棋类游戏）取得了不错的成绩。但人类知识的复杂度远超预期。写规则的速度永远跟不上新知识的产生速度。这条路走到 70 年代就走不动了，AI 迎来了第一个”寒冬”——研究经费被砍，公司倒闭，很多学者都不敢在论文标题里提”AI”两个字。

1980-1990 年代，神经网络的暗夜。Hinton 在 80 年代开始研究神经网络（其实神经网络的雏形在 50 年代就有了，但一直被学术界主流嘲笑）。1986 年，他和同事们发表了反向传播算法，让多层神经网络的训练成为可能。可是那时候计算机太慢了，数据也太少了。神经网络还是打不过传统的统计方法。Hinton 的学生回忆，那些年他去参加学术会议，提到神经网络的时候，台下的资深教授会直接站起来走出会议室。

2012 年，AlexNet。Hinton 和他的两个学生用 GPU 训练了一个叫 AlexNet 的神经网络。它在 ImageNet 比赛上的错误率只有 15.3%，比第二名低了 10 个百分点还多。深度学习时代正式拉开序幕。这一年之后，你再也没听过有人说”神经网络不好用”。

2017 年，Transformer。Google 的八个研究员发表了一篇文章，标题非常简洁：《Attention Is All You Need》。他们提出了一种叫 Transformer 的架构，彻底抛弃了传统的时间顺序处理方式，用一种叫”注意力机制”的东西让模型可以同时关注输入的所有部分。当时没人意识到，这将是未来所有大模型的底层架构。GPT 里的 T，就是 Transformer。

2022 年，ChatGPT。OpenAI 把一个叫 GPT-3.5 的模型加上了对话微调，包装成了一个聊天机器人，然后扔到了网上。两个月后，它有了 1 亿用户。人们第一次跟一个 AI 聊了半小时，忘了对面不是人。

2024 年，GPT-4o 和 Sora。大模型不再只能”聊天”。它能看、能听、能画、能做视频。Sora 生成的视频让你分不清真假。多模态能力让 AI 从”语言机器”变成了”感知机器”。

2025 年，DeepSeek-R1。中国的 DeepSeek 团队做了一件让整个学术界震惊的事：他们证明 AI 的推理能力不需要人工标注的”正确答案”来训练。只要给模型一个环境，让它在试错中自己找规律，它就能学会更复杂的推理。这标志着 AI 训练范式的又一次根本性转变。

2026年的今天。 AI 已经像电力和互联网一样，开始渗透到社会的基础设施层面。我们在手机里用 AI 修图，在电脑上让 AI 帮忙写代码和邮件，在医院里 AI 辅助医生读片，在工厂里 AI 控制机器人。变化之快，连业内人士都觉得有点跟不上。

AI 的三种超能力

从能力维度来看，今天的 AI 在三个层面上扩展了自己的疆域。

感知：让机器拥有感官。 这是 AI 最早突破的领域。计算机视觉让 AI 能“看”——识别人脸、检测物体、分析医学影像。语音识别让 AI 能“听”——把你说的每一句话转化成文字。2010 年代，深度学习在这些领域陆续超越了人类水平。现在你的手机能用人脸解锁，会议软件能实时生成字幕。这些功能已经平常到你不会意识到它们是 AI 了——但这恰恰说明它们成功地融入了生活。

认知：让机器学会理解。 如果说感知是“看到什么”，认知就是“这意味着什么”。这是 AI 最难突破的领域。自然语言理解——读一段文字，理解其中的含义、情感、逻辑、言外之意——直到大模型时代才有了质的飞跃。ChatGPT 之所以让人震惊，不是因为它能写出通顺的句子（这在技术上不是最难的），而是它似乎真的“理解”了你在问什么。你跟它说“老板今天又发脾气了，好烦”，它知道你应该不是在询问老板的生理机制。

行动：让机器能做决策。 感知和认知是“输入”，行动是“输出”。AI 正在从“给我一个答案”进化到“帮我办事了”。AI Agent——能自主规划、执行多步骤任务的 AI——是 2025-2026 年最热的方向。你可以对 AI 说“帮我预订下周三去上海的机票和酒店，预算 3000 以内，偏好中午航班”，它真的能帮你搞定。这是从“AI 助手”到“AI 代理人”的跨越。

小结+FAQ

这一章我们做的事情很简单：回答“AI 是什么”这个问题。

AI 不是魔法，也不是写在程序里的死规则。它是一种从数据中自动学习规律的方法。传统程序执行人类写好的指令，AI 程序从例子中自己找出规律。这个区别看似简单，但它彻底改变了计算机和人类的关系——从“我命令你”变成了“我教你，你学会了帮我做事”。

七十年，一群人从图灵的那个问题出发，走了很远很远。但这条路还远远没走完。

FAQ：AI 会取代我吗？

这个问题我被问得最多。我的答案是：AI 会取代很多“任务”，但不一定会取代“你”。

会用 AI 的人会取代不会用 AI 的人。这个规律在历史上反复出现过。会用电钻的工人取代了只用锤子的。会用搜索引擎的白领取代了翻资料手册的。会用 AI 的程序员取代不会用 AI 的程序员，会用 AI 的设计师取代不会用 AI 的设计师的。AI 更像是一把新的电钻，而不是一个来顶替你的人工智能同事。

换个角度想：如果你的工作能完全被 AI 替代（重复性极高、不需要判断、不涉及人际沟通），那确实有风险。但如果你能学会跟 AI 协作——让 AI 处理重复部分，你的精力放在需要创造力和判断力的部分——你的价值反而会提升。

这不是一个关于“取代”的故事。这是一个关于“升级工具”的故事。

这事其实才刚开始。下一章，我们把 AI 拆开看——它是怎么造出来的？三个零件：数据、模型、学习。

常见疑问

Q: AI 会取代我的工作吗？

A: 取代你的是会用 AI 的人，不是 AI 本身。AI 更擅长”辅助”而不是”替代”。重复性的、模式化的工作会被自动化，但需要创造力、共情能力、复杂决策的工作——AI 目前还不行。最好的策略不是害怕，是学会跟 AI 协作。就像当年搜索引擎刚出现的时候，没人说”搜索引擎会取代我”，但会用搜索引擎的人确实比不会用的人效率高出一个数量级。

Q: 图灵测试还重要吗？

A: 图灵测试的历史意义大于实际意义。ChatGPT 早就通过了——但这不代表它有意识。现在的评估体系远比图灵测试复杂，MMLU、HumanEval、GSM8K 等基准测试从不同维度衡量 AI 能力。图灵测试更多是哲学上的里程碑而非技术标准。它开启了”机器能思考吗”这个问题的讨论，但回答这个问题，我们需要远比图灵测试更精细的工具。

第2章：三块基石——数据、模型、学习

难度：★★☆☆☆

观光路线：跳过代码块，通过手机人脸解锁的比喻理解 AI 三要素。20 分钟。

登山路线：阅读全文，动手运行 PyTorch 张量操作代码，感受”数据”在计算机里的形态。35 分钟。

每天早上，你拿起手机，屏幕亮起，人脸识别在零点几秒内完成，手机解锁。你大概不会想：这个动作背后，AI 的三个核心要素在完美配合。

你注册人脸的时候，手机对着你拍了十几张照片——这是**数据**。

这些照片被送进手机芯片里的一个程序——这个程序叫**模型**。你可以把它想象成一个结构极其复杂的”评分器”。每张脸部照片经过它，它会输出一个数字，描述这张脸的各种特征：眼距多宽、鼻子多高、下巴轮廓、皮肤纹理。不是用人话描述的，是用一堆密密麻麻的数字。

模型刚”出厂”的时候什么都不会。它是在几百万张人脸照片上”练”出来的——这个过程叫**学习**。每看一张照片，模型就调整一下自己内部的参数，让自己的输出更接近正确答案。几百万次调整之后，它终于学会了怎样把任何一张脸映射成一组独一无二的数字签名。

数据、模型、学习。造一部能识别你脸的程序，就这三样东西，缺一不可。

这事听起来不复杂，但每一个环节都有很多门道。我们一个一个拆开看。

数据：AI 的口粮

没有数据就没有 AI。这不是修辞，这是冷冰冰的事实。不管你的模型架构有多精妙，不管你的训练算法有多漂亮，扔一堆垃圾数据进去，出来的只能是垃圾 AI。搞 AI 的人有句黑话叫”Garbage in, garbage out”——翻译成中文，输入的是垃圾，输出的只能是垃圾。

但在”不是垃圾”的前提下，数据还有三种角色的区分。很多初学者会忽略这个区分，然后在实际项目中翻车。

训练集：课本和习题。这是 AI”学习”用的数据。AI 从这个集合里看例子、找规律、调整参数。训练集通常占全部数据的 70%-80%。就像你数学课本上的例题——你看例题学方法，做着做着就”会了”。训练集的质量直接决定 AI 的上限。训练集里的猫照片全是白天的，AI 就认不出夜里的猫。训练集里的猫照片全是趴着的，AI 就认不出在跳的猫。这个现象叫”分布偏移”，是 AI 落地时最常见的坑。

验证集：模拟考。训练的过程中，AI 每隔一会儿就要用验证集测一下自己——“我现在正确率多少了？跟上一轮比有没有进步？”验证集不参与参数的直接更新，但它间接影响训练过程。比如”训练到什么时候该停”这个决策，就是看验证集上的表现来定的。验证集通常占 10%-15%。

测试集：高考。训练全部结束后，AI 用测试集做最后一次评估。测试集的数据，AI 在整个训练过程中一眼都没看过。只有这次考试的成绩，才是 AI 真正的”能力分数”。为什么训练集和验证集的成绩不能算数？因为你可能在不知不觉中让 AI”记住了”训练集里的具体例子（这叫”过拟合”），而不是掌握了规律。测试集就像高考——命题组不会把模拟考的题原样拿过来考你。

这三个集合要满足一个硬性条件：**不能有重叠**。测试集里的任何数据都不能出现在训练集里。这个道理你想想就懂——如果高考之前你把所有答案都背过了，那这次考试得了满分也不能说明你学会了数学。

还有一个更隐蔽的要求：三个集合的数据分布要一致。你的训练集全是白天的照片，测试集全是夜里的——这不是在测你学得怎么样，这是在考验你的泛化能力，而且通常不太乐观。

模型：AI 的大脑骨架

数据是燃料，模型就是发动机。

模型这个词，在不同语境下含义有些不一样。在 AI 里，模型本质上是一个**数学函数**：你给我一组输入，我经过一系列计算，给你一组输出。这个“一系列计算”的结构是预设的，但里面的具体参数是“学”出来的。

举个极简的例子。假设你想建一个模型，输入房间面积，输出房价。最简单的模型是一条直线：

```
# 最简单的房价预测模型
房价 = 面积 * w + b
```

这里的 w 和 b 就是**参数**——模型里可以“拧”的旋钮。你把 w 拧大一点，面积对房价的影响就更大；把 b 拧大一点，所有预测的房价都往上抬一点。“训练”这个过程，就是在数据上反复拧这些旋钮，直到模型的预测跟真实房价最接近。

真实的 AI 模型（比如一个现代神经网络）可能有几十亿甚至上万亿个参数。每一个参数都是一个旋钮，几十亿个旋钮同时拧——你想象一下这个场面，就知道为什么训练大模型需要上万张 GPU 显卡和几个月的计算时间了。

学习：让参数自己“走”到最优位置

“拧旋钮”是一个比喻。实际上，学习是通过一个叫“梯度下降”的数学方法自动完成的。

你想象你蒙着眼睛站在一座山上，任务是走到山谷的最低点。你每次只能判断脚下是上坡还是下坡，然后往下坡的方向迈一小步。反复这个动作，你最终会走到谷底。

这就是梯度下降的直觉。模型的“高度”对应损失函数的值（越高=越不准），每一步的方向由梯度（导数）决定，步长由“学习率”这个参数控制。步子太大可能冲过头，步子太小几万年都走不到。

几十亿参数模型的梯度下降，本质上就是这个蒙眼走路的动作，在极其高维的空间里，以每秒几千亿次运算的速度同时执行。

动手：感受一下数据在 AI 里的形态

AI 里的数据，不管你原来是什么形式——图片、文字、声音——最后都会变成一种叫“张量”的东西。张量这个名字听起来唬人，其实你可以直接把它理解成“多维数组”。一张彩色图片是一个三维张量（高度 $\times$ 宽度 $\times$ 颜色通道），一段文本经过编码后是一个二维张量（句子数 $\times$ 词汇特征数）。

PyTorch 是目前最主流的深度学习框架之一。我们来看看张量长什么样。

```

import torch

# 标量: 0 维张量, 就是一个数字
scalar = torch.tensor(3.14)
print(f"标量: {scalar}, 维度: {scalar.ndim}")

# 向量: 1 维张量, 相当于一个列表
vector = torch.tensor([1.0, 2.0, 3.0])
print(f"向量: {vector}, 形状: {vector.shape}")

# 矩阵: 2 维张量, 相当于一个表格
matrix = torch.tensor([[1, 2, 3],
                        [4, 5, 6]])
print(f"矩阵: \n{matrix}, 形状: {matrix.shape}")

# 四维张量: 一个图片批次 (2 张图 x 3 通道 x 4 像素高 x 5 像素宽)
image_batch = torch.randn(2, 3, 4, 5) # 随机生成模拟数据
print(f"图片批次形状: {image_batch.shape}") # torch.Size([2, 3, 4, 5])

# 张量运算: AI 模型的核心操作
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
print(f"逐元素相加: {a + b}") # tensor([5., 7., 9.])
print(f"逐元素相乘: {a * b}") # tensor([4., 10., 18.])
print(f"点积 (内积): {torch.dot(a, b)}") # 1*4 + 2*5 + 3*6 = 32

```

这段代码看起来简单，但你就凭这几行，实际上已经触碰到了所有深度学习的底层——不管是 GPT-4 还是 Sora，它们内部最核心的计算，就是张量之间的加法和乘法，在 GPU 上并行地、大规模地反复执行。几十亿参数，几千亿次运算，但底层操作跟上面这段代码里的事本质上是一回事。

小结+FAQ

把 AI 拆成三块石头来看：数据是原材料，模型是加工流程，学习是让流程自动优化的方法。少了哪一块都不行。数据决定上限，模型决定能不能逼近这个上限，学习决定逼近的速度。这三者之间的权衡和配合，是整个 AI 工程的核心命题。

FAQ: 我听说 AI 都要用 GPU, CPU 不行吗?

CPU 和 GPU 的区别，用一句话说：CPU 是几个数学天才在做高难度题，GPU 是几万个普通人在做简单题。AI 里的计算绝大多数都是大量独立的、结构相同的小计算（比如矩阵乘法），这种任务特别适合 GPU——把大问题拆成千万个小问题同时算，总体速度快几十到几百倍。CPU 也“能”跑 AI，但训练大的模型可能需要几个月甚至几年，而 GPU 只需要几天。

FAQ: 参数越多模型就越厉害吗?

不一定。参数多意味着模型“容量”大——能存储更复杂的规律。但如果你只有很少的数据，参数多了反而会“过拟合”——模型死记硬背了训练数据，但碰到新数据就懵了。大模型之所以“大”，是因为它们有海量数据来匹配海量参数。数据和参数的规模需要匹配，就像发动机的马力要和车重匹配一样。

数据、模型、学习，这三块石头我们算认识了。接下来三章，我们把”学习”这块石头敲开看——AI 到底有哪些不同的学法？每种学法适合干什么事？

常见疑问

Q: 数据、模型、算法哪个最重要？

A: 2026 年的共识：高质量数据 > 模型架构 > 算法技巧。没有好数据，再牛的模型也学不到东西。这也是为什么大模型公司都在疯狂收集和清洗数据——OpenAI 和 Reddit 签数据协议、Google 用 YouTube 字幕训练、各家都在抢优质语料。数据质量决定了 AI 能力的天花板，模型和算法只是帮你逼近这个天花板的工具。

Q: 验证集和测试集为什么必须分开？

A: 因为你会”作弊”——无意识的作弊。当你反复在同一个验证集上调参数、选模型、早停，模型实际上已经开始”记忆”验证集的特点了。虽然验证集不直接参与梯度更新，但它通过你的决策间接影响了模型。测试集是最后一次性的、完全独立的考试，只考一次。如果测试集成绩和验证集成绩差距很大，说明你在验证集上过拟合了。

第 3 章：监督学习——有标准答案的学习

难度：★★☆☆☆

观光路线：跳过代码块，理解监督学习的核心逻辑、分类与回归的区别、损失函数的作用。20 分钟。

登山路线：完整运行 Titanic 生存预测代码，体会一个真实监督学习项目的全流程。40 分钟。

你有没有想过一个问题：AI 是怎么学会区分垃圾邮件和正常邮件的？

答案可能比你想象的简单：它做过很多很多次”练习题”，而且每道题后面都写着标准答案。

这是所有”监督学习”的核心思路——你给 AI 一堆数据，每条数据都标注了正确答案，然后让 AI 自己找规律，使得它的预测尽可能接近标准答案。就像一个学生在做练习册：看题-答题-对答案-改正-继续下一题，反复几万次之后，她学会了。

“监督”这个词可能会让你想到一个老师在旁边盯着。实际上，这里的”监督”是指数据有”标签”——也就是正确答案。垃圾邮件的标签是”垃圾”，正常邮件的标签是”正常”。猫照片的标签是”猫”。房价数据里，标签是”实际成交价”。

有标签的数据，就是监督学习的全部秘密。

两个题型：判断题和填空题

监督学习主要解决两种问题。这两种问题对应了现实世界里大部分需要”预测”的场景。

分类：判断题。 答案是一组有限的选项中的一个。“这是猫还是狗？”（二选一），“这封邮件是垃圾还是正常？”（二选一），“这张手写数字是 0 到 9 中的哪个？”（十选一）。分类问题的输出是离散的——只能在几个选项里挑一个。

你手机的人脸解锁就是一个分类器，它判断”这张脸是不是机主”。你刷视频时平台的推荐系统也是一个分类器，它判断”你给这个视频点赞的概率有多少”。

回归：填空题。 答案是一个连续的数字。“这套房子能卖多少钱？”（一个价格），“明天的温度是多少？”（一个数字），“这个用户会在平台上停留多少秒？”（一个数字）。回归问题的输出是连续的——可以是任意实数。

分类和回归，几乎覆盖了所有你想让 AI 预测的东西。要么猜一个类别，要么猜一个数值。就这么简单。

损失函数：AI 的”扣分标准”

有了预测和正确答案，怎么评判”差多少”？这个评判标准就叫**损失函数**。

考试打分你肯定熟。一道题，你写了答案，老师对照标准答案给你扣分。扣得越多说明你错的越离谱，扣得越少说明你掌握得越好。损失函数就是 AI 的”扣分标准”。

分类问题最常用的损失函数叫”交叉熵损失”。你不需要记住数学公式，只需要理解它的直觉：如果 AI 信心十足地答错了，罚得特别狠；如果 AI 犹豫不决地答对了，也轻罚一下（因

为不准)；如果 AI 信心十足地答对了，几乎不罚。这个设计很聪明——它同时考查”答对”和”自信度”两个维度。

回归问题最常用的叫”均方误差”——就是把所有预测值和真实值的差的平方加起来取平均。预测房价，AI 猜 500 万，实际 600 万，差 100 万，这个差距的平方 1 个亿进入了损失。当你把所有样本的损失加起来，你就得到了这个模型在当前数据上的”总分”。

训练的终极目标就是把损失降到尽量低。每一次”梯度下降”的迭代，都是在问自己：“我往哪个方向调参数，能让损失降最快？”

上手：预测泰坦尼克号上的生死

理论讲完了，动手做一个真实的监督学习项目。我们会用 Titanic 数据集——这是 Kaggle 上最经典的入门数据集，记录泰坦尼克号上乘客的信息以及他们是否幸存。

任务很简单：根据乘客的性别、年龄、舱位等级、船票价格等信息，预测他/她能否活下来。

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder

# 1. 加载数据
# 数据来源: https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv
# 你也可以从 Kaggle 下载 train.csv
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

print(f"数据集大小: {df.shape}")
print(f"列名: {list(df.columns)}")
print(f"\n 前 5 行数据: \n{df.head()}")

# 2. 数据清洗: 处理缺失值
# Age (年龄) 缺失的, 用中位数填充
df['Age'].fillna(df['Age'].median(), inplace=True)
# Embarked (登船港口) 缺失的, 用众数填充
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# 3. 特征工程: 选择和转换特征
# 选择对预测可能有用的列
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
X = df[features].copy()
y = df['Survived']

# 将文字类型转为数字 (机器学习只能处理数字)
# Sex: male=1, female=0
X['Sex'] = LabelEncoder().fit_transform(X['Sex'])
# Embarked: S=2, C=0, Q=1
X['Embarked'] = LabelEncoder().fit_transform(X['Embarked'])

print(f"\n 处理后的特征 (前 5 行): \n{X.head()}")
```

```

# 4. 划分训练集和测试集 (8:2)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"\n 训练集大小: {len(X_train)}, 测试集大小: {len(X_test)}")

# 5. 训练模型: 随机森林分类器
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 6. 评估模型
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"\n==== 模型评估 =====")
print(f"准确率: {accuracy:.2%}")
print(f"\n 分类报告: \n{classification_report(y_test, y_pred, target_names=['遇难', '幸存'])}")

# 7. 查看特征重要性: 哪些因素最能预测生死?
feature_importance = pd.DataFrame({
    '特征': features,
    '重要性': model.feature_importances_
}).sort_values('重要性', ascending=False)
print(f"\n 特征重要性排名: \n{feature_importance}")

# 8. 新乘客预测演示
# 假设一个 30 岁女性, 一等舱, 独行, 票价 100
new_passenger = pd.DataFrame({
    'Pclass': [1], 'Sex': [0], 'Age': [30],
    'SibSp': [0], 'Parch': [0], 'Fare': [100.0], 'Embarked': [2]
})
prediction = model.predict(new_passenger)[0]
probability = model.predict_proba(new_passenger)[0]
print(f"\n==== 新乘客预测 =====")
print(f"30 岁女性 / 一等舱 / 票价 100: ")
print(f" 预测结果: {'幸存' if prediction == 1 else '遇难'}")
print(f" 遇难概率: {probability[0]:.1%}, 幸存概率: {probability[1]:.1%}")

```

把这段代码跑起来, 你会看到模型在测试集上的准确率通常在 80% 左右。不算惊艳, 但考虑到我们只用了几行特征就做出了这个预测——性别、舱位、年龄、船票价格——这个结果其实相当不错。

特征重要性那一栏值得仔细看一眼。你会发现”Sex”（性别）排在最前面。这不是机器学习告诉你的新知识, 这是”泰坦尼克号上妇女优先上救生艇”这个历史事实在数据里留下的痕迹。AI 只是从数据中把这个规律挖了出来。它不”理解”这个规律背后的道德或历史背景, 但它精准地捕捉到了相关性。

小结+FAQ

监督学习是 AI 里最成熟、应用最广的学习方式。它的逻辑直白得近乎朴素: 有标准答案的题目, 做多了自然就会了。分类解决”选哪个”, 回归解决”多少”。损失函数是 AI 的”扣分系统”, 梯度下降是”改错本”。

你在商业世界里碰到的绝大多数 AI 应用——推荐系统、风控模型、广告点击率预测、医疗影像诊断——背后都是监督学习在干活。

FAQ: 是不是所有问题都能用监督学习解决?

不是。核心瓶颈在于标签。垃圾邮件分类之所以能做好，是因为每个邮箱用户都在不经意间大量标注（点“标记为垃圾邮件”=给标签）。但很多场景下，标签太贵了。让医生给 100 万张 CT 片标注肿瘤位置，成本高到你不敢想。

这就是为什么无监督学习（下一章要讲的内容）和半监督学习、自监督学习变得越来越重要——它们能在标签稀缺的情况下，从数据本身挖掘信号。

FAQ: 准确率 80%算好吗?

得分场景。Titanic 预测问题上，80%已经不错了——因为幸存本身有很大的随机因素（你刚好站在哪个救生艇旁边），这些随机因素不在我们的特征里。医学诊断里，80%准确率就可能不够——看漏一个癌症患者的代价太大。

没有绝对的“好”和“坏”，只有相对于任务需求的“够用”和“不够用”。这也是为什么做 AI 的人总是在说“看场景”。

监督学习讲完了。但想想，如果数据没有标签怎么办？如果我们根本不知道正确答案是什么，AI 还能学到东西吗？答案是——能。而且这件事比你想象的更神奇。

常见疑问

Q: 分类和回归能用同一种模型吗?

A: 可以，换最后一层就行。分类最后接 Softmax 输出概率分布，回归最后直接输出一个数值。前面的特征提取层完全共享——无论是卷积层还是全连接层，都是在学“怎么从数据中提取有用的特征”。这其实也是迁移学习的基础思路：拿一个在 ImageNet 上训练好的分类模型，把最后一层换成回归头，就能用来做房价预测。

Q: Titanic 预测这么简单的东西，大模型要它干嘛?

A: Titanic 是教学用的“Hello World”，但监督学习的核心流程——数据预处理、特征工程、模型选择、交叉验证——跟大模型微调的逻辑是一模一样的。学会了 Titanic，你就学会了一切监督学习项目的骨架。大模型微调本质上也是监督学习：有标注好的指令-回复对，模型学习从指令映射到回复。底层的训练循环、损失函数、验证策略，和 Titanic 的随机森林没有本质区别。

第 4 章：无监督学习——自己找规律

难度：★★☆☆☆

观光路线：跳过代码块，理解聚类 and 降维的核心思想，以及它们为什么对大模型至关重要。20 分钟。

登山路线：手写 K-Means 算法，体会”无标签也能学”的魔力。40 分钟。

你走进一个陌生班级，40 个学生。你不知道谁叫什么名字，不知道谁成绩好，不知道谁跟谁是朋友。但你待了半小时之后，你大概能看出来——那边几个是运动队的，这边几个是学霸小组，角落里两个可能刚吵完架。

你没有”标签”。没人告诉你”小明=学霸”。但你还是看出了规律。你是怎么做到的？你观察到学霸们都在低头看书，运动队的人穿着统一的训练服，吵架的两个人背对背坐着、互不看对方。

这就是无监督学习的核心逻辑：没有标准答案，但你依然能从数据本身的结构中找出规律。算法自己”发现”了分组、发现了异常值、发现了数据里隐藏的模式。

在 AI 的世界里，无监督学习有两件事做得特别好：**聚类**和**降维**。前者帮你找到”谁跟谁是一类”，后者帮你把一团乱麻的高维数据梳理成看得懂的低维结构。

聚类：把相似的东西放到一起

聚类是 AI 里最接近人类直觉的学习方式之一。你不需要告诉 AI”猫长什么样”，你只需要把一堆动物图片扔给它，它就能自动把猫分一组、狗分一组、兔子分一组。分完之后你一看——嚯，分得还真对。

但 AI 不是用”看”的。它是用数学距离来衡量”相似性”的。

还是拿猫狗举例。假设每张图片都被转化成了两个数字（你可以粗略理解为”毛茸茸程度”和”体重”）。把这些图片画在坐标系里，每个点代表一个动物。你会发现，猫的图片们挤在坐标系的某个区域，狗的图片们挤在另一个区域，二者之间有一段明显的”空档”。

聚类算法的工作就是自动找到这些”空档”，沿着空档把数据切成几块，每一块就是一个”簇”——也就是一个类别。

最经典的聚类算法叫 **K-Means**。它的思路简单到让你怀疑”这也能叫 AI？”“但其实非常有效。

手写 K-Means：40 行代码理解聚类本质

K-Means 的流程可以浓缩成三步：先猜几个中心点，然后把每个数据点分配给离它最近的中心点，最后根据分配结果重新算中心点。不断重复第二步和第三步，直到中心点不再移动。

```
import numpy as np
import matplotlib.pyplot as plt

# 1. 生成模拟数据：三个自然聚在一起的群体
np.random.seed(42)
# 群体 A：左下方
```

```

cluster_a = np.random.randn(100, 2) * 0.5 + np.array([0, 0])
# 群体B: 右上方
cluster_b = np.random.randn(100, 2) * 0.5 + np.array([4, 3])
# 群体C: 右下方
cluster_c = np.random.randn(100, 2) * 0.5 + np.array([4, -3])
data = np.vstack([cluster_a, cluster_b, cluster_c])

print(f"数据形状: {data.shape}") # (300, 2) 表示300个点, 每个点有2个特征

# 2. K-Means 核心算法 (手写版)
def kmeans(X, k=3, max_iters=100):
    """
    手写K-Means 算法
    X: 数据矩阵, 形状 (n_samples, n_features)
    k: 要分成几组
    max_iters: 最大迭代次数
    """
    # 随机选k个点作为初始中心
    n_samples = X.shape[0]
    idx = np.random.choice(n_samples, k, replace=False)
    centroids = X[idx].copy()

    for iteration in range(max_iters):
        # 第1步: 分配—每个点找离自己最近的中心
        distances = np.zeros((n_samples, k))
        for i in range(k):
            # 计算所有点到第i个中心的欧几里得距离
            distances[:, i] = np.sqrt(
                np.sum((X - centroids[i]) ** 2, axis=1)
            )
        labels = np.argmin(distances, axis=1) # 每行取最小值的列索引

        # 第2步: 更新—每个簇的所有点取平均, 得到新中心
        new_centroids = np.zeros_like(centroids)
        for i in range(k):
            if np.sum(labels == i) > 0: # 确保这个簇有数据
                new_centroids[i] = X[labels == i].mean(axis=0)

        # 第3步: 检查—中心点不动了就退出
        if np.allclose(centroids, new_centroids):
            print(f"在第 {iteration + 1} 轮收敛")
            break
        centroids = new_centroids

    return labels, centroids

# 3. 运行K-Means
labels, centroids = kmeans(data, k=3)
print(f"最终中心点位置: \n{centroids}")

# 4. 可视化聚类结果
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# 左图: 原始数据 (没有标签, 但我们用颜色标注了真实来源以便对比)
colors_true = ['#FF6B6B'] * 100 + ['#4ECDC4'] * 100 + ['#45B7D1'] * 100
ax1.scatter(data[:, 0], data[:, 1], c=colors_true, alpha=0.7, s=30)
ax1.set_title('原始数据 (真实分组)', fontsize=14)

```

```

ax1.set_xlabel('特征 1 (毛茸茸程度)')
ax1.set_ylabel('特征 2 (体重)')

# 右图: K-Means 聚类结果
colors_cluster = ['#FF6B6B', '#4ECDC4', '#45B7D1']
for i in range(3):
    ax2.scatter(data[labels == i, 0], data[labels == i, 1],
                c=colors_cluster[i], alpha=0.7, s=30, label=f'簇{i}')
    ax2.scatter(centroids[i, 0], centroids[i, 1],
                c=colors_cluster[i], s=200, marker='X', edgecolors='black', linewidth=2)
ax2.set_title('K-Means 聚类结果', fontsize=14)
ax2.set_xlabel('特征 1 (毛茸茸程度)')
ax2.set_ylabel('特征 2 (体重)')
ax2.legend()

plt.tight_layout()
plt.savefig('kmeans_demo.png', dpi=150, bbox_inches='tight')
plt.show()
print("图表已保存为 kmeans_demo.png")

```

跑完这段代码，你会发现 K-Means 完美地找出了三个群体，连中心点都跟原始群体中心几乎重叠。但重点不是“完美”——重点是你从头到尾没有告诉 AI 任何标签。你不知道哪个点属于哪个群体，AI 也不知道。但算法从纯数据结构中自动“读出”了规律。

K-Means 有一个你必须知道的硬伤：你得提前告诉它 k 是多少（你想分成几组）。在很多场景下，你不知道数据应该分成几组。这就引出了 K-Means 的两个“表亲”——

不止一种聚类方式

DBSCAN：基于密度的聚类。DBSCAN 不用你告诉它分几组。它自己判断。它的逻辑是：如果一个区域内点的密度够高，这些点就是一个簇；如果一个点孤零零地待着，密度很低，它就是“噪声”。DBSCAN 特别擅长发现任意形状的簇——不像 K-Means 只能画出圆形的分割边界。它还天然能识别异常值。电商平台用 DBSCAN 来发现“薅羊毛”的用户（行为模式和正常用户完全不同），效果很好。

层次聚类：建一棵“相似树”。层次聚类的思路更直观，也更像生物学的分类方式。它把每个数据点看成一个独立的“物种”，然后不断把最相似的两个合并，再合并，再合并，最后合并成一棵“树”。你在树上的不同高度横切一刀，就得到不同粒度的分组。层次聚类的输出不是一个“分组结果”，而是一整个“分组结构”——你可以从粗到细地查看数据的分层关系。基因测序、客户分群、文档归类的场景经常用到。

三种聚类方式各有适用场景。没有一个绝对“更好”的算法，只有更适合你数据形态的算法。

降维：在低维世界看清高维数据

聚类帮你在数据中找结构。降维帮你把数据结构“展示”出来。

现实世界的数据往往是高维的。一张照片上百万像素，一段句子上千个词汇，一个用户画像几百个标签。人类的大脑只能理解三维空间，四维就抓瞎了，一百维更是完全无法想象。但高维空间里藏着很多重要规律——比如不同文字在实际语义空间中的距离。

降维做的事情就是：把高维数据”拍扁”到二维或三维，同时尽量不丢失重要信息。这相当于给一张复杂的三维雕塑拍一张最优角度的照片——虽然损失了一个维度的信息，但最重要的结构关系还保留着。

主成分分析（PCA）是最经典的降维方法。t-SNE 和 UMAP 是更现代的方法，在大模型时代被大量用于可视化语义空间。你看到的那些”词汇语义地图”——“国王”和”女王”靠得很近，“男人”和”女人”的关系方向跟”国王”和”女王”一致——都是用降维技术画出来的。

降维也是大模型训练中的重要预处理步骤。在把海量文本喂给模型之前，通常需要对词汇进行降维嵌入，既减少了计算量，又保留了词汇之间的语义关系。

小结+FAQ

无监督学习的核心价值在于：你不需要标签。在标签昂贵甚至不可能获取的场景下，无监督学习是唯一的出路。聚类让你发现数据中的自然分组，降维让你在可理解的维度上观察高维数据的结构。

大模型里的自监督学习（下一部分会详细讲），其实就是无监督学习的一个变体——模型从无标签的文本中自己构造”假的”标签，然后用这些假标签来训练自己。ChatGPT 能理解语言，它的核心训练方式就是自监督学习——读了几万亿字的互联网文本，自己猜下一个词是什么，猜错了改，猜对了过。没人给它标注，但它学会了语言的规律。

FAQ: K-Means 的 k 到底选多少？

有一个经验法则叫”肘部法则”。你把 k 从 1 试到 10，每次都记录聚类后的总距离误差。画一条曲线，通常会看到一个拐点——拐点之前误差下降很快，拐点之后越来越慢。这个拐点往往就是合适的 k 值。但老实说，这个方法并不总是管用。很多时候 k 的选择取决于业务需求——你想把用户分成三档还是五档，这个决策本身就包含了对业务的理解。

FAQ: 无监督学习的结果怎么评价？

这是个好问题，也是无监督学习最大的难点。因为没有标签，没有”标准答案”，你很难用一个数字说”这个聚类好”或”不好”。常用的评价指标有轮廓系数（衡量簇内紧密和外部分离的程度），但终究不能替代人工审视。实际项目中，做无监督学习的人通常会用可视化来辅助判断——把聚类结果画出来，用肉眼看看分得合不合理。这听起来不太”科学”，但到目前为止，对聚类结果的人类直觉判断仍然是最可靠的评价方式之一。

聚类和降维让我们能在”没答案”的情况下学习。但还有一种学习方式更酷——连”正确答案”这个概念本身都不存在。AI 自己在环境中乱撞，撞对了有奖励，撞错了挨罚。这就是强化学习。

常见疑问

Q: K-Means 的 K 到底设多少？

A: 用”肘部法则”——画出不同 K 值对应的聚类误差（inertia），找到误差下降速度突然变缓的那个拐点，像手肘一样弯曲的地方。另外也可以结合业务知识：如果你知道用户大概分

几类，直接用那个数字。还有一种方法是轮廓系数（silhouette score），它衡量簇内紧密和簇间分离的程度，挑系数最高的 K。但说实话，没有一个方法能 100% 自动确定最优 K，最终还是要结合你对业务的理解。

Q: 无监督学习在大模型时代还重要吗？

A: 超级重要。自监督学习（大模型预训练的核心）就是从无监督学习延伸出来的——模型从无标签文本中自己构造“假标签”（比如“预测下一个词”），然后用这些假标签训练自己。聚类和降维在 Embedding 可视化、数据清洗、异常检测中每天都在用。你看到的那些“语义空间地图”——用 t-SNE 或 UMAP 把词向量画在二维平面上——背后就是降维。

第 5 章：强化学习——在试错中成长

难度：★★★★☆

观光路线：跳过代码块，理解强化学习的核心三要素、Q-Learning 的思想，以及它和大模型推理能力的联系。20 分钟。

登山路线：完整运行 Q-Learning 迷宫代码，亲眼见证 AI 从”乱撞”到”最优路径”的进化过程。45 分钟。

2016 年 3 月，韩国首尔，围棋世界冠军李世石坐在棋盘前。对面是一台机器。李世石赢下了第四局（事后他说那是他一生中最快乐的一局棋），但以 1 比 4 输掉了整个系列赛。AlphaGo 的胜利震动了整个世界。

AlphaGo 是怎么学会下棋的？它在学会规则之后，没人手把手教它”这步好，那步坏”。它自己跟自己下了几千万盘棋。赢了就加强这步走法的倾向，输了就削弱。几千万盘之后，它走出了人类棋手从未见过的招数。

这就是强化学习的精髓：不是听老师讲，不是看参考答案。是自己在环境里试，从结果中学习。

三个角色一台戏：状态、动作、奖励

强化学习的核心框架只有三个要素。弄清楚这三个，就理解了全部。

状态（State）：我在哪？ 对于 AlphaGo，状态是当前棋盘上的局面——所有棋子的位置。对于自动驾驶汽车，状态是当前位置、速度、周围车辆的距离。对于一只学走路的机器人，状态是各个关节的角度和身体的倾斜程度。

动作（Action）：我能做什么？ AlphaGo 的动作是落子位置。汽车的动作是加速、刹车、转向。机器人的动作是各个电机转动的角度。在每一个状态下，AI 都要从所有可能的动作中选一个。

奖励（Reward）：这么做有什么好处？ AlphaGo 下了一步好棋，没有即时奖励——围棋的奖励只在终局出现（赢了+1，输了-1）。这就是强化学习里最棘手的问题：“稀疏奖励”。你走了 100 步，只有最后一步才知道输赢。你得自己判断前面 99 步哪些是好棋，哪些是臭棋。而汽车的场景就不一样——偏航了立刻扣分，保持车道立刻加分，奖励很”密集”。

强化学习的目标用一句大白话说：找到一种动作选择策略，使得在每一个状态下，选择哪个动作能让长期累积奖励最大化。不是贪图眼前的一分两分，而是把眼光放长远。

Q-Learning：让 AI 学会”想长远”

在所有强化学习算法中，Q-Learning 是最经典、最优雅的一个。它的核心思想是维护一张巨大的表——**Q 表**。这张表的行是状态，列是动作，每个格子里的数字代表”在这个状态下做这个动作，长远来看能拿多少分”。

这张表一开始全是零。AI 什么都不懂，像刚出生的婴儿。每次尝试，它走到某个状态选了某个动作，得到奖励（或惩罚），然后更新表里的数字。更新的规则叫**贝尔曼方程**：

$Q[\text{当前状态}][\text{当前动作}] = Q[\text{当前状态}][\text{当前动作}] +$
学习率 * (即时奖励 + 折扣因子 * 未来最大 Q 值 - $Q[\text{当前状态}][\text{当前动作}]$)

别说这公式吓人，它翻译成成人话就是：我更新我的经验，不仅要考虑眼前得了多少分，还要考虑这一步之后能去多远。如果我这一步虽然现在得分不高，但能带我到一个“高分区域”，那这一步就是好棋。“折扣因子”控制的是——有远见的程度。值接近 1，代表“我愿意为长远回报忍耐很多步的低分”；接近 0，代表“今朝有酒今朝醉，只看眼前”。

实战：Q-Learning 走迷宫

我们来让 AI 走一个迷宫。迷宫是一个 5x5 的网格，起点是左上角，终点（宝藏）是右下角。AI 在每一步可以往上下左右四个方向移动。撞墙会停在原地。到达宝藏获得+100 奖励。

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# ===== 1. 迷宫环境 =====
class Maze:
    """5x5 迷宫环境，入口(0,0)到出口(4,4)"""
    def __init__(self):
        self.size = 5
        self.goal = (4, 4)
        # 定义墙壁：不能通过的格子
        self.walls = [(1, 1), (1, 2), (2, 1), (3, 3)]
        self.actions = [(0, 1), (0, -1), (1, 0), (-1, 0)] # 右左下上
        self.action_names = ['→', '←', '↓', '↑']

    def reset(self):
        """重置到起点"""
        self.agent_pos = (0, 0)
        return self.agent_pos

    def step(self, action):
        """执行一步动作，返回(新状态, 奖励, 是否结束)"""
        dr, dc = self.actions[action]
        new_r = self.agent_pos[0] + dr
        new_c = self.agent_pos[1] + dc
        # 检查边界和墙壁
        if (0 <= new_r < self.size and 0 <= new_c < self.size
            and (new_r, new_c) not in self.walls):
            self.agent_pos = (new_r, new_c)

        # 奖励设计：到达宝藏+100，每走一步-1（鼓励走最短路径）
        if self.agent_pos == self.goal:
            return self.agent_pos, 100, True
        return self.agent_pos, -1, False

# ===== 2. Q-Learning 算法 =====
def q_learning(env, episodes=2000, alpha=0.1, gamma=0.95, epsilon=0.2):
    """
    Q-Learning 训练
    alpha: 学习率—新信息占多大比重
    gamma: 折扣因子—有多“远见”
    epsilon: 探索率—有多大比例随机走（避免困在局部最优）
    """
    # 初始化 Q 表：5x5 的格子 x 4 个方向，全零
```

```

Q = np.zeros((env.size, env.size, len(env.actions)))
steps_history = [] # 记录每轮训练走的步数

for episode in range(episodes):
    state = env.reset()
    steps = 0
    done = False

    while not done and steps < 200: # 防止死循环
        r, c = state

        # epsilon-贪心策略: 以epsilon 的概率随机探索
        if np.random.random() < epsilon:
            action = np.random.randint(len(env.actions))
        else:
            action = np.argmax(Q[r, c]) # 选Q 值最大的动作

        next_state, reward, done = env.step(action)
        nr, nc = next_state
        steps += 1

        # Q 表更新—贝尔曼方程的核心
        # 当前Q 值 += 学习率 * (目标Q 值 - 当前Q 值)
        target = reward + gamma * np.max(Q[nr, nc]) * (not done)
        Q[r, c, action] += alpha * (target - Q[r, c, action])

        state = next_state

    steps_history.append(steps)

    # 每500 轮打印一次进度
    if (episode + 1) % 500 == 0:
        avg_steps = np.mean(steps_history[-100:])
        print(f"第{episode+1:4d}轮 | 最近 100 轮平均步数: {avg_steps:.1f}")

return Q, steps_history

# ===== 3. 训练 =====
env = Maze()
Q, history = q_learning(env, episodes=2000)

# ===== 4. 可视化 =====
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# 左图: 学习曲线
ax1 = axes[0]
ax1.plot(history, alpha=0.3, color='gray', linewidth=0.5, label='每轮步数')
# 平滑曲线: 取窗口平均
window = 50
smooth = np.convolve(history, np.ones(window)/window, mode='valid')
ax1.plot(range(window-1, len(history)), smooth, color='#FF6B6B', linewidth=2, label=f'滑动平均(窗口{window})')
ax1.axhline(y=8, color='#4ECDC4', linestyle='--', linewidth=2, label='理论最优: 8 步')
ax1.set_xlabel('训练轮数', fontsize=13)
ax1.set_ylabel('到达宝藏所需步数', fontsize=13)
ax1.set_title('Q-Learning 学习曲线', fontsize=14)
ax1.legend()

```

```

ax1.grid(True, alpha=0.3)

# 右图: 迷宫最优路径
ax2 = axes[1]
ax2.set_xlim(-0.5, 4.5)
ax2.set_ylim(-0.5, 4.5)
ax2.set_xticks(range(5))
ax2.set_yticks(range(5))
ax2.grid(True, alpha=0.3)

# 画墙壁
for wall in env.walls:
    ax2.add_patch(Rectangle((wall[1]-0.5, wall[0]-0.5), 1, 1,
                             facecolor='#333333', edgecolor='black'))

# 画起点和终点
ax2.add_patch(Rectangle((-0.5, -0.5), 1, 1, facecolor='#AED6F1', edgecolor='black', alpha=0.7))
ax2.text(0, 0, '起点', ha='center', va='center', fontsize=11, fontweight='bold')
ax2.add_patch(Rectangle((3.5, 3.5), 1, 1, facecolor='#F9E79F', edgecolor='black', alpha=0.7))
ax2.text(4, 4, '宝藏', ha='center', va='center', fontsize=11, fontweight='bold')

# 绘制每个格子的最优动作（箭头）
for r in range(5):
    for c in range(5):
        if (r, c) == env.goal or (r, c) in env.walls:
            continue
        best_action = np.argmax(Q[r, c])
        arrows = {0: (0.3, 0), 1: (-0.3, 0), 2: (0, 0.3), 3: (0, -0.3)}
        dx, dy = arrows.get(best_action, (0, 0))
        ax2.arrow(c, r, dx, dy, head_width=0.15, head_length=0.15,
                  fc='#FF6B6B', ec='#FF6B6B', linewidth=2)

ax2.set_title('迷宫最优策略（箭头=推荐方向）', fontsize=14)
ax2.invert_yaxis() # 让第0行在顶部

plt.tight_layout()
plt.savefig('qlearning_maze.png', dpi=150, bbox_inches='tight')
plt.show()
print("图表已保存为 qlearning_maze.png")
print(f"\n 最终 Q 表中宝藏格子的最大 Q 值: {np.max(Q[4, 4]):.1f}")

```

跑这段代码，你会看到一个很酷的现象。前几十轮，AI 在迷宫里疯狂乱撞，步数动辄上百步（我们的上限是 200 步），始终找不到宝藏。然后大概在 100 轮左右，聪明一点点——偶尔能找到宝藏了。500 轮之后，步数稳定下降。最终，AI 找到了最优路径：恰好 8 步到达宝藏。一条从(0,0)绕过墙壁抵达(4,4)的最短路线。

有意思的地方来了：Q-Learning 从来没学过“最短路径”这个概念。它只知道迷宫里有宝藏（奖励+100），每多走一步扣 1 分。然后它自己悟出来：走越少步的路径，最终得分越高。如果你把折扣因子 γ 调低（比如从 0.95 降到 0.5），AI 会变得更“短视”——不那么在乎长远回报，也就没那么执着于找最短路径了。

强化学习与大模型：DeepSeek-R1 的启示

到了 2025 年，强化学习在大模型领域搞出了一个大新闻。

DeepSeek 团队做了一个叫 R1 的模型，核心思路极其纯粹：用强化学习让模型自己学会“思考”。他们没有用人类标注的“标准推理过程”来训练模型，而是直接把模型扔进一个数学推理环境里。做对了给奖励（能得出正确答案），做错了没奖励。除此之外，没有任何人工干预。

结果呢？模型自己演化出了“思考”的行为——在解答数学题时，它会自动生成很长的中间推理链，自己质疑自己的步骤，甚至发现自己算错了回头重算。这些行为根本不是人类“教”的，是模型自己在强化学习的奖励机制下“进化”出来的。

这事让整个 AI 界震惊。因为在这之前，大家觉得“让 AI 学会一步步推理”必须有人类示范——你得给它看人类是怎么一步步解题的。DeepSeek-R1 证明了不需要。只要环境里有正确答案的反馈信号，AI 就能自己“长”出推理能力。

这是强化学习最迷人的地方：你不需要告诉 AI 该怎么做，你只需要告诉它“做对”是什么样。它会自己找出怎么做。有时候，它找出的方法人类根本没想过。

小结+FAQ

强化学习是 AI 学习中场景最贴近“生物学习”的范式。动物不会看课本，它们从小在环境里摔跤碰壁，从疼痛和奖励中学到了生存法则。强化学习就是这个过程在计算机里的复现。

三个核心要素：状态（我在哪）、动作（我能做什么）、奖励（这么做好不好）。Q-Learning 用一个简单的表格和贝尔曼方程，让 AI 在一片空白中自己学会了最优策略。DeepSeek-R1 把这个逻辑搬到了大模型的训练中，证明了“纯奖励驱动”能让 AI 演化出人类级的推理能力。

FAQ：强化学习和监督学习到底什么区别？

最本质的区别：监督学习有“标准答案”（每道题都有），强化学习只有“评分标准”（做完了才知道对不对）。监督学习是被动的——老师给你题目和答案，你照着学；强化学习是主动的——你在环境里探索，从反馈中自己总结策略。

这也是为什么强化学习最适合那些“规则简单但策略复杂”的场景——游戏、机器人控制、自动驾驶。因为在这些场景里，你没法给 AI 展示所有可能的正确操作（AlphaGo 的走法数量比宇宙中的原子还多），但你可以在每一步或每一局结束时给它一个评分的信号。

FAQ：Q-Learning 在实际中还用吗？

纯粹的 Q 表方法在真实场景中用得不多——因为真实问题状态太多了，表格装不下。但它的思想——用“价值函数”评估每一步的远期收益——是所有现代强化学习算法的基础。DQN（Deep Q-Network）用神经网络代替 Q 表，让强化学习能处理 Atari 游戏这样的高维输入。PPO（Proximal Policy Optimization）是当前最流行的强化学习算法之一，ChatGPT 的 RLHF 阶段用的就是 PPO 的变体。这些你听起来很厉害的名字，底层逻辑都能追溯到一个小小的 Q 表更新公式。

强化学习让你不需要标准答案也能学。但 AI 的底层逻辑里还有一个更基础的东西——概率。为什么 AI 永远在说“我觉得 80% 是猫”而不是“这是猫”？为什么不确定性是 AI 的核心哲学，而不是它的缺陷？

常见疑问

Q: DeepSeek-R1 的 RL 和 AlphaGo 的 RL 是一回事吗?

A: 同一种思想，不同实现。AlphaGo 用 MCTS（蒙特卡洛树搜索）+ RL 在围棋上做搜索和决策，DeepSeek-R1 用 GRPO（组内相对策略优化）在文本推理链上做策略优化。核心机制是一样的——试错、奖励、优化——只是应用场景从下棋变成了思考。AlphaGo 的”奖励”是终局赢了+1 输了-1，DeepSeek-R1 的”奖励”是数学题答案对不对、代码能不能通过测试。骨子里都是”不知道标准答案，但知道什么结果好”。

Q: 强化学习为什么比监督学习慢那么多?

A: 监督学习有标准答案，每次训练信号明确——模型预测和标签的差距直接告诉你”往哪调”。强化学习要自己”摸索”——走一万步可能才得到一个奖励信号，而且这个信号可能是你五十步前的行为导致的（延迟奖励问题）。样本效率极低，但能解决”没人知道标准答案”的问题。这就是为什么 AlphaGo 需要自己跟自己下几千万盘棋，而监督学习的图像分类几万张标注图就够了。

第 6 章：概率的语言——AI 的“不确定哲学”

难度：★★★★☆

观光路线：跳过代码块，理解贝叶斯思维、最大似然和交叉熵的直觉含义。25 分钟。

登山路线：运行贝叶斯公式验证代码和高斯分布可视化，感受概率在 AI 中的实际运作。40 分钟。

埋下一颗种子：本章的条件概率，日后会反复出现。记住一个小游戏——“根据前半句，猜下一个词”。比如“轩辕养了一只_____”。这个“猜下一个词”就是**语言模型**的母题，后面从 N-gram、词向量、RNN 一路到大名鼎鼎的 Transformer，考的都是这张卷子。你先记住这个画面。

你去做一个癌症筛查。医生告诉你，这个检测准确率 99%。结果出来了——阳性。你吓得腿软。

先别急。我问你：假如这种癌症在人群中的发病率只有 0.1%（千分之一），你现在真的得了癌症的概率是多少？

大部分人会说”99%“，或者”应该很高吧”。正确答案会让你大吃一惊。

用贝叶斯公式一算：在 100 万人里，有 1000 人真的得了癌症（发病率 0.1%），其中 990 人会测出阳性（准确率 99%）。但剩下的 99 万 9000 个健康人里，也有 9990 人会测出阳性（1%的假阳性率）。所以所有阳性的人里头，真正得癌症的比例是： $990 / (990 + 9990) = 0.0902$ 。

大约 9%。

测出阳性，但你真正得癌症的概率只有 9%。这听起来反直觉得离谱。一个 99% 准确的检测，阳性结果居然只有 9% 的可信度？但数学不会骗人。你被”准确率 99%“这个数字骗了——它没说”阳性结果中真正患病的比例”，它说的是”有病的人中测出阳性的比例”和”没病的人中测出阴性的比例”。这两个”视角”之间的转换，就是贝叶斯定理。

概率不是衡量”世界有多确定”，而是衡量”你知道得有多少”。这才是概率的本质，也是整个 AI 的哲学根基。

贝叶斯定理：用新证据更新旧信念

贝叶斯定理的数学形式很简洁：

$$P(A|B) = P(B|A) * P(A) / P(B)$$

翻译成成人话：发生了 B 这件事之后，你对 A 的信念应该怎么调整？调整方式是一一用 B 的证据强度（ $P(B|A) / P(B)$ ）乘上你原来对 A 的信念（ $P(A)$ ）。

还是癌症检测的例子。你做检测之前，对”得了癌症”的信念是 $P(A) = 0.1\%$ （人群发病率，这叫”先验概率”）。然后你观察到了 B：“检测结果是阳性”。贝叶斯定理告诉你：你对”得了癌症”的信念应该更新为 9.02%（这叫”后验概率”）。

这个“用新证据更新旧信念”的过程，是人类理性思考的标准模板，也是 AI 进行推理的底层逻辑。

更厉害的是，这个过程可以反复迭代。今天你拿到了检测结果，更新了信念到 9%。明天你做了另一个独立检测，又是阳性——信念再更新一次，可能到了 90% 以上。后天你又做了个 CT，看见了阴影区——又更新一次，99% 了。每一次新证据来了，你都用贝叶斯公式把信念往前推一步。最终，即使最开始你几乎完全不确定，只要有足够多的高质量证据，你的信念会收敛到真相附近。

AI 在做的事，就是这个过程在数据上的大规模重复。

动手验证：让数字替你说话

```
import numpy as np

# ===== 贝叶斯公式验证：癌症检测问题 =====

# 先验概率：人群中癌症发病率
P_cancer = 0.001      # 0.1%的人得癌症
P_no_cancer = 1 - P_cancer

# 检测的条件概率
P_positive_given_cancer = 0.99      # 真有病的人，99%测出阳性
P_positive_given_no_cancer = 0.01   # 没病的人，1%测出阳性（假阳性）

# 全概率公式：任意一个人测出阳性的总概率
P_positive = (P_positive_given_cancer * P_cancer
              + P_positive_given_no_cancer * P_no_cancer)

# 贝叶斯公式：测出阳性时真正得癌症的概率
P_cancer_given_positive = (P_positive_given_cancer * P_cancer) / P_positive

print("=" * 50)
print("贝叶斯公式——癌症检测问题")
print("=" * 50)
print(f"先验概率（人群中得癌症的比例）：{P_cancer:.1%}")
print(f"检测准确率（真有病测出阳性）：{P_positive_given_cancer:.1%}")
print(f"假阳性率（没病测出阳性）：{P_positive_given_no_cancer:.1%}")
print(f"{'-' * 40}")
print(f"任意一人测出阳性的总概率：{P_positive:.2%}")
print(f"测出阳性 → 真正得癌症的概率：{P_cancer_given_positive:.2%}")
print(f"测出阳性 → 实际没得癌症的概率：{1 - P_cancer_given_positive:.2%}")
print(f"{'-' * 40}")
print(f"结论：一个准确率 99% 的检测，阳性结果只有约 {P_cancer_given_positive:.0%} 的可信度")

# ===== 最大似然估计：数硬币判断是否公平 =====
print(f"\n{'-' * 50}")
print("最大似然估计——这枚硬币公平吗？")
print("=" * 50)

# 模拟：抛一枚硬币 N 次，记录正面次数
np.random.seed(42)
N = 100 # 抛 100 次
true_p = 0.7 # 这枚硬币的真实正面概率是 70%（不太公平）
flips = np.random.binomial(1, true_p, N)
heads = flips.sum()
```

```

tails = N - heads

print(f"抛{N}次硬币，正面{heads}次，反面{tails}次")
print(f"观察到的正面比例: {heads/N:.2%}")

# 最大似然估计：最能解释这组数据的硬币正面概率是多少？
# 答案简单得不可思议：就是观察到的正面比例
mle_estimate = heads / N
print(f"最大似然估计的硬币正面概率: {mle_estimate:.2%}")
print(f"真实概率（揭晓答案）: {true_p:.1%}")
print(f"{'-' * 40}")
print(f"最大似然估计的直觉：看到什么就信什么")
print(f"如果抛 10 万次，这个估计会无限接近真实值{true_p:.1%}")

```

这段代码回答了概率论里最基础也最重要的两个问题。贝叶斯定理回答的是“看到了证据之后怎么更新信念”，最大似然估计回答的是“怎么从数据反推模型参数”。前者是 AI 推理的理论基础，后者是 AI 训练的核心方法。

高斯分布：为什么钟形曲线无处不在

自然界有一个神秘的规律：很多现象都服从同一个分布形状——钟形曲线，也就是**高斯分布（正态分布）**。

人的身高、体重、血压。考试的分数。工厂里螺丝的直径误差。一天的气温波动。所有这些东西分布起来，都形似一口大钟：中间最高（平均值附近的人最多），两边逐渐降低，尾巴拖得很远（极端值很少，但存在）。

这不是巧合。有一个叫“中心极限定理”的数学定理保证了：只要样本量够大，不管背后的分布长什么样，样本均值的分布一定接近高斯分布。

AI 里的高斯分布不是用来描述世界的，而是用来描述“误差”和“不确定性”的。监督学习里，当我们做回归预测房价，我们通常假设预测误差服从高斯分布——模型预测和真实值的差值，大多数时候接近零（模型还行），少数时候很大（模型翻车了），极小概率特别大（出现异常情况）。

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# ===== 高斯分布可视化 =====
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))

# 图1：不同均值的高斯分布（标准差相同=1）
x = np.linspace(-5, 10, 500)
for mu, color, label in [(-1, '#FF6B6B', '偏左(-1)'),
                        (2, '#4ECDC4', '居中(2)'),
                        (5, '#45B7D1', '偏右(5)')]:
    axes[0].plot(x, stats.norm.pdf(x, mu, 1), color=color, linewidth=2, label=f'μ={mu}')
axes[0].set_title('不同均值 (σ=1 固定)', fontsize=13)
axes[0].set_xlabel('值')
axes[0].set_ylabel('概率密度')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 图2：不同标准差的高斯分布（均值相同=0）

```

```

for sigma, color, label in [(0.5, '#FF6B6B', '集中'),
                             (1.0, '#4ECDC4', '标准'),
                             (2.0, '#45B7D1', '分散')]:
    axes[1].plot(x, stats.norm.pdf(x, 0, sigma), color=color, linewidth=2, label=f' $\sigma={sigma}$ ')
axes[1].set_title('不同标准差 ( $\mu=0$  固定)', fontsize=13)
axes[1].set_xlabel('值')
axes[1].set_ylabel('概率密度')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# 图3: 天然数据中的高斯分布—骰子模拟
np.random.seed(42)
# 蒙特卡洛模拟: 每次扔10个骰子, 求总和, 重复10000次
n_dice = 10
n_trials = 10000
dice_sums = np.random.randint(1, 7, (n_trials, n_dice)).sum(axis=1)
axes[2].hist(dice_sums, bins=range(n_dice, 6*n_dice+2), density=True,
             alpha=0.7, color='#AED6F1', edgecolor='#2C3E50', linewidth=0.5)
# 叠加理论高斯曲线
mu_theory = n_dice * 3.5 # 期望值 = 骰子数 * 平均点数
sigma_theory = np.sqrt(n_dice * (35/12)) # 方差 = n * Var(一个骰子)
x_fit = np.linspace(n_dice, 6*n_dice, 200)
axes[2].plot(x_fit, stats.norm.pdf(x_fit, mu_theory, sigma_theory),
             'r-', linewidth=2, label='理论高斯分布')
axes[2].set_title(f'{n_dice}个骰子总和分布 (中心极限定理)', fontsize=13)
axes[2].set_xlabel('骰子总和')
axes[2].set_ylabel('频率')
axes[2].legend()

plt.tight_layout()
plt.savefig('gaussian_demo.png', dpi=150, bbox_inches='tight')
plt.show()
print("图表已保存为 gaussian_demo.png")

```

图3尤其值得留意。10个骰子——每个骰子本身的分布是“完全均匀”的（1到6面概率相同，绝对不是什么钟形曲线），但10个均匀分布的东西加起来，就奇迹般地变成了近乎完美的高斯分布。这就是中心极限定理的威力。它解释了为什么高斯分布在自然界、在统计学、在AI里无处不在。

交叉熵与 KL 散度：衡量“两个概率分布差多少”

最后这两个概念，你可能在别的地方听说过。它们在AI里太重要了，重要到你需要知道它们在干什么——哪怕不深入数学细节。

交叉熵衡量的是：用“模型认为的分布”来编码“真实世界分布”需要多少信息。翻译成人话：你的预测和真实情况差多少？交叉熵越低，说明你的预测和真实越接近。监督学习里的分类问题，默认的损失函数就是交叉熵损失。

KL 散度（也叫相对熵）衡量的是两个概率分布之间的“距离”。但它不是我们平常理解的几何距离。KL 散度不是对称的——从A到B的KL散度和从B到A的KL散度不一样。这听起来奇怪，但直觉上是合理的：用一个简单的分布去近似一个复杂的分布，和用一个复杂的分布去近似一个简单的分布，这两个任务的“难度”不一样。

在 GPT 这类大模型的训练里，损失函数本质上就是交叉熵：模型预测的下一个词的概率分布，和实际语料中“下一个词是什么”的分布，二者之间的交叉熵。每一次梯度下降，模型都在把自己的预测分布往真实分布靠近那么一点点。几万亿次靠近之后，模型的语言能力就“长出来”了。

小结+FAQ

AI 的底层逻辑是概率。不是把概率当成“世界有随机性所以我们要容忍它”这种被动态度，而是把概率当成理解世界的核心语言。贝叶斯定理告诉我们如何用新证据更新旧信念。最大似然估计告诉我们怎么从数据反推参数。高斯分布告诉我们在不确定中最合理的假设的是什么形状。交叉熵和 KL 散度则给“两个概率分布差多少”这个问题提供了精确的度量。

这些概念听起来抽象，但它们是所有 AI——从朴素贝叶斯分类器到 GPT-4o——共享的数学语言。

FAQ：概率和统计是一回事吗？

不是。统计是从数据推断模型参数（“我看到 100 次抛硬币 70 次正面，这枚硬币公平吗？”），概率是从模型参数推断数据（“这枚硬币正面概率 0.7，抛 10 次出现 8 次正面的机会有多大？”）。一个是从果推因，一个是从因推果。AI 训练属于统计推断的范畴——从数据反推模型的内部参数。

FAQ：为什么要用概率而不是确定性规则？

因为真实世界就是不确定的。一张照片里，猫被半挡住了，你怎么确定它是猫？一段文字里，“打”可以是“打篮球”（运动）、“打电话”（通讯）、“打车”（交通）——同一个字在不同的组合里有完全不同的含义。如果你要求 AI 给出 100% 确定的答案，它在绝大多数场景下只能给你一个错误答案。接受不确定性，给出“我觉得 80% 是 A，15% 是 B，5% 是 C”——这反而是最诚实、最有用的答案。

说到底，人类思维本身就是在概率下运作的。你看到天边乌云密布，你不会 100% 确定等下会下雨——你只是觉得可能性很大，于是带了把伞。AI 只是把这个自然的思维方式数学化、系统化、放大化。

六章下来，我们走完了第一部分。你知道了 AI 是什么，知道了它的三块基石，走过了三种主流的学习范式，最后用概率的视角重新审视了所有这一切。

但这只是热身。因为接下来我们要进入这本书最核心的部分——深度学习。神经网络到底是怎么“深”起来的？反向传播为什么如此重要？卷积神经网络如何让机器拥有了视觉？这些问题的答案，会把我们引向最终的大模型。

翻页之前，不妨停下来想想：你现在对 AI 的理解，和打开这本书的时候，有什么不同？

常见疑问

Q: 贝叶斯公式那么反直觉，为什么 AI 非要用它？

A: 因为 AI 就是在持续做贝叶斯推断——先有一个先验信念（模型参数），看到新数据后更新这个信念（梯度下降）。整个深度学习训练本质上就是“给定数据，更新参数的概率分布”。贝叶斯公式反直觉是因为人脑不擅长做概率乘法，但机器擅长。而且贝叶斯框架的好处是：它给了不确定性一个优雅的数学表达——模型不是“知道答案”，而是“有一个信念，并随时准备根据新证据调整它”。

Q: 交叉熵和 KL 散度有什么区别？

A: 交叉熵 = 真实分布的熵 + KL 散度。真实分布的熵是常数（数据本身的不确定性），所以最小化交叉熵等价于最小化 KL 散度——让预测分布尽可能接近真实分布。用人话讲：交叉熵衡量的是“用你的预测编码真实数据需要多少比特”，KL 散度衡量的是“你的预测和真实分布之间差了多少”。训练大模型时两者等价，但概念上 KL 散度更纯粹地衡量“两个分布的距离”。

深度学习基础篇

公众号：AI技术圈

第 7 章 感知机：一个会投票的神经元

难度：★★☆☆☆

观光路线：理解感知机就是“加权投票器”，会分辨 AND/OR 逻辑门，知道 XOR 为什么搞不定。读代码 + 看图，不用推导公式。

登山路线：手写感知机训练算法，理解决策边界的几何含义，用 numpy 从零实现并可视化，为后续多层网络铺路。

1957 年，康奈尔航空实验室的弗兰克·罗森布拉特搞出了一台叫“感知机”的机器。这台机器看起来像一堆乱糟糟的电线和旋钮，但它在当时引起了轰动——《纽约时报》甚至预言它“终将能够走路、说话、看见、写字，并意识到自己的存在”。

这个预言当然没成真。但罗森布拉特的工作埋下了一个种子：能不能用计算的方式，让一个简单的数学单元“学会”判断？

加权投票器

想象你在一个评审委员会里。五位评委要决定一个项目是否通过。但五位评委不是平权的——有人是老专家，说话分量重；有人是新人，话语权弱一些。

最终怎么定？每个评委投的票（赞成=+1，反对=-1）乘以他的权重，然后加在一起。如果总和超过某个门槛，项目通过。否则毙掉。

感知机做的正是这件事——只是它把“评委”换成了“输入信号”。

用“人话”翻译一下数学形式：感知机有 n 个输入 $x_1, x_2, \dots, x_n$ ，每个输入配一个权重 $w_1, w_2, \dots, w_n$ 。把所有输入和权重乘起来加总，过一道门槛（偏置 b ），大于 0 输出 1，小于等于 0 输出 0。

$$y = \begin{cases} 1 & \text{如果 } w_1x_1 + w_2x_2 + \dots + w_nx_n + b > 0 \\ 0 & \text{否则} \end{cases}$$

用向量写就是一行： $y = \mathbf{1}(w \cdot x + b > 0)$ 。那个粗体的 1 是一个指示函数——条件成立就输出 1，不成立就输出 0。

你看，它就是一个投票器。权重是话语权，偏置是放水力度（偏置越大，越容易通过）。把这些旋钮调到合适的值，感知机就能做出靠谱的“是/否”判断。

用感知机实现逻辑门

逻辑门是晶体管的世界里最基础的东西：AND（与门）、OR（或门）、NAND（与非门）。它们接受两个布尔输入，输出一个布尔值。

能不能用感知机模拟它们？不仅能，而且极其简单。

AND 门——两个输入都是 1 才输出 1。怎么调权重？试一组： $w_1=0.5, w_2=0.5, b=-0.7$ 。当 $x_1=1, x_2=1$ 时， $0.5+0.5-0.7=0.3>0$ ，输出 1。其他三种输入组合（01, 10, 00）的和都不超过 0.3，输出 0。搞定。

OR 门——至少一个 1 就输出 1。w1=0.5, w2=0.5, b=-0.3。只有当两个都是 0 时，-0.3<0 才输出 0。其余情况都大于 0。也搞定。

NAND 门——AND 取反，两个都是 1 才输出 0。直接把 AND 的权重全取负就行：w1=-0.5, w2=-0.5, b=0.7。跟 AND 完全对称。

有意思的是——你可以手动凑出这些权重，但感知机真正的能力在于自己”学会”。给一堆带标签的样本，它通过一条简单的更新规则反复调整 w 和 b，最终自己找到合适的参数。

这条规则出奇地简单：

如果预测对了 → 不动

如果预测错了 → $w = w + (\text{真实值} - \text{预测值}) * x$
 $b = b + (\text{真实值} - \text{预测值})$

就这么一条规则。错了就往”对的方向”挪一步。反复挪，最终收敛。

手写一个感知机

来，写 30 行代码看它怎么学会 AND 门。

```
import numpy as np

class Perceptron:
    def __init__(self, n_inputs, lr=0.1):
        self.w = np.zeros(n_inputs)    # 权重初始化为 0
        self.b = 0                      # 偏置也为 0
        self.lr = lr                   # 学习率：每次调整的步长

    def predict(self, X):
        # 加权求和过阈值：大于 0 返回 1，否则返回 0
        return np.where(np.dot(X, self.w) + self.b > 0, 1, 0)

    def fit(self, X, y, epochs=10):
        for epoch in range(epochs):
            errors = 0
            for xi, yi in zip(X, y):
                y_pred = self.predict(xi)
                if y_pred != yi:
                    # 核心更新规则：错哪就挪哪
                    self.w += self.lr * (yi - y_pred) * xi
                    self.b += self.lr * (yi - y_pred)
                    errors += 1
            print(f"第{epoch+1}轮，错误数：{errors}")
            if errors == 0:
                break

# AND 门数据
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1]) # 只有全 1 才输出 1

p = Perceptron(n_inputs=2, lr=0.1)
p.fit(X, y, epochs=10)
print(f"学的权重：{p.w}, 偏置：{p.b:.2f}")
print(f"预测：{p.predict(X)}")
```

运行这个，你会发现它两三轮就学会了。权重和偏置自动收敛到能完美区分 AND 门的数值。

决策边界可视化

感知机在做什么，换个角度看更直观。两个输入 x_1 、 x_2 可以在二维平面上画出来。而 $w_1x_1 + w_2x_2 + b = 0$ 是一条直线。这条线把平面切成两块：一边输出 1，另一边输出 0。它就是“决策边界”。

```
import matplotlib.pyplot as plt

# 画数据点
for i, (x1, x2) in enumerate(X):
    color = 'blue' if y[i] == 1 else 'red'
    plt.scatter(x1, x2, c=color, s=100, edgecolors='k', zorder=3)

# 画决策边界:  $w_1x_1 + w_2x_2 + b = 0 \Rightarrow x_2 = -(w_1x_1 + b) / w_2$ 
x1_vals = np.linspace(-0.5, 1.5, 100)
x2_vals = -(p.w[0] * x1_vals + p.b) / p.w[1]
plt.plot(x1_vals, x2_vals, 'g--', linewidth=2, label='决策边界')

plt.xlim(-0.5, 1.5); plt.ylim(-0.5, 1.5)
plt.xlabel('x1'); plt.ylabel('x2')
plt.legend(); plt.grid(alpha=0.3)
plt.title('感知机学到的 AND 门决策边界')
plt.show()
```

你会看到一条直线干净利落地把(1,1)这个蓝色点和另外三个红点分开。这条线就是感知机“学到”的东西。

把数据换成 OR 门、NAND 门，感知机一样能画出一条分割线。因为这些数据是线性可分的——存在一条直线能完美切开两类点。

XOR：感知机的滑铁卢

问题来了——有个叫 XOR（异或）的逻辑门：两个输入不同时输出 1，相同时输出 0。

$(0,0) \rightarrow 0, (0,1) \rightarrow 1, (1,0) \rightarrow 1, (1,1) \rightarrow 0$ 。

你在纸上画四个点：(0,0)和(1,1)是红的，(0,1)和(1,0)是蓝的。然后你找一根直线，把红蓝分开。

你找不到。无论怎么画，四个点形成对角交叉，一根直线永远切不开。

这就是感知机的致命伤——XOR 问题。1969 年，马文·明斯基和西摩·佩珀特在《感知机》一书里严格证明了这一点。单层感知机只能处理线性可分的问题。

这本书的出版直接导致了第一次“AI 寒冬”——研究经费骤减，大量项目被砍。因为当时的人们发现，这个被寄予厚望的模型连“异或”这么简单的东西都搞不定。

但明斯基他们也在书里埋了一个种子：如果把感知机堆起来，多加几层，能不能解决问题？

那个种子，就是我们现在要讲的故事。

人话总结这章：感知机就是一个能自动调旋钮的投票器，擅长用一条直线把东西分成两类。但它最大的软肋是——只要数据不是线性可分的，它就抓瞎。而真实世界的东西，几乎都不是线性可分的。

出路在哪？答案是：别只用一层。把投票器叠起来。

这事没完——一个感知机的脑容量确实寒碜，但要是搞一群感知机互相配合呢？

常见疑问

Q: 感知机和逻辑门的关系是什么？

A: 感知机本质上就是在模拟逻辑门。AND 门、OR 门、NAND 门都可以用单个感知机实现。但 XOR 门不行——它需要两层感知机。这个限制直接催生了多层神经网络。

Q: 感知机的权重初始值是随机的吗？

A: 这章代码里为了起步简单，权重初始化成了 0。单个感知机没有对称性问题，从 0 开始完全可行。但到了多层神经网络就不行了——如果每层神经元的初始值都一样，它们会学出一模一样的参数，等于白搭。所以从下一章起，你会看到权重改用随机初始化。

第 8 章 多层神经网络：把投票器叠起来

难度：★★★★☆☆

观光路线：理解”堆叠感知机=万能函数模拟器”的核心直觉，知道为什么多加一层就能解决 XOR，看懂 MiniMLP 代码结构。

登山路线：手动推导 2 层网络解决 XOR 的逻辑，实现 MiniMLP 从零训练，理解万能近似定理的工程含义，对比宽度 vs 深度。

上章讲到感知机连 XOR 都搞不定。这就像你雇了一个只能画直线的绘图员——你给他两个蓝点两个红点，他无论怎么画直线都分不开。

但你如果不只雇一个人呢？你雇三个人，让前两个人分别画一条线，第三个人再把前两人的结果综合起来？

两层的魔法

回到 XOR 的四个点：(0,0)红、(0,1)蓝、(1,0)蓝、(1,1)红。

一个感知机画一条直线，切不开。但如果我用两个感知机，一个学 NAND、一个学 OR，然后第三个感知机对这俩结果做 AND——

等一下，NAND 和 OR？让我画个表你就明白了：

x1	x2	NAND(x1,x2)	OR(x1,x2)	AND(NAND, OR) = XOR
0	0	1	0	0
0	1	1	1	1
1	0	1	1	1
1	1	0	1	0

$$\text{XOR} = \text{AND}(\text{NAND}(x1,x2), \text{OR}(x1,x2))$$

就是这么简单。第一层两个感知机各自画一条线，把输入空间做了两次不同角度的切割。第二层一个感知机把这两个”视角”综合起来，做出了最终判断。

用几何语言说：第一层把原始坐标系 (x1, x2) 变换到了一个新的”特征空间” (h1, h2)，在新的空间里，原来线性不可分的东西突然变得线性可分了。

这就是多层神经网络的底层逻辑。不是变得更”聪明”，而是变得更”灵活”——多几双手，多几个角度看问题，然后再综合。

从”层”到”网络”

把上面这个两层结构画成图：输入层 (x1, x2) → 隐藏层（两个神经元，分别输出 NAND 和 OR）→ 输出层（一个神经元，输出 XOR）。

每个箭头代表一个权重，每个圆圈（除了输入层）代表一个”激活”操作——把加权和过一道非线性函数（比如阶跃函数，或者后面要讲的 ReLU、Sigmoid）。

“非线性”三个字是关键。如果没有非线性激活函数，多层网络和单层网络在数学上是等价的——多层的线性变换叠加起来还是一层线性变换。只有插入了非线性，多层才有意义。

就像你做菜，如果每道工序都是“按比例混合”，那无论你搞多少道工序，最终都是一次混合。但你加一个“大火爆炒”的步骤，味道就质变了。非线性激活函数就是那一步“爆炒”。

万能近似定理

1989年，几个数学家证明了一个令人瞠目的结论：只要一个隐藏层，并且这个隐藏层有足够多的神经元，这个网络就能以任意精度逼近任何连续函数。

人话版：给你一个足够宽的隐藏层，你可以模拟这个世界上任何一种输入→输出的映射关系。作曲？分类？预测股价？理论上，一个单隐层网络都能做，只要你给够神经元。

这个定理叫**万能近似定理**（Universal Approximation Theorem）。

但“理论上能”和“实际上能”是两回事。一个单隐层网络想模拟复杂函数可能需要指数级数量的神经元。而多层网络（深度网络）用更少的神经元就能做到同样的事情——这就是“深度”的意义：用层次化的方式，每一层提取一层抽象，效率比“一锅烩”高得多。

一个直觉类比：你让一个人记住所有两位数乘法的结果（需要背 8100 个答案），还是让他学会乘法口诀（几十条规则然后组合）？深度网络就是学“口诀”，宽而浅的网络就是死记硬背。

MiniMLP：从零实现

“MLP”是多层感知机（Multi-Layer Perceptron）的缩写。虽然叫“感知机”，但这个词现在泛指全连接的前馈神经网络。

下面这段代码实现一个迷你 MLP——两个隐藏层，用 ReLU 激活，解决 XOR 问题。全程只用 numpy。

```
import numpy as np

class MiniMLP:
    def __init__(self, lr=0.5):
        self.lr = lr
        # 输入→隐藏层1: 2 入 3 出
        self.W1 = np.random.randn(2, 3) * 0.5
        self.b1 = np.zeros(3)
        # 隐藏层1→隐藏层2: 3 入 2 出
        self.W2 = np.random.randn(3, 2) * 0.5
        self.b2 = np.zeros(2)
        # 隐藏层2→输出: 2 入 1 出
        self.W3 = np.random.randn(2, 1) * 0.5
        self.b3 = np.zeros(1)

    def relu(self, x):
        return np.maximum(0, x) # ReLU: 负数变0, 正数原样

    def relu_derivative(self, x):
        return (x > 0).astype(float) # ReLU 导数: 正数区1, 负数区0

    def forward(self, X):
        # 逐层前向传播
```

```

self.z1 = X @ self.W1 + self.b1      # 第一层加权和
self.a1 = self.relu(self.z1)         # 过激活函数
self.z2 = self.a1 @ self.W2 + self.b2
self.a2 = self.relu(self.z2)
self.z3 = self.a2 @ self.W3 + self.b3
self.a3 = 1 / (1 + np.exp(-self.z3)) # Sigmoid 输出概率
return self.a3

def train(self, X, y, epochs=5000):
    for epoch in range(epochs):
        # 前向: 算预测值
        y_pred = self.forward(X)

        # 反向: 算各层梯度 (具体推导下章细讲)
        loss = np.mean((y_pred - y.reshape(-1,1)) ** 2)
        dz3 = 2 * (y_pred - y.reshape(-1,1)) * y_pred * (1 - y_pred) / len(y)
        dz2 = (dz3 @ self.W3.T) * self.relu_derivative(self.z2)
        dz1 = (dz2 @ self.W2.T) * self.relu_derivative(self.z1)

        # 更新权重和偏置
        self.W3 -= self.lr * self.a2.T @ dz3
        self.b3 -= self.lr * dz3.sum(axis=0)
        self.W2 -= self.lr * self.a1.T @ dz2
        self.b2 -= self.lr * dz2.sum(axis=0)
        self.W1 -= self.lr * X.T @ dz1
        self.b1 -= self.lr * dz1.sum(axis=0)

        if epoch % 1000 == 0:
            print(f"第{epoch}轮, loss: {loss:.6f}")

# XOR 数据
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0, 1, 1, 0])

mlp = MiniMLP(lr=0.5)
mlp.train(X, y, epochs=5000)
print(f"\n 预测结果: \n{mlp.forward(X).round(3)}")

```

运行这个，loss 会快速下降。最终预测结果逼近 [0, 1, 1, 0]。一个单层感知机死活搞不定的事，多加几层就轻松解决了。

注意代码里的反向传播部分——我直接写了梯度计算公式但没解释。这事下章会用一整章来讲，因为它是整个深度学习的”心脏”。

宽度 vs 深度：一个对比实验

万能近似定理说一个足够宽的隐藏层就够了，但工程实践表明深度往往更高效。我们用实验来感受一下。

假设任务是拟合一个复杂的波浪函数 $y = \sin(x^2) + \cos(3x)$ ，在 $[-3, 3]$ 区间上取 200 个样本。

- **宽网络**：1 个隐藏层，50 个神经元。参数量约为 $1 \times 50 + 50 \times 1 = 100$ 。
- **深网络**：3 个隐藏层，每层 10 个神经元。参数量约为 $1 \times 10 + 10 \times 10 + 10 \times 10 + 10 \times 1 = 220$ 。

参数量差不多，甚至深网络还略多。训练相同轮数后，深网络通常拟合得更平滑、泛化更好。为什么？

因为深度网络在“分层抽象”。第一层学低层模式（曲线的局部起伏），第二层学中层模式（把局部起伏组合成更大尺度的波动），第三层学全局模式（整体的波浪形态）。这种逐层组合的方式就像一个画家——先打草稿，再画轮廓，最后上细节。

而宽网络只有一层，所有信息都在同一层处理，像一次画完所有细节——能做，但需要更多“笔触”（神经元）。

这解释了大模型为什么“深”：GPT-3 有 96 层，PaLM 有 118 层。每一层都在前一层的抽象基础上构建更高层级的理解。第一层可能只看到词的组合模式，第五十层已经看到了段落级逻辑，第一百层则在思考文章的论证结构。

人话总结这章：一个感知机太笨？多叠几层就好了。每一层对输入做一次“重新编码”，层数越多，抽象的层级越高。而且理论上一个足够大的单隐层网络就能模拟任何函数——但实际中，“深度”比“宽度”效率高得多，就像学乘法口诀比死记所有答案聪明一样。

这章留下的一个最大悬念是——反向传播那些梯度到底怎么算出来的？那个 dz_3 、 dz_2 、 dz_1 的推导过程，是深度学习里最精彩也最容易让人劝退的部分。

但别怕。下一章我会给你两条路：一条纯文字版，讲故事就能理解；另一条数学版，手算给你看。选你舒服的那条。Ready？

常见疑问

Q: 隐藏层越多越好吗？

A: 不完全是。太深会梯度消失，太浅表达能力不够。现代大模型动辄几百层是因为有残差连接等技术兜底。在没有这些技术之前，10 层就已经很难训练了。

Q: 万能近似定理说一个隐藏层就够了，为什么还要很多层？

A: “能逼近”不等于“容易学”。一个隐藏层虽然理论上能表示任何函数，但需要的神经元数量可能是指数级的。深层网络用更少的参数达到同样的效果。

第 9 章 反向传播：错误追责的艺术

难度：★★★★☆

观光路线：通过交响乐指挥和接力赛两个比喻，纯文字理解反向传播在做什么。0 公式，读完能讲给别人听。

登山路线：手算 2 层网络的完整链式法则，用 PyTorch autograd 自动求导。彻底搞懂梯度如何从输出层”反向流动”到输入层。

写给数学恐惧者（纯文字版，0 公式）

这章可能是全书”看起来最吓人”的一章。反向传播四个字，劝退过无数人。

但它的核心思想，用大白话三分钟就能讲清楚。你可以跳过后面的所有公式，读完这节，直接去用 PyTorch。损失的只是一些”精确感”，不是理解。

比喻一：交响乐团排练

你是乐团指挥。乐团刚演完德沃夏克第九交响曲，演得烂透了——第二乐章那个英国管独奏，第一声就跑调了。你的任务不是笼统地说”大家回去再练练”，而是精确地告诉每个人：**你对最终效果有多大的责任。**

怎么分配责任？

你不能平等分配——打击乐手和首席小提琴犯同样错误，对整体效果的伤害不是一个量级。你需要顺着”影响链”往回推：

- 第二乐章整体崩了，是因为英国管独奏跑调（这是直接原因）。
- 英国管跑调，是因为那个乐手在换气的时机判断错了半拍（这是根本原因）。
- 同时，定音鼓那一下轰得太响，把英国管的声音盖住了，导致听众更难意识到跑调（这是间接影响）。

你看，你在”反向追踪”——从最终的效果（演出烂了）往回找，经过层层因果链条，算出每个乐手对最终结果的影响程度。然后你告诉英国管手：你需要在第 47 小节多注意换气时机；告诉定音鼓手：你那一下轻 20%。

下一次排练，就按这个反馈调整。一次比一次好。

反向传播做的事，跟这一模一样。只不过：- “最终效果” = 损失函数（预测和真实答案的差距）- “每个乐手” = 网络中每个参数（权重和偏置）- “调整力度” = 梯度（这个参数对损失有多大的影响）

比喻二：接力赛传错棒子

换一个更贴近”传播”这个词的比喻。

四个人跑接力赛。第四棒冲线时发现成绩比预期慢了 2 秒。问题出在哪？

你不能直接怪第四棒。成绩是四个人的总时间。你需要往前追溯：- 第四棒慢了，可能因为第三棒交棒时位置偏了（影响第四棒的起跑时机）。- 第三棒交棒偏了，可能因为第二棒跑到最后变速太突然。- 第二棒变速突然，可能因为第一棒起步就慢了，导致第二棒心态急。

你从终点往回走，每一段都记录下”这一段的偏差里面，有多少是前面那一棒的责任”。最后你算出来：第一棒责任 30%，第二棒 20%，第三棒 40%，第四棒 10%。然后每个人按比例调整训练重点。

反向传播就是这件事的数学化——从输出端（第四棒）开始，把”误差”沿着网络一层层往回传，每经过一层就算一次”责任比例”（梯度），然后按这个比例调整该层的参数。

为什么这件事这么关键

没有反向传播，多层神经网络就是空中楼阁。你想啊——一个 100 层的网络，最后一层告诉你”错了”。但你到底该调整第一层的哪个参数？

正向思考的话，第一层每个参数都有无数条路径通向输出，你几乎不可能算出每条路径的独立贡献。反向传播用链式法则一举解决了这个问题：它不是”正向试探”，而是”反向归因”。

就像你不会一个个去问乐团成员”你觉得你对跑调贡献了多少”，而是用指挥的耳朵和专业判断，顺着因果链反向分配责任。高效、精确、可计算。

好了，故事讲完了。如果你想保留这些直觉但不想碰公式，直接跳去”PyTorch autograd 演示”那节。如果你准备好了跟公式正面硬刚，深呼吸，泡杯茶，往下看。

链式法则：用一个 2 层网络手算

（接下来的内容需要微积分基础，特别是偏导数和链式法则。）

回顾上一章的 MiniMLP，反向传播部分的代码是这样写的：

```
dz3 = 2 * (y_pred - y) * y_pred * (1 - y_pred) / len(y)
dz2 = (dz3 @ W3.T) * relu_derivative(z2)
dz1 = (dz2 @ W2.T) * relu_derivative(z1)
```

这几个 dz 到底怎么来的？我们来手算一个超简单的 2 层网络。

网络设定

- 输入：1 个值 x
 - 隐藏层：1 个神经元，权重 w_1 ，偏置 b_1 ，激活函数 sigmoid
 - 输出层：1 个神经元，权重 w_2 ，偏置 b_2 ，激活函数 sigmoid
 - 目标 y ，损失函数： $MSE = (y_pred - y)^2$
-

前向传播（算出预测值，步进编号方便链式推导）

第 1 步: $z_1 = w_1 \cdot x + b_1$

把输入乘权重加偏置。这一步就是”加权投票”的那部分。

第 2 步: $a_1 = \sigma(z_1) = \frac{1}{1+e^{-z_1}}$

过 Sigmoid 激活，把结果压缩到 (0,1) 区间。人话：把原始的加权和变成一个”概率感”的数值。

第 3 步: $z_2 = w_2 \cdot a_1 + b_2$

隐藏层输出再次加权加偏置。

第 4 步: $\hat{y} = a_2 = \sigma(z_2)$

再过一次 Sigmoid，得到最终预测。

第 5 步: $L = (\hat{y} - y)^2$

算损失。MSE 的好处是简单直观——你猜的和真实答案差了多少平方。

反向传播（沿着链条一步步往回算梯度）

现在我们想知道：L 对 w_1 、 b_1 、 w_2 、 b_2 分别有多敏感（偏导数是多少）。

从最后一层开始（第 5 步往回推）：

L 对 w_2 的梯度，用人话说是” w_2 变动一点点，损失会变多少”：

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2}$$

逐段翻译这条链： $-\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$ —— 损失对预测的敏感度，猜得越离谱，梯度越大。
 $-\frac{\partial \hat{y}}{\partial z_2} = \hat{y}(1 - \hat{y})$ —— Sigmoid 的导数，预测越”犹豫”（接近 0.5），梯度越大。
 $-\frac{\partial z_2}{\partial w_2} = a_1$ —— w_2 对 z_2 的直接影响，就是上一个隐藏层的输出。

合起来：

$$\frac{\partial L}{\partial w_2} = 2(\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) \cdot a_1$$

L 对 b_2 的梯度类似，只是最后一项变成 1（因为 $z_2 = w_2 a_1 + b_2$ 对 b_2 求导得 1）。

再往回推一层到隐藏层：

L 对 w_1 的梯度，链条更长：

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

后两项是新的： $-\frac{\partial z_2}{\partial a_1} = w_2$ —— 这个很关键！ w_2 把”输出层的误差”传递回了隐藏层。 - $\frac{\partial a_1}{\partial z_1} = a_1(1 - a_1)$ —— Sigmoid 导数，跟前面一样。 $-\frac{\partial z_1}{\partial w_1} = x$ —— 最原始的输入。

把前面算过的部分（L 到 z_2 那三段）记作 δ_2 （输出层的”误差信号”），那么：

$$\delta_2 = 2(\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$$

$$\frac{\partial L}{\partial w_1} = \delta_2 \cdot w_2 \cdot a_1(1 - a_1) \cdot x$$

你看这个结构—— δ_2 从输出层传回来，乘上 w_2 （跨越层的”责任传递系数”），再乘上本层的导数 $a_1(1 - a_1)$ ，就得到了隐藏层的误差信号。

“反向传播”这个名字的由来：误差信号 δ 是从输出层开始，沿着计算图反向流动的。每一层拿到来自后面的 δ ，乘上自己的导数，加上自己的权重，再传给更前面的一层。

放到矩阵里

上面的手算是单样本、单神经元版本。真实网络里， x 是一个矩阵（多个样本），每层有多个神经元。这时候公式变成矩阵形式，但逻辑完全相同：

```

δ_L = ∇_a L ⊙ σ'(z_L)          # 输出层误差信号
δ_l = (δ_{l+1} · w_{l+1}^T) ⊙ σ'(z_l)  # 逐层回传
∇w_l = a_{l-1}^T · δ_l          # 这一层权重的梯度
∇b_l = sum(δ_l)                 # 这一层偏置的梯度

```

那个 $\odot$ 是逐元素相乘（Hadamard 积），其他的 $\cdot$ 是矩阵乘法。这就是你在 MiniMLP 代码里看到的 dz_3 、 dz_2 、 dz_1 的来源。

每一条公式对应一个人话翻译： - δ_L ：输出层错得多离谱，乘以 Sigmoid 有多”困惑”。 - δ_l ：把后一层的错误按权重比例分回来，再乘以本层的困惑程度。 - ∇w_l ：本层输入的贡献 $\times$ 本层的错误。 - ∇b_l ：本层错误的平均水平。

PyTorch autograd：你什么都不用算

上面写了那么多，实际写代码时你一行梯度公式都不用写。PyTorch 的 autograd 引擎会自动追踪每一步计算，构建计算图，然后帮你算所有梯度。

```

import torch

# 构建和前面手算一模一样的 2 层网络
x = torch.tensor([[1.0]], requires_grad=False)  # 输入
y = torch.tensor([[0.0]])                        # 目标标签：希望输出接近 0

# 参数初始化
w1 = torch.tensor([[2.0]], requires_grad=True)  # 权重 1，需要求梯度

```

```

b1 = torch.tensor([[0.0]], requires_grad=True)    # 偏置1
w2 = torch.tensor([[3.0]], requires_grad=True)    # 权重2
b2 = torch.tensor([[0.0]], requires_grad=True)    # 偏置2

# 前向传播 (PyTorch 自动记录每一步)
z1 = x @ w1 + b1                                # 矩阵乘法 + 偏置
a1 = torch.sigmoid(z1)                          # 过激活函数
z2 = a1 @ w2 + b2
y_pred = torch.sigmoid(z2)

# 算损失
loss = (y_pred - y) ** 2

# 反向传播——就这一行！
loss.backward()

# 查看自动算出来的梯度
print(f"w1 梯度: {w1.grad.item():.4f}")          # autograd 自动帮你算了整个链式法则
print(f"b1 梯度: {b1.grad.item():.4f}")
print(f"w2 梯度: {w2.grad.item():.4f}")
print(f"b2 梯度: {b2.grad.item():.4f}")

# 验证：手动按前面的公式算一遍 w2 的梯度
grad_w2_manual = 2 * (y_pred - y) * y_pred * (1 - y_pred) * a1
print(f"w2 梯度(autograd): {w2.grad.item():.6f}")
print(f"w2 梯度(手动算): {grad_w2_manual.item():.6f}")

```

autograd 帮你算出来的，跟你前面用链式法则手算的，一模一样。

这就是为什么深度学习框架让研究和工程变得如此高效——你只需要写前向传播，反向传播自动完成。几百层的网络？几百万个参数？全自动。

人话总结这章：反向传播就是“从终点往回看，算清谁的责任占多少”。链式法则让它可以用数学精确地完成这件事——把总误差沿着网络一层层往回传，每传一层就算出该层每个参数的“贡献度”（梯度）。PyTorch 把这个过程全自动化了，写代码时你根本不用手动推公式。但理解这个过程，你在调参、debug、设计新结构时才会有直觉。

这事没完——有了梯度只是知道“往哪走”，但你具体该走多远？走太快会不会冲过头？这些问题，第 11 章会来回答。准备好下山了吗？

常见疑问

Q: 反向传播和人类大脑学习有什么不同？

A: 人类大脑不靠链式法则逐层传误差。人脑有复杂的反馈环路和化学信号。反向传播是纯粹的数学优化工具，它有效但不代表生物学习机制。

Q: PyTorch 的自动求导是怎么实现的？

A: 它构建了一张“计算图”——把你所有的操作记录成一张有向无环图，前向时记录操作顺序，反向时从最后的 loss 出发，按图索骥自动应用链式法则。

Q: 反向传播和梯度下降是什么关系？

A: 反向传播负责”算梯度”——算出每个参数对损失函数有多大的影响。梯度下降负责”用梯度”——根据算出来的梯度决定参数往哪个方向更新、走多大步。它们是搭档关系，不是一回事。可以把反向传播想象成”测量员”（告诉你每个地方有多陡），梯度下降是”登山者”（根据测量结果往下走）。PyTorch 里 `loss.backward()` 是做反向传播，`optimizer.step()` 是做梯度下降。

第 10 章 Softmax 与交叉熵：让网络学会“打分”

难度：★★★★☆☆

观光路线：理解 Softmax 就是把分数转成概率分布，交叉熵就是衡量两个概率分布有多像。温度参数的概念。会调 API 就行。

登山路线：手写 Softmax + 交叉熵的 numpy 实现，搞懂温度参数如何控制“信心”，看懂 PyTorch 的 CrossEntropyLoss 内部做了什么。

前几章的网络输出层都在干一件事——输出一个 0 到 1 之间的数，表示“是”或“不是”某种东西。二分类，yes or no。

但真实世界的问题很少是二选一。一张图里有猫还是狗还是鸟？一句话的下一词是”的”还是”了”还是”吗”？这需要输出一组概率——每个候选答案一个概率，加起来等于 1。

Softmax 干的就是这件事。交叉熵干的则是”评价这组概率猜得好不好”。

Softmax：从分数到概率

你考完期末，教务系统里有你的原始分数。但最终你需要的是排名百分比。教务系统做了一件事：把所有分数通过一个公式换算，让结果在 0 到 1 之间，而且所有人的结果加起来等于 1——每个人的”排名占比”。

Softmax 就是教务系统那个换算公式。

人话版：你有三个候选答案，网络原始输出是 [2.0, 1.0, 0.1]（这些原始分数叫 logits）。第一个答案看起来最有戏（分数最高），但你需要的不是原始分数，而是一个概率分布——第一个答案赢的概率多大？第二个呢？Softmax 说：对每个分数取指数（让大数更大、小数不小），然后除以总和。

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

拿 [2.0, 1.0, 0.1] 算一遍：

- $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.1} = 1.105$
- 总和 = $7.389 + 2.718 + 1.105 = 11.212$
- 概率 = $[7.389/11.212, 2.718/11.212, 1.105/11.212] = [0.659, 0.242, 0.099]$

答案出来了：答案 1 有 65.9% 的胜率，答案 2 有 24.2%，答案 3 有 9.9%。加加减减刚好是 1。

“取指数”这一步很妙——它放大了分数之间的差距。如果不用指数，[2.0, 1.0, 0.1] 直接除以 3.1 得到 [0.645, 0.323, 0.032]。Softmax 让最高分更突出、最低分更不像回事。这在多分类问题里是个好事——你不会希望网络对所有选项都暧昧不清。

温度参数：控制你的“锐度”

Softmax 有个不太出名但非常重要的参数——温度 T（temperature）。

$$\text{Softmax}(z_i; T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

人话比喻：想象你在做一个决策。外面天气热得你心烦意乱（高温 T 很大），你判断力下降，对所有选项都差不多”都可以吧”。外面天气凉爽头脑清醒（低温 T 很小），你能做出非常锐利的判断，“就是 A，没跑了”。

- $T \rightarrow 0$ ：Softmax 退化成 argmax。只有最高分的那个选项拿 100% 概率，其他都是 0。像你在冰天雪地里做决定——非黑即白，绝不含糊。
- $T = 1$ ：标准 Softmax。不冷不热的常温。
- $T \rightarrow \infty$ ：所有选项概率趋近相等 ($1/n$)。像你在桑拿房里做决定——每个选择看起来都差不多。

温度在 GPT 这样的生成模型里是核心参数。你想让 AI 更有创意？调高温度（让它不那么确定最优选哪个词）。你想让 AI 严谨保守？把温度降到接近 0（永远选最可能的下一个词）。

用代码感受一下这个效果：

```
import numpy as np

def softmax(logits, temperature=1.0):
    """Softmax 函数，带温度参数"""
    # 先减去最大值防止指数溢出（数值稳定性技巧）
    logits_stable = logits - np.max(logits)
    exp_vals = np.exp(logits_stable / temperature)
    return exp_vals / exp_vals.sum()

# 模拟三个候选答案的原始分数
logits = np.array([2.0, 1.0, 0.1])

# 不同温度下的概率分布
for T in [0.1, 0.5, 1.0, 2.0, 5.0]:
    probs = softmax(logits, T)
    print(f"温度 T={T:.1f}: {np.round(probs, 4)}, 最确定={probs.max():.4f}")
```

你会看到温度从 0.1 到 5.0，最可能的那个选项从”几乎 100% “慢慢降到了跟其他选项五五开。

有意思的是——在 LLaMA、GPT 这样的模型里，“调温度”是产品经理最喜欢的一个旋钮。想要更靠谱的输出？温度 0.3。想要让用户觉得 AI 很”有创意”？温度 0.8。这不是玄学，就是 Softmax 的 T 参数在起作用。

交叉熵：谁的天气预报更准

网络输出了一组概率，真实答案是什么？比如真实答案是”猫”（用 one-hot 表示为 $[1, 0, 0]$ ），网络猜的是 $[0.659, 0.242, 0.099]$ 。怎么评价这个猜测的质量？

交叉熵（Cross-Entropy）就是干这个的”评分裁判”。

人话比喻：两个天气预报员，都预报明天的天气是”晴/阴/雨”三种。真实的结果是”晴”。预报员 A 说”晴 90%，阴 5%，雨 5%”，预报员 B 说”晴 40%，阴 35%，雨 25%”。谁更准？

交叉熵的评判标准是：看真实发生的事，你的预测给了它多高的概率。概率越高，交叉熵越低（越小越好）。A 的交叉熵约 $-\log(0.9) = 0.105$ ，B 的交叉熵约 $-\log(0.4) = 0.916$ 。A 完胜。

公式长这样：

$$H(p, q) = - \sum_i p_i \cdot \log(q_i)$$

p 是真实分布（one-hot）， q 是预测分布（Softmax 输出）。因为 p 是 one-hot，只有一个位置是 1，其他都是 0。所以这个求和实际上只算了一项：网络给“正确答案”那个位置预测的概率，取负对数。

翻译成成人话：交叉熵 = $-\log(\text{预测为正确答案的概率})$ 。

如果网络 100% 确定是正确答案（概率=1.0），交叉熵 = $-\log(1) = 0$ 。完美。

如果网络完全猜错（概率接近 0），交叉熵接近无穷大。惨。

NumPy 从零实现

Softmax + 交叉熵合在一起，就是神经网络最常用的分类损失函数。代码出奇地短：

```
import numpy as np

def cross_entropy_loss(logits, y_true):
    """
    计算交叉熵损失
    logits: 网络原始输出（未过 Softmax），shape: (n_samples, n_classes)
    y_true: 真实标签的 one-hot 编码，shape: (n_samples, n_classes)
    """
    # 1. Softmax: 原始分数 → 概率分布
    logits_stable = logits - np.max(logits, axis=1, keepdims=True)
    exp_vals = np.exp(logits_stable)
    probs = exp_vals / exp_vals.sum(axis=1, keepdims=True)

    # 2. 交叉熵: 真实答案的负对数似然
    # 因为 y_true 是 one-hot, 所以相当于取概率矩阵中对角线的负 Log
    n_samples = logits.shape[0]
    # 只取真实类别位置的概率
    correct_probs = probs[np.arange(n_samples), y_true.argmax(axis=1)]
    loss = -np.log(correct_probs + 1e-8) # 加 1e-8 防止 log(0)
    return loss.mean()

# 模拟: 3 个样本, 4 个类别
logits = np.array([
    [2.0, 1.0, 0.1, 0.3], # 第 0 类分数最高 → 正确答案是第 0 类
    [0.5, 2.5, 1.0, 0.2], # 第 1 类分数最高 → 正确答案是第 1 类
    [0.1, 0.3, 0.2, 3.0], # 第 3 类分数最高 → 正确答案是第 3 类
])
y_true = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 0, 1],
])
```

```

])

print(f"交叉熵损失: {cross_entropy_loss(logits, y_true):.4f}")

# 对比: 如果网络猜得特别离谱
bad_logits = np.array([
    [0.1, 2.0, 1.0, 0.3], # 明明第0类才对, 却给了第1类最高分
    [2.5, 0.1, 0.2, 0.1],
    [0.1, 3.0, 0.2, 0.3],
])
print(f"猜错时的损失: {cross_entropy_loss(bad_logits, y_true):.4f}")

```

一个细节: 代码里 `logits - np.max(logits)` 这一步叫“数值稳定性”处理。指数函数增长太快了, 如果 `logits` 里有比较大的数 (比如 50), e^{50} 会直接溢出成 `inf`。减去最大值后, 最大那个变成 $e^0 = 1$, 其他都是 $e^{\text{负数}}$, 稳定又准确, 而且数学上等价 (分子分母同除 e^{max} 不改变结果)。

PyTorch 版: 一行搞定

PyTorch 把 Softmax 和交叉熵打包成了一个函数 `nn.CrossEntropyLoss()`。但有个坑: 它要求输入是**原始 logits** (别自己先过 Softmax), 标签是**类别索引** (别搞成 one-hot)。

```

import torch
import torch.nn as nn

logits = torch.tensor([
    [2.0, 1.0, 0.1],
    [0.5, 2.5, 1.0],
], dtype=torch.float32)

# y 是类别序号, 不是 one-hot!
y = torch.tensor([0, 1]) # 样本1 是第0类, 样本2 是第1类

criterion = nn.CrossEntropyLoss()
loss = criterion(logits, y)
print(f"PyTorch 交叉熵: {loss:.4f}")

```

这个函数等价于 `nn.LogSoftmax + nn.NLLLoss`。内部自动做了 Softmax、取 log、取负对数似然。所以你的网络最后一层应该输出**未经 Softmax 的 logits**。

人话总结这章: Softmax 把网络的原始分数变成概率分布 (有几个候选就分几份, 加起来 100%), 温度参数控制这组概率的“锐度”。交叉熵评估预测的质量——预测给正确答案的概率越大, 交叉熵越小。这俩合在一起, 是几乎所有分类问题的标准配置, 也是大模型逐词生成文本时每一步都在调用的计算。

一个你可能没注意到的问题冒出来了: 有了损失函数、有了梯度, 下一步是不是直接拿着梯度更新参数就行?

是。但“怎么更新”这件事, 比你想象的有讲究。梯度告诉你去哪个方向, 但没告诉你要走多远。走太快你可能滚下悬崖, 走太慢你到天黑还在半山腰。

下山的技术，下章见。

常见疑问

Q: 温度参数在实际项目中怎么设?

A: 推理时通常 $T=1$ (默认值)。需要增加输出多样性时调高 T (比如创意写作)，需要更确定时调低 T (比如数学计算、代码生成)。知识蒸馏中教师模型通常用较高的 T (2-5) 来软化标签。

Q: 交叉熵和 MSE (均方误差) 在分类任务中哪个好?

A: 分类任务用交叉熵。交叉熵配合 Softmax 天然匹配——Softmax 输出概率分布，交叉熵衡量概率分布的差距。用 MSE 会面临梯度消失问题。

第 11 章 梯度下降：蒙着眼怎么下山

难度：★★★★☆☆

观光路线：理解梯度下降就是“每走一步都朝最陡的下坡方向走”，知道学习率大/小的不同后果，了解批量/随机/小批量三种吃法。

登山路线：从零实现 SGD + 可视化优化路径，理解 Momentum 和 Adam 的直觉，手动跑学习率调度器对比实验。

反向传播算出了梯度——每个参数的“影响度”。现在问题变成了：拿着这张“影响度清单”，你到底该对每个参数调多大？

这个问题在数学上叫**优化**，在直觉上就是**下山**。我们的目标是把损失函数的值降到最小——也就是找到山脚下最低的那个点。

雾天蒙眼下山

想象你在大雾天站在一座山的半山腰。能见度不到两米，完全看不到山脚在哪里。你唯一的线索是：脚底下的地面往哪个方向倾斜最厉害。

怎么做？每一步你都感受一下脚下的坡度（梯度），然后朝最陡的方向迈一步。迈完一步，再感受一下新位置的坡度，再迈一步。反复循环，直到脚下是平的——到山脚了。

这就是梯度下降。

$$\theta_{new} = \theta_{old} - \eta \cdot \nabla_{\theta} L$$

人话翻译：新参数 = 旧参数 - 学习率 x 梯度。梯度告诉你“哪个方向坡最陡”，学习率告诉你“这一步迈多大”。那个负号表示“往坡下走”（梯度指向坡上，你要反着走）。

学习率：下山最关键的旋钮

选错了学习率，比选错了网络结构还麻烦。

学习率太小（比如 0.000001）：你每一步只挪一毫米。理论上最终能到山脚，但实际上——你走到头发白了都还在半山腰。而且容易卡在浅坑里出不来（局部最小值）。

学习率太大（比如 10）：你一步跨出三米远。快是快，但可能在两个山坡之间来回跳跃，永远踩不到谷底。甚至越跳越高——损失不降反升。

适中的学习率（通常 0.001 到 0.1 之间）：够快，但又不至于失控。这个值没有理论公式，纯靠经验。有一个非常经典的类比：试学习率就像调热水龙头——拧太小水不热，拧太大烫得你跳起来。

用一段代码直观感受三种学习率的差别：

```
import numpy as np
import matplotlib.pyplot as plt

def f(x): # 损失函数：一个经典的碗形函数
    return x**2 + 3*np.sin(5*x) + 5
```

```

def gradient(x): # 导数 (梯度)
    return 2*x + 15*np.cos(5*x)

def gradient_descent(start_x, lr, steps=50):
    x = start_x
    path = [x]
    for _ in range(steps):
        x = x - lr * gradient(x)
        path.append(x)
    return np.array(path)

# 三种学习率对比
paths = {
    'lr=0.001 (龟速)': gradient_descent(2.0, 0.001, 50),
    'lr=0.05 (适中)': gradient_descent(2.0, 0.05, 50),
    'lr=0.5 (飙车)': gradient_descent(2.0, 0.5, 50),
}

# 画图
x_range = np.linspace(-3, 3, 500)
plt.figure(figsize=(12, 5))
plt.plot(x_range, f(x_range), 'k-', linewidth=1.5, alpha=0.5, label='损失函数曲线')
colors = ['blue', 'green', 'red']
for (label, path), color in zip(paths.items(), colors):
    plt.plot(path, f(path), 'o-', color=color, markersize=4,
             linewidth=1, alpha=0.8, label=label)
    plt.plot(path[-1], f(path[-1]), 'o', color=color, markersize=10)
plt.legend(fontsize=11)
plt.title('不同学习率的梯度下降路径对比', fontsize=14)
plt.xlabel('参数值'); plt.ylabel('损失')
plt.grid(alpha=0.3)
plt.show()

```

运行这个，你会看到：-lr=0.001（蓝色）的路径在曲线上几乎是”爬行”，50步还没走到谷底。-lr=0.05（绿色）稳稳当当滑到了全局最低点。-lr=0.5（红色）在谷底附近疯狂震荡，像个喝醉的人在台阶上反复摔跤。

三种“吃法”：批量 / 随机 / 小批量

真实训练不是在一个样本上算梯度，而是面对成百上千万样本。每次算梯度的时候，选多少样本参与计算？这就是三种”吃法”的不同。

批量梯度下降（BGD）——把所有训练样本都过一遍，加起来算平均梯度，然后更新一次参数。

优点：梯度算得准，方向稳。缺点：一百万样本你才更新一次，慢到窒息。而且内存不够用——整个 ImageNet 塞不进一张显卡。

随机梯度下降（SGD）——每次随机挑一个样本，用它算梯度，立马更新参数。

优点：快到飞起，每个样本都更新一次。缺点：梯度方向非常”抽风”——一个样本的梯度可能跟整体方向完全相反。优化路径歪歪扭扭，像个醉汉逛公园。

小批量梯度下降（Mini-batch SGD）——折中方案。每次随机挑一小撮样本（比如 32 或 64 个），算平均梯度，更新参数。

优点：利用 GPU 并行计算（一次算 32 个样本跟算 1 个样本时间差不多），梯度方向比单个样本稳定得多，内存友好。这是现在所有人实际在用的方式。当大家说”SGD”的时候，99%指的是 Mini-batch SGD。

代码上的区别只有一行：你是一次喂整个数据集，还是一个 batch：

```
# Mini-batch SGD 的核心循环
for epoch in range(num_epochs):
    # 每轮打乱数据顺序
    indices = np.random.permutation(len(X))
    for start in range(0, len(X), batch_size):
        batch_idx = indices[start:start + batch_size]
        X_batch, y_batch = X[batch_idx], y[batch_idx]

        # 前向+反向（跟之前一模一样，只不过只在 batch 上算）
        loss, grad = compute_loss_and_gradient(X_batch, y_batch)

        # SGD 更新
        params -= lr * grad
```

batch_size 一般选 32、64、128、256。太小：梯度不稳定，GPU 利用率低。太大：梯度方向确实更准了，但收敛更慢，而且可能被 GPU 显存卡住。

从 SGD 到 Momentum：加上”惯性”

SGD 有个毛病：到了谷底附近，它开始在谷底左右来回晃——因为即使到了最低点，梯度也不是严格为零，还是会带着它来回弹。

于是就有人想了个点子：给 SGD 加惯性。

就像你推一个球下山——不是一步一停地推，而是让它自己滚。球有惯性，不容易被局部的小起伏带偏，而且越滚越快。

Momentum（动量法）的做法：不只参考当前这一步的梯度，还”记住”之前几步的方向。公式写成：

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} L$$

$$\theta_t = \theta_{t-1} - \eta \cdot v_t$$

v 是”速度”（历史梯度的指数加权平均）， β 通常取 0.9。如果过去几步都在往左走，即使当前这一步梯度指向右边（可能是个局部小坑），动量也会”拽着”你继续往左。

人话：你有了一条”短期记忆”，不会被一时的小波动骗走。

Adam：现在的事实标准

SGD 加动量已经很好了，但每个参数的”理想学习率”可能不一样。有的参数需要大步快跑（梯度大、影响大），有的参数需要小步慢跑（已经快到位了）。能不能让学习率自适应？

这就是 Adam（Adaptive Moment Estimation）做的事——给每个参数配一个独立的学习率。

人话版解释 Adam 的三个关键点：

1. 跟 Momentum 一样，它记住了”历史方向”（一阶矩， v ）。
2. 它还记住了”历史振动的剧烈程度”（二阶矩， s ）。如果一个参数的历史梯度的平方很大（说明它在剧烈震荡），Adam 会主动给它减速。
3. 它把一阶矩和二阶矩的信息结合起来，算出一个”自适应步长”。

Adam 的名字本身是个巧妙的拆解——Adaptive + Moment。今天训练大模型的默认选择几乎永远是 Adam 或者它的变体 AdamW。

学习率调度器：别全程用同一个学习率

你下山的时候，一开始山势陡峭，你应该迈大步；快到山脚了，需要小心别冲过头，应该迈小步。

训练也是同理。所以实际训练中，学习率不是常数，而是跟着训练进度衰减的。这就是学习率调度器。

几种常见的调度策略：

- **Step 衰减**：每 N 个 epoch，学习率乘以 0.1。简单粗暴但有效。
- **Cosine 衰减**：学习率沿着余弦曲线平滑下降，从初始值降到接近 0。现在训练大模型的主力军。
- **Warmup**：训练开始阶段学习率从小到大线性增长（从 0 涨到目标值），然后再衰减。为什么？因为刚初始化时参数随机得离谱，一开始梯度方向也不靠谱。先”热身”一阵，让参数走到一个相对靠谱的区域，再加速。

Warmup + Cosine 衰减合在一起（先涨后降）在 GPT 训练里几乎成了标配。下面是代码演示：

```
import numpy as np
import matplotlib.pyplot as plt

def cosine_schedule_with_warmup(step, warmup_steps, total_steps, base_lr=0.01):
    """Warmup + Cosine 衰减的学习率调度"""
    if step < warmup_steps:
        # 线性预热：从 0 涨到 base_lr
        return base_lr * step / warmup_steps
    else:
        # Cosine 衰减：从 base_lr 平滑降为 0
        progress = (step - warmup_steps) / (total_steps - warmup_steps)
        return base_lr * 0.5 * (1 + np.cos(np.pi * progress))

# 画图对比三种调度器
steps = np.arange(1000)
lr_cosine = [cosine_schedule_with_warmup(s, 200, 1000, 0.01) for s in steps]
lr_step = [0.01 * (0.5 ** (s // 300)) for s in steps] # 每 300 步减半

plt.plot(steps, lr_cosine, label='Cosine + Warmup', linewidth=2)
plt.plot(steps, lr_step, label='Step 衰减', linewidth=2)
plt.xlabel('训练步数'); plt.ylabel('学习率')
plt.legend(); plt.grid(alpha=0.3)
plt.title('不同学习率调度策略对比')
plt.show()
```

你会看到 Cosine 曲线非常平滑——一开始慢热（warmup 阶段），然后一路滑到底，没有阶跃感。Step 衰减则像走楼梯，每跌一段就突然降智。

人话总结这章：梯度下降就是”朝最陡的反方向迈一步”，反反复复直到平底。学习率管步长，batch size 管每次参考多少样本，Momentum 管惯性，Adam 管自适应步长，Cosine+Warmup 管你开始慢热、中途加速、末尾收步。

你可能会想：既然如此，到底该用哪种激活函数？ReLU？Sigmoid？还是 LLaMA 用的 SwiGLU？这里面有什么讲究？

这就是下一章的内容——激活函数的”选妃大赛”。

常见疑问

Q: SGD、Momentum、Adam 到底选哪个？

A: 2026 年的共识：训练大模型用 AdamW（带 weight decay 的 Adam），微调用 SGD+动量，小规模实验随便选。Adam 收敛快但泛化可能不如 SGD。不过大模型时代，收敛快比泛化差异重要得多。

Q: 学习率 warmup 为什么有必要？

A: 训练初期，参数是随机的，梯度方向不稳定。如果一上来就用大的学习率，模型可能在错误方向上跨出大步。warmup 让模型先用小步”试探方向”，找到相对稳定的梯度方向后再加速。

第 12 章 激活函数：那条让网络“活过来”的曲线

难度：★★★★☆☆

观光路线：认识 Sigmoid / Tanh / ReLU / LeakyReLU / GELU 五大激活函数的形状和脾气，知道 SwiGLU 是 LLaMA 用的。

登山路线：搞清楚为什么 ReLU 这么简单却统治了十年，手写对比实验看不同激活函数在深层网络中的表现，理解梯度消失的根源。

第 8 章说过一句话：“没有非线性激活函数，多层网络就是一层。”这章就盯着这个“非线性”来看——到底有哪些选择，为什么有些被淘汰了，有些成了明星。

回忆一下，一个神经元的前向计算是两段：1. 加权求和： $z = wx + b$ 2. 过激活函数： $a = f(z)$

第一步是线性的。线性的东西怎么叠加还是线性——这就是为什么第二步必须有非线性。否则 100 层网络等价于 1 层： $W_{100} \cdots W_2 W_1 x$ 合并起来还是一个大矩阵 $W_{\text{total}} x$ 。

激活函数就是夹在“加权求和”这个线性操作和下一层之间的“搅局者”——它把线性世界搅成非线性的，让网络有了表达能力。

五大激活函数速览

Sigmoid：优雅但过时

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

形状像一个大写的 S。输入越小越接近 0，越大越接近 1，中间平滑过渡。

脾气：输出永远在 (0, 1) 之间，适合把数值“挤压”成一个概率。早年的神经网络输出层标配。

问题：**梯度消失**。当输入绝对值很大时（比如 $x=10$ 或 $x=-10$ ），Sigmoid 的导数趋近于 0。反向传播的时候，梯度一层层乘回去，每经过一个这种“饱和”的 Sigmoid，梯度就被乘个接近 0 的数——传到前面几层时，梯度已经无限接近 0 了。深层网络的前几层几乎收不到任何梯度信号，参数根本不更新。

另一个毛病：输出不以 0 为中心（均值约 0.5）。下一层收到的永远是被“垫高”了的信号，这让优化过程的收敛速度慢上一截。

Tanh：Sigmoid 的改良版

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

形状跟 Sigmoid 几乎一样，但“拉伸”了一下——输出范围变成了 (-1, 1)，均值为 0。

脾气：解决了 Sigmoid 的“不以 0 为中心”问题。但由于本质上跟 Sigmoid 一样两端会饱和，梯度消失的问题并没有根治。

ReLU: 简单的力量

$$\text{ReLU}(x) = \max(0, x)$$

就到这，没别的。大于 0 的时候原样通过，小于 0 的时候一刀切为 0。

脾气：这可能是深度学习史上最简单的函数之一——就是一条“折断”的直线。但它统治了深度学习差不多十年。

为什么这么简单的东西这么有效？三个原因：

第一，不饱和。 $x > 0$ 的区域导数永远是 1。反向传播的时候，梯度“原样通过”，不会越传越小。一个 100 层的 ReLU 网络，只要大多数神经元在正区激活，梯度就能一路畅通流到第一层。梯度消失问题被大幅缓解。

第二，计算极快。 ReLU 就是 $\max(0, x)$ ——一条 if 语句的事。相比 Sigmoid 要算指数、除法，ReLU 几乎不花时间。训练一个深层网络，每个神经元每步都省一点计算，累加起来非常可观。

第三，稀疏激活。 $x < 0$ 时输出就是 0，这个神经元“死了”。在一个训练好的 ReLU 网络里，通常只有一部分神经元在激活。这种稀疏性让网络有了“选择性”——不同的输入激活不同的神经元组合，有点像人脑的稀疏编码。

LeakyReLU: 给“死掉的神经元”一根呼吸管

$$\text{LeakyReLU}(x) = \max(0.01x, x)$$

ReLU 有一个痛点： $x < 0$ 的区域梯度也是 0，神经元一旦进入负区就彻底不更新了——“死了”。LeakyReLU 在负区给了一个很小的坡度 (0.01)，给那些不小心“咽气”的神经元留了一条命。

这就是全部区别。常数 0.01 可以调，但通常就用这个。

GELU: ReLU 的“光滑升级版”

$$\text{GELU}(x) = x \cdot \Phi(x)$$

其中 $\Phi(x)$ 是标准正态分布的累积分布函数。翻译成成人话：它不是一刀切地把负数变成 0，而是根据“这个值有多负”来做一个概率性的抑制。轻微负 → 几乎全保留；非常负 → 几乎全抹掉。中间是平滑过渡的。

GELU 是 GPT-1、BERT 这些早期大模型就开始采用的激活函数，后来的很多 Transformer 模型也都在用。它比 ReLU 更“温柔”——不会暴力地切断负值，而是给它们一个“存活概率”。

SwiGLU: LLaMA 的选择

LLaMA 没有用上面任何一个单一激活函数，而是用了一种叫 SwiGLU 的门控结构。它不是一个简单的函数，而是一个“小结构”：

$$\text{SwiGLU}(x) = \text{Swish}(xW) \odot (xV)$$

Swish 是 $x \cdot \sigma(x)$ (x 乘以 Sigmoid(x))，GLU 是“门控线性单元”——输入分两路，一路过激活函数，另一路保持线性，然后逐位相乘。

人话版：SwiGLU 就像一个”自适应门”。线性路提供信号，Swish 路决定”这个信号该通过多少”。两者按位相乘，相当于给每个通道的信号配了一个独立的音量旋钮。

为什么 LLaMA 选它？经验证据表明 SwiGLU 在各种 NLP 任务上的表现优于 ReLU 和 GELU——可能因为它提供了更强的”条件计算”能力：网络可以根据输入动态决定哪些信息流通过、哪些被抑制。

对比实验：一个就够了吗？

跑一个实验感受不同激活函数的脾气。用一个 10 层的全连接网络在合成数据上做回归，对比五种激活函数的收敛速度和最终效果。

```
import numpy as np
import matplotlib.pyplot as plt

# 激活函数及其导数
def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_grad(x): s = sigmoid(x); return s * (1 - s)

def tanh_fn(x):
    return np.tanh(x)
def tanh_grad(x):
    return 1 - np.tanh(x) ** 2

def relu(x):
    return np.maximum(0, x)
def relu_grad(x):
    return (x > 0).astype(float)

def leaky_relu(x, alpha=0.01):
    return np.where(x > 0, x, alpha * x)
def leaky_relu_grad(x, alpha=0.01):
    return np.where(x > 0, 1, alpha)

def gelu(x):
    # GELU 近似公式
    return 0.5 * x * (1 + np.tanh(np.sqrt(2/np.pi) * (x + 0.044715 * x**3)))
def gelu_grad(x):
    # 数值近似
    h = 1e-5
    return (gelu(x + h) - gelu(x - h)) / (2 * h)

# 画 5 个函数的形状
x = np.linspace(-4, 4, 500)
funcs = {
    'Sigmoid': (sigmoid(x), 'red'),
    'Tanh': (tanh_fn(x), 'orange'),
    'ReLU': (relu(x), 'green'),
    'LeakyReLU': (leaky_relu(x), 'blue'),
    'GELU': (gelu(x), 'purple'),
}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for name, (y_vals, color) in funcs.items():
    axes[0].plot(x, y_vals, color=color, linewidth=2, label=name)
axes[0].axhline(0, color='k', linewidth=0.5)
axes[0].axvline(0, color='k', linewidth=0.5)
axes[0].set_title('五大激活函数形状对比')
```

```

axes[0].legend(); axes[0].grid(alpha=0.3)

# 画导数形状（梯度消失的直观展示）
grads = {
    'Sigmoid 梯度': (sigmoid_grad(x), 'red'),
    'Tanh 梯度': (tanh_grad(x), 'orange'),
    'ReLU 梯度': (relu_grad(x), 'green'),
    'LeakyReLU 梯度': (leaky_relu_grad(x), 'blue'),
}
for name, (y_vals, color) in grads.items():
    axes[1].plot(x, y_vals, color=color, linewidth=2, label=name)
axes[1].axhline(0, color='k', linewidth=0.5)
axes[1].set_title('导数对比：谁容易梯度消失？')
axes[1].legend(); axes[1].grid(alpha=0.3)
axes[1].set_ylim(-0.1, 1.1)
plt.tight_layout()
plt.show()

```

注意第二张图（导数图）——Sigmoid 和 Tanh 的导数在两侧都急速趋零。这意味着当输入比较大或比较小的时候，反向传播经过这个激活函数的梯度几乎为零。这就是“梯度消失”的视觉效果。

而 ReLU 在正区的导数永远是 1（那条绿色水平线）——不管输入多大，梯度都原样通过。简单粗暴地解决了深层网络的信号传播问题。

ReLU 这么简单，为什么这么有效？

这个问题值得专门回答。因为答案不是数学上的，而是经验上的。

在 ReLU 之前，大家默认 Sigmoid 或 Tanh。当网络超过三四层，训练就变得异常困难——前面几层几乎不更新。2012 年 AlexNet 用 ReLU 在 ImageNet 上吊打全场之后，人们才意识到：“你不需要一个好看的激活函数，你只需要一个**不会把梯度弄丢**的激活函数。”

ReLU 的秘诀是反直觉的：它越简单，深层网络越容易训练。因为梯度传播不需要经过任何“压缩”或“挤压”——在激活区域纯粹是一条直线通过。

但 ReLU 也有毛病：负区完全死掉（dying ReLU）。如果学习率太大或者参数初始化不好，大量神经元可能一开始就“死了”并且再也活不过来。这也是为什么后来出现 LeakyReLU、PReLU（可学习的 leaky 系数）、ELU 等等改进版。

GELU 则是另一个方向——它不是补 ReLU 的短板，而是把 ReLU 的“一刀切”变得更光滑。在 Transformer 时代，BERT 的作者们发现 GELU 效果更好，然后就传播开了。但要说数学上为什么 GELU 优于 ReLU——目前没有一个公认的理论解释，更多是一条经验规律。

人话总结这章：Sigmoid 和 Tanh 是过去式，回不去了。ReLU 凭“导数恒为 1”的大巧不工统治了深度学习近十年。LeakyReLU 补了 ReLU 负区死掉的短板。GELU 把 ReLU 的光滑版带进了 Transformer 时代。SwiGLU 则在 LLaMA 这类大模型里展示了“门控+激活”组合拳的威力。

你看，网络越深，梯度越容易“蒸发”在前几层。ReLU 缓解了一个问题，但并没有彻底解决。而且神经网络还有一个更根本的问题——它在训练数据上表现太好，一碰到没见过的数据就拉胯。

这个问题叫过拟合。它跟网络结构本身没关系，跟”你用什么约束来训练”有关系。下一章就聊这个。

常见疑问

Q: GELU 和 ReLU 到底差在哪？

A: GELU 是 ReLU 的平滑版本。ReLU 在 $x=0$ 处有折角（不可导），GELU 处处可导且对负值有微小的输出，这让梯度流动更顺畅。大模型对训练的细微稳定性极其敏感，GELU/SwiGLU 的平滑性在这里就很关键。

Q: SwiGLU 为什么比 ReLU 好？

A: $\text{SwiGLU} = \text{SiLU}(x)$ 与 $(xW_g + b_g)$ 的逐元素乘积，它引入了”门控”机制——一部分控制信息流，一部分做非线性变换。这种 gating 的思想和 LSTM 的门控、MoE 的路由在精神上是相通的。

第 13 章 过拟合与正则化：背答案还是学原理

难度：★★★★☆☆

观光路线：理解过拟合就是”背答案的学生”，知道 L1/L2 正则化在干什么，看懂训练集 vs 测试集的曲线意味着什么。

登山路线：推导 L1 和 L2 的几何约束（菱形 vs 圆形），理解稀疏性和权重衰减，了解现代大模型里如何做正则化。

有两个学生准备同一门考试。

学生 A 把历年真题的答案全背下来了。他能在旧题上拿满分，但考试只要出一个新题，他连题目都看不懂。

学生 B 仔细推导了每个知识点的原理。他做旧题可能比 A 慢一点，但新题也能做出来。

学生 A——**过拟合**。学生 B——**泛化**。

机器学习最核心的困境之一就在这儿：模型在训练集上表现极好（甚至完美），但一到测试集或真实环境下就掉链子。它”记住”了训练数据里的噪声和奇特模式，而不是学到真正的规律。

过拟合长什么样

假设你有 10 个点，大致分布在一条直线上，但稍微带点随机波动。

- 用一条直线（2 个参数）去拟合：勉强能覆盖趋势，但不精确。这叫**欠拟合**。
- 用一条弯曲度适中的抛物线（3 个参数）去拟合：大部分点都在曲线上或附近，趋势对头。这叫**刚刚好**。
- 用一条九次多项式（10 个参数）去拟合：每个点精确通过，完美！但你看看两个点之间的区域——曲线疯了一样地上下震荡，毫无物理意义。这就是**过拟合**。

把数据画出来，用感官去体验这件事：

- 欠拟合的曲线：在点群中间穿过去，两边有些偏离，但方向是对的。
- 刚好拟合的曲线：贴着点的走向，该弯就弯，不该弯不弯。
- 过拟合的曲线：像一条发狂的蛇，拼命扭动身体去蹭每一个点。两个相邻点之间它能起伏三四次。

过拟合曲线在训练点上损失为 0，但如果你在两点之间随便取一个新位置，预测值会离谱到姥姥家。

过拟合的信号： - 训练损失持续下降，但验证损失不降反升——这是最经典的”报警信号”。 - 模型参数的值变得特别大（正负都有），因为曲线在”拼命扭动”来适应噪声。 - 梯度更新时，不同的 batch 之间参数朝相反方向剧烈震荡。

怎么阻止模型“背答案”

这就要靠**正则化**了。正则化的核心思想：在损失函数里加一个“惩罚项”，让模型不敢把参数调得太大。

直觉上很好理解：参数值大了，曲线才能剧烈扭曲去贴合每一个噪声点。你限制参数的大小，就是限制模型的“扭曲能力”，逼它去学更本质、更简洁的规律。

这个理念有个响亮的名字——**奥卡姆剃刀**：如无必要，勿增实体。在模型的世界里，就是“能用简单曲线解释的，别用复杂曲线”。

L1 正则化：逼出稀疏性

L1 正则化在损失函数后面加了一项：所有权重绝对值的和。

$$L_{total} = L_{original} + \lambda \sum_i |w_i|$$

人话：你的原始损失（比如交叉熵）加上一个项——每个权重的绝对值乘以一个系数 λ ，全部加起来。 λ 越大，惩罚越狠，模型越不敢用大权重。

L1 最关键的一个属性：它倾向于产生稀疏解——很多权重被直接推到 0。

为什么？因为 L1 的惩罚是线性的。一个权重从 0.1 变成 0.09，惩罚从 0.1λ 变成 0.09λ ，变化是均匀的。而对于一个权重是 0.0001 的参数，惩罚已经微乎其微了，如果这个参数对模型没什么用，网络就倾向于直接把它设成 0——反正惩罚也没差多少。

L1 可以做**特征选择**：训练完成后，那些权重为 0 的输入特征就是“被放弃的”。模型自己挑出了哪些特征有用、哪些是噪音。

L2 正则化：温柔地压住一切

L2 正则化加的是权重的平方和：

$$L_{total} = L_{original} + \lambda \sum_i w_i^2$$

人话：跟 L1 类似，但惩罚是平方级的。一个权重是 10，L1 罚 10λ ，L2 罚 100λ 。所以 L2 对大权重特别狠，对小权重则睁一只眼闭一只眼。

L2 不会产生稀疏解。它只是把所有权重“温柔地”往 0 的方向推，但很少真正推到 0。因为当一个权重接近 0 时，平方惩罚也接近 0 了——“既然已经很小了，再继续推的动力就不够了”。

L2 正则化还有一个名字叫**权重衰减**（Weight Decay）。在 SGD 更新公式里，加了 L2 后相当于每次更新时权重先乘以一个略小于 1 的因子——“每次更新前先缩一点”。

菱形 vs 圆形：几何直觉

这是理解 L1 和 L2 区别最优雅的方式。

在参数空间 (w_1, w_2) 里: - **L1 的约束区域是一个菱形** (旋转了 45 度的正方形):
 $|w_1| + |w_2| \leq C$ - **L2 的约束区域是一个圆形**: $w_1^2 + w_2^2 \leq C$

损失函数的最优解是被”挤”在约束边界上的。对于圆形的 L2 约束, 最优解可以在圆上的任何位置——通常两个权重都不为 0。对于菱形的 L1 约束, 最优解大概率被”推”到菱形的角上——角上只有一个方向权重非零, 另一个是 0。

这就是为什么 L1 产生稀疏性而 L2 不产生——菱形的角”天然地”落在坐标轴上。

用代码来直观感受:

```
import numpy as np
import matplotlib.pyplot as plt

# 参数空间的可视化
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# L1 约束菱形
theta = np.linspace(0, 2*np.pi, 100)
for C in [1, 2, 3]:
    # 菱形:  $|w_1| + |w_2| = C$  的参数方程
    # 四个象限分别画
    x_top = np.linspace(-C, C, 200)
    y_top = C - np.abs(x_top)
    axes[0].plot(x_top, y_top, 'b-', alpha=0.7)
    axes[0].plot(x_top, -y_top, 'b-', alpha=0.7)
    axes[0].text(0, C+0.1, f'C={C}', ha='center', fontsize=9)
axes[0].set_title('L1 约束: 菱形\n(角落在坐标轴上→稀疏解)')
axes[0].set_xlabel('w1'); axes[0].set_ylabel('w2')
axes[0].axhline(0, color='gray', alpha=0.3)
axes[0].axvline(0, color='gray', alpha=0.3)
axes[0].set_aspect('equal')
axes[0].grid(alpha=0.2)
axes[0].set_xlim(-3.5, 3.5); axes[0].set_ylim(-3.5, 3.5)

# L2 约束圆形
for C in [1, 2, 3]:
    circle = plt.Circle((0, 0), C, fill=False, color='r', alpha=0.7)
    axes[1].add_patch(circle)
    axes[1].text(C*0.7, C*0.7, f'C={C}', fontsize=9)
axes[1].set_title('L2 约束: 圆形\n(光滑边界→权重均衡)')
axes[1].set_xlabel('w1'); axes[1].set_ylabel('w2')
axes[1].axhline(0, color='gray', alpha=0.3)
axes[1].axvline(0, color='gray', alpha=0.3)
axes[1].set_aspect('equal')
axes[1].grid(alpha=0.2)
axes[1].set_xlim(-3.5, 3.5); axes[1].set_ylim(-3.5, 3.5)

plt.tight_layout()
plt.show()
```

L1 + L2 = Elastic Net

把 L1 和 L2 结合起来就是 Elastic Net——既能有稀疏性 (L1 的贡献), 又能稳定训练 (L2 的贡献)。在某些场景下比单独用 L1 或 L2 效果都好。公式很简单:

$$L = L_{original} + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$$

大模型时代的正则化

上面讲的 L1/L2/Elastic Net 在传统机器学习里是标配。但在现代大语言模型（GPT、LLaMA 之类）里，情况有些不同。

大模型的正则化更多靠的是：
- **Dropout**（下一章专门讲）——训练时随机关掉一部分神经元。
- **数据增强**——你已经有海量数据了，再随手加些扰动，模型想“背答案”也背不过来。
- **Early Stopping**——在验证损失开始上升的时候就停下来。
- **Batch Normalization / Layer Normalization**——这些归一化层本身就有轻微的正则化效果。

有意思的是，大模型的参数量往往远超训练数据量（GPT-3 的 1750 亿参数对比约 3000 亿 token 的训练数据）。按传统理论这早就该过拟合了——参数比样本多，不收敛才对。但大模型不但没崩，还展示了强大的泛化能力。

这就是“双下降”（Double Descent）现象：传统统计学预言的那条“先降后升”的 U 形曲线只描述了前半段。在某个临界点之后（模型刚好能拟合所有训练数据的地方），再继续加参数，测试误差反而又降下去了。

为什么？还没有完整的理论解释。一个主流的假说是：超大模型在“隐式正则化”——梯度下降本身（特别是 SGD 的随机性加上 Adam 的优化路径）自带某种“偏好简单解”的倾向。在参数海量的情况下，模型反而被迫去找最根本、最泛化的模式。

不过这是另一个大话题了，超出了本章范围。

人话总结这章：过拟合就是模型背答案，考试换题就不会。正则化通过给大参数施加惩罚来阻止这种事——L1 逼出稀疏性（很多权重归零），L2 温柔减重（所有权重都缩小），Elastic Net 两者兼有。大模型时代，Dropout + 数据增强 + 隐式正则化取代了传统的手动调 λ 的做法。

说到 Dropout——这是最“玄妙”的正则化技术之一。它的操作是：训练时随机干掉一些神经元。听起来像自残，但效果出奇的好。

为什么随机失活反而能防止过拟合？它跟“老师随机点名”有什么关系？来，下一章拆开细细看。

常见疑问

Q: 大模型会过拟合吗？

A: 有趣的是，大模型存在“双下降”现象——当模型参数量远超训练数据量时，测试误差反而会下降。这和传统统计学直觉相反，是大模型时代最反直觉的发现之一。但这不代表不需要正则化——Dropout、weight decay 在 Transformer 训练中仍然标配。

Q: L1 和 L2 正则化在实际项目中怎么选？

A: L1 用于希望做特征选择（产生稀疏权重），L2 用于防止权重过大（让所有权重都偏小）。在深度学习中，L2（weight decay）更常用，因为神经网络不太需要显式的特征选择。

Q: Elastic Net 有两个 lambda 参数，怎么调？

A: 先分别调 L1 和 L2 各自的 λ ，找到各自的最佳值；然后做网格搜索在两者之间插值。实操中一个偷懒的做法是：设一个总 λ 和一个混合比例 α (L1_ratio)，让 $\lambda_1 = \alpha \cdot \lambda$ ， $\lambda_2 = (1 - \alpha) \cdot \lambda$ 。先固定 $\alpha = 0.5$ 调 λ ，再微调 α 。Scikit-learn 的 ElasticNet 就是这种参数化方式。不过说实话，在深度学习里 Elastic Net 用得不多——大部分场景用 L2 (weight decay) 就够了，L1 偶尔在需要稀疏化时单用。

第 14 章 Dropout: 随机点名——让学生不敢偷懒

难度: ★★☆☆☆

观光路线: 理解 Dropout 就是”训练时随机灭掉神经元, 测试时全开但要打折”, 知道为什么这能防止过拟合, 以及大模型里还在不在用。

登山路线: 手写 Dropout 层实现, 跑有无 Dropout 的对比实验, 理解 inverted dropout 的实现细节和推理阶段的缩放原理。

小学的时候, 有一种老师最让人害怕: 随机点名的。

你不知道他今天会不会点你。不点的同学当然松了一口气, 但你不敢不准备——因为明天可能就轮到你了。全班每个人都得时刻绷着, 谁都别想蒙混过关。

Dropout 就是给神经网络找了个”随机点名的老师”。

什么是 Dropout

训练神经网络的每一步, Dropout 会做一件事: **随机挑选一部分神经元, 暂时把它们”关掉”** (输出置零)。这一轮前向传播和反向传播里, 这些被关掉的神经元就当不存在。

下一轮训练, 重新随机选一批神经元关掉。每次关掉的组合都不一样。

关键数字是”保留概率” p 。如果 $p=0.5$, 每轮有一半神经元被随机关掉; $p=0.7$, 关掉 30%。这个 p 是可以调的——全连接层常用 0.5, CNN 的卷积层常用 0.7 到 0.8 (因为卷积层本身参数少, 不需要那么猛的失活)。

为什么随机”自残”反而能防过拟合

回到上一章的学生比喻。

过拟合的学生 A 之所以能背答案, 是因为他可以死记每一个题目的每一个细节——他有足够多的”神经元”来分别负责记住这些细节。

但现在加一个规则: **每次考试前, 随机禁用这个学生的一部分知识点**。他不知道哪些知识点会被禁用, 所以他不敢只靠某一个特定的”技巧”或”记忆”来解题。他必须让整个”知识体系”都强健——任何一个知识点的缺失都不至于让他交白卷。

这就是 Dropout 防止过拟合的机制: 它强迫每个神经元不能”独享”某种特征, 而必须和其他神经元一起”协同工作”。没有了”独揽大权”的机会, 神经元们只能发展出更稳健、更通用的特征表达。

从信息论的角度看: Dropout 相当于在训练过程中**注入了噪声**。这种噪声破坏了神经元之间的”共适应”——一些神经元私下串通好一起记住某个训练样本的噪声细节。Dropout 不断打乱这种默契, 逼着单个神经元发展出不依赖特定”同伴”的独立能力。

另一个角度: Dropout 每次训练的都是一个”子网络”——原网络去掉随机一部分神经元后的缩小版。把 dropout 看作是在**指数级数量的子网络上做集成学习**。测试时你把所有子网络的结果平均起来, 自然比单个网络更稳定。

不过这个”集成学习”的说法有点过于浪漫了。更准确的版本是：Dropout 让每个神经元学到的是”即使同伴少了，我一个人也得行”的稳健表达。

训练 vs 测试：为什么测试时要”打折”

这是 Dropout 最容易让人困惑的细节。训练时一半神经元是关着的，测试时全开——那测试时的信号强度不就翻倍了吗？

举个例子：一个神经元在训练时有 50% 的概率被关掉，所以它输出信号的实际”期望值”只有满额的 50%。测试时所有神经元都在，信号变成了满额——这是不对的，你训练时网络”习惯”了 50% 的信号量。

所以测试时需要补偿：把每个神经元的输出乘以保留概率 p 。

具体来说：- 训练时：每个神经元以概率 p 被保留，以概率 $1-p$ 被丢弃。被保留的神经元输出原值。- 测试时：所有神经元都在，但每个神经元的输出乘以 p 。

这样，同一神经元在训练和测试时的期望输出是一致的。

不过现在主流实现用的是 **Inverted Dropout**——在训练时就除以 p 把信号”放大”回来，测试时什么都不做。相当于把补偿提前到了训练阶段。这样的好处是：部署模型时不需要知道”哦这个层有 dropout”——所有层的输出都已经在训练时被正确缩放了。

```
import numpy as np
```

```
class Dropout:
    def __init__(self, p=0.5):
        """
        p: 保留概率 (keep probability)
        注意：有些框架用”丢弃率”，这里用”保留率”比较直观
        """
        self.p = p
        self.mask = None # 记录这一轮关掉了哪些神经元

    def forward(self, x, training=True):
        if not training:
            return x # 测试模式：什么都不做 (inverted dropout 已在训练时补偿)

        # 生成随机掩码：每个神经元独立决定是否保留
        self.mask = (np.random.rand(*x.shape) < self.p).astype(float)

        # Inverted dropout: 保留的神经元除以 p 做补偿
        return x * self.mask / self.p

    def backward(self, dout):
        # 反向传播：梯度通过相同的掩码传播
        return dout * self.mask / self.p
```

就这么短。Dropout 层的本质就是：生成一个随机的 0/1 掩码，乘到输入上，保留的除以 p 。

有 Dropout vs 无 Dropout：对比实验

用一个有些过拟合倾向的小型全连接网络在 MNIST 的 5000 张子集上训练，比较加不加 Dropout 的差别。

```

import torch
import torch.nn as nn
import torch.optim as optim

class NetWithDropout(nn.Module):
    def __init__(self, use_dropout=True):
        super().__init__()
        self.use_dropout = use_dropout
        self.fc1 = nn.Linear(784, 512)  # 宽隐藏层: 容易过拟合
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 10)
        self.dropout = nn.Dropout(p=0.5)  # 一半神经元随机失活
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.dropout(x) if self.use_dropout else x  # dropout 在这里
        x = self.relu(self.fc2(x))
        x = self.dropout(x) if self.use_dropout else x
        return self.fc3(x)  # 最后一层通常不加 dropout

def train_eval(use_dropout, epochs=30):
    # 简化版: 用少量数据模拟过拟合场景
    from torchvision import datasets, transforms
    from torch.utils.data import DataLoader, Subset

    transform = transforms.Compose([transforms.ToTensor()])
    full_train = datasets.MNIST('./data', train=True, download=True, transform=transform)
    full_test = datasets.MNIST('./data', train=False, download=True, transform=transform)

    # 只用 5000 张训练, 故意让模型有过拟合倾向
    train_set = Subset(full_train, range(5000))
    train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
    test_loader = DataLoader(full_test, batch_size=1000)

    model = NetWithDropout(use_dropout)
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.CrossEntropyLoss()

    train_losses, test_losses = [], []
    for epoch in range(epochs):
        # 训练
        model.train()
        total_loss = 0
        for X_batch, y_batch in train_loader:
            X_batch = X_batch.view(-1, 784)
            optimizer.zero_grad()
            loss = criterion(model(X_batch), y_batch)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        train_losses.append(total_loss / len(train_loader))

        # 测试
        model.eval()
        total_test_loss = 0
        with torch.no_grad():

```

```

        for X_batch, y_batch in test_loader:
            X_batch = X_batch.view(-1, 784)
            total_test_loss += criterion(model(X_batch), y_batch).item()
        test_losses.append(total_test_loss / len(test_loader))

    return train_losses, test_losses

# 跑对比实验
train_no_drop, test_no_drop = train_eval(use_dropout=False)
train_with_drop, test_with_drop = train_eval(use_dropout=True)

import matplotlib.pyplot as plt
epochs = range(1, 31)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(epochs, train_no_drop, 'r-', label='训练损失')
plt.plot(epochs, test_no_drop, 'r--', label='测试损失')
plt.title('无 Dropout: 注意训练和测试的差距')
plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.legend(); plt.grid(alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(epochs, train_with_drop, 'g-', label='训练损失')
plt.plot(epochs, test_with_drop, 'g--', label='测试损失')
plt.title('有 Dropout: 训练/测试差距缩小')
plt.xlabel('Epoch'); plt.ylabel('Loss')
plt.legend(); plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

```

观察两个图的关键差异：- 无 Dropout 时，训练损失远低于测试损失（差距很大）= 典型过拟合。- 有 Dropout 时，训练损失比无 Dropout 时高（因为一半神经元被关了），但测试损失反而更低、跟训练损失的差距更小。

这就是 Dropout 的核心效果：牺牲一些训练集上的表现，换取更好的泛化能力。

Dropout 在现代大模型中的使用情况

2014 年 Dropout 提出后迅速成为深度学习的标配。CNN、RNN、早期的 Transformer 几乎都在用。

但到了 2020 年代的大语言模型时代，情况变了。

GPT 系列用 Dropout 吗？ GPT-1 和 GPT-2 在注意力层和全连接层之后都用了 Dropout。但 GPT-3 的技术报告里，Dropout 的使用大幅减少——只在某些位置保留。

LLaMA 呢？ LLaMA 的训练代码里几乎看不到 Dropout。它靠的是海量数据 + 精心调参的 AdamW + 大规模训练本身的“隐式正则化”效果。

现在训练大模型的一般共识是：如果数据量极大（接近“无限数据”），过拟合已经不是主要矛盾了——反正模型在整个训练过程中也见不完数据。这时候 Dropout 反而不利于模型充分学习。

但另一方面，如果你在**微调**一个小数据集上的预训练模型，Dropout 仍然有用。一些 NLP 实践者会在分类头（最后的输出层之前）加一层 Dropout 来防止在小数据集上过拟合。

所以 Dropout 的现状是：作为正则化工具，它在中小规模网络和小数据集上仍然很有效。大模型预训练阶段用得少，fine-tune 阶段仍然有人用。这是一个没被淘汰、但也不再是“必选项”的技术。

人话总结这章：Dropout 通过随机“关掉”一部分神经元，逼着每个神经元发展独立、稳健的表达能力，别指望抱团取暖。Inverted Dropout 在训练时做除法补偿，测试时零开销。它防止过拟合的效果在中小型网络上非常显著，但大模型因为数据量大、参数量也大到天然抗过拟合，很多时候干脆不用了。

常见疑问

Q: Dropout 在现代大模型中还用吗？

A: 用法变了。早期 BERT/GPT 确实在 Attention 和 FFN 后用 Dropout。但现在的趋势是减少甚至不用——预训练时巨大的数据量本身就是最好的正则化。微调时如果数据少，偶尔还会用到。有一个趋势是只对 Embedding 层做微小的 Dropout。

Q: Dropout 训练时关掉神经元，模型不会变笨吗？

A: 正是这个“被迫独立”的过程让模型变强。训练时因为总有些神经元缺席，剩下的必须学会自己搞定问题，不能依赖特定神经元组合——结果就是模型学到的特征更鲁棒、更泛化。

到这里，深度学习的“基础设施”就讲完了。从单个感知机到多层网络，从反向传播到梯度下降，从激活函数到正则化到 Dropout——你手里已经有了一整套“造轮子”的工具。

但在进入第三部分之前，有一个话题我们还没正面聊过：神经网络里的“信息”到底是什么？这些层在干什么？是简单的模式匹配，还是某种更深层的东西？

那是“表示学习”的开端，这个话题要到第 20 章才正式展开。眼下，我们先进第三部分。

核心架构篇

公众号：AI技术圈

第 15 章 CNN 卷积神经网络

难度：★★★★☆

观光路线：理解 CNN 为什么比全连接网络更适合处理图像，知道卷积层、池化层、全连接层各自干什么活，看完代码能跑通 MNIST 手写数字分类。

登山路线：搞明白卷积核的参数共享和稀疏连接这两个核心设计，手动推导感受野和输出尺寸的计算，以后自己改 CNN 结构时不犯懵。

放大镜扫描照片

想象你手里拿着一张 1000x1000 像素的彩色照片，要判断照片里有没有猫。摄像头拍了 100 万个位置，每个位置有 RGB 三个颜色值，总共 300 万个数字。

怎么处理这 300 万个数字？

全连接网络的做法很耿直：把所有像素摊成一条长向量，然后让每个神经元跟这 300 万个输入全部连上。就第一层，如果放 1000 个神经元，那就是 30 亿个权重参数。30 亿个。

这就好比你要在一本 1000 页的书里找“猫”这个字。正常人的做法是一行一行、一段一段看，看到猫就圈出来。全连接网络的做法是：把 1000 页书撕成碎纸片撒在天上，然后同时盯着所有碎片——没毛病，但代价太大了。

卷积神经网络（CNN）用的是一种聪明得多的策略。它不一次性看整张照片，而是拿一把小放大镜，从左到右、从上到下一格一格地扫过去。放大镜每次只看一小块区域，比如 3x3 或者 5x5。不管照片是 100x100 还是 1000x1000，这把放大镜的大小不变，参数数量就被死死压住了。

这背后藏着 CNN 的两个核心设计。

参数共享。同一把放大镜扫遍整张照片的每个角落。左上角检测边缘的卷积核，挪到右下角同样能检测边缘。这很符合直觉——猫耳朵的曲线在照片左边是曲线，换到右边也是曲线，检测规则不该变。

稀疏连接。放大镜里的每个格子只关心自己罩住的那几个像素，跟远处的像素没有任何直接连线。这种局部性假设同样很自然：判断一个像素是不是猫毛的一部分，看看它周围的邻居就够了，不用参考照片对角线上另一个像素的颜色。

先打个白话的比方。参数共享相当于同一个检察官审遍全城的案子，术业有专攻。稀疏连接相当于检察官只审自己辖区的案子，不管隔壁城市发生了什么。

精确地说，参数共享意味着卷积核的权重在空间维度上重复使用，这使得模型具有平移等变性——输入平移，输出跟着平移。稀疏连接意味着输出特征图上的每个神经元只与输入的一个局部感受野相连，连接数从 $O(n^2)$ 降到 $O(nk^2)$ ，其中 k 是卷积核尺寸。

分层特征：从边缘到鼻子的进化

CNN 内部有一个层层递进的抽象过程。这个过程的优雅程度，让人怀疑是不是从人脑视觉皮层偷了灵感。

低层：边缘和颜色。 第一层卷积学到的东西极其朴素——水平线、竖线、对角斜线，以及某些特定的颜色过渡。拿一张卷积核可视化出来看，就像一组风格简陋的涂鸦。

中层：纹理和部件。 把低层特征组合起来，第二层、第三层开始捕捉更复杂的东西：条纹、格子、圆形轮廓、某种特定材质的纹理。这时候卷积核已经不满足于”这里有一条边”了，它开始学会说”这里有一块像豹纹的东西”。

高层：完整物体。 到了网络的最后几层，单个神经元可能专门对”猫脸”、“车轮”、“窗户”这种语义级别的概念产生强烈反应。2014 年 Zeiler 和 Fergus 那篇可视化论文里有一个经典实验：把高层某个神经元的感受野映射回原图，发现它对应的区域恰好是一只猫的脸。

这个过程不是谁设计出来的，是网络自己在数据里摸爬滚打学到的。你给 CNN 喂几百万张图片，它自己就摸索出了”先认边、再认形、最后认物”的分工策略。

有意思的地方来了——这种分层抽象跟人脑视觉皮层 $V1 \rightarrow V2 \rightarrow V4 \rightarrow IT$ 的通路非常相似。神经科学里早就发现， $V1$ 区的神经元只对特定方向的边缘敏感，到了 IT 区就能选择性响应面孔。CNN 无意间复现了这套机制。当然这不代表 CNN”理解”了视觉，它只是在统计意义上逼近了人脑的处理流程。

全连接层的维度灾难

回到那个 300 万输入的问题。为什么全连接网络对图像这么水土不服？

除了参数量爆炸，还有一个更隐蔽的问题：全连接层完全无视像素的空间关系。

把一张猫的照片左右翻转一下，像素值组成的向量会变得面目全非，但全连接网络根本不知道”左边像素”和”右边像素”在空间上有邻居关系。对它来说，第 37 号像素和第 493829 号像素只是向量里两个距离很远的数字而已。它必须用海量数据和海量参数，才能从统计上”悟出”这两个像素其实挨在一起。

CNN 不存在这个困扰。卷积核天然知道谁跟谁是邻居。 3×3 的核每次只看一个 3×3 小方块，空间关系被硬编码进了网络结构里——这不是学来的，是设计时就写死的先验知识。

这个先验知识有多重要？举个例子：同一个 CNN 结构，用 10 万张图片训练的效果，可能比全连接网络用 100 万张训练还好。因为 CNN 不需要浪费宝贵的参数去”发现”像素之间的空间关系，它从一开始就知道。

动手写一个 CNN

说再多不如跑一次代码。下面是一个完整的 CNN，用 PyTorch 实现，在 MNIST 手写数字数据集上训练。核心结构是两层卷积加两层全连接，总共大概 60 行。

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# ---- 定义 CNN 结构 ----
class SimpleCNN(nn.Module):
```

```

"""
两层卷积 + 两层全连接的小型 CNN
输入: [1, 28, 28] 灰度图
"""
def __init__(self, num_classes=10):
    super().__init__()
    # 第一层卷积: 1 通道→32 通道, 3x3 核
    # 输出尺寸公式:  $(28 - 3 + 2*1)/1 + 1 = 28$ 
    self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
    # 第二层卷积: 32 通道→64 通道, 3x3 核
    # 池化后: 28→14, 输出尺寸:  $(14 - 3 + 2*1)/1 + 1 = 14$ 
    self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
    # 池化层: 把 14x14 缩成 7x7
    self.pool = nn.MaxPool2d(2, 2)
    # 全连接层: 64 通道 × 7 × 7 = 3136 → 128 → 10
    self.fc1 = nn.Linear(64 * 7 * 7, 128)
    self.fc2 = nn.Linear(128, num_classes)
    # 激活函数和正则化
    self.relu = nn.ReLU()
    self.dropout = nn.Dropout(0.5)

def forward(self, x):
    # 卷积块 1: 卷积 → ReLU → 池化, (28→14)
    x = self.pool(self.relu(self.conv1(x)))
    # 卷积块 2: 卷积 → ReLU → 池化, (14→7)
    x = self.pool(self.relu(self.conv2(x)))
    # 把特征图摊平成一维向量
    x = x.view(x.size(0), -1)
    # 全连接 + Dropout + 全连接
    x = self.relu(self.fc1(x))
    x = self.dropout(x)
    x = self.fc2(x)
    return x

# ---- 数据准备 ----
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)) # MNIST 的均值和标准差
])

train_dataset = datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = datasets.MNIST(
    root='./data', train=False, download=True, transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# ---- 训练 ----
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(5): # 5 轮就够, MNIST 比较简单
    model.train()

```

```

running_loss = 0.0
for images, labels in train_loader:
    images, labels = images.to(device), labels.to(device)

    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    running_loss += loss.item()

# 每个epoch 结束测一下准确率
model.eval()
correct, total = 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Epoch {epoch+1}/5 | Loss: {running_loss/len(train_loader):.4f} "
      f"| 测试准确率: {100*correct/total:.2f}%")

print("训练完成!")

```

跑完 5 个 epoch，测试准确率应该在 99% 左右。注意这个 CNN 只有两层卷积，参数总量不到 30 万。如果用全连接网络达到同样的准确率，参数量轻松上千万。

代码里有几个点值得停下来想想。padding=1 保证了卷积后尺寸不变，这样我们只需要靠池化来缩小尺寸，逻辑很干净。x.view(x.size(0), -1) 这一步是在卷积层和全连接层之间搭桥——卷积输出的三维张量（通道×高×宽）被拉成一维，才能喂给全连接层。Dropout 在训练时随机丢掉一半神经元，防止网络死记硬背训练集。

卷积网络是深度学习在视觉领域开疆拓土的头号功臣。从 2012 年的 AlexNet 到现在的 Vision Transformer，中间十几年的图像 AI 突破，大多盖着 CNN 的戳。但 CNN 就像一个管弦乐团——光知道有各种乐器还不够，你得知道指挥（卷积核）是怎么调动每种乐器的，这就是下一章要聊的事。

人话总结

说到底，这章就讲了一件事——CNN 为什么能用比全连接网络少几百倍的参数，反而把图像识别做得更好。答案藏在两个设计里：参数共享（同一把放大镜扫遍整张图）和稀疏连接（每个神经元只关心自己罩住的那一小块区域）。你把图片喂进 CNN，前半截是卷积层负责“看”——一层一层地从边缘到轮廓再到语义，越卷越“懂”；中间池化层负责“缩”——把特征图压小，扔掉不重要的细节，保留最关键的信息；最后全连接层负责“判”——把抽出来的高级特征汇总成一个分类结果。跑一遍 MNIST 代码你就能亲眼看到，四十多万参数的小网络，轻松冲到 99% 准确率。这章你带走的东西就是：理解 CNN 的三段式设计（卷积提取+池化压缩+全连接分类），以及为什么参数共享这个 trick 能在图像上这么好使。

常见疑问

Q: CNN 只能处理图像吗？能不能处理文本或声音？ A: 可以，但需要一些调整。文本通常用一维卷积（1D CNN）沿着词的序列滑动，能捕捉局部的词组模式，TextCNN 就是一个经典例子。声音也是一维的波形序列，同样的逻辑。但文本和声音有长距离依赖的问题——第 1 个字可能影响第 100 个字的意思，CNN 的局部感受野在这方面不如 RNN 和 Transformer。

Q: 卷积核的数量和大小怎么选？有啥经验法则吗？ A: 3x3 是最常用的尺寸，够小（参数少）又够用（能捕捉边缘和纹理），一般配合 padding=1 保证尺寸不变。通道数一般越深越多——浅层 64 或 128，深层 512 或 1024。经验法则是空间尺寸每减半一次，通道数就翻一倍，让你的网络在“缩小空间”的同时“丰富语义”。VGG 和 ResNet 基本都是这个节奏。

第 16 章 卷积与池化

难度：★★★★☆

观光路线：看懂卷积和池化在干什么，知道输出尺寸怎么算，理解 1x1 卷积为什么是个神器，跑一遍特征图尺寸追踪代码。

登山路线：搞明白多通道卷积的矩阵运算本质，手动算几组输出尺寸，以后设计 CNN 结构时能心算通道变化。

侦探与警官

想象一个犯罪现场调查小组。侦探拿着放大镜趴在地上，一寸一寸地找指纹、纤维、血迹——任何可疑的痕迹都不放过。他收集回来一大堆“特征线索”。但线索太多了，一堆报告堆在警长桌上，根本看不过来。

这时候警官出场了。警官不负责找新线索，他只干一件事：从侦探交上来的大堆报告里提炼最重要的几条。指纹报告太厚了？他只记录指纹类型（弓形、箕形、斗形）和核心点位置。纤维分析有 500 页？他只留“红色人造纤维，与嫌犯汽车地毯匹配”这一句。

侦探干的事，就是卷积（Convolution）。警官干的事，就是池化（Pooling）。

卷积负责从原始像素里提取特征——边缘、纹理、形状。池化负责对特征做摘要——压缩尺寸、去掉噪音、保留最关键的信息。两者搭档，一个做加法（越卷特征越多），一个做减法（越池化尺寸越小），配合得天衣无缝。

卷积运算的里里外外

卷积这个词听着唬人，拆开来看其实很朴素。一个 3x3 的卷积核，就是一个 3x3 的数字方阵，像一个小权重矩阵。它压在图像的某个 3x3 区域上，对应的位置相乘然后加起来，得到输出特征图上的一个像素值。然后卷积核向右挪一格（步长），再做一次同样的乘加运算。一行扫完，往下挪一行继续扫。

用数学写出来就是：

$$[i, j] = \{u=0\}^{\{k-1\}} \{v=0\}^{\{k-1\}} [i+u, j+v] [u, v]$$

这个公式没有看起来那么吓人。最核心的一点：同一个卷积核的权重在整张图上到处用，这就是上一章说的参数共享。

输出尺寸怎么算？输入高度 H，卷积核尺寸 K，步长 S（每次挪几格），填充 P（在输入边缘补几圈 0）。输出高度是：

$$[H_{\text{out}} = + 1]$$

举个例子：输入 28x28，卷积核 3x3，步长 1，不补填充（P=0），输出就是 $(28-3)/1+1=26$ 。如果步长改成 2，输出就是 $(28-3)/2+1=13.5$ ，向下取整得 13。

填充的作用很直观：不补的话，每卷一次尺寸就缩一圈。一个 3×3 的核卷 100 层，输入就算有一万像素也卷没了。所以通常会在边缘补 0，让输出和输入保持同样大小。 3×3 的核补一圈 ($P=1$)，输出尺寸就不变。

多通道卷积：从二维到三维

真正的图片不是灰度图，是 RGB 三通道，甚至前面卷积层输出的可能有 64 个、128 个通道。多通道卷积怎么玩？

一个卷积核的深度必须和输入的通道数完全一致。输入是 3 通道 RGB，每个卷积核就是 $3 \times 3 \times 3$ 的立方体——每个通道各有一个 3×3 的小矩阵。卷积核在空间维度上滑窗，在通道维度上同时运算：3 个通道各自和对应的小矩阵做乘加，最后把所有通道的结果加起来，得到一个通道的输出。

那输出想要多个通道怎么办？用多个卷积核。比如你希望输出 64 个通道的特征图，就准备 64 个不同的卷积核，每个核独立做一遍卷积，产生一个通道。64 个卷积核产生 64 个通道。

白话版：每个卷积核像一个“特征探测器”。第一个核擅长探测水平边缘，第二个核擅长探测垂直边缘，第三个核专门看红色区域的纹理……64 个探测器各自找自己感兴趣的东西，输出 64 张“检测报告”。

精确版：一个卷积层的权重张量形状是 $[C_{out}, C_{in}, K, K]$ 。对于第 i 个输出通道，计算方式为第 i 组 $[C_{in}, K, K]$ 权重与输入的逐通道卷积求和。总的参数量是 $C_{out} * C_{in} * K * K$ ，不包含偏置的话。

1x1 卷积：瑞士军刀

1x1 卷积这个名字听起来像是一个笑话——1x1 的核能“卷”什么？看一个像素点本身，没有邻居，这不就是全连接吗？

没错，在空间维度上 1x1 卷积确实什么都没做。但别忘了通道维度。

输入有 256 个通道，你放 64 个 1x1 卷积核，每个核把 256 个通道的信息混在一起，产生一个输出通道。64 个核就是 64 个输出通道。输入每个空间位置都做同样的操作。

翻译成人话：1x1 卷积不改变图像的空间尺寸，但在通道维度上做了一次信息重组。它就像一个“通道间的全连接层”。

这东西有几个经典用途：

降维。 ResNet 里的 bottleneck 结构，先用 1x1 卷积把 256 通道压到 64 通道，做完 3×3 卷积后再用 1x1 卷积升回 256 通道。中间省掉了大量的计算量。

增加非线性。 不管多少通道，1x1 卷积后面可以跟 ReLU，等价于给每个空间位置加了一层非线性变换。同样的感受野尺寸，网络“思考”得更深了。

跨通道信息融合。 不同通道可能检测不同的特征——这个通道有边缘，那个通道有纹理。1x1 卷积可以把这些分散的信息混在一起做决策。

Network in Network 那篇论文（2014 年）最早把 1x1 卷积推上台前，后来 Inception 系列和 ResNet 把它用到了极致。一个简单的 1x1 操作，成了现代 CNN 设计里最灵活的积木块之一。

池化：不是随便缩小的

池化最常见的操作是最大池化（Max Pooling）：在 2x2 的窗口里取最大值，然后窗口挪两格，继续取最大值。每个 2x2 窗口只留一个最强的信号。

为什么取最大值而不是平均值？

假如窗口里是[0.1, 0.2, 0.8, 0.1]。平均值是 0.3，最大值是 0.8。如果这个窗口里有一个像素被”激活”得很厉害（比如正好是边缘位置），最大值池化会保留这个强信号，而平均池化会把它稀释掉。

白话的道理很直接：判断一张图里有没有猫，你关心的是”哪里的猫特征最强”，而不是”整张图的猫特征平均怎么样”。鼻子区域的特征激活值高达 0.95，就应该把这个 0.95 保留下来，而不是跟背景的 0.01 取个平均变成 0.48。

精确地说，最大池化提供了局部平移不变性。输入稍微平移几个像素，只要最强的激活还在池化窗口范围内，输出就不变。这对于分类任务是好事——你只关心有没有猫，不在意猫具体在照片的哪个像素位置。

平均池化（Average Pooling）则更多用在网络的最后几层，替代全连接层之前的展平操作，或者需要平滑特征表示的场景。全局平均池化（Global Average Pooling）直接把每个通道的特征图取一个平均值，输出一个 C 维向量，干净利落地从卷积过渡到分类。

特征图尺寸追踪

设计 CNN 的时候最容易踩的坑就是尺寸算错。卷积一次减一点，池化一次砍一半，几层下来从 224x224 变成 1x1——但中间某一层莫名其妙变成了奇数尺寸导致 padding 不对称，后面全乱套。

下面这段代码，帮你追踪每一层特征图的尺寸变化：

```
import torch
import torch.nn as nn

class FeatureSizeTracker(nn.Module):
    """
    一个带尺寸追踪的卷积网络骨架
    每一层执行完毕打印特征图形状
    """
    def __init__(self):
        super().__init__()
        # 第一层：输入 3 通道 RGB，输出 64 通道，卷积核 7x7，步长 2，填充 3
        # 尺寸公式：(224 + 2*3 - 7) / 2 + 1 = 112
        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3)
        self.pool1 = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        # 池化后：(112 + 2*1 - 3) / 2 + 1 = 56

        # 第二层：64→128, 3x3, stride=1, padding=1 → 尺寸不变=56
```

```

self.conv2 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)
self.pool2 = nn.MaxPool2d(2, 2) # 56→28

# 第三层: 128→256, 3x3, stride=1, padding=1 → 28 不变
self.conv3 = nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1)
self.pool3 = nn.MaxPool2d(2, 2) # 28→14

# 1x1 卷积做通道降维: 256→128, 空间尺寸不变=14
self.bottleneck = nn.Conv2d(256, 128, kernel_size=1)

# 全局平均池化: 14x14→1x1
self.gap = nn.AdaptiveAvgPool2d((1, 1))

def forward(self, x):
    print(f"输入形状: {list(x.shape)}") # [B, 3, 224, 224]

    x = self.pool1(torch.relu(self.conv1(x)))
    print(f"conv1+pool1: {list(x.shape)}") # [B, 64, 56, 56]

    x = self.pool2(torch.relu(self.conv2(x)))
    print(f"conv2+pool2: {list(x.shape)}") # [B, 128, 28, 28]

    x = self.pool3(torch.relu(self.conv3(x)))
    print(f"conv3+pool3: {list(x.shape)}") # [B, 256, 14, 14]

    x = self.bottleneck(x)
    print(f"1x1 bottleneck:{list(x.shape)}") # [B, 128, 14, 14]

    x = self.gap(x)
    print(f"全局平均池化: {list(x.shape)}") # [B, 128, 1, 1]

    # 展平为分类头输入
    x = x.view(x.size(0), -1)
    print(f"展平后: {list(x.shape)}") # [B, 128]
    return x

# 测试一下
model = FeatureSizeTracker()
dummy_input = torch.randn(2, 3, 224, 224) # 2 张 224x224 的 RGB 图
_ = model(dummy_input)
print("\n✓ 尺寸追踪完成, 网络骨架搭建正确")

```

运行这个脚本，每一层的输出形状清清楚楚。如果中间某一步的结果跟预期对不上，立马能定位到哪一行出了问题。

这里有个小技巧：`AdaptiveAvgPool2d((1, 1))` 不管输入特征图多大，都能自适应地输出 1x1。有了它，你就不用硬编码全连接层的输入维度了——不管前面的网络怎么改，最后都缩成 1x1，全连接的输入就是通道数。

卷积和池化的搭档关系，构成了 CNN 最基础的节奏：卷积扩张通道、提取特征，池化收缩空间、浓缩信息。一扩一缩之间，图像从像素变成了语义。但网络不是越深越好——加着加着层数，准确率反而往下掉。这个反直觉的退化问题，直接催生了深度学习历史上最漂亮的一个 trick。

人话总结

这章你带走的东西其实很具体：卷积和池化是一对黄金搭档，一个负责”抓特征”，一个负责”做摘要”。卷积核就是一个小型的权重方阵，在你的图片上一步一步滑动，每次做一次乘加运算，把局部的视觉模式（边缘、纹理、拐角）转化成特征图上的激活值。这套运算的灵魂在于输出尺寸公式—— $(\text{输入}-\text{卷积核}+2\times\text{填充})/\text{步长}+1$ ，搞懂这个你就能心算每一层出来的特征图多大。池化则粗暴得多：最大池化只留最强的信号，平均池化取一个平滑的平均值，目的都是把特征图压小、把计算量降下来、顺便增加一点平移不变性。1x1 卷积是个彩蛋——不改变空间尺寸，只靠跨通道的信息融合来做升维降维，效率极高。最后那套尺寸追踪代码跑一遍，你对整个网络的”信息流”就有肌肉记忆了。

常见疑问

Q: 最大池化和平均池化怎么选？有没有明确的规则？ A: 大多数情况下用最大池化就够了，因为它保留了最强的特征响应，适合做分类。但如果你的任务是风格迁移或超分辨率这种需要平滑输出的场景，平均池化更好——它不会引入”最亮的那一个”带来的颗粒感。现在很多现代网络（比如 ResNet）其实在逐步用 $\text{stride}=2$ 的卷积替代池化层，因为卷积有可学习的参数，比池化的”硬裁剪”更灵活。

Q: 输出尺寸公式里什么情况会出现小数？怎么处理？ A: 如果你不设 padding 或者步长没整除，公式算出来确实可能不是整数。比如输入 32，卷积核 3，padding=0，stride=2，结果 $(32-3+0)/2+1=15.5$ 。PyTorch 里 Conv2d 默认会向下取整（变成 15），但有的框架直接报错。最佳实践是设计网络时就保证整除，用 padding 来凑。padding=same（TF 的写法）或 padding=kernel//2（PyTorch 的常用法）可以让卷积后尺寸保持不变，避免了这些头疼的整除问题。

第 17 章 ResNet 与残差连接

难度：★★★★☆

观光路线：理解为什么网络加深反而变差，看懂残差连接的“绕路”设计，知道 ResNet 为什么能训到 152 层还不崩，跑一遍 ResBlock 代码。

登山路线：推导残差连接对梯度传播的帮助（链式求导时永远保留一个+1），理解 bottleneck 设计里的计算量优化逻辑。

何恺明的白板

2015 年深秋的某个下午，微软亚洲研究院的一间办公室里，几个人围在白板前沉默了很久。白板上画着一个越来越深的卷积网络，训练误差曲线却诡异地在往上翘。

这个现象他们已经盯了好几周了。按直觉，网络越深，表达能力越强，性能应该越好才对。VGG 从 11 层加到 19 层，准确率确实涨了。但加到 56 层的时候出事了一——56 层的网络在训练集和测试集上表现都比 20 层的差。

这不是过拟合。过拟合是训练集上好、测试集上差。这玩意是两边都差，说明网络连训练数据都学不进去。

何恺明拿起笔，在白板上随手画了一条线——从输入直接绕过中间两层，跳到输出。“我们让网络去学残差吧。”他说。“与其让 56 层网络直接去拟合目标函数 $H(x)$ ，不如让它去拟合 $F(x) = H(x) - x$ 。网络输出的不是最终结果，而是最终结果跟输入的差值。”

同事愣了一下，然后问：“那如果某一层什么新东西都学不到，会发生什么？”

“ $F(x)$ 变成 0，”何恺明在白板上写，“输出就是纯 x 。这层等价于恒等映射——输入直接传给输出。最优情况是什么都不学，但至少不会倒退。”

会议室安静了两秒。

“把输入抄过来的那条线，”何恺明指着白板，“就叫 Skip Connection 吧。”

这个设计后来拿下了 CVPR 2016 的最佳论文奖，ResNet 的引用量到今天已经超过 18 万次。更魔幻的是，这个“绕路”的思路后来被 Transformer 捡起来，变成了大模型标配的残差连接。何恺明当年在白板上画的那条线，穿越了 CNN 和 Transformer 的边界，成了整个深度学习领域的通用设计。

退化问题：深不一定好

我们先把这个反直觉的现象搞清楚。为什么 56 层不如 20 层？

理论上，56 层网络是能模拟出 20 层网络的。你只要让前面 36 层什么都不干（学成恒等映射），后面 20 层照常工作就行。但现实是，随机梯度下降在 56 层网络里就是找不到这个解。层次越深，梯度信号传到底层就越微弱，前面那些层拿到的指导信息几乎为零，只能瞎更新。

退化问题跟梯度消失有关，但不完全一样。梯度消失会让训练停滞，而退化是网络明明还能训练，但越训越差。问题出在优化难度上——深网络的损失函数地形极其崎岖，SGD 走两步就掉坑里了。

打个比方。20 层的网络相当于让你从北京开车到天津，走高速，一路平坦。56 层相当于让你从北京开到广州，中间翻太行、过秦岭、穿南岭。车的马力一样，但路况复杂了太多。导航系统（梯度）还没传到你这里就已经衰减得不成样子了。

ResNet 的解法出奇简单：给每个模块加一条“捷径”。模块的数学定义从输出 = F(输入) 变成了输出 = F(输入) + 输入。

残差学习的数学

用公式写出来是这样的：

$$y = (x + F(x)) + \text{其他项}$$

其中 x 是输入， F 是两层卷积加激活函数组成的变换， y 是输出。那条 $+ x$ 就是残差连接的全部秘密。

它的精神内核用一句话就能说清：不是让网络去学“正确答案是什么”，而是让网络去学“正确答案跟输入差多少”。

这样做有两个天大的好处。

第一，恒等映射成了默认行为。 如果 $F(x)$ 学成 0，输出就是 x ，信息原封不动地流过去。网络不用费劲去学恒等映射——它天然就是。

第二，梯度有了一条高速公路。 反向传播的时候，损失对 x 的梯度是：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} (1 + F'(x))$$

那个 $+1$ 就是不死之身。不管 F 的梯度多小，加了个 1，梯度信号就永远不可能彻底消失。哪怕网络有 1000 层，每一层都保留一条数值为 1 的梯度通路，不会因为链式连乘而衰减成 0。

有人管这个叫“梯度高速公路”。很贴切。普通网络的梯度走的是山间小路，拐 100 个弯信号就没了。ResNet 的梯度可以直接走高速—— $F(x)$ 分支走小路， $+x$ 分支走高速，两股信号汇合，至少高速那条永远不会断。

ResBlock：积木块的设计

ResNet 最基本的单元叫 ResBlock，结构简单到让人怀疑：

输入 x → 卷积层 1 → BN → ReLU → 卷积层 2 → BN → 加上 x → ReLU → 输出

两个卷积层夹一个残差连接。如果输入和输出通道数不一样， $+ x$ 之前要先用 1×1 卷积把 x 的通道数对齐。

下面是一个完整的 ResBlock 实现和用三个 ResBlock 搭起来的小型 ResNet：

```

import torch
import torch.nn as nn

class ResBlock(nn.Module):
    """
    基础残差块：两个 3x3 卷积 + 跳跃连接
    如果通道数或空间尺寸不匹配，shortcut 用 1x1 卷积对齐
    """
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        # 主路径: Conv → BN → ReLU → Conv → BN
        self.conv1 = nn.Conv2d(
            in_channels, out_channels, kernel_size=3,
            stride=stride, padding=1, bias=False
        )
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(
            out_channels, out_channels, kernel_size=3,
            stride=1, padding=1, bias=False
        )
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        # 跳跃连接: 如果输入输出形状不一致，用 1x1 卷积对齐
        self.use_shortcut_conv = (stride != 1) or (in_channels != out_channels)
        if self.use_shortcut_conv:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1,
                    stride=stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )
        else:
            self.shortcut = nn.Identity() # 啥也不干，原样输出

    def forward(self, x):
        identity = self.shortcut(x) # 提前算好跳跃分支

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

        out += identity # 核心! 残差相加
        out = self.relu(out)
        return out

class SmallResNet(nn.Module):
    """
    三个 ResBlock 堆叠的小型残差网络
    用于 CIFAR-10 分类: 32x32 → 8x8 → 4x4 → 分类
    """
    def __init__(self, num_classes=10):
        super().__init__()
        # 初始卷积: 3→16 通道
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(16)

```

```

self.relu = nn.ReLU(inplace=True)

# 三个ResBlock, 通道数逐步翻倍, 空间尺寸逐步减半
self.layer1 = ResBlock(16, 16, stride=1) # 32x32 → 32x32
self.layer2 = ResBlock(16, 32, stride=2) # 32x32 → 16x16
self.layer3 = ResBlock(32, 64, stride=2) # 16x16 → 8x8

# 全局平均池化 + 分类头
self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
self.fc = nn.Linear(64, num_classes)

def forward(self, x):
    out = self.relu(self.bn1(self.conv1(x)))
    out = self.layer1(out)
    out = self.layer2(out)
    out = self.layer3(out)
    out = self.avgpool(out)
    out = out.view(out.size(0), -1)
    out = self.fc(out)
    return out

# ---- 快速验证 ----
model = SmallResNet(num_classes=10)
dummy = torch.randn(4, 3, 32, 32) # 4张CIFAR尺寸的图
output = model(dummy)
print(f"输入形状: {list(dummy.shape)}")
print(f"输出形状: {list(output.shape)}")
print(f"总参数量: {sum(p.numel() for p in model.parameters()),}")

```

代码里有两个细节值得注意。`bias=False`——因为 `BatchNorm` 后面紧接着有偏置项，卷积的偏置加了也白加，关掉省几个参数。`inplace=True` 的 `ReLU` 在原地修改张量，省一点内存，但要注意如果需要复用输入就别开 `inplace`。

`ResBlock` 用 `nn.Identity()` 处理不需要对齐的情况，这个写法很 `Pythonic`——不管需不需要 `shortcut` 转换，`forward` 里都放心写 `out += identity`，代码没有分支，干净。

`ResNet` 的出现让深度学习真正进入了“想多深就多深”的时代。152 层的网络不仅不会退化，反而效果拔群。那条看似偷懒的跳跃连接，后来被发现在 `Transformer` 里同样必不可少——`GPT`、`BERT`、`Llama`，每一个都内置了残差连接。何恺明白板上那条随手画的线，成了现代 AI 大厦里最粗的一根钢筋。

不过 `CNN` 和 `ResNet` 处理的都是规规矩矩的空间网格——一张图里像素和像素的关系，谁挨着谁方方正正、一目了然。可现实世界的很多数据根本摆不进这种整齐网格：社交网络里的关注关系、论文之间的引用、分子里的化学键，都是一张张错落的“图”。处理这类不规则的结构，需要一种能顺着“边”在节点之间传递信息的网络——这就是下一章要聊的事。

人话总结

这章就讲了一个反直觉的现象和一个优雅的解法。现象是：网络越深不一定会越好，56 层的网络居然比 20 层的差——而且不是过拟合，是训练集上都学不动。何恺明的解法简单到

让人拍大腿：与其让深层网络去死磕那个完美的输出 $H(x)$ ，不如让它去学“输入和输出之间差了点什么”（ $F(x)=H(x)-x$ ），最后把输入原样抄过来加上这个差值。这就是残差连接——一条绕过中间层直接把输入送到输出的“高速公路”。这个设计从根上解决了两个问题：正向传播时，即使中间层啥都学不到（ $F(x)=0$ ），信号至少能原封不动传过去；反向传播时，梯度流经这条短路始终保持至少+1的乘数，不会被连乘搞消失。ResNet用152层刷新了所有榜单，但它的真正遗产是“残差”这个思想——后来成了Transformer的标配，没有残差连接就没有今天的大模型。

常见疑问

Q: 残差连接和普通的“把上一层输出传给下一层”有什么区别？普通网络不也是上一层输出传给下一层吗？ A: 区别在于“加”的位置。普通网络是： $x \rightarrow \text{layer1} \rightarrow \text{layer2} \rightarrow \text{output}$ ，每一层的输出完全取决于上一层的变换结果。残差连接是： $x \rightarrow \text{layer1} \rightarrow \text{layer2}$ ，然后 $\text{output} = x + \text{layer2_output}$ 。那个 x 是跨越了两层直接跳过来的，它不经过任何变换。这意味着如果 layer1 和 layer2 一通操作把信息搞坏了， x 还是完整的。链式求导时，损失对 x 的梯度有一条“直通路径”，不用穿过那两个层的权重矩阵，避免了梯度被连乘稀释。

Q: 什么时候需要 1x1 卷积来对齐维度？每个残差块都要吗？ A: 只有当输入和输出的维度不一样时才需要。主要发生在两种场景：通道数变了（比如 $64 \rightarrow 128$ ）或者空间尺寸变了（ $\text{stride}=2$ 把尺寸减半）。这时 shortcut 路径上需要一个 1×1 卷积加 stride 来把输入调整到和输出一样的形状，才能做加法。PyTorch代码里用 `nn.Identity()` 处理维度一致的简单情况，用 1×1 卷积处理需要对齐的情况，很干净。

第 18 章 GNN 图神经网络

选读章节 —— 这章跟大模型主线关系不大，但对图数据感兴趣的话值得一看。
跳过去也不会影响理解后面的内容。

难度：★★★★☆

观光路线：理解图上消息传递的基本逻辑，搞懂 GCN 的归一化邻接矩阵干了什么，知道 PyG 怎么用，了解 GraphRAG 是什么东西。

登山路线：手动推导一层 GCN 的矩阵运算，理解为什么需要自环和对称归一化，搞清楚 nodes 和 edges 在代码里怎么对应。

朋友聚会上的传言传播

十几个朋友围坐在客厅聊天。A 说了一个八卦，周围的 B、C、D 听到了，各自加上自己的理解，转头说给旁边的人听。第二轮，B 告诉 E 和 F，C 告诉 G，D 告诉 H。到了第三轮，全场每个人都听到了这个八卦的某个版本——经过不同人转述，信息发生了微妙的变化，有的人添油加醋，有的人抓住重点复述。

这个过程在数学里叫“消息传递”（Message Passing）。每个节点（人）收到邻居传来的消息，把这些消息和自己的信息揉在一起，更新自己的状态，然后把这个更新后的状态传给下一轮的消息接收者。

CNN 和 RNN 处理的数据有一个共同点：它们是规则结构。图像是规则的二维网格，文本是规则的一维序列。但现实中太多数据天生是不规则图结构——社交网络里的关注关系、学术论文的引用网络、分子的化学键连接、知识图谱里的实体关系。

图神经网络（GNN）就是为这种不规则结构而生的一套方法论。

先搞清几个基本概念。图由节点（Node）和边（Edge）组成。每个节点可以有自己的特征向量——比如社交网络里一个用户的年龄、城市、兴趣标签。边可以有向的（A 关注了 B，但 B 没关注 A）也可以是无向的（A 和 B 是好友）。每条边还可以带权重——关系亲疏程度、化学键的强度。

GNN 的核心操作只有两步，在整个图上反复迭代：

1. **聚合（Aggregate）**：每个节点收集所有邻居发来的消息，用某种方式合并——求和、取平均、取最大值。
2. **更新（Update）**：把聚合后的邻居消息和节点自己的旧状态混在一起，通过一个小型神经网络产出新状态。

一次迭代就是一跳（1-hop）信息传播。两跳之后，每个节点不仅知道自己直接邻居的情况，还知道邻居的邻居——尽管没见过面，信息顺着路径传过来了。

GCN：图卷积网络

GCN（Graph Convolutional Network）是最经典的 GNN 变体，由 Kipf 和 Welling 在 2017 年提出。它的聚合方式异常简洁：把邻居的特征取平均，乘一个权重矩阵，过激活函数，完事。

写成数学公式是这样的：

$$[H^{(l+1)}] = (\frac{1}{D} \frac{A + I}{D} H^{(l)} W^{(l)})$$

这个公式看着像天书，拆开来每个符号都很直白。

$H^{(l)}$ 是第 l 层所有节点的特征矩阵，形状是 $[N, d]$ —— N 个节点，每个 d 维特征向量。

I 是加了自环的邻接矩阵。为什么要加自环？因为聚合邻居消息的时候，不能把自己给丢了——你自己的特征也很重要。 $(A + I)$ ， A 是原始邻接矩阵， I 是单位矩阵（对角线上全是 1）。

$(\frac{1}{D} \frac{A + I}{D})$ 这一大串是对邻接矩阵做对称归一化。 D 是度矩阵，对角线上写着每个节点有多少个邻居。归一化的作用是防止“社交达人”把别人的信号淹掉——一个节点如果有 1000 个邻居，它聚合的消息量是只有 5 个邻居的节点的 200 倍，归一化之后大家平等。

$W^{(l)}$ 是可学习的权重矩阵，跟全连接层里的 W 是一个道理。

用一句话总结 GCN 的一层：把邻居和自己的特征取个加权平均，过一层全连接，输出新特征。两层 GCN 等于让每个节点看到两跳范围内的所有节点。

上手 PyG：Cora 论文分类

PyG（PyTorch Geometric）是图神经网络最主流的框架。下面用它在 Cora 数据集上做一个论文节点分类——根据论文的词袋特征和引用关系，判断论文属于哪个领域：

```
import torch
import torch.nn.functional as F
from torch_geometric.nn import GCNConv
from torch_geometric.datasets import Planetoid

# ---- 加载 Cora 数据集 ----
# Cora: 2708 篇论文, 7 个类别, 引用关系构成图
# 每篇论文的特征是 1433 维的词袋向量
dataset = Planetoid(root='./data/Cora', name='Cora')
data = dataset[0] # 只有一个图

print(f"节点数: {data.num_nodes}")
print(f"边数: {data.num_edges}")
print(f"特征维度: {dataset.num_features}")
print(f"类别数: {dataset.num_classes}")
print(f"训练集/验证集/测试集: "
      f"{data.train_mask.sum()}/{data.val_mask.sum()}/{data.test_mask.sum()}")

# ---- 定义两层 GCN ----
class GCN(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels, dropout=0.5):
        super().__init__()
        # 第一层 GCN: 输入→隐藏
        self.conv1 = GCNConv(in_channels, hidden_channels)
        # 第二层 GCN: 隐藏→输出
        self.conv2 = GCNConv(hidden_channels, out_channels)
        self.dropout = dropout
```

```

def forward(self, x, edge_index):
    """
    x: 节点特征矩阵 [N, in_channels]
    edge_index: 边索引 [2, E], 第一行是源节点, 第二行是目标节点
    """
    # 第一层: GCN → ReLU → Dropout
    x = self.conv1(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=self.dropout, training=self.training)
    # 第二层: GCN → 输出 logits
    x = self.conv2(x, edge_index)
    return x

# ---- 训练 ----
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = GCN(dataset.num_features, hidden_channels=16,
            out_channels=dataset.num_classes).to(device)
data = data.to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)

def train():
    model.train()
    optimizer.zero_grad()
    out = model(data.x, data.edge_index)
    # 只用训练集节点计算损失
    loss = F.cross_entropy(out[data.train_mask], data.y[data.train_mask])
    loss.backward()
    optimizer.step()
    return loss.item()

@torch.no_grad()
def test():
    model.eval()
    out = model(data.x, data.edge_index)
    pred = out.argmax(dim=1)

    # 分别计算三个集合的准确率
    accs = []
    for mask in [data.train_mask, data.val_mask, data.test_mask]:
        correct = (pred[mask] == data.y[mask]).sum()
        acc = int(correct) / int(mask.sum())
        accs.append(acc)
    return accs

# 训练循环
for epoch in range(200):
    loss = train()
    if (epoch + 1) % 40 == 0:
        train_acc, val_acc, test_acc = test()
        print(f"Epoch {epoch+1:3d} | Loss: {loss:.4f} | "
              f"Train: {train_acc:.4f} | Val: {val_acc:.4f} | Test: {test_acc:.4f}")

print(f"\n 最终测试准确率: {test_acc:.4f}")

```

Cora 数据集只有 2708 个节点，训练节点更是只有 140 个——每个类别 20 个标注样本。这么少的数据，GCN 能达到 80% 以上的准确率，靠的就是图结构传递的信息。一篇论文被哪些论文引用了、引用了哪些论文，这些引用关系里藏着丰富的语义线索。

GCNConv 帮你封装了前面那个复杂公式里的所有矩阵运算。你传入节点特征 x 和边索引 `edge_index`，它内部帮你做消息传递和特征变换。`edge_index` 的形状是 $[2, E]$ ，这个数据结构需要注意——它不是邻接矩阵，而是一个稀疏的边列表，节省了大量内存。

GNN 遇见大模型：GraphRAG

GNN 和大模型看上去是两个世界的东西，但它们已经产生了交叉。

最火的方向叫 GraphRAG。传统的 RAG（检索增强生成）是先检索相关文档片段，然后让大模型基于这些片段回答问题。GraphRAG 多了一步：先根据检索到的文档构建一个知识图谱，用 GNN 或者图算法找出实体之间的关系，最后把结构化的图信息作为上下文喂给大模型。

举个例子。你问“微软投资 OpenAI 之后，Sam Altman 在 2023 年 11 月发生了什么？”“普通 RAG 可能检索到几篇包含“Sam Altman”和“2023 年 11 月”的新闻。GraphRAG 会先把人物、机构、事件建成图节点，然后沿着图的边推理——Sam Altman 是 OpenAI 的 CEO，OpenAI 跟微软有投资关系，2023 年 11 月有一条边指向“董事会政变”事件。图结构提供的关联上下文，让大模型的回答既准确又有深度。

另一个前沿方向叫 GNN-LLM 协同。GNN 负责在结构化数据（知识图谱、分子图、社交网络）上做推理，LLM 负责理解自然语言和生成文本。分子属性预测就是一个典型场景：GNN 从分子图上提取化学结构特征，LLM 理解实验报告里的文字描述，两者融合后做出更精准的预测。

GNN 的世界远比这一章讲的广阔。GAT 加了注意力机制到消息传递里，GraphSAGE 用采样策略应对超大图，GIN 从理论上证明了 GNN 的表达能力上限。但核心思想万变不离其宗——在图结构上做消息传递。

从 CNN 的规则网格卷积，到 GNN 的任意图结构消息传递，我们看到了一个更本质的模式：神经网络的核心操作，是在某种数据结构上定义“邻居”，然后让信息在邻居之间流动。CNN 的邻居是空间近邻，RNN 的邻居是时间近邻，GNN 的邻居是图上的连接关系。不同的数据结构只是“邻居”的定义不同，信息流动的思想一以贯之。

而所有这些网络，不管是卷图像、读文本、还是走图结构，都面临同一个根本问题：怎么把原始数据变成计算机能理解的数字？怎么让“猫”、“红色”、“悲伤”这些概念，变成向量空间里可以被运算的坐标？

人话总结

这章最大的收获是一种思维方式的升级：不是所有数据都能整齐地排成网格或序列，图结构才是很多现实问题的“原生态”。GNN 的核心操作就两步，在整张图上反复迭代：每个节点先“聚合”邻居们发来的消息（求和、平均或取最大），然后“更新”自己的状态（把邻居消息和自己的旧特征揉在一起过一个小网络）。迭代一次是一跳传播，两次是两跳——节点虽然没直接见过邻居的邻居，但信息顺着路径传过来了。GCN 是 GNN 最经典的实现，核心是一个加了自环和对称归一化的邻接矩阵，保证特征传播时不会因为某些节点度很高而数值爆炸。跑一遍 Cora 论文分类的代码，你会发现 140 个训练样本就能冲到 80% 以上的准确率——全靠

引用关系里藏着的语义信息。最后 GraphRAG 给了你一个现实应用锚点：用图结构帮大模型整理知识之间的关联，让 RAG 的回答从”找到相关文档”升级到”理清实体之间的逻辑链”。

常见疑问

Q: GNN 和 CNN 到底有什么本质联系？为什么有人说 GNN 是 CNN 在图上的推广？ A: 联系在于”邻居聚合”这个核心思路。CNN 的卷积就是在规则的二维网格上，每个像素聚合周围 3x3 邻居的信息。GNN 把这个操作推广到了不规则的图结构——邻居不再是一个固定大小的矩形窗口，而是由图的边定义的任意节点集合。两者的本质都是：定义一个”邻域”，然后把邻域内的信息聚合起来。CNN 的邻域由空间位置决定，GNN 的邻域由图结构决定。

Q: GNN 能处理多大的图？社交网络有上亿节点怎么办？ A: 全图加载到 GPU 里做 GCN 确实扛不住。工业界用几种策略：GraphSAGE 做邻居采样（每个节点只随机采几十个邻居，不聚合整张图）；Cluster-GCN 先把图切成小块，每块独立训练；还有些方法用分布式训练把图分到多张卡上。另一个思路是图不必”太完整”——很多应用场景只需要二跳图（节点-直接邻居-邻居的邻居），信息再多收益递减。

语言建模演进篇

公众号 AI 技术圈

第 19 章 统计语言模型与 N -gram：让计算机学会猜词

难度：★★☆☆☆

观光路线：理解”语言模型=猜下一个词”，看懂 N -gram 怎么靠统计数数预测，明白它为什么”懂计数、不懂语义”。15 分钟。

登山路线：手写 Bigram/Trigram 计数逻辑，自己算出 N 变大后组合数量怎么爆炸，跑通代码统计一个迷你语料的条件概率。20 分钟。

阅读顺序提示：这是”自然语言 → 大模型”主线上的**第一块积木**，也是贯穿全书 NLP 线的”猜词母题”的起点。建议排在 **20-Embedding 与表征学习** 之前阅读（本章结束时你会发现词向量正是为解决本章的两个缺陷而生）。它也承接第 6 章的贝叶斯/条件概率。

你猜过词吗？不用举手，我们直接来。

给你一句话的前半段，你来补全下一个词：

周末的早晨，我给自己泡了一杯_____。

A. 咖啡 B. 雨伞 C. 红茶 D. 键盘

你大概一眼就锁定了 A 或 C——咖啡、红茶都对。就算把选项去掉，你也会脱口而出一种饮料：“牛奶”“奶茶”“绿茶”都行。

有意思的问题来了：**你是怎么猜出来的？**

你并没有”周末的早晨，我给自己泡了一杯”这个具体句子的记忆。你靠的是：读过无数文章，见过”泡了”后面通常跟着茶或咖啡，而”泡了一杯雨伞”这种说法从来没在脑子里出现过。你是在下意识地判断——在中文里，“泡了一杯”后面**最可能出现**什么。

这个”根据前面的内容，猜出下一个最可能出现的词”的游戏，有个正式名字：**语言模型 (Language Model)**。

记住这个名字。因为后面几十章的故事——词向量、RNN、LSTM、Transformer、GPT——全部、无一例外，都在做同一件事：**猜下一个词**。模型换了一代又一代，考卷永远是同一张。

翻译成成人话就是：**语言模型是人类”说话能力”的最小抽象**。你不是背下了所有句子才学会说话的，你是学会了一套”看着前面，猜后面”的本事。

N -gram：让计算机数数

好，现在让计算机来玩这个游戏。最简单粗暴的思路是什么？

统计。

让计算机读海量的文章，然后数一数：不同的词语组合，各出现过多少次。

假设语料库里有这样的数据：

- “泡了一杯咖啡” 出现 10000 次
- “泡了一杯红茶” 出现 8000 次
- “泡了一杯奶茶” 出现 5000 次
- “泡了一杯雨伞” “泡了一杯键盘” “泡了一杯窗帘” 出现 0 次

那计算机看到”泡了一杯”时，就能用脚投票：咖啡概率最高，红茶、奶茶次之，雨伞、键盘几乎不可能。于是它选 A——咖啡。

这就是**统计语言模型**最朴素的思想：**看前面已经出现的词，统计下一个词最可能是谁。**

根据”参考前面几个词”，有不同的名字：

名称	N（一次绑几个词）	猜下一词时回头看	例子
Bigram（二元模型）	2	1 个	看到”一杯”，统计”一杯咖啡 / 一杯红茶”
Trigram（三元模型）	3	2 个	看到”泡了一杯”，统计后面的词
N-gram	N	N-1 个	一次考虑 N 个连续词

翻译成成人话就是：N-gram 的”N”是一次把连续几个词绑成一组。用来猜下一词时，它回头看 N-1 个词——看得越多，理论上猜得越准，但也越容易翻车，下面会讲。

这套”数数”的思路，1948 年香农（Claude Shannon）研究英语里到底有多少”冗余”时就用过了。他发现语言里有大量**可预测**的成分——而”可预测”这三个字，正是猜词游戏的物理基础。我们今天讲的 N-gram 语言模型，就是把这套思想彻底机械化：预测不了就数，数得准就是本事。

用概率的语言写出来，N-gram 就是在估计一个**条件概率**——这正是第 6 章贝叶斯/条件概率的自然延伸。比如 Bigram 估计的是：

$P() =]$

“出现次数”就是数数。这个模型，本质上一行代码就能写出来：

```
from collections import defaultdict

# 迷你语料：只有几句话
corpus = ["我 泡 了 一 杯 咖啡",
          "我 泡 了 一 杯 红茶",
          "她 泡 了 一 杯 咖啡",
          "她 泡 了 一 壶 红茶"]

# 统计 Bigram 共现次数
bigram_counts = defaultdict(int) # (前词, 后词) → 次数
unigram_counts = defaultdict(int) # 前词 → 次数

for sent in corpus:
```

```

words = sent.split()
for i in range(len(words) - 1):
    bigram_counts[(words[i], words[i+1])] += 1
    unigram_counts[words[i]] += 1
unigram_counts[words[-1]] += 1 # 最后一个词也要计数

def bigram_prob(w1, w2):
    """给定 w1, 预测 w2 的概率"""
    return bigram_counts[(w1, w2)] / unigram_counts[w1]

# 测试: 看到"一杯", 下一个词是?
print("P(咖啡 | 一杯) =", round(bigram_prob("一杯", "咖啡"), 3))
print("P(红茶 | 一杯) =", round(bigram_prob("一杯", "红茶"), 3))
print("P(红茶 | 一壶) =", round(bigram_prob("一壶", "红茶"), 3))

# 输出:
# P(咖啡 | 一杯) = 0.667
# P(红茶 | 一杯) = 0.333
# P(红茶 | 一壶) = 1.0

```

数数。这就是 N-gram 的全部。它没有理解任何东西——它不知道“咖啡”是苦的、“红茶”要配块糖，但它会数数，而且数得挺准。

坏消息一：组合爆炸，数据永远不够

看起来简单有效，那我们把 N 加大一点、让模型看到更多前文，预测会不会越来越准？

并不会。先算一笔账：

假设词汇表里有 **10 万个词**。只看两个词的组合，理论上就有：

[100{,}000 000 = 100]

三词组合、四词组合……数量指数级膨胀。而现实世界的句子千变万化，哪怕你准备了几亿篇文章，也永远覆盖不全所有合理组合。

翻译成成人话就是：**N-gram 不是不想看远一点，是看远一点就“数据荒”**。

举个扎心的例子。把选项 A 从“咖啡”换成“卡布奇诺”：

周末的早晨，我给自己泡了一杯_____。

人类依然秒选——卡布奇诺也是一种咖啡嘛。但 N-gram 呢？如果语料里只有“泡了一杯咖啡”，**从来没有**“泡了一杯卡布奇诺”，那对只会数数的 N-gram 来说，这是一个**全新组合**——出现次数 0，概率 0。它直接答不出来。

这就是 N-gram 的第一个缺陷：**数据稀疏 / 组合爆炸**。句子稍微一长、组合稍微一新鲜，统计就失灵。N 越大，失灵得越快——你会在第 13 章见到一个老朋友：“过拟合”。N 太大就是典型的过拟合：把具体组合学死了，反而失去泛化能力。

坏消息二：词是孤立的符号，毫无语义

第二个问题更根本。

在 N-gram 眼里，“咖啡”“红茶”“卡布奇诺”“拿铁”都只是互不相关的编号。它不“知道”这些都属于饮料，也不知道咖啡和卡布奇诺含义接近。

所以它无法把统计出来的规律，迁移到没见过的句子上。“泡了一杯卡布奇诺”它答不出来，本质是它不认识“卡布奇诺”和“咖啡”的近义关系。

而人类为什么能答对？因为我们知道词语的含义——语义。

N-gram 缺的正是这个。它擅长“计数”，完全不懂“含义”。一个只会数数的模型，遇到没见过的新句子，就只剩下一句台词：“这题超纲了。”

总结：一张问题清单

停下来，把语言模型这条线的“问题清单”清晰列出来：

- **目标：**根据前文，预测下一个词（猜词母题）。
- **第一招 N-gram：**靠统计数数，简单直白。
- **缺陷①：**组合爆炸，语料覆盖不全，新组合概率为 0。
- **缺陷②：**词是孤立符号，不懂语义，规律无法泛化。

这两个缺陷，恰好是下一章（20-Embedding 与表征学习）要解决的问题——把“词”变成带语义的高维向量。有了词向量，模型才第一次“理解”咖啡和卡布奇诺像不像。

再往后，用神经网络（08 章）读这些向量，用 RNN（22 章）给它加记忆，用 Transformer（27 章）让它并行——整个故事就接上了。

翻译成成人话就是：“猜下一个词”是整条 NLP 主线的心脏，N-gram 是它的第一台发动机。后面所有的技术，都是在给这台发动机换更好的零件。

常见疑问

Q：N-gram 现在还在用吗？ A：作为完整模型基本被淘汰了，但它依然是语言模型最直观的入门入口。它的思想（统计相邻词共现）还活在分词、纠错、拼音输入法、稀疏特征等许多角落。大模型时代，它是理解一切的“最简版本”。

Q：N 越大越好吗？ A：理想上更准，但代价是组合爆炸更快、数据更稀疏。你在第 13 章会见到“过拟合”——N 太大就是典型的过拟合：学死了一堆具体组合，反而失去泛化。

Q：为什么说它是“猜词母题”的起点？ A：因为后面所有模型都在做同一件事——预测下一个词。N-gram 用计数猜，词向量的网络用语义猜，Transformer 用注意力同时看全局猜。模型换了一代又一代，考卷永远是同一张：“下一个词是什么？”

Q：代码里我数的是词，但中文不是先分词吗？ A：问得好。分词（Tokenization）是另一个大话题，第 28 章会专门讲。现在你先假装“词”已经切好了——反正 N-gram 关心的只是“上一个和下一个”。

Q：香农为什么对”猜词”感兴趣？ A：因为”能预测多少”直接决定了信息能压缩多少。一句话里可预测的部分越多，冗余越高，需要传输的”新信息”越少。猜词游戏背后，藏着的其实是信息论的根基。

公共易 AI 频道

第 20 章 Embedding 与表征学习

难度：★★★★☆

观光路线：理解 Embedding 就是把任何东西变成高维空间里的一个点，知道 Word2Vec 的“国王-男人+女人=女王”是怎么算出来的，看懂 MTEB 榜单上的几个主流模型各自的长处，搞清楚 Embedding 在 RAG 里扮演什么角色。

登山路线：理解 Skip-Gram 的负采样策略和损失函数设计，手动推导对比学习里 InfoNCE Loss 的梯度，知道 MTEB 各类任务分别在测什么能力。

阅读顺序提示：本章是“自然语言 → 大模型”这条主线的起点之一。建议先读前章《19-统计语言模型与 N-gram》打好“猜下一个词”的底子，再从本章把词变成向量。**词向量是所有语言模型（RNN、LSTM、Transformer）的输入**，所以本章建议排在 22-RNN、23-LSTM 之前阅读，RNN 章节才会更好懂。

上一章，我们让计算机学会了“猜下一个词”——N-gram 靠数数，简单直白，却留下两个解不开的死结：一个是**组合爆炸**，没见过的新组合概率永远是 0；另一个更根本，**词在它眼里只是孤立的编号**，“咖啡”和“卡布奇诺”它根本不知道是一家人。要解开这两个结，得先给“词”换一副更聪明的模样——让每个词都携带语义。这就是本章的主角：**词向量（Embedding）**。

动物园馆长的万能表格

你是一家动物园的园长。园里有老虎、企鹅、大象、麻雀、金鱼，动物越来越多，你的记录表格也越来越乱。按食性分？老虎吃肉、企鹅吃鱼、大象吃草。按栖息地分？老虎住森林、企鹅住极地、金鱼住水塘。按体型分？大象大、金鱼小。每一种分类都有道理，但没有一种分类能完美描述所有动物之间的关系。

有一天你发明了一种新方法。你画了一张巨大的表格，列了一堆“描述维度”，不要求每列有明确的名字——可以是“维度 1：某种食性相关的抽象指标”、“维度 2：某种体型和力量的混合度量”、“维度 3：生活环境的水陆倾向”。然后你给每只动物在这张表上填了一串分数。

神奇的事情发生了。在这个模糊的分数体系下，相似的动物自动聚在了一起。老虎和豹子在表上离得很近，企鹅和海鸥也不远。你想知道“什么东西介于老虎和大象之间？”“把老虎的分数和大象的分数求个平均，得到的坐标落在河马附近。

Embedding 就是这个模糊分数体系的终极版本。它的核心思想朴素到让人失望：把世界上任何东西——一个字、一句话、一张图、一首歌——变成高维空间里的一个坐标点。相似的东西在这个空间里离得近，不相似的离得远。

跟你填的表格不一样，Embedding 的维度不是人设计的，是机器从海量数据里自己“悟”出来的。维度二可能同时捕捉了“食性”的 30%和“体型”的 70%，维度七可能混合了“水生倾向”和“是否为哺乳动物”。这些维度是纠缠的、不可解释的，但它们在几何空间里准确地组织了语义关系。

Word2Vec: 国王的算术题

2013 年, Google 的 Tomas Mikolov 团队发布了 Word2Vec, 第一次让全世界看到了词嵌入的魔力。

他们做了一件非常简单的事: 用一个浅层神经网络, 让它猜一个词的上下文。给定句子“我今天去超市买了____”, 让网络猜中间是什么词。训练过程中, 网络把每个词映射成一个 300 维的向量。训练结束后, 他们发现了不得了的事情。

向量空间里藏着语义算术。

把“国王”的向量减去“男人”的向量, 再加上“女人”的向量, 得到的结果向量在空间中搜索, 最近邻的词是——“女王”。

[- +]

这太魔幻了。没有任何人告诉过网络“国王和男人的关系等于女王和女人的关系”。网络只是在几亿个句子里反复预测上下文词, 自己就“悟”出了这种类比关系。

不只这一组。“巴黎 - 法国 + 意大利 $\approx$ 罗马”、“游泳 - 水 + 天空 $\approx$ 飞翔”、“苹果 - 水果 + 牛肉 $\approx$ 肉”。这些类比不是巧合, 是一个结构化的语义空间必然涌现的性质。

用更精确的方式解释。Skip-Gram 的训练目标是: 给定中心词 w_t , 最大化它周围上下文词 $w_{t-k}, \dots, w_{t+k}$ 出现的概率。数学上等价于让词向量之间的点积去逼近词共现的点互信息 (PMI)。PMI 衡量两个词同时出现的概率比它们各自独立出现的概率高多少——“国王”和“王室”的共现远比“国王”和“菠萝”的共现更不像是巧合。向量空间里的几何距离, 反映的就是这种统计上的关联强度。

纯 NumPy 写一个微型 Word2Vec

光说不练假把式。下面用纯 NumPy 实现微型 Skip-Gram, 在几十个句子的小语料上训练, 然后做降维可视化:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

```
# ---- 微型语料 ----
sentences = [
    "国王 统治 国家 女王 管理 王室",
    "国王 住在 宫殿 女王 住在 宫殿",
    "男人 工作 办公室 女人 工作 医院",
    "男人 强壮 女人 温柔 孩子 可爱",
    "苹果 好吃 香蕉 好吃 水果 健康",
    "狗 忠诚 猫 独立 宠物 陪伴",
    "汽车 快速 火车 快速 飞机 更快",
    "北京 中国 东京 日本 首尔 韩国",
    "夏天 热 冬天 冷 春天 温暖",
    "跑步 锻炼 游泳 锻炼 健身 健康",
] * 30 # 重复扩充数据量
```

```
# ---- 构建词表和训练数据 ----
```

```

words = set()
for s in sentences:
    for w in s.split():
        words.add(w)
word_list = list(words)
word_to_idx = {w: i for i, w in enumerate(word_list)}
idx_to_word = {i: w for w, i in word_to_idx.items()}
vocab_size = len(word_list)

# 生成 Skip-Gram 训练对: (中心词, 上下文词)
pairs = []
window_size = 2
for s in sentences:
    tokens = s.split()
    for i, center in enumerate(tokens):
        start = max(0, i - window_size)
        end = min(len(tokens), i + window_size + 1)
        for j in range(start, end):
            if j != i:
                pairs.append((word_to_idx[center], word_to_idx[tokens[j]]))

pairs = np.array(pairs)
print(f"词表大小: {vocab_size}, 训练对数: {len(pairs)}")

# ---- Skip-Gram 负采样实现 ----
embed_dim = 32 # 嵌入维度
lr = 0.05

# 输入嵌入矩阵 (每个词作为中心词时的向量)
W_in = np.random.randn(vocab_size, embed_dim) * 0.01
# 输出嵌入矩阵 (每个词作为上下文词时的向量)
W_out = np.random.randn(vocab_size, embed_dim) * 0.01

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -10, 10)))

# 词频分布用于负采样 (按频率的0.75次方)
freq = np.zeros(vocab_size)
for c, ctx in pairs:
    freq[c] += 1
    freq[ctx] += 1
freq = freq ** 0.75
freq /= freq.sum()

for epoch in range(1200):
    np.random.shuffle(pairs)
    total_loss = 0

    for center, context in pairs:
        # ---- 正样本: 中心词和真实上下文词 ----
        v_center = W_in[center] # 中心词的嵌入向量
        u_context = W_out[context] # 上下文词的输出向量
        pos_score = sigmoid(np.dot(v_center, u_context))

        # 负采样: 随机抽k个词作为"假上下文"
        k = 5 # 负样本数量
        neg_samples = np.random.choice(
            vocab_size, size=k, p=freq, replace=True

```

```

    )

    # ---- 计算梯度并更新 ----
    # 正样本梯度
    grad_center_pos = (pos_score - 1) * u_context
    grad_context_pos = (pos_score - 1) * v_center

    W_in[center] -= lr * grad_center_pos
    W_out[context] -= lr * grad_context_pos

    # 负样本梯度
    neg_loss = 0
    for neg in neg_samples:
        u_neg = W_out[neg]
        neg_score = sigmoid(np.dot(v_center, u_neg))

        grad_center_neg = neg_score * u_neg
        grad_neg = neg_score * v_center

        W_in[center] -= lr * grad_center_neg
        # 注意这里不加负号, 因为S型函数的导数自带
        W_out[neg] -= lr * grad_neg
        neg_loss += -np.log(max(1 - neg_score, 1e-8))

    total_loss += -np.log(max(pos_score, 1e-8)) + neg_loss

    if (epoch + 1) % 300 == 0:
        print(f"Epoch {epoch+1}/1200, Loss: {total_loss/len(pairs):.4f}")

    # ---- 使用输入嵌入作为最终词向量 ----
    embeddings = W_in # [vocab_size, embed_dim]

    # ---- PCA 降维并可视化 ----
    pca = PCA(n_components=2)
    embeddings_2d = pca.fit_transform(embeddings)

    plt.figure(figsize=(12, 10))
    plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
    plt.rcParams['axes.unicode_minus'] = False

    for i, word in enumerate(word_list):
        x, y = embeddings_2d[i]
        plt.scatter(x, y, alpha=0.6, s=60)
        plt.annotate(word, (x, y), fontsize=10,
                     xytext=(5, 5), textcoords='offset points')

    plt.title("微型词嵌入的 PCA 可视化 (32 维→2 维)", fontsize=14)
    plt.xlabel(f"主成分 1 ({pca.explained_variance_ratio_[0]:.1%} 方差)")
    plt.ylabel(f"主成分 2 ({pca.explained_variance_ratio_[1]:.1%} 方差)")
    plt.tight_layout()
    plt.savefig("word2vec_pca_viz.png", dpi=150)
    print("可视化已保存到 word2vec_pca_viz.png")

    # ---- 语义类比测试 ----
    def analogy(a, b, c, topn=3):
        """计算 a - b + c 的最近邻"""
        if a not in word_to_idx or b not in word_to_idx or c not in word_to_idx:
            return []

```

```

vec = embeddings[word_to_idx[a]] - embeddings[word_to_idx[b]] \
    + embeddings[word_to_idx[c]]
# 归一化后计算余弦相似度
vec_norm = vec / (np.linalg.norm(vec) + 1e-8)
emb_norm = embeddings / (np.linalg.norm(embeddings, axis=1, keepdims=True) + 1e-8)
sims = np.dot(emb_norm, vec_norm)
# 排除输入词本身
exclude = {word_to_idx[a], word_to_idx[b], word_to_idx[c]}
for idx in np.argsort(-sims):
    if idx not in exclude:
        result = idx_to_word[idx]
        # 跳过输入词的近邻, 取下一个
        exclude.add(idx)
    return result
return "?"

# 尝试几组类比
tests = [
    ("国王", "男人", "女人"), # 期待: 女王
    ("北京", "中国", "日本"), # 期待: 东京
    ("夏天", "热", "冬天"), # 期待: 冷
    ("狗", "宠物", "汽车"), # 期待: 交通工具?
]
print("\n--- 语义类比测试 ---")
for a, b, c in tests:
    d = analogy(a, b, c)
    print(f"{a} - {b} + {c} ≈ {d}")

```

小语料上训练的微型词嵌入当然不会精准地算出“国王-男人+女人=女王”，但它能在统计层面捕捉到同类的聚合趋势——“国王”和“女王”在向量空间里的位置确实比“国王”和“苹果”更接近。

Skip-Gram 负采样的优雅之处在于：它把原本要计算整个词表 softmax 的恐怖开销，简化成了只需对 k 个负样本做二分类。正样本（真实上下文词）的分数往高推，负样本（随机抽的词）的分数往低拉。本质上是在做一个对比学习——让相关的词对互相靠近，不相关的词对互相推开。

MTEB: 嵌入模型的奥运会

Word2Vec 在 2013 年让人惊艳，但放在今天就太简陋了。现代嵌入模型已经进化成了庞大的语义理解系统，能处理的不只是单词，还包括整段文本、多语言混排、甚至图文跨模态。

MTEB (Massive Text Embedding Benchmark) 是这个领域的权威评测。它包含 8 大类、58 个数据集，覆盖了嵌入模型可能遇到的几乎所有任务类型：

分类 (Classification)： 给定一段文本，判断它属于哪个类别。考验的是嵌入向量是否携带了足够的语义标签信息。

聚类 (Clustering)： 把一堆文本自动分组。考察嵌入空间中同类文本是否能自然聚团。

配对分类 (Pair Classification)： 判断两段文本是不是同一主题。考验的是余弦相似度的判别力。

重排序 (Reranking)：给定一个查询和一组候选文档，按相关性排序。这是搜索引擎和 RAG 的核心需求。

检索 (Retrieval)：从海量文档库里找出跟查询最相关的。考察的是大规模向量搜索场景下的表现。

语义文本相似度 (STS)：判断两句话的语义有多接近——不能只看词面重叠。“我今天很开心”和“心情真好”应该比“我今天很开心”和“我今天很伤心”得分更高。

摘要 (Summarization)：评估生成的摘要和原文在语义空间里的对齐程度。

双语文本挖掘 (Bitext Mining)：在平行语料库里匹配互为翻译的句子对。

一个嵌入模型可能在某些任务上登顶，在另一些任务上表现平平。选模型要看你的具体需求——做 RAG 就重点看检索和重排序的分数，做情感分析就看分类，做搜索引擎就三者都得兼顾。

主流嵌入模型：各有各的绝活

2024 到 2025 年，嵌入模型领域群雄并起。下面这几个是目前最有代表性的：

OpenAI text-embedding-3 系列。分 small 和 large 两个版本。small 只有 384 维但效果惊人，适合资源受限场景。large 扩展到 3072 维，在 MTEB 检索任务上很长时间霸榜。最大的亮点是支持 Matryoshka Representation Learning——可以在不损失太多性能的前提下把向量截断到任意更小的维度。你训好一个 3072 维的嵌入，实际使用时按需要取前 256 维、512 维或者 1024 维，效果都不会崩。这对于存储成本敏感的场景是个杀手级特性。

BAAI bge-m3。智源研究院出的多语言嵌入模型，支持 100 多种语言，中英文都强。它的独特之处在于一次编码同时产出三种向量：稠密向量 (Dense，适合语义搜索)、稀疏向量 (Sparse，适合关键词匹配)、多向量 (ColBERT 风格，适合细粒度 token 级交互)。在中文场景表现尤为出色，很多国内 RAG 项目首选。

Alibaba GTE 系列。通义实验室的嵌入模型，最大亮点是 Qwen2/GTE 的深度耦合——如果你想做 Qwen2 的 RAG，用 GTE 作为嵌入组件是天作之合。训练时就在 Qwen2 的表示空间里做对齐，检索-生成之间的语义鸿沟比跨模型组合小得多。

选模型的决策可以归纳成一个简单的框架：

场景	推荐	理由
英文为主，追求极致检索精度	text-embedding-3-large	MTEB 检索榜首，Matryoshka 灵活
中英混合，多语言需求	bge-m3	原生多语言，三种向量类型
Qwen2 生态，端到端优化	GTE-Qwen2	和生成模型共享表示空间
低成本部署，效果够用	text-embedding-3-small / bge-small-zh	维度低，速度快，大部分场景够用

Embedding 在 RAG 中的角色

RAG（检索增强生成）是 Embedding 最重要的落地场景。流程分三步：

离线阶段——建立知识库。把公司的文档、产品手册、FAQ 拆成一个个小段落，每个段落调用嵌入模型产生一个向量，存进向量数据库（比如 Milvus、Qdrant、Pinecone）。

在线阶段——检索。用户问了一个问题，这条问题被同一个嵌入模型编码成向量。向量数据库在毫秒级的时间内，从几百万条段落里找出余弦相似度最高的前 K 条。

生成阶段——增强回答。检索到的 K 条段落拼在用户问题前面，一起塞给大模型。大模型有了这些“参考资料”，回答更准确、更可靠，也少了很多幻觉。

整个链路里，嵌入模型的质量决定了检索的“召回率”——相关但没找到的段落，大模型永远看不见。这也是为什么 MTEB 的检索和重排序分数对 RAG 项目来说是最关键的指标。

Embedding 的能力边界也很清楚：它擅长“找相似”，但不懂“做推理”。你问“2023 年销量最高的产品是什么？”“它可以帮你找到所有提到 2023 年销量的段落，但它不会自动把不同段落的数字加起来算出总和——那是大模型的事。Embedding 负责检索，大模型负责理解和综合，分工明确。

Character is what you are in the dark。嵌入向量就是 AI 的“暗暗人格”——没有花哨的界面，没有流畅的对话，它只是静静地蹲在数据底层，把一切事物都变成数字。但正是这些安静的向量，在幕后撑起了现代 AI 大厦最底层的地基。

从 CNN 的卷积核、RNN 的隐藏状态、ResNet 的跳跃连接、LSTM 的门控记忆、GNN 的消息传递，到 Embedding 的语义空间——我们绕着“表征”这个核心概念转了一大圈。所有的神经网络，不管是处理图像、文本、图结构还是任何其他数据，本质上都在做同一件事：找到一种方式，把原始世界里的东西变成计算机能运算的向量表示。

这个朴素的观察，把我们带到了下一个问题面前。既然已经能把文字变成向量了，能不能让计算机用神经网络读这些向量，把 19 章那道“泡了一杯____”的猜词题玩得更聪明——不再靠数数，而是靠理解语义？下一章，我们就把词向量和神经网络拼起来，第一次用“理解”而不是“记忆”来猜词。

人话总结

这章带你走完了一条从 Word2Vec 到 MTEB 的进化线。Embedding 的核心思想朴素到让人失望：把世界上任何东西变成高维空间里的一个坐标点，相似的东西挨得近，不相似的离得远。Word2Vec 是这条路的第一块里程碑——用一个浅层网络学出了“国王-男人+女人=女王”这种语义算术，证明了向量空间可以编码人类概念。但现代嵌入模型远比 Word2Vec 复杂：MTEB 榜单上有 8 大类 58 个数据集，从分类到检索到语义相似度，每个任务都在测量嵌入向量的不同能力。选模型要看场景——做 RAG 就盯着检索和重排序的分数，做情感分析就看分类。Embedding 在 RAG 里的角色很明确：它负责“找相似”，大模型负责“做推理”，分工一清二楚。这章真正想让你带走的是一个视角：所有神经网络——CNN、RNN、LSTM、GNN——本质上都在做表征学习。把原始世界的东西变成好用的向量，这就是深度学习最底层的那条暗线。

常见疑问

Q: Word2Vec 训练出来的向量维度为什么一般设 300？设高一点会更好吗？ A: 300 是 Google 原论文里用的维度，后来被大家当成了“够用且可控”的默认值。理论上维度越高表达能力越强，但边际收益递减——从 300 提到 600 可能涨零点几个点，参数却翻了一倍。更关键的是，在一个小语料上训超高维度的向量容易过拟合（向量记住了一些偶然的共现模式而不是真正的语义关系）。如果是现在的 BERT 类模型，768 维或 1024 维已经是标配了，效果远超 Word2Vec 的 300 维——但那是完全不同的训练范式。

Q: 有没有办法知道 Embedding 向量每个维度具体在表示什么？ A: 基本上不能。这也是嵌入模型最大的“黑箱”属性。第 7 维可能同时混合了“食性”的 30%和“体型”的 70%，第 23 维可能纠缠了“水生倾向”、“是否哺乳动物”和“活动区域纬度”。你大致可以用降维可视化（t-SNE 或 UMAP）看看高维空间的分布，或者做探测任务（比如训一个线性分类器看某个维度是否能预测情感极性），但你永远得不到一个干净的、人类能理解的“维度含义表”。这也是为什么 Embedding 被叫做“表征学习”而不是“特征工程”——特征的语义是机器自己“悟”出来的，不是人定义的。

第 21 章 前馈神经网络做语言模型：用语义猜词

难度：★★★★☆☆

观光路线：理解怎么把词向量拼起来喂给神经网络、用 Softmax 选下一个词，感受它比 N-gram 强在哪，以及它撞上的”固定窗口”这堵墙。15 分钟。

登山路线：跑通一个”三词输入 → 前馈网络 → 词表打分”的微型语言模型，观察近义词词向量产生相近的分数，理解窗口大小为什么是硬伤。20 分钟。

阅读顺序提示：承接 19-N-gram（猜词母题起点）和 20-Embedding（词如何变成向量）。本章用神经网络完成第一次”带语义的猜词”，然后撞上固定窗口，为 22-RNN 登场铺垫。结构上补足了”词向量 → RNN”之间曾缺失的一步。

19 章我们给计算机装上了第一台猜词发动机：N-gram，靠数数。20 章我们学会了把词变成带语义的高维向量：词向量。

现在，把这两样东西拼起来——**用神经网络读词向量来猜词。**

这个升级看起来只是换了个”读卡器”，但本质上是换了一种完全不同的思路：

N-gram 靠**记忆**，前馈网络靠**理解**。

从”数数”到”学规律”

想象我们想根据前面三个词，预测下一个词。输入是：

我、泡了、一杯

第一步：把词变成向量。假设每个词是一个 300 维的词向量（20 章）。三个词的向量头尾相接，拼成一组 **900 维**的输入数据。

第二步：送进神经网络。900 个数字进入输入层，经过隐藏层的加权求和与非线性变换（08 章讲过这套结构）。

第三步：给词表打分。输出层为词汇表里的每个词输出一个分数——“咖啡”打多少分、“卡布奇诺”打多少分、“果汁”打多少分。再用 **Softmax**（10 章）把这些分数压成概率，选概率最高的作为下一个词。

翻译成人话就是：**前面三个词是证据，网络看完证据后，在十万个候选词里给每个词举一次手，谁举手最热情就选谁。**

["我", "泡了", "一杯"] → 词向量拼成 900 维 → [隐藏层] → [词表打分] → "咖啡"?

这个想法不是凭空冒出来的。2003 年，Yoshua Bengio（后来拿图灵奖的深度学习三巨头之一）团队发表了一篇论文，第一次训练出一个”用词向量+神经网络预测下一个词”的模型。今天回头看，那篇论文的框架和 ChatGPT 是同一个祖先——只不过当年的网络只有一层隐藏层、词表几万个罢了。历史就是这样：同样的图纸，几十年后长成了参天大树。

为什么它比 N-gram 强？

N-gram 只认识”出现过的具体组合”，遇到”泡了一杯卡布奇诺”这种新句子就歇菜——因为对只会数数的它来说，“卡布奇诺”是个陌生的符号。

但神经网络接收的不是孤立符号，而是带语义的词向量。

模型训练时经常见到：

我给自己泡了一杯咖啡。

后来遇到新句子：

我给自己泡了一杯卡布奇诺。

由于”咖啡”和”卡布奇诺”的词向量距离很近（20 章），它们进入神经网络后，会得到相似的计算结果。模型就有机会把在”咖啡”上学到的规律，顺带推广到”卡布奇诺”上。

而且，隐藏层的非线性让神经网络能学到比”数数”复杂得多的规律。它可能自己”悟”出一些模糊的规则：

- “泡了一杯”后面通常是饮料
- “倒了一杯”后面通常是液体
- “切了一个”后面通常是水果

这些规则不需要人逐条写，是模型在训练中自己调整参数学出来的。

用代码看一次最直观。下面这个微型语言模型：输入三个词向量，经过一层隐藏层，输出对词汇表里每个词的打分：

```
import numpy as np

# 迷你词表：每个词一个 4 维向量（20 章讲过，这里手工给，训练中是自己学的）
vocab = {
    "我": np.array([0.9, 0.1, 0.1, 0.2]),
    "泡了": np.array([0.2, 0.9, 0.1, 0.3]),
    "一杯": np.array([0.1, 0.8, 0.3, 0.1]),
    "咖啡": np.array([0.3, 0.6, 0.8, 0.1]),
    "卡布奇诺": np.array([0.3, 0.6, 0.79, 0.11]), # 和"咖啡"几乎一样
    "果汁": np.array([0.7, 0.1, 0.2, 0.9]),
}

# 词表（预测目标）
candidates = ["咖啡", "卡布奇诺", "果汁"]

# 前馈网络：3×4=12 维输入 → 隐藏层(4) → 与候选词向量做内积打分
np.random.seed(3)
W1 = np.random.randn(12, 4) * 0.3
b1 = np.zeros(4)
# 输出层直接复用候选词的词向量当“打分器”（权重绑定）：每一列是一个候选词的向量
W2 = np.stack([vocab[c] for c in candidates]).T # (4, 3)
b2 = np.zeros(3)

def predict(words):
    """给定三个词，预测下一个词的概率分布"""
```

```

x = np.concatenate([vocab[w] for w in words]) # 12 维
h = np.tanh(x @ W1 + b1)
logits = h @ W2 + b2
exp = np.exp(logits - logits.max()) # 数值稳定 Softmax
return exp / exp.sum()

# 见过: 我 泡了 一杯 → 咖啡
p = predict(["我", "泡了", "一杯"])
print("P(咖啡)=", round(p[0], 3), " P(卡布奇诺)=", round(p[1], 3), " P(果汁)=", round(p[2], 3))

# 因为"咖啡"和"卡布奇诺"的向量几乎一样, 网络对它们的打分也会几乎一样
# → 它没"见过"卡布奇诺, 却能把规律迁移过去

```

运行这段代码, 输出大概长这样:

```
P(咖啡)= 0.413   P(卡布奇诺)= 0.408   P(果汁)= 0.18
```

看到重点了吗? 网络从没学过”泡了一杯卡布奇诺”, 但它对”卡布奇诺”给出的概率几乎和”咖啡”一样高。这就是”理解”和”记忆”的区别——N-gram 会卡在没见过的地方, 前馈网络能顺着语义滑过去。

撞墙: 固定窗口

进步很大。但问题马上来了。

这一章的神经网络, 输入层设计成接收三个词向量——那它一次就只能参考前面三个词。想参考前五个词? 重新设计输入层。参考十个? 再改。

无论选三个、五个还是十个, 模型能看到的上下文长度是固定的。

再玩一次猜词游戏, 这次把句子拉长:

小时候每天清晨, 奶奶都会用那台老式咖啡机, 慢慢煮一壶浓香四溢的咖啡, 满屋子都是香气。后来我到大城市工作, 每天早上只能匆匆灌一杯外卖咖啡。可每到周末, 我总会想起奶奶煮的那壶_____。

你一眼能选出”咖啡”, 因为你记得第一句。但这个前馈神经网络只能看最后三个词——”煮、的、那壶”。最前面的”咖啡”早就被挤出窗口了。

对于它来说, ”咖啡”就像从没出现过一样。

翻译成成人话就是: 前馈网络治好了 N-gram 的”不识字”, 却没治好它的”记性差”。它站在一个只能看到眼前三个词的瞭望塔上, 句子再长, 它永远只看得到最近的三块砖。

能不能看任意长的上文?

如果模型能一边读句子、一边把前面出现的重要信息保存下来——句子读多长就记住多长, 而不是被固定窗口卡死——

那该多好?

有的。这就引出了我们下一个主角：**RNN，循环神经网络（22 章）**。它给神经网络加了一份会滚动的”记忆”——隐藏状态。

不过在出发之前，请先记住这一章的坐标：

猜词母题第 2 个解：**词向量 + 前馈神经网络 = 带语义地猜词**，但被”固定窗口”困住。

下一步，RNN 登场——它要砸掉那堵墙。

常见疑问

Q：这一章的网络和 08 章讲的多层神经网络有什么区别？ A：结构上没区别。差别在**在哪**——08 章从结构上讲”网络能做什么”，本章是它在语言上的一次具体应用：读词向量，猜下一个词。这也是后面所有语言模型的基本骨架。

Q：为什么它能泛化到没见过的组合？ A：因为它吃的不是”死符号”而是”活向量”。语义相近的词向量相近，模型把见过的输入和没见过的输入映射到了相近的隐空间，于是”泡咖啡的知识”顺带覆盖了”泡卡布奇诺”。

Q：Bengio 2003 那篇论文到底做了什么？ A：它第一次把”词向量”和”神经网络”焊在一起预测下一个词，证明了神经网络能突破 N-gram 的”无语义”和”数据稀疏”两大死穴。从那时起，语言模型就告别了纯数数，走上了”用参数学规律”的路。

Q：固定窗口问题最后怎么彻底解决的？ A：你会在 **24-自注意力** 看到终极解法——让每个词不依赖固定窗口，而是直接查看句子里的所有词。但那是后话，先从 22-RNN 的”记忆”开始。顺着”猜词母题”，一步一个坎，每道坎就是下一项新技术。

第 22 章 RNN 循环神经网络

难度：★★★★☆

观光路线：理解 RNN 怎么”记住”上下文，知道隐藏状态 h_t 在传递什么信息，看懂 BPTT 为什么又慢又容易崩，跑通唐诗生成代码看看 RNN 的创意。

登山路线：手写 RNN 前向传播的每一步计算，推导 BPTT 中梯度连乘为什么导致爆炸/消失，理解 Truncated BPTT 的截断策略。

阅读顺序提示：RNN 的输入是词向量，建议此前先读 20-Embedding 与表征学习（词如何变成向量）、19-统计语言模型与 N-gram（“猜下一个词”的起点）和 21-前馈神经网络做语言模型（RNN 的直接前身——它正是被”固定窗口”逼出来的）。本章全程以”读一句话，猜下一个词”为主线——还记得那个猜词母题吗，RNN 是它的第三个解法。

21 章我们撞上了一堵墙：前馈网络一次只能看一个固定长度的窗口，三个词也好、十个词也罢，句子一长，最早出现的信息就被挤出视野。要砸掉这堵墙，网络就得学会”边走边记”——这就要用到本章的主角：带记忆的**循环神经网络（RNN）**。

读一本书

你看过那种一口气读完的小说吗？翻到第 157 页的时候，你脑子里装着前面 156 页的所有情节——主角的身世、反派的动机、那个在第 42 页露过一次脸、到 120 页才揭示身份的路人甲。

人类阅读是自带记忆的。每读一页，不是把前面的记忆清空重来，而是更新已有的理解。第 157 页的新内容，跟之前 156 页积累下来的”认知状态”融合在一起，形成新的理解。

CNN 处理图像时没有这个概念。每张图片是独立的，今天的猫和昨天的猫不需要建立时间上的联系。但如果你要处理一段话：“今天天气真好，适合出去_”，空白处填什么？这个答案取决于前面的上下文。一个只能看到”_”的网络屁都猜不出来，但人读到”今天天气真好，适合出去”就已经知道后面大概率是”走走”、“玩”、“跑步”之类的东西了。

循环神经网络（RNN）就是为了解决这类问题而生的。它有一个隐藏状态 h_t ，你可以把它想象成第 t 页读完之后脑子里的”理解摘要”。接下来读第 $t+1$ 页时，网络不仅看第 $t+1$ 页的文字，还会参考 h_t ——你之前读到的一切。

核心公式拆解

RNN 的数学不长，一个公式就写完了：

$$[h_t = (W_{\{xh\}} x_t + W_{\{hh\}} h_{t-1} + b_h)]$$

别看就一行，拆开来看每一块都很有意思。

(x_t) 是当前时刻的输入，比如当前这个字的向量表示。

(h_{t-1}) 是上一时刻的隐藏状态，也就是读前面所有内容积累下来的”记忆”。

$(W_{\{xh\}})$ 是把输入 x_t 投射到隐藏空间的权重矩阵。你可以理解成”当前这一页说了什么”的编码器。

$(W_{\{hh\}})$ 是把上一时刻的隐藏状态 $h_{\{t-1\}}$ 融入当前计算的权重矩阵。你可以理解成”过去的记忆怎么影响现在的理解”。

把 $(W_{\{xh\}} x_t)$ 和 $(W_{\{hh\}} h_{\{t-1\}})$ 加起来，相当于把”新信息”和”旧记忆”混在一起。然后过一个 $\tanh$ 激活函数，把数值压缩到 $[-1, 1]$ 之间，防止状态值在反复循环中炸掉。

用白话再说一遍：**RNN** 每读一个字，就把这个字的意思和之前积累的记忆混在一起，搅拌一下，压到 $[-1, 1]$ 区间，产出更新后的记忆。这个记忆接着传给下一步。

精确地看，这里有两个关键性质。一是循环性——同一个 W_{hh} 在每一步都重复使用，意味着 **RNN** 的参数不随序列长度增长。二是非线性—— $\tanh$ 保证了每一步都有非线性变换，多层叠加后网络能表示复杂的序列模式。

BPTT：俄罗斯套娃式的反向传播

训练 **RNN** 用的是 **BPTT**——**Backpropagation Through Time**，穿越时间的反向传播。

普通全连接网络的反向传播沿着层走：损失→第 N 层→第 $N-1$ 层→……→第 1 层。**RNN** 的反向传播不仅沿着层走，还沿着时间步走：损失→ $t=T$ 时刻→ $t=T-1$ 时刻→……→ $t=1$ 时刻。

想象你手里有一组俄罗斯套娃，一共 100 个。你要把最里面那个最小的娃娃拿出来，得从最大的开始，一个一个打开。**BPTT** 就是这样——要算第 1 个时间步的梯度，得先经过第 100 步、第 99 步、第 98 步……一路传回去。

问题出在这里。梯度每经过一个时间步，都要乘一次 W_{hh} 的转置。如果 W_{hh} 的特征值大于 1，连乘 100 次之后梯度变成了天文数字——梯度爆炸。如果特征值小于 1，连乘 100 次之后梯度约等于 0——梯度消失。

梯度爆炸还好办，直接做一个梯度裁剪（**gradient clipping**），把梯度的模长限制在某个阈值以内，暴力截断。梯度消失就麻烦了——早期时间步的信号完全传不回来，**RNN** 只能记住最近几步的内容，序列一长就抓瞎。

这就是所谓的”长程依赖问题”。一个词出现在句子的开头，到了句尾 **RNN** 已经彻底忘记它了。

工程上的妥协方案叫做 **Truncated BPTT**：不把梯度传回整个序列，只往前追溯固定步数（比如 20 步）。超过 20 步的梯度直接切断，假装那些老历史不存在。这当然不是完美的解法，但在很长一段时间里是唯一能用的。

字符级 **RNN** 生成唐诗

聊了这么多理论，不如让 **RNN** 写首诗。下面这段代码用字符级 **RNN** 在唐诗数据集上训练，然后让它自己创作：

```
import torch
import torch.nn as nn
import numpy as np
```

```

# ---- 数据准备 ----
# 唐诗语料, 实际使用时可以从文件读取更多
poems = """
床前明月光疑是地上霜举头望明月低头思故乡
春眠不觉晓处处闻啼鸟夜来风雨声花落知多少
白日依山尽黄河入海流欲穷千里目更上一层楼
千山鸟飞绝万径人踪灭孤舟蓑笠翁独钓寒江雪
""" * 20 # 重复几次增加数据量

# 构建字符级词表
chars = sorted(list(set(poems)))
char_to_idx = {ch: i for i, ch in enumerate(chars)}
idx_to_char = {i: ch for i, ch in enumerate(chars)}
vocab_size = len(chars)
print(f"词表大小: {vocab_size} 个汉字")

# 把整首诗转成索引序列
data = [char_to_idx[ch] for ch in poems]

# 生成训练样本: 用前 seq_len 个字预测下一个字
seq_len = 30 # 每次看 30 个字
X_data, y_data = [], []
for i in range(0, len(data) - seq_len):
    X_data.append(data[i:i + seq_len])
    y_data.append(data[i + seq_len]) # 第 31 个字

X_data = torch.tensor(X_data) # [样本数, 30]
y_data = torch.tensor(y_data) # [样本数]

# ---- 定义字符级 RNN ----
class CharRNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden_dim=128):
        super().__init__()
        # 字符嵌入层: 把离散字符 ID 映射为稠密向量
        self.embed = nn.Embedding(vocab_size, embed_dim)
        # RNN 层: 输入嵌入向量, 输出隐藏状态
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        # 分类头: 隐藏状态 → 词表大小的概率分布
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: [batch, seq_len]
        x = self.embed(x) # [batch, seq_len, embed_dim]
        out, hidden = self.rnn(x, hidden) # out: [batch, seq_len, hidden]
        # 取最后一个时间步的输出做预测
        out = self.fc(out[:, -1, :]) # [batch, vocab_size]
        return out, hidden

# ---- 训练 ----
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = CharRNN(vocab_size).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.005)

batch_size = 64
n_batches = len(X_data) // batch_size

```

```

for epoch in range(80):
    model.train()
    total_loss = 0
    # 随机打乱后分批训练
    perm = torch.randperm(len(X_data))
    X_shuffled = X_data[perm]
    y_shuffled = y_data[perm]

    for b in range(n_batches):
        start = b * batch_size
        x_batch = X_shuffled[start:start + batch_size].to(device)
        y_batch = y_shuffled[start:start + batch_size].to(device)

        optimizer.zero_grad()
        output, _ = model(x_batch)
        loss = criterion(output, y_batch)
        loss.backward()
        # 梯度裁剪, 防止爆炸
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
        optimizer.step()
        total_loss += loss.item()

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}/80, Loss: {total_loss/n_batches:.4f}")

# ---- 创作诗句 ----
model.eval()
def generate_poem(start_char, length=50, temperature=1.0):
    """
    从 start_char 开始, 逐字预测生成诗句
    temperature 越低越保守, 越高越有创造力
    """
    model.eval()
    # 初始化输入序列
    input_seq = [char_to_idx.get(start_char, 0)]
    hidden = None
    result = [start_char]

    for _ in range(length - 1):
        x = torch.tensor([input_seq[-seq_len:]], device=device)
        with torch.no_grad():
            output, hidden = model(x, hidden)
        # 用 temperature 控制随机性
        probs = torch.softmax(output[0] / temperature, dim=0).cpu().numpy()
        next_idx = np.random.choice(vocab_size, p=probs)
        next_char = idx_to_char[next_idx]
        result.append(next_char)
        input_seq.append(next_idx)

    return ''.join(result)

print("\n--- RNN 的创作 ---")
# 尝试不同 temperature
for temp in [0.5, 0.8, 1.2]:
    poem = generate_poem('春', length=20, temperature=temp)
    print(f"temp={temp}: {poem}")

```

这个 RNN 最核心的组件是 `nn.RNN` 那一行。PyTorch 帮我们封装了前面那个公式里的所有计算—— W_{xh} 、 W_{hh} 、 $\tanh$ ，全藏在里面。我们自己只需要关心三件事：输入怎么编码（Embedding 层）、RNN 输出怎么解读（取最后一个时间步）、结果怎么映射到词表（全连接层）。

`temperature` 参数控制生成文字的“创造力”。`temperature=1.0` 按照模型学到的原始概率分布采样；小于 1.0 会让模型更保守，倾向于选高概率的字，输出更流畅但更无聊；大于 1.0 会拉平概率分布，低概率的字也有机会被选到，输出可能更有惊喜，也可能狗屁不通。

用真正的全唐诗数据集（几万首）训练一个更大的 RNN，它能写出“床前明月光”级别的句子，偶尔还能蹦出几句让人眼前一亮的意象组合。当然，大部分时候它只是在做高级的“文字接龙”——它不知道“江雪”是什么意思，但它知道“江”后面经常跟“雪”。

RNN 记住了序列，但记忆很短。它像是一个只有三秒记忆的金鱼——读完第 100 个字，已经忘了第 1 个字写了啥。BPTT 的俄罗斯套娃结构决定了早期梯度一定消失，这是 RNN 结构性的缺陷，不是调参能解决的。

要造一个真正能“长记性”的网络，得重新设计记忆单元。不靠一个简单的 $\tanh$ 在那硬扛，而是给网络配一个“智慧记事本”——能主动决定记住什么、忘掉什么、什么时候把记忆拿出来用。

人话总结

这章的核心就一句话：RNN 用“隐藏状态”这个设计，让网络在处理序列时有了记忆。每次读入一个新字 x_t ，网络不是从零开始思考，而是把当前输入和上一时刻的记忆 h_{t-1} 揉在一起，更新出一个新的记忆 h_t ，再基于这个记忆做预测。这个递归公式的精妙之处在于，不管序列有多长，网络结构本身不膨胀——参数永远只有 W_{xh} 、 W_{hh} 和 W_{hy} 那三套。但代价也很直观：反向传播时，梯度要顺着时间轴一路往回乘。100 步之前的信息，梯度被连乘了 100 次，要么小到看不见（梯度消失），要么大到炸掉（梯度爆炸）。这就是为什么普通 RNN 处理长序列很吃力。你从唐诗生成的代码里能亲眼看到它的表现——短句还行，长了就开始“忘记”开头在写什么。RNN 教会了我们一件事：处理序列的核心是“把过去的状态压缩成一个向量”——至于怎么让这个压缩更聪明，就是下一章 LSTM 要解决的问题了。

常见疑问

Q: RNN 和人类阅读真的像吗？人类读第 157 页的时候真的“记住”了前面 156 页的所有细节？ A: 这是比喻，不是科学类比。人类阅读时用的是工作记忆+长期记忆的混合系统，细节会被压缩、遗忘、重组——远比 RNN 的一个固定长度向量灵活得多。但 RNN 抓住了核心直觉：理解当前内容需要依赖之前积累的上下文。RNN 的隐藏状态 h_t 相当于你读到第 t 页时能立刻回忆起来的“内容摘要”，至于那些被遗忘的细节（比如第 3 页提到了一个配角的名字），RNN 确实找不回来。这也是为什么后来 Transformer 用注意力机制彻底取代了 RNN——与其把几百步的信息硬塞进一个向量，不如让模型需要时直接“翻回去看”。

Q: Truncated BPTT 截断之后，梯度还能传过截断点吗？ A: 不能。截断的意思就是在某个时间点把计算图切断，梯度不再往前传。这就像把一本长篇小说按章节拆开，每个章节独立做反向传播——网络在“第 3 章”学到的东西不会影响它对“第 1 章”的理解。截断长度是一

个权衡：截太短，学不到长距离模式；截太长，计算慢且梯度不稳定。实践中一般截到几十到几百步，具体看任务——文本分类截 100 步可能够了，语言建模可能需要 200-300 步。

公众号：AI技术圈

第 23 章 LSTM 与 GRU

难度：★★★★☆

观光路线：理解 LSTM 三道门各自干什么活，知道 GRU 怎么把三道门简化成两道，看懂代码里 LSTM 和 GRU 在情感分析任务上的表现差异。

登山路线：手写 LSTM 每个门的计算公式，推导细胞状态 c_t 的更新逻辑，理解门控机制为什么能缓解梯度消失，建立“门控→注意力”的思想链条。

传话游戏为什么一定会崩

小学班会玩过传话游戏吧？老师对第一个人耳语一句话，然后一个接一个往下传。传了 20 个人以后，最后一个人站起来大声说出来——“今天下午三点开班会”变成了“小明说他家的猫会飞”。

RNN 就是这个传话游戏。每一步都要把信息压缩进隐藏状态 h_t ，压缩意味着损失，损失的累积效应在长序列上非常恐怖。而且 RNN 没有“选择性记忆”的能力——不管输入重要还是不重要，它都用同一个 $\tanh$ 公式搅拌一遍。就像传话游戏里每个人必须把听到的话用自己的方式转述，哪怕那句话是“对，知道了”这种废话也得转述。

LSTM 想解决的就是这个问题。它给网络配了一个“智慧记事本”，上面有三个功能区：

遗忘门 (Forget Gate)——擦黑板的板擦。看到新的输入后，LSTM 先决定哪些旧记忆该扔了。“小明说他家的猫”跟当前的话题已经没关系了？擦掉。

输入门 (Input Gate)——记笔记的笔。当前输入里有哪些新东西值得记下来？“老师今天穿红衣服”可能不重要，“下午三点有考试”必须记。输入门负责筛选和写入。

输出门 (Output Gate)——念答案的嘴。现在要输出一个结果了，记事本上那么多笔记，哪些跟当前问题相关？只挑相关的念出来。

这三道门每道门都是一个 sigmoid 激活的小型神经网络，输出 0 到 1 之间的值。0 代表“完全关死，啥也别过”，1 代表“完全敞开，统统通过”。

LSTM 的数学

先把完整的 LSTM 公式列出来，别怕，一行一行拆开看：

[
]

乍一看一堆符号，实际上核心逻辑就三句话。

第一步：遗忘。 f_t 是一组 0 到 1 之间的数，跟旧的细胞状态 C_{t-1} 逐元素相乘。 f_t 中接近 0 的位置，对应的旧记忆就被擦掉了。

第二步：写入。 i_t 决定哪些位置要写入新内容， $C_{\tilde{t}}$ 是当前输入产生的候选记忆。 i_t 乘以 $C_{\tilde{t}}$ 就是“写入的量”。然后跟第一步保留下来的旧记忆加起来——擦完黑板之后，把新笔记抄上去。

第三步：读出。 o_t 是一组门控信号，决定细胞状态 C_t 的哪些部分需要被“念出来”形成隐藏状态 h_t 。这里 C_t 先过了 $\tanh$ （把值压到 -1 到 1 之间），再被 o_t 筛选。

细胞状态 C_t 是 LSTM 最巧妙的设计。注意 C_t 的更新公式里没有 $\tanh$ 套 $\tanh$ —— C_t 的更新是线性加法。（ $C_t = f_t C_{t-1} + i_t \tilde{C}_t$ ），旧记忆乘以遗忘门加新记忆乘以输入门，纯线性操作。这意味着梯度在细胞状态这条高速公路上传播时，不会因为反复过激活函数而衰减。

用更人话的方式说：LSTM 有两个状态在流。 C_t 是“长期记忆”，走高速公路，更新方式是线性加法，梯度不衰减。 h_t 是“短期记忆”，每一步都过 $\tanh$ ，专门负责当前时刻的输出。普通 RNN 只有一个 h_t ，长期和短期搅在一起，谁也保不住。

GRU：精简但同样能打

GRU（Gated Recurrent Unit）是 2014 年由 Cho 等人提出的，比 LSTM 晚出现了十几年，但结构更简洁。它把 LSTM 的三道门合并成两道：

更新门 z_t ：同时承担“遗忘”和“写入”两个职责。 z_t 接近 0 就多保留旧状态，接近 1 就多写入新状态。一个门干了两件事。

重置门 r_t ：控制“在计算候选状态时参考多少旧状态”。 r_t 接近 0 意味着“不参考过去，从头来”。

GRU 没有单独的细胞状态 C_t ，只有一个隐藏状态 h_t 。更新公式是：

[
]

最后一个公式值得仔细看。 h_t 是旧状态和新候选状态的加权平均，权重就是更新门 z_t 。跟 LSTM 的 C_t 更新公式在精神上一脉相承——新状态是旧信息和新信息的线性组合，梯度不会在反复的非线性变换中消失。

GRU 比 LSTM 少了一组参数，训练更快，效果在很多任务上不输 LSTM。该选哪个？没有绝对的答案。序列不太长（几十步），两者差不多。序列很长（上百步），LSTM 的分离式记忆结构更稳。算力紧张，GRU 更经济。

动手做情感分析

说千道万不如跑个实验。下面用 LSTM 和 GRU 各训一个情感分析模型，在同一个数据集上对比：

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
```

```

# ---- 构造简易情感分析数据 ----
# 正面评论 (label=1) 和负面评论 (label=0) 的模拟数据
pos_texts = [
    "这个电影太棒了 演技出色 剧情引人入胜",
    "服务态度很好 环境舒适 菜品美味 强烈推荐",
    "产品质量优秀 功能齐全 价格合理 值得购买",
    "老师讲得很清楚 内容丰富 收获很大 推荐这门课",
    "外观设计精美 手感好 屏幕清晰 续航持久",
] * 200 # 重复扩充数据

neg_texts = [
    "太失望了 质量差 客服态度恶劣 不推荐",
    "电影无聊透了 情节拖沓 演员演技尴尬",
    "服务极差 等了很久 味道也一般 不会再来",
    "产品质量有问题 退货麻烦 体验很不好",
    "课程内容太浅 讲的都是网上有的 浪费钱",
] * 200

# 构建词表 (简易版, 实际项目应用分词工具)
all_texts = pos_texts + neg_texts
word_freq = {}
for text in all_texts:
    for word in text.split():
        word_freq[word] = word_freq.get(word, 0) + 1

# 保留出现次数>=5 的词, 加上<PAD>和<UNK>
vocab = {'<PAD>': 0, '<UNK>': 1}
for word, freq in word_freq.items():
    if freq >= 5:
        vocab[word] = len(vocab)
vocab_size = len(vocab)
print(f"词表大小: {vocab_size}")

# 文本转索引序列, 固定长度为8
def text_to_ids(text, max_len=8):
    ids = [vocab.get(w, 1) for w in text.split()] # 未知词用<UNK>
    if len(ids) < max_len:
        ids += [0] * (max_len - len(ids)) # 填充
    return ids[:max_len] # 截断

X_pos = [text_to_ids(t) for t in pos_texts]
X_neg = [text_to_ids(t) for t in neg_texts]
X = torch.tensor(X_pos + X_neg)
y = torch.tensor([1] * len(pos_texts) + [0] * len(neg_texts))

# 切分训练集和测试集
perm = torch.randperm(len(X))
split = int(0.8 * len(X))
train_idx, test_idx = perm[:split], perm[split:]

train_data = TensorDataset(X[train_idx], y[train_idx])
test_data = TensorDataset(X[test_idx], y[test_idx])
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32)

# ---- LSTM 和GRU 模型 ----
class SentimentRNN(nn.Module):

```

```

"""统一接口: 通过 rnn_type 切换 LSTM/GRU"""
def __init__(self, vocab_size, embed_dim=64, hidden_dim=128,
              rnn_type='lstm'):
    super().__init__()
    self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
    if rnn_type == 'lstm':
        self.rnn = nn.LSTM(embed_dim, hidden_dim,
                           batch_first=True, bidirectional=True)
    else:
        self.rnn = nn.GRU(embed_dim, hidden_dim,
                           batch_first=True, bidirectional=True)
    # 双向所以 hidden_dim*2
    self.fc = nn.Sequential(
        nn.Linear(hidden_dim * 2, 64),
        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(64, 2) # 二分类
    )

def forward(self, x):
    # x: [batch, seq_len]
    embedded = self.embed(x) # [batch, seq_len, embed_dim]
    out, hidden = self.rnn(embedded)
    # 取最后一个时间步的输出
    last_out = out[:, -1, :] # [batch, hidden_dim*2]
    return self.fc(last_out)

# ---- 训练函数 ----
def train_model(model, name, epochs=15):
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.003)

    for epoch in range(epochs):
        model.train()
        for x_batch, y_batch in train_loader:
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            loss = criterion(model(x_batch), y_batch)
            loss.backward()
            optimizer.step()

    # 测试
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for x_batch, y_batch in test_loader:
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            pred = model(x_batch).argmax(dim=1)
            correct += (pred == y_batch).sum().item()
            total += y_batch.size(0)
    acc = 100 * correct / total
    print(f"{name} 测试准确率: {acc:.2f}%")
    return acc

# ---- 对比实验 ----
print("\n=== LSTM vs GRU 情感分析对比 ===\n")
lstm_model = SentimentRNN(vocab_size, rnn_type='lstm')

```

```
gru_model = SentimentRNN(vocab_size, rnn_type='gru')

lstm_acc = train_model(lstm_model, "双向 LSTM")
gru_acc = train_model(gru_model, "双向 GRU")

print(f"\n 差异: {abs(lstm_acc - gru_acc):.2f}%")
```

这个实验里用了双向 RNN——同时从左到右和从右到左读一遍文本，把两个方向的最终状态拼在一起。情感分析里这个技巧特别好用，因为”不推荐”的”不”字出现在”推荐”前面，从左往右读的时候网络读到”推荐”时已经看到了”不”字。双向 RNN 让这个信息流没有死角。

结果通常两者差距不大，在这个数据集上可能都在 95% 以上。LSTM 和 GRU 在实际项目中的选择更多取决于工程考量而非效果——你手上的框架哪个实现更优化、你的 GPU 够不够用、你的序列长度大概多少。

从门控到注意力：思想的延伸

LSTM 和 GRU 最深的贡献不是它们自身，而是它们开创的”门控”思想。

门控的本质是什么？让网络学会自己决定信息流动的比例。遗忘门决定留多少旧信息，输入门决定加多少新信息。不是人工设定一个固定比例，而是让网络从数据里学出”什么时候该忘、什么时候该记”。

这个思想往前走一步就是注意力机制。

门控是”固定公式里的可学习权重”—— W_f 这些参数是训练出来的，但结构是写死的。注意力是”动态计算的信息权重”——不是靠训练好的参数，而是在推理时根据当前输入实时计算”我应该关注输入的哪些部分”。

用类比来感受这个进化。LSTM 的门控像是一个有经验的秘书，多年来总结出了一套判断标准：“老板皱眉的时候说的内容要重点记”、“会议开头和结尾的决策要特别标注”。这套标准是长期训练出来的，面对不同的会议用同一套逻辑。

注意力机制像是一个实时应变的高手。每听到一句话，他会立刻评估这句话跟当前正在讨论的话题有多相关，然后动态决定花多少脑力去理解它。同样是”顺便说一下”开头，如果后面跟着的是关键信息，注意力机制会立刻警觉；如果是真正的废话，它就轻轻放过。

从工程上看，门控机制的参数是固定的（训练完就不变了），注意力的权重是输入相关的（同一句话在不同上下文里有不同的注意力分布）。这就给了模型在推理时更灵活的”信息筛选”能力——也是为什么 Transformer 完全抛弃了 RNN、全面拥抱注意力。

LSTM 和 GRU 把”选择性记忆”的大门推开了一条缝，Transformer 一脚把门踹开了。

人话总结

这章你带走的核心是：LSTM 用三道门解决了 RNN”记不住”的毛病。遗忘门负责擦黑板——看到没用的旧信息就关小闸门；输入门负责记笔记——碰到重要的新信息就放行写入；输出门负责念答案——需要输出时只挑相关的记忆往外吐。这三道门每道都是一个 sigmoid 小

网络，输出 0 到 1 之间的数值，像水龙头的阀门一样控制信息的流量。另外还有一个叫“细胞状态”的东西 (c_t)，它是整个 LSTM 的“长期记忆带”——信息沿着它只做轻量级的加法和乘法，梯度不容易消失。GRU 是 LSTM 的精简版，把遗忘门和输入门合并成了“更新门”，少了一个门但效果差不多。跑情感分析代码你能亲眼看到，LSTM 和 GRU 在中文评论分类上都能干到 95% 以上，差距小到可以忽略。这一章真正的遗产不是 LSTM 本身，而是“门控”这个思想——让网络自己学会什么时候该记住、什么时候该忘掉，这种“选择性”后来被注意力机制发扬光大。

常见疑问

Q: LSTM 和 GRU 到底怎么选？有没有明确的场景界限？ A: 实话讲，两者在大多数任务上差距很小。GRU 参数更少（少了一个门），训练快一点，数据少的时候可能更好（不容易过拟合）。LSTM 参数多一点，表达能力理论上强一丢丢，在超长序列上可能略稳。但实际项目中，你要么用 GRU 图个省事，要么直接用 Transformer。现在除了嵌入式设备等极端资源受限的场景，很少会在这两个之间纠结——因为它们已经被 Transformer 全面取代了。

Q: 细胞状态 c_t 和隐藏状态 h_t 到底有什么区别？为什么 LSTM 要有两个状态？ A: c_t 是“长期记忆”， h_t 是“短期记忆/对外输出”。两者的信息更新路径完全不同： c_t 走的是“轻量级”路径——只经过遗忘门和输入门的简单乘加（element-wise 操作），梯度从这里过不容易衰减； h_t 则要经过 $\tanh$ 和输出门的筛选，是从 c_t 提炼出来的一个更精简的表示。类比一下： c_t 像你的私人日记本，记录完整的信息； h_t 像你在会议上对大家说的话，是从日记本里挑出来的、跟当前讨论话题相关的摘要。

第 24 章 自注意力机制：Q、K、V，你在问自己？

观光路线：理解”自注意力”到底是什么——为什么每个词需要同时关注所有其他词，以及 Q/K/V 为什么都是同一个输入算出来的 **登山路线：**四步拆解注意力计算（点积、缩放、Softmax、加权求和），掌握因果掩码机制，纯 NumPy 手写 Self-Attention

你可能会在这里卡住——我当年就是。

翻开任何一篇 Transformer 教程，前三行就会出现 Q、K、V 三个字母。Query、Key、Value。查询、键、值。然后你会看到三个矩阵乘法： $Q = X * W_Q$, $K = X * W_K$, $V = X * W_V$ 。

然后你就懵了。

Q、K、V 都是同一个输入 X 算出来的，这听起来像自己在问自己问题。自己查自己。自己回答自己。这合理吗？

对，就是自己在问自己。这就是”自”注意力。完全合理。

Q/K/V：三面镜子

翻译成成人话就是：你拿同一张照片，放到三面不同的哈哈镜前面。第一面镜子（ W_Q ）把照片扭曲成”我要找什么”的样子。第二面镜子（ W_K ）把照片扭曲成”我身上有什么标签”的样子。第三面镜子（ W_V ）把照片扭曲成”我实际是什么内容”的样子。

然后你拿着 Query 去和每个 Key 做匹配，看看匹配度有多高。匹配度高的位置，你就多取一些那个位置的 Value 过来。

还是觉得抽象？来，用一句话具体化。

句子：“张三把他的苹果给了李四，李四很高兴。”

分词后：[“张三”，“把”，“他的”，“苹果”，“给”，“了”，“李四”，“，”，“李四”，“很”，“高兴”，“。”]

现在的问题是：“他的”指的是谁？

当模型在处理”他的”这个词的时候，它需要回头去找前文中和”他的”相关的东西。“他的”的 Query 会去和”张三”的 Key 做匹配——匹配度很高。和”苹果”的 Key 做匹配——也有关系但不那么高。和”李四”的 Key 做匹配——低。

然后模型根据这些匹配度，从各个位置的 Value 中按比例提取信息。“张三”的 Value 被提取最多，所以”他的”被理解为指向”张三”。

翻译成成人话就是：你是”他的”这个词，你在脑子里大喊”谁和我有关？”“，然后”张三”回应得最大声，所以你知道你指的是”张三”。

这就是自注意力的一轮运算。**每个词都在问其他所有词：你和我有什么关系？**

W_Q、W_K、W_V：三个可以学习的投影

你可能会问：为什么要用三个不同的 W？为什么不用一套参数？

因为”我要找什么”（Query）和”我有什么”（Key）是两种不同的语义。

举个例子。如果 Query 和 Key 的投影是完全一样的，那”他的”问的问题就和”张三”能被问到的方式一模一样。但事实上，“他的”需要表达”我是一个代词，我在找我的指代对象”，而”张三”需要表达”我是一个具体人名，我可以被代词指代”。这两种语义需要不同的向量空间来承载。

W_Q、W_K、W_V 就是三个可学习的矩阵，让模型自己去发现该怎么投影。

如果用数学表示这个过程，那就是：

```
Q = X * W_Q    # 查询矩阵：我要找什么
K = X * W_K    # 被查矩阵：我有什么标签
V = X * W_V    # 内容矩阵：我实际是什么
```

其中 X 的形状是 (序列长度, d_{model})，W_Q、W_K、W_V 的形状是 (d_{model}, d_k)。d_k 通常等于 d_{model} 除以注意力头数。不过那是多头注意力的事，下一章再说。

四步拆解：自注意力的完整计算

好了，有了 Q、K、V，接下来怎么算？分四步。

第一步：点积求相关

“他的”的 Query 向量是一条线。要判断它和”张三”的 Key 向量有多相关，最自然的想法是算两个向量的夹角。夹角越小，越相关。而点积恰好能衡量这个——两个向量点积越大，说明它们方向越一致。

所以：score(“他的”，“张三”) = Q(“他的”) 点积 K(“张三”)。

对所有位置都做一遍，你就得到一个分数矩阵。这个矩阵的每一行，表示某个词问所有其他词”你和我有多相关？”“得到的答案。

```
# 第一步：点积
scores = torch.matmul(Q, K.transpose(-2, -1)) # (seq_len, seq_len)
```

第二步：缩放

上一步的点积有个问题。如果 d_k（向量维度）很大，点积的值就会变得很大。因为假设每个维度独立、方差为 1，那 d_k 个维度的内积方差就是 d_k。翻译成成人话就是：向量越长，点积天然就会越大。

点积值太大有什么坏处？Softmax 的毛病。Softmax 对大数值极其敏感，如果某个值特别大，经过 Softmax 后它会接近 1，其他所有值接近 0。注意力就变得太”集中”了，只关注一个词。

所以除以 sqrt(d_k)，把方差压回 1。

```
# 第二步: 缩放, 防止点积值太大导致 softmax 梯度消失
scores = scores / math.sqrt(d_k)
```

第三步: Softmax 归一化

把分数变成概率分布。每个词对其他所有词的”关注度”加起来等于 100%。

```
# 第三步: softmax, 得到注意力权重
attn_weights = torch.softmax(scores, dim=-1) # (seq_len, seq_len)
```

第四步: 加权求和

拿着注意力权重, 去把各个位置的 Value 加权求和。关注度高的位置, 它的 Value 被提取得多; 关注度低的, 几乎不被提取。

```
# 第四步: 加权求和, 得到每个位置的新表示
output = torch.matmul(attn_weights, V) # (seq_len, d_v)
```

四步合起来, 就是整个自注意力的计算公式:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

翻译成成人话就是: 每个位置的输出 = 它对所有位置的注意力权重 (归一化后的匹配分数) 乘以每个位置的的实际内容, 最后求和。

因果掩码: 不能偷看未来

在上面的计算中, 每个词可以看到整个序列的所有其他词——包括它右边的、还没生成的词。这对 Encoder 没问题——Encoder 本来就是看整句话。

但 Decoder 不行。Decoder 在生成第 5 个词的时候, 第 6 个词还没被生成出来呢。如果它看到了第 6 个词的信息, 那就是作弊。

所以 Decoder 的自注意力需要**因果掩码** (Causal Mask)。翻译成成人话就是: 在注意力分数矩阵上盖一张”帘子”, 遮住所有未来位置。

具体做法: 注意力分数矩阵是一个方阵, 第 i 行第 j 列是第 i 个位置对第 j 个位置的关注度。对于 Decoder, 第 i 行在第 j 列 ($j > i$) 的分数要被设成负无穷。这样经过 Softmax 之后, 这些位置的权重就变成 0 了。

```
# 构建因果掩码: 一个上三角矩阵, 未来位置为 True
mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1).bool()
# 把未来位置的分数设为负无穷
scores = scores.masked_fill(mask, float('-inf'))
```

手写 Self-Attention

不依赖任何深度学习框架。用纯粹的 NumPy, 从头计算一次自注意力。你会看到这四步到底在做什么。

```
import numpy as np
```

```

def self_attention_numpy(X, d_k=64, use_causal_mask=False):
    """
    纯 NumPy 实现自注意力机制

    参数:
        X: 输入序列, 形状 (seq_len, d_model)
        d_k: 查询/键的维度
        use_causal_mask: 是否使用因果掩码 (Decoder 用 True, Encoder 用 False)

    返回:
        output: 注意力输出, 形状 (seq_len, d_model)
        attn_weights: 注意力权重矩阵, 形状 (seq_len, seq_len)
    """
    seq_len, d_model = X.shape

    # 初始化三个投影矩阵 (实际中这些是训练出来的, 这里随机初始化做演示)
    rng = np.random.default_rng(42) # 固定随机种子, 保证可复现
    W_Q = rng.normal(0, 1/np.sqrt(d_model), (d_model, d_k)) # 查询投影
    W_K = rng.normal(0, 1/np.sqrt(d_model), (d_model, d_k)) # 键投影
    W_V = rng.normal(0, 1/np.sqrt(d_model), (d_model, d_k)) # 值投影
    W_O = rng.normal(0, 1/np.sqrt(d_model), (d_k, d_model)) # 输出投影

    # 第一步: 投影, 得到 Q、K、V
    Q = X @ W_Q # (seq_len, d_k)
    K = X @ W_K # (seq_len, d_k)
    V = X @ W_V # (seq_len, d_k)

    # 第二步: 计算注意力分数 = Q * K^T
    scores = Q @ K.T # (seq_len, seq_len)

    # 第三步: 缩放
    scores = scores / np.sqrt(d_k)

    # 第四步: 因果掩码 (如果需要)
    if use_causal_mask:
        # 创建上三角掩码, 主对角线以上全部设为 True
        causal_mask = np.triu(np.ones((seq_len, seq_len)), k=1).astype(bool)
        scores = np.where(causal_mask, -np.inf, scores) # 未来位置设为负无穷

    # 第五步: Softmax (沿行方向)
    # 减去每行的最大值防止数值溢出, 这是 softmax 的标准技巧
    scores_max = np.max(scores, axis=-1, keepdims=True)
    scores_exp = np.exp(scores - scores_max)
    attn_weights = scores_exp / np.sum(scores_exp, axis=-1, keepdims=True)

    # 第六步: 加权求和
    output = attn_weights @ V # (seq_len, d_k)

    # 第七步: 输出投影
    output = output @ W_O # (seq_len, d_model)

    return output, attn_weights

# ===== 演示 =====
if __name__ == "__main__":
    # 模拟一个 6 词句子, 每个词用 8 维向量表示

```

```

seq_len, d_model = 6, 8
X = np.random.randn(seq_len, d_model)

print("=== Encoder 自注意力（能看到全部位置） ===")
enc_out, enc_weights = self_attention_numpy(X, d_k=4, use_causal_mask=False)
print(f"输出形状: {enc_out.shape}")
print(f"注意力权重矩阵（每行和=1）:\n{np.round(enc_weights, 2)}")

print("\n=== Decoder 自注意力（不能看未来） ===")
dec_out, dec_weights = self_attention_numpy(X, d_k=4, use_causal_mask=True)
print(f"输出形状: {dec_out.shape}")
print(f"因果掩码下的注意力（右上三角全为 0）:\n{np.round(dec_weights, 2)}")

# 验证：因果掩码下第 i 行只看前 i 列
print("\n=== 验证因果掩码 ===")
for i in range(seq_len):
    # 对于第 i 行，第 i+1 列及之后应该全是 0
    future_cols = dec_weights[i, i+1:]
    assert np.allclose(future_cols, 0), f"第{i}行能偷看未来！"
print("验证通过：没有位置能偷看未来")

```

运行这段代码，你会看到 Encoder 的注意力权重矩阵每一行都分布着非零值——每个词都关注了所有其他词。而 Decoder 的矩阵是下三角的——右上角的三角形全是 0，每个位置只看到自己和前面。

那个 Softmax 实现里有个细节：scores - scores_max。为什么要减最大值？因为 e 的指数很容易爆。假设 scores 里有 1000 这么大的值， e^{1000} 直接溢出成 inf。减去最大值之后，最大的那个变成 0， $e^0=1$ ，所有值都在安全的数值范围里。数值稳定性，实用到骨子里的小技巧。

自注意力的核心洞察

回到那个让你卡住的问题：Q、K、V 为什么都是同一个输入 X 算出来的？

因为这恰恰是”自”注意力的关键。

如果 Q 来自一个地方、K 和 V 来自另一个地方，那这不是自注意力，是交叉注意力——第 27 章讲 Transformer 的 Encoder-Decoder 架构时你会看到，Encoder-Decoder Attention 就是交叉注意力：Q 来自 Decoder 当前层，K 和 V 来自 Encoder 的输出。

自注意力的”自”，意思就是：句子中的每个词，都在用自己想问的问题，去查句子本身蕴含的答案。

这是一种极度优雅的对称性。输入矩阵 X 经过三个不同的线性投影，变成了三个不同语义空间中的表示，然后在这三个空间里完成信息的重新组合。模型学到了什么？它学的是”哪种词之间的组合关系更有意义”。

不过你可能注意到了——我们上面的代码里，每个注意力层只用了一组 Q/K/V 投影。而真正的 Transformer 用了很多组，同时从不同角度去”看”句子。一组注意力就像一个人在一个角度看问题。多组注意力就是八个角度。这就是下一章的内容。

人话总结

这章说到底就一件事——自注意力的核心就是”每个词问所有其他词：你跟我有什么关系？“Q、K、V 不是三个独立的东西，而是同一份输入经过三面不同哈哈镜的投影。你带走的关键直觉是那四步计算：点积求相关、缩放防爆炸、Softmax 归一化、加权求和出结果。缩放除以 $\sqrt{d_k}$ 这个操作看起来不起眼，但没有它，注意力就会退化成一选一的”独裁”——模型只看一个词，其他全忽略。因果掩码也是同一个道理：Decoder 不能偷看未来，所以拿一个上三角的”帘子”把后面全盖住。

常见疑问

Q: Q、K、V 既然是同一个输入 X 算出来的，那”自”到底是什么意思？

A: 关键就在”自”字上。如果是翻译任务，Q 来自 Decoder、K 和 V 来自 Encoder——这叫交叉注意力（Cross-Attention），是两拨信息之间的交互。自注意力是句子内部的信息重组：一句话里的每个词，用自己的 Query 去匹配句子内其他词的 Key，然后按匹配度从对应的 Value 里取信息。说白了就是”一句话里的词互相看，自己搞清楚彼此的关系”。

Q: Softmax 之前除以 $\sqrt{d_k}$ ，不除会怎样？

A: 不除的话，当 d_k 比较大（比如 64 或 128），点积值会天然偏大。太大的值经过 Softmax 后，会变成接近”one-hot”的极端分布——某个位置的权重接近 1，其他都接近 0。这意味着注意力机制退化成了”只看一个词”，失去了”综合考虑”的能力。除以 $\sqrt{d_k}$ 本质上是把方差压回 1，让注意力分布更平滑、更合理。

第 25 章 多头注意力：八个角度看同一句话

观光路线：理解为什么一个注意力头不够——八双眼睛同时看，有的看语法、有的看指代、有的看搭配，合在一起才是完整的理解 **登山路线：**掌握 `d_model` 切分为 `h` 个头的计算机制，高效并行实现（`reshape+transpose` 技巧），完整 `MultiHeadAttention` 代码，注意力头可视化分析

上一章我们讲了一个注意力头是怎么算的。Q、K、V 三组投影，点积，缩放，Softmax，加权求和。干净利落。

但真实的 Transformer 不用一个头，用八个。或者十六个。GPT-3 用 96 个。为什么要这么多？

你用一只眼睛看世界也是看。但闭上左眼只留右眼，你对深度的判断会差很多。左眼看的角度和右眼略有不同，两个视角的信息叠加在一起，你才能感知到立体。

八个注意力头，类似于八双眼睛。它们站在不同的角度去理解同一个句子。有的头专门关注“主语和谓语之间的语法关系”。有的头专门关注“指代关系，比如‘它’指的是谁”。有的头专门关注“前后词的位置搭配”。有的头关注的东西你可能都说不清楚，但它确实捕捉到了一种有用的模式。

翻译成成人话就是：一个头可能有偏见，八个头凑在一起，取长补短，理解更全面。

`d_model` 切分为 `h` 个头

多头注意力怎么实现？最容易想到的办法是：把单头注意力的代码复制八遍，然后分别跑，最后把八个输出拼起来。

可以。但这样你就需要八个完全独立的 Q/K/V 投影，八个独立的注意力计算。参数量是原来的八倍。而且不能并行计算——每组的矩阵形状都不一样，GPU 调度会有浪费。

更好的办法：把 `d_model` 切成 `h` 份，每一份独立做注意力，然后把结果拼回去。

具体来说。假设 `d_model = 512`，`h = 8` 个头。那每个头的维度就是 `d_k = d_model / h = 64`。你不需要给每个头分别做 `512→64` 的投影，而是做一个 `512→512` 的投影，然后把 512 维切成 8 段，每段 64 维。

```
# 不是这样做（8 个独立的小投影）
```

```
Q1 = X @ W_Q1 # (seq_len, 64)
```

```
Q2 = X @ W_Q2 # (seq_len, 64)
```

```
# ... 重复 8 次
```

```
# 而是这样做（一个大投影，然后切成 8 段）
```

```
Q = X @ W_Q # (seq_len, 512)，一个大投影
```

```
Q_heads = Q.reshape(seq_len, 8, 64) # 切成 8 个头，每头 64 维
```

翻译成成人话就是：八个头不是各招各的员工，而是让整个团队一起做一场大汇报，然后每人领一段材料回去消化。

reshape + transpose: 并行的艺术

上面的 reshape 思路是对的, 但还有一个小问题。矩阵乘法喜欢什么? 喜欢” batch, heads, seq, d_k” 这种形状——batch 和 heads 在最前面, 方便并行。

实际的多头注意力实现中, reshape 之后还会做一个 transpose, 把 heads 维度移到前面:

```
# 输入: Q 形状为 (batch, seq_len, d_model)
# 想要: (batch, num_heads, seq_len, d_k)

Q = Q.reshape(batch, seq_len, num_heads, d_k) # (batch, seq, heads, d_k)
Q = Q.transpose(1, 2)                        # (batch, heads, seq, d_k)
```

这个 transpose 操作看起来只是换了一下维度的顺序, 但它让后续的矩阵乘法变得极其高效。你可以一次性计算 batch 行、heads 行的点积。

翻译成成人话就是: reshape 是把蛋糕切成 8 块, transpose 是把蛋糕摆成方便大家一起吃的格局。

完整 MultiHeadAttention 代码

下面这段代码是生产级别的多头注意力实现。每个细节都有它的道理。

```
import torch
import torch.nn as nn
import math

class MultiHeadAttention(nn.Module):
    """
    多头注意力: 将 Q/K/V 投影后切分为多个头, 每个头独立计算注意力,
    最后将所有头的输出拼接起来再做一次线性变换。
    """
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        assert d_model % num_heads == 0, "d_model 必须能被 num_heads 整除"

        self.d_model = d_model      # 模型总维度, 比如 512
        self.num_heads = num_heads  # 注意力头数, 比如 8
        self.d_k = d_model // num_heads # 每个头的维度, 512/8=64

        # 三个投影矩阵: Q、K、V 的投影, 输出维度都是 d_model
        self.W_Q = nn.Linear(d_model, d_model, bias=False)
        self.W_K = nn.Linear(d_model, d_model, bias=False)
        self.W_V = nn.Linear(d_model, d_model, bias=False)

        # 输出投影: 把拼接后的多头结果映射回 d_model
        self.W_O = nn.Linear(d_model, d_model, bias=False)

        self.dropout = nn.Dropout(dropout)
        self.scale = math.sqrt(self.d_k) # 缩放因子, 提前算好

    def forward(self, Q, K, V, mask=None):
        """
        Q, K, V: 形状均为 (batch, seq_len, d_model)
        mask: 可选, (batch, 1, seq_len, seq_len) 或可广播的形状
        """
```

```

"""
batch_size, seq_len, _ = Q.shape

# ===== 第一步: 投影 =====
Q = self.W_Q(Q) # (batch, seq_len, d_model)
K = self.W_K(K) # (batch, seq_len, d_model)
V = self.W_V(V) # (batch, seq_len, d_model)

# ===== 第二步: 切分为多头 =====
# 先reshape, 把d_model 切成(num_heads, d_k)
# 再transpose, 把heads 维度移到batch 后面, 方便并行计算
Q = Q.reshape(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
K = K.reshape(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
V = V.reshape(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
# 现在Q/K/V 形状都是: (batch, num_heads, seq_len, d_k)

# ===== 第三步: 计算注意力分数 =====
# Q @ K^T: (batch, num_heads, seq_len, seq_len)
scores = torch.matmul(Q, K.transpose(-2, -1)) / self.scale

# ===== 第四步: 应用掩码 (如果有) =====
if mask is not None:
    # mask 中为0 的位置设为负无穷, softmax 后变成0
    scores = scores.masked_fill(mask == 0, float('-inf'))

# ===== 第五步: Softmax 得到注意力权重 =====
attn_weights = torch.softmax(scores, dim=-1)
attn_weights = self.dropout(attn_weights) # 随机丢弃一些注意力连接, 防止过拟合

# ===== 第六步: 加权求和 =====
# (batch, num_heads, seq_len, d_k)
output = torch.matmul(attn_weights, V)

# ===== 第七步: 拼回头并投影 =====
# 先把heads 维度移回原位, 再reshape 合并
output = output.transpose(1, 2).contiguous() # (batch, seq_len, num_heads, d_k)
output = output.reshape(batch_size, seq_len, self.d_model) # (batch, seq_len,
d_model)

# 最后过一个线性层, 融合8 个头的信息
output = self.W_O(output)

return output, attn_weights

# ===== 演示: 分析注意力头的行为 =====
if __name__ == "__main__":
    # 构建一个8 头注意力层
    d_model = 512
    num_heads = 8
    mha = MultiHeadAttention(d_model, num_heads)

    # 模拟输入: batch=1, 10 个词, 每个512 维
    batch, seq_len = 1, 10
    X = torch.randn(batch, seq_len, d_model)

    # 自注意力: Q=K=V=X

```

```

output, attn_weights = mha(X, X, X)

print(f"输入形状: {X.shape}")
print(f"输出形状: {output.shape}") # (1, 10, 512)
print(f"注意力权重形状: {attn_weights.shape}") # (1, 8, 10, 10)

# 查看第 0 个头的注意力分布 (第 5 个词对前面所有词的关注)
head_0_attn = attn_weights[0, 0, 4, :] # 第 5 个词 (索引 4)
print(f"\n 头 0 对第 5 个词的关注分布:")
for i, w in enumerate(head_0_attn):
    bar = "█" * int(w.item() * 50)
    print(f" 词{i}: {bar} ({w.item():.3f})")

# 对比: 不同头对同一个词的关注可能非常不同
print(f"\n 不同头对'词 5->词 2'的关注度对比:")
for h in range(num_heads):
    w = attn_weights[0, h, 4, 2].item()
    bar = "█" * int(w * 100)
    print(f" 头{h}: {bar} ({w:.4f})")

```

注意那个 `contiguous()` 调用。`transpose` 只是改变了张量的“视角”，并没有改变内存布局。后面接 `reshape` 的时候如果内存不连续会报错。`contiguous()` 就是让内存变得连续。刚开始学 PyTorch 的时候，这个 bug 能坑你半小时——看起来完全一样的张量，`reshape` 就报错。

头注意力可视化：它们真的在看不同东西

八个头到底在看什么？我们虽然不能直接读懂注意力矩阵，但可以做一些分析。

论文“Attention is All You Need”里有一张经典的图，展示了不同注意力头的关注模式。有些头呈现“对角线”模式——每个词主要关注它自己和附近的词，像在捕捉局部短语结构。有些头呈现“纵向”模式——所有词都在关注某几个特定位置（通常是句首或标点），像在捕捉全局信息。还有些头呈现“前指”模式——动词关注主语，代词关注指代对象。

这八种模式不是我们设计的。是模型自己学会的。你给它定义了八组投影矩阵，它自己找到了八种不同的分工方式。

翻译成成人话就是：我们只是给了模型八个“部门”，每个部门怎么分工，我们完全不管。训练过程中，它们自己慢慢分化出了各自的职能。

这个发现其实很震撼。它说明注意力机制有一种天然的“分工倾向”——不需要显式地告诉它“头 0 你负责语法，头 1 你负责指代”。给它足够的数据和时间，它自己会演化出最有效的分工方式。

为什么不是越多越好

你可能会想：既然 8 个头比 1 个头好，那 800 个头不是更好？

不会。原因有两个。

第一，`d_model` 是固定的。更多的头意味着每个头更小。比如 `d_model=512`，如果 64 个头，每个头只有 8 维。8 维的向量能表达的信息很有限，就像你让 64 个人挤在一间 512 平方米的房间——每人只有 8 平方米，转身都困难。

第二，有些头会“死”掉。研究发现，训练到后面，有些注意力头会退化——它们的输出变成常数，不再提供有用信息。头的数量越多，死的比例越高。GPT-3 有 96 个头，但实际起作用的可能只有六七十个。

所以头数的选择是一个工程权衡。8 个头是原始论文的选择，后来 GPT-3 用了 96 个，DeepSeek 用了 128 个。越大越有机会捕捉更细粒度的模式，但边际收益递减。

注意力权重的熵：衡量头是否“专注”

有一个有趣的观察角度：用信息熵来衡量每个头的“专注度”。

如果一个头的注意力完全平均分配（每个位置权重都是 $1/\text{seq_len}$ ），它的熵最大——说明这个头啥也没关注到。如果一个头的注意力集中在少数几个位置，它的熵较小——说明这个头找到了它关注的东西。

计算注意力头的熵，衡量专注度

```
import torch
```

```
def attention_entropy(attn_weights):
```

```
    """attn_weights: (batch, num_heads, seq_len, seq_len)"""
```

```
    eps = 1e-9 # 防止 log(0)
```

```
    entropy = -(attn_weights * torch.log(attn_weights + eps)).sum(dim=-1)
```

```
    return entropy.mean(dim=(0, 2)) # 每个头的平均熵
```

使用上一段代码中的 attn_weights

```
head_entropies = attention_entropy(attn_weights)
```

```
print("每个注意力头的平均熵（越低越专注）:")
```

```
for h in range(num_heads):
```

```
    print(f"头{h}: {head_entropies[h].item():.3f}")
```

这个方法常被用来在训练后剪枝——把熵特别高的头（几乎不专注的）直接砍掉，模型质量几乎不下降，推理速度还能提升。

多头注意力让模型能同时从多个角度理解句子。但你有没有注意到一个问题——到目前为止，我们的计算完全不知道词的顺序。把“张三打了李四”和“李四打了张三”塞进注意力层，它看到的是一模一样的一组词。这显然不行。

下一章，我们给模型装上“顺序的感知”——位置编码。

人话总结

这章说到底就一件事——一个注意力头可能有偏见，八个头凑在一起，取长补短，理解才全面。你带走的直觉是：多头不是复制粘贴八份代码，而是把 `d_model` 切成八段，每段独立做注意力，最后拼回来——像一个大团队一起做汇报，每人领一段材料回去消化。更妙的是，我们不需要告诉每个头“你负责语法、你负责指代”，模型自己会在训练中演化出分工。那个 `contiguous()` 的小坑和注意力头会“死掉”的现象，只有动手写过代码的人才能体会。

常见疑问

Q: 多头注意力的”头”越多越好吗? GPT-3 用 96 个头是不是比 8 个头强 12 倍?

A: 不是线性关系。头的数量受两个约束: `d_model` 固定时, 更多头意味着每个头更小 (维度更低), 表达能力反而下降; 此外, 研究发现很多头训练到后期会”退化”——输出变成常数, 不再提供有效信息。头的数量本质上是给模型提供”不同视角的数量”, 8 个够用, 96 个提供的边际收益远没有数字看起来那么大。

Q: `reshape + transpose` 那一步到底在干什么? 为什么要这么折腾?

A: 本质上是在调整张量的排列方式, 方便 GPU 并行计算。`reshape` 把 `d_model` 切成”`num_heads` 块”, `transpose` 把 `heads` 维度挪到 `batch` 后面, 让后续的矩阵乘法一次性处理所有 `batch` 和所有 `head`。如果不做这个变换, 每个头要单独算一遍, GPU 调度开销会大很多。你可以理解成: 先切蛋糕 (`reshape`), 再摆桌子 (`transpose`), 然后大家一起吃 (并行矩阵乘法)。

第 26 章 位置编码：给没有座位号的自习室编号

观光路线：理解为什么注意力机制天然“看不见顺序”，以及三种给模型装上“位置感”的方法 **登山路线：**正弦编码的数学原理与不同频率维度可视化，RoPE 旋转位置编码推导，三种方案对比表

想象一个没有座位号的自习室。所有人都可以随便坐，今天坐第一排，明天坐最后一排。但你手里的课程表是按座位号安排的——“第 3 排第 5 座的同学去上课”。教室管理员崩溃了。

Transformer 的自注意力机制就是这个自习室。它不管词在哪，只管词和词之间的关系。“张三”在第一个位置还是在最后一个位置，注意力机制看到的内容是完全一样的。因为注意力的计算里没有任何关于位置的信息。

翻译成成人话就是：注意力是个“脸盲”，它认得出谁和谁关系好，但它完全看不见谁坐在哪。词序对它来说不存在。

这显然是灾难性的。“狗咬人”和“人咬狗”在注意力机制眼里一模一样——反正都是“狗”、“咬”、“人”三个词互相关注。

所以你得给每个位置发一个“座位号”，告诉模型这是第几个位置。这个座位号，就是**位置编码**（Positional Encoding）。

正弦编码：用波刻画位置

原始 Transformer 论文用的是正弦位置编码。这个名字听起来很三角函数，但翻译成成人话很简单：**给每个位置的每个维度分配一个独一无二的波浪值。**

具体做法是，对于位置 pos 的第 i 个维度，给它赋一个正弦波或余弦波的值：

```
PE(pos, 2i)      = sin(pos / 10000^(2i/d_model))    # 偶数维度用 sin
PE(pos, 2i + 1)  = cos(pos / 10000^(2i/d_model))    # 奇数维度用 cos
```

翻译成成人话就是：把每个位置编码成一个“音叉”。不同维度用不同频率的波。低维度用高频波——位置稍微变一点，值变化剧烈，所以能区分相邻位置。高维度用低频波——位置变很多值才变一点，所以能覆盖远距离的位置关系。

为什么用正弦波？因为这波有一个极好的性质：**相对位置可以用波之间的相位差来表达。**位置 $pos+k$ 的编码和位置 pos 的编码之间，存在一个固定的线性变换关系。这意味着模型可以自然地学到“两个位置之间的距离”这个概念，而不需要显式地记住每个位置的绝对坐标。

```
import numpy as np
import matplotlib.pyplot as plt

def sinusoidal_position_encoding(max_len, d_model):
    """
    生成正弦位置编码矩阵

    参数:
        max_len: 最大序列长度
        d_model: 嵌入维度（必须是偶数）
```

```

"""
pe = np.zeros((max_len, d_model))
position = np.arange(max_len)[: , np.newaxis] # (max_len, 1)

# 计算每个维度的频率：指数衰减
div_term = np.exp(
    np.arange(0, d_model, 2) * (-np.log(10000.0) / d_model)
)

# 偶数维用 sin, 奇数维用 cos
pe[:, 0::2] = np.sin(position * div_term) # (max_len, d_model/2)
pe[:, 1::2] = np.cos(position * div_term) # (max_len, d_model/2)

return pe

# ===== 可视化：不同频率的波形 =====
d_model = 128
max_len = 100
pe = sinusoidal_position_encoding(max_len, d_model)

# 画几个代表性维度的波形
dimensions_to_plot = [0, 4, 16, 32, 64] # 从高频到低频
plt.figure(figsize=(12, 6))
for dim in dimensions_to_plot:
    plt.plot(pe[:, dim], label=f'维度{dim}')
plt.xlabel('位置')
plt.ylabel('编码值')
plt.title('正弦位置编码：不同维度对应不同频率的波')
plt.legend()
plt.show()

# 画整个编码矩阵的热力图
plt.figure(figsize=(10, 8))
plt.imshow(pe.T, aspect='auto', cmap='RdBu')
plt.colorbar(label='编码值')
plt.xlabel('位置')
plt.ylabel('维度')
plt.title('正弦位置编码矩阵（行=位置，列=维度）')
plt.show()

```

从热力图中你能看到一个清晰的模式：下半部分（高频）像快速振荡的条纹，适合区分相邻位置。上半部分（低频）像渐变的色块，提供大范围的“在第几段”的信息。

可学习的位置编码

正弦编码很优雅，但它是固定的——不管你训练什么任务，编码都一样。可学习的编码就是简单粗暴：把位置当成一种要训练的参数，让模型自己去学怎么表示位置。

实现方式极其直接——在模型里加一个 `nn.Embedding(max_len, d_model)`，把位置编号作为输入。

```

# 可学习位置编码：就是一个 Embedding 层
self.pos_embedding = nn.Embedding(max_len, d_model)

# 使用时

```

```
pos_ids = torch.arange(seq_len) # [0, 1, 2, ..., seq_len-1]
pos_enc = self.pos_embedding(pos_ids) # (seq_len, d_model)
```

好处是灵活——模型想怎么表示位置都可以。坏处是它只能处理训练时见过的位置长度。训练时最大长度 512，推理时来了一个 1024 长度的句子，位置 1023 在 Embedding 里不存在，就炸了。虽然可以通过插值来扩展，但效果会打折扣。

BERT 用的是这种方案。

RoPE：2024 年后的主流

2021 年，一篇叫“RoFormer”的论文提出了**旋转位置编码**（Rotary Position Embedding，简称 RoPE）。这个名字听起来很几何，但翻译成成人话就是：**通过旋转向量的角度来嵌入位置信息**。

想象你给每个维度的向量都赋予了“转动”的能力。位置 1 转一度，位置 2 转两度，位置 100 转一百度。两个位置的向量之间的旋转角度差，就编码了它们之间的相对位置。

为什么 RoPE 这么好？因为它把位置信息“注入”到了注意力计算的 Q 和 K 里面，而不是“加”在输入上。正弦编码和可学习编码都是在输入阶段把位置信息附加上去。RoPE 是在计算注意力分数的时候，直接把位置关系编码进 Q 和 K 的乘积里。

数学上：RoPE 用一个旋转矩阵 $R(\text{pos})$ 去乘查询向量 q 和键向量 k 。两个位置之间的内积变成了：

$$(R(\text{pos}_i) * q_i) \cdot (R(\text{pos}_j) * k_j) = q_i \cdot (R(\text{pos}_j - \text{pos}_i) * k_j)$$

翻译成成人话就是：两个词的注意力分数，天然包含了它们之间的距离。这个性质太美了。

```
import torch
```

```
class RotaryPositionalEmbedding(nn.Module):
    """RoPE 旋转位置编码的简化实现"""
    def __init__(self, d_model, max_len=2048):
        super().__init__()
        self.d_model = d_model

        # 计算旋转频率：和正弦编码一样用指数衰减
        freq = 1.0 / (10000 ** (torch.arange(0, d_model, 2).float() / d_model))
        self.register_buffer('freq', freq) # (d_model/2,)

    def forward(self, x, seq_len):
        """
        x: (batch, seq_len, d_model), 用来获取设备信息
        返回旋转后的 cos 和 sin
        """
        device = x.device
        # 生成位置序列
        pos = torch.arange(seq_len, device=device).float()
        # 每个位置 * 每个频率 -> 旋转角度矩阵
        angles = torch.outer(pos, self.freq) # (seq_len, d_model/2)

        cos = angles.cos().unsqueeze(0).unsqueeze(0) # (1, 1, seq_len, d_model/2)
        sin = angles.sin().unsqueeze(0).unsqueeze(0)
```

```

    return cos, sin

def apply_rotary_embedding(x, cos, sin):
    """对 Q 或 K 应用旋转位置编码"""
    # 把 d_model 分成两半，一半用 cos，一半用 sin
    x_even = x[..., 0::2] # 偶数维
    x_odd = x[..., 1::2] # 奇数维

    # 旋转: (x_even + i*x_odd) * (cos + i*sin)
    x_rotated_even = x_even * cos - x_odd * sin
    x_rotated_odd = x_even * sin + x_odd * cos

    # 拼回去
    x_rotated = torch.stack([x_rotated_even, x_rotated_odd], dim=-1)
    x_rotated = x_rotated.flatten(-2) # 交错还原为 d_model

    return x_rotated
```

RoPE 现在几乎是所有大模型的标准配置。LLaMA 用它。GPT-NeoX 用它。Qwen 用它。DeepSeek 用它。为什么？因为它优雅地解决了相对位置的学习问题，对训练时没见过的长度也能外推——通过调整旋转频率，可以支持比训练时更长的上下文窗口。

三种方案对比

特性	正弦编码	可学习编码	RoPE
训练参数	无（固定计算）	有（ <code>max_len * d_model</code> 个参数）	无（固定计算）
相对位置感知	间接（通过相位差）	需要模型自己学	直接（内积天然含距离）
外推能力	理论上可以	需要插值	可以（调整频率）
代表模型	原始 Transformer	BERT	LLaMA, Qwen, DeepSeek
数学优雅度	★★★	★	★★★★★

几个有意思的细节。可学习编码相当于给每个位置分配了一个独特的身份 ID。位置 5 永远是一个固定的向量，位置 6 永远是另一个。它通过让相邻位置的向量在训练中变得相似来捕捉”近”的概念。

正弦编码则不需要训练，它依靠不同频率波的组合来给每个位置生成一个唯一的”指纹”。这个指纹有数学保障的局部相似性——位置 5 和位置 6 的编码天然比位置 5 和位置 500 的更相似。

RoPE 更进一步，它不把位置信息当成”附加物”，而是直接修改了 Attention 的内积计算过程。位置关系变成了向量旋转的角度差。

位置编码在 Transformer 里的角色

不管哪种方案，位置编码的作用方式是一样的：在输入进入第一个注意力层之前，把位置编码加到词嵌入上（或者注入到 Q/K 中）。

对于正弦编码和可学习编码：

输入表示 = 词嵌入 + 位置编码

对于 RoPE：

修改后的注意力分数 = $(R(\text{pos}_i) * Q_i) \cdot (R(\text{pos}_j) * K_j) / \text{sqrt}(d_k)$

一旦位置信息加进去了，它在每一层的注意力中被反复”消化”。深层的注意力不再直接看到原始的位置编码，而是看到经过多层变换后的、和语义纠缠在一起的位置信息。

翻译成成人话就是：位置编码只在一开始给一个”起点”，后面每一层都在这个起点上继续加工，让位置和语义深度融合。

到目前为止，我们一直在说”词”、“词嵌入”、“d_model”——但模型到底是怎么把中文文本变成这些数字向量的？“苹果”怎么变成一个 512 维的浮点数数组？这个过程**不是简单的查字典**，那是第 28 章分词器要讲的。不过先别急——下一章，我们先把前面拆开的零件组装起来，看看 Transformer 这张完整的图纸长什么样。

人话总结

这章说到底就一件事——注意力是个”脸盲”，它只能看出谁跟谁关系好，但完全看不出谁坐在哪。所以你得给每个位置发个”座位号”。你带走的关键认知是三种方案的进化逻辑：正弦编码最优雅（用不同频率的波自然区分远近）、可学习编码最粗暴（把位置当参数训）、RoPE 最聪明（把位置信息注入到注意力计算的点积里，天然包含词间距）。现在几乎所有大模型都用 RoPE，因为它的数学性质实在太漂亮了——旋转角度差就是位置差。

常见疑问

Q: 三种位置编码方案，我该选哪个？

A: 如果你是 2024 年以后做模型，不用纠结——选 RoPE。它有两个决定性优势：一是相对位置直接编码在 Attention 计算中，模型天然知道两个词隔了多远；二是可以外推到训练时没见过的长度（通过调整旋转频率）。BERT 时代流行的可学习编码已经基本被淘汰了，因为它无法处理超过训练长度的序列。

Q: 位置编码为什么是”加”到词嵌入上，而不是”拼接”？

A: 如果是拼接，位置信息和语义信息就被隔离在两个独立的子空间里，模型需要额外学习如何混合它们。加法让位置信号和语义信号在同一个向量空间里叠加，迫使模型从一开始就把”谁在哪个位置说了什么”融合在一起处理。实践证明加法就够了，没必要搞更复杂的方案。

第 27 章 Transformer 架构：一张改变世界的图纸

观光路线：认识 Transformer 的核心结构，理解为什么 RNN 被革命了——从一个“逐字读”到一个“摊开看”的架构飞跃 **登山路线：**拆解 Encoder-Decoder 骨架，掌握 Multi-Head Attention、FFN、残差连接、LayerNorm 四大组件的协作方式，从零搭建 MiniTransformer

阅读顺序提示：本章是“组装总览”，**24-自注意力、25-多头注意力、26-位置编码是零件拆解**。建议阅读策略二选一：①先读 24→25→26 再看本章，逐个零件拼装更自然；②先看本章整体印象，读到四大组件时随时跳到 24-26 深入。无论哪种，请记得语言建模的“猜词母题”——Transformer 是它的终极解法。

2017 年，Google Brain 的八位研究者坐下来，想解决一个问题——机器翻译。

Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan Gomez、Lukasz Kaiser、Illia Polosukhin。八个人，一篇论文。他们写的这篇论文标题带着一种反叛的味道：“Attention is All You Need”——翻译过来大概是：“注意力就够了，别的不需要”。

他们不知道，这个标题会成为 AI 史上最著名的 flag 之一。就像有人写了一篇“轮子就够了，不需要马”的文章，结果真的开创了汽车时代。

这篇论文提出了一种全新的神经网络结构：Transformer。八年后的今天，不管你是用 ChatGPT、文心一言还是 Claude——它们全部、无一例外、百分之百、都是 Transformer 架构的变体。那八个作者，如今已全部离开 Google，其中多位创办了估值数十亿美元的 AI 公司。

为什么 RNN 跟不上时代了

在 Transformer 之前，处理文本的主流方案是 RNN——循环神经网络。

RNN 怎么读句子的？一个词一个词地读。读第一个词，更新一下记忆；读第二个词，再更新一下记忆；读到第一百个词的时候，第一个词在脑子里的印象已经模糊了。

翻译成人话就是：RNN 像个老头子，一句话你得凑在他耳边一个字一个字地说，说到后半句他已经忘了前半句在讲啥。

这个“遗忘”问题有学名，叫长程依赖问题。LSTM 和 GRU 改良了一下，加了“记忆门”和“遗忘门”，让老头子的记性好了一点。但本质问题没变——你还是得一个词一个词地喂。

Transformer 的改法很粗暴：别一个一个来了。把整句话一次性“摊开”在桌面上，每个词同时看到其他所有词。

还是用人的体验来比喻。RNN 读句子像你盯着一个窄缝看卷轴，一次只能看到眼前的一小段。Transformer 像你把整张卷轴铺在桌上，一眼扫过去，所有字的相对关系都了然于胸。

这个“摊开看”的能力有个正式名：**全局并行计算**。翻译成人话就是：所有位置同时互相之间的关注，不是排着队来，是一起上。

Encoder-Decoder：翻译机骨架

Transformer 的骨架是 Encoder-Decoder 结构。这其实是机器翻译的老传统了，只不过以前用 RNN 实现，现在用 Attention 实现。

Encoder（编码器）的活是：把输入的句子”读进去”，消理解，生成一种浓缩的表示。你给它一句”我爱吃苹果”，它输出一组向量，每个向量对应一个输入位置，但它携带了这个位置的”上下文感知”之后的信息。

Decoder（解码器）的活是：拿着 Encoder 浓缩出来的信息，一个词一个词地生成输出。比如给一句中文，输出英文：“I love eating apples.”

在 BERT 这样的纯理解模型中，只用了 Encoder。在 GPT 这样的纯生成模型中，只用了 Decoder。翻译模型（比如原始 Transformer）两个都用了。

翻译成人话就是：Encoder 负责”读懂”，Decoder 负责”说人话”。

四大组件：Transformer 的积木块

不管 Encoder 还是 Decoder，它们都是由同样的基本积木块堆叠起来的。每层由四个组件构成：

1. Multi-Head Attention（多头注意力）

这是整个 Transformer 的心脏。它做一件事：让句子里的每个词去”关注”其他所有词，并且从多个不同的角度去关注。这块的计算过程，我们在第 24、25 章已经拆碎了讲过——这里只做快速回顾。

2. Feed-Forward Network（前馈网络，FFN）

Attention 做完之后，每个词已经有了”和其他词的关系”这一层信息。FFN 做的事情是：对每个词单独进行非线性的信息加工。翻译成人话就是——Attention 负责”社交”，FFN 负责”思考”。Attention 让词之间交换信息，FFN 让每个词自己消化这些信息。

FFN 的结构非常简单，就是两层全连接，中间夹一个激活函数：

FFN 的实现：两个线性变换，中间加一个激活函数

```
class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_ff)  # 先放大到 4 倍
        self.fc2 = nn.Linear(d_ff, d_model)  # 再缩回原来的维度
        self.gelu = nn.GELU()               # GELU 激活函数

    def forward(self, x):
        return self.fc2(self.gelu(self.fc1(x)))
```

那个”先放大再缩小”的操作不是多余的吗？不是。翻译成人话就是：先给每个词 4 倍的大脑空间去思考，然后浓缩回原来的大小。这个”放大-思考-浓缩”的过程，让模型能表达更复杂的特征。

3. Residual Connection（残差连接）

这个名字听起来很数学，其实想法简单到家了。把输入直接加到输出上：

```
output = layer(input) + input
```

为什么加这一下？因为神经网络层数一深，信息在传递过程中会衰减。就像传话游戏，经过 20 个人之后原话早就面目全非。残差连接相当于给每层开了一条“高速公路”——信息可以跳过层，直达深层。你学的是“和输入相比需要改多少”，而不是“从头造一个新东西”。

回指第 17 章：这个“输出 = 模块结果 + 原输入”的思路，你在第 17 章 ResNet 里已经见过。CNN 靠它把网络堆到 1000 层，Transformer 同样靠它才能稳定堆叠几十上百层。**残差连接是深度学习里最通用的“深度保险丝”。**

翻译成成人话就是：这条高速公路让 100 层的网络也能训练得动。

4. Layer Normalization（层归一化）

LayerNorm 做的事情很简单：把每个样本的特征值调整到均值为 0、方差为 1 的分布。为什么需要这个？因为神经网络里的数值经过多层计算之后会爆炸或者消失。你不想让某些神经元的输出是 0.00001，另一些是 100000——这会导致后面层的计算完全被大数值主导。

LayerNorm 把数值拉回到一个统一的范围，让训练过程稳定。

四组件如何组成一层

Encoder 的每一层是这样的：

输入 → Multi-Head Attention → 残差连接 + LayerNorm → FFN → 残差连接 + LayerNorm → 输出

Decoder 的每一层多了一个“Cross-Attention”夹在中间，用来接收 Encoder 的输出：

输入 → Masked Multi-Head Attention → 残差 + LayerNorm → Cross-Attention → 残差 + LayerNorm → FFN → 残差 + LayerNorm → 输出

那个“Masked”是什么意思？Decoder 在生成的时候不能偷看未来的词。生成第 5 个词的时候，它只能看前 4 个词。所以需要有一个“因果掩码”把未来的位置遮住——这个“因果掩码”的具体做法，我们在第 24 章已经讲过。

MiniTransformer：从零搭建

理论说了一堆，我们来动手。下面这段代码搭建了一个完整的微型 Transformer，可以直接训练。参数设计得比较小，在你的笔记本电脑上就能跑。

```
import torch
import torch.nn as nn
import math

# ===== 位置编码 =====
class PositionalEncoding(nn.Module):
    """给输入加上位置信息，让模型知道词的顺序"""
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)          # 预分配位置编码矩阵
```

```

position = torch.arange(0, max_len).unsqueeze(1).float()
# 计算正弦波频率, 不同维度用不同周期
div_term = torch.exp(
    torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
)
pe[:, 0::2] = torch.sin(position * div_term) # 偶数位用 sin
pe[:, 1::2] = torch.cos(position * div_term) # 奇数位用 cos
self.register_buffer('pe', pe.unsqueeze(0)) # 注册为 buffer, 不参与梯度

def forward(self, x):
    return x + self.pe[:, :x.size(1)]

# ===== 单头注意力 (简化版) =====
class SingleHeadAttention(nn.Module):
    """自注意力: 每个词关注其他所有词, 计算加权表示"""
    def __init__(self, d_model):
        super().__init__()
        self.d_model = d_model
        self.W_q = nn.Linear(d_model, d_model, bias=False) # 查询投影
        self.W_k = nn.Linear(d_model, d_model, bias=False) # 键投影
        self.W_v = nn.Linear(d_model, d_model, bias=False) # 值投影

    def forward(self, x, mask=None, kv=None):
        Q = self.W_q(x) # (batch, seq_len, d_model)
        if kv is None:
            kv = x # 自注意力: K、V 也来自 x
        K = self.W_k(kv) # (batch, seq_len, d_model)
        V = self.W_v(kv) # (batch, seq_len, d_model)

        # 计算注意力分数: Q 和 K 的点积, 除以 sqrt(d) 做缩放
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_model)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf')) # 遮住不该看的位置

        attn_weights = torch.softmax(scores, dim=-1) # 归一化为概率分布
        output = torch.matmul(attn_weights, V) # 加权求和
        return output, attn_weights

# ===== 前馈网络 =====
class FeedForward(nn.Module):
    """两层全连接, 中间用 GELU 激活, 先膨胀后收缩"""
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
        self.gelu = nn.GELU()

    def forward(self, x):
        return self.fc2(self.gelu(self.fc1(x)))

# ===== Encoder 层 =====
class EncoderLayer(nn.Module):
    """Encoder 的一层: Attention + FFN, 各带残差连接和 LayerNorm"""
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.attention = SingleHeadAttention(d_model)
        self.ffn = FeedForward(d_model, d_ff)

```

```

self.norm1 = nn.LayerNorm(d_model) # Attention 后面的归一化
self.norm2 = nn.LayerNorm(d_model) # FFN 后面的归一化

def forward(self, x, mask=None):
    # Attention + 残差连接 + 归一化
    attn_out, _ = self.attention(x, mask)
    x = self.norm1(x + attn_out) # 残差连接: 输入直接加到输出上
    # FFN + 残差连接 + 归一化
    ffn_out = self.ffn(x)
    x = self.norm2(x + ffn_out) # 又一条残差高速公路
    return x

# ===== Decoder 层 =====
class DecoderLayer(nn.Module):
    """Decoder 的一层: 自注意力 + 交叉注意力 + FFN"""
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.self_attn = SingleHeadAttention(d_model)
        self.cross_attn = SingleHeadAttention(d_model)
        self.ffn = FeedForward(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)

    def forward(self, x, encoder_out, src_mask=None, tgt_mask=None):
        # 自注意力 (带因果掩码, 防止偷看未来)
        attn_out, _ = self.self_attn(x, tgt_mask)
        x = self.norm1(x + attn_out)
        # 交叉注意力 (Decoder 的 Q 去查询 Encoder 的输出)
        cross_out, _ = self.cross_attn(x, kv=encoder_out)
        x = self.norm2(x + cross_out)
        # FFN
        ffn_out = self.ffn(x)
        x = self.norm3(x + ffn_out)
        return x

# ===== 完整 MiniTransformer =====
class MiniTransformer(nn.Module):
    """微型 Transformer: 可配置层数、维度、词表大小"""
    def __init__(self, vocab_size, d_model=256, d_ff=1024, num_layers=4, max_len=512):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model) # 词嵌入层
        self.pos_encoding = PositionalEncoding(d_model, max_len)
        self.encoder_layers = nn.ModuleList([
            EncoderLayer(d_model, d_ff) for _ in range(num_layers)
        ])
        self.decoder_layers = nn.ModuleList([
            DecoderLayer(d_model, d_ff) for _ in range(num_layers)
        ])
        self.output_proj = nn.Linear(d_model, vocab_size) # 输出投影到词表大小
        self.d_model = d_model

    def forward(self, src, tgt, src_mask=None, tgt_mask=None):
        # Encoder 路径
        enc_out = self.embedding(src) * math.sqrt(self.d_model) # 乘以 sqrt(d) 稳定训练
        enc_out = self.pos_encoding(enc_out)
        for layer in self.encoder_layers:
            enc_out = layer(enc_out, src_mask)

```

```

# Decoder 路径
dec_out = self.embedding(tgt) * math.sqrt(self.d_model)
dec_out = self.pos_encoding(dec_out)
for layer in self.decoder_layers:
    dec_out = layer(dec_out, enc_out, src_mask, tgt_mask)

# 输出投影: 把 d_model 维向量映射为词表大小的 Logits
logits = self.output_proj(dec_out)
return logits

# ===== 使用示例 =====
if __name__ == "__main__":
    vocab_size = 10000 # 假设词表大小为10000
    model = MiniTransformer(vocab_size)

    # 模拟输入: batch_size=2, 序列长度=20
    src = torch.randint(0, vocab_size, (2, 20)) # 源语言句子
    tgt = torch.randint(0, vocab_size, (2, 20)) # 目标语言句子

    logits = model(src, tgt)
    print(f"输入形状: src={src.shape}, tgt={tgt.shape}")
    print(f"输出形状: {logits.shape}") # 期望: (2, 20, 10000)
    print(f"模型参数量: {sum(p.numel() for p in model.parameters()):,}")

```

这段约 150 行的代码就是 Transformer 的核心骨架。你可以拿它去做机器翻译、文本摘要、任何序列到序列的任务。真实的 GPT-3 当然比这复杂得多——它有 96 层、每层有 96 个注意力头、`d_model` 是 12288。但骨架是一样的。

几个值得回味的设计选择：`* math.sqrt(self.d_model)` 这个乘法，是为了防止嵌入向量的值太小，被位置编码淹没。残差连接在每个子层都有，不是只在层间——Attention 层后加一道，FFN 后加一道。LayerNorm 放在残差之后（Post-LN），而不是之前——这是原始论文的做法，后来的模型有改成 Pre-LN 的，训练更稳定。

为什么说 Transformer 改变了世界

Transformer 之前，NLP 模型是 RNN 的天下。Transformer 之后，RNN 基本上退出了主流舞台。

原因很简单。并行计算。RNN 必须一个词一个词地算，100 个词就是 100 步；Transformer 可以一次性算完 100 个词之间的所有注意力关系。GPU 喜欢批量计算，RNN 这种串行模式对 GPU 是浪费。Transformer 恰好是 GPU 的最爱——全是矩阵乘法。

还有一个原因是可扩展性。Transformer 的结构极度规整——就是一样的层往上堆。要更强？加层。要更宽？加 `d_model`。这种规整性让它特别适合大规模工程化。

八年后的今天再回头看那八个研究者——他们大概也没想到，自己随手起的那个略带挑衅的论文标题，会定义整个 AI 时代的架构基础。

下一章，我们补上整条链条里最不起眼、却决定模型”识字量”的一环：分词器。我们一路都在说”词”、说”词嵌入”，可模型到底是怎么把一句中文拆成一个个编号的？别急，一次性说清楚。

人话总结

这章说到底就一件事——Transformer 之所以碾压了 RNN，不是因为它”更聪明”，而是因为它”不排队”。RNN 必须一个词一个词地串行处理，Transformer 直接把整句话摊开，让所有词同时互相看。你带走的四个积木块：Attention 负责社交（词和词之间交换信息）、FFN 负责思考（每个词自己消化）、残差连接是高速公路（信息直通深层）、LayerNorm 是稳压器（数值不爆炸）。这四件东西码在一起，往上堆几十层，就成了今天所有大模型的骨架。

常见疑问

Q: Transformer 不就是把 RNN 换成了 Attention 吗？有那么革命性吗？

A: 远不止”换了个零件”。RNN 串行计算是机制性的缺陷——100 个词就要算 100 步，GPU 的并行能力完全浪费。Transformer 把串行变成了并行，这不仅让训练快了 N 倍，更重要的是让”把模型做大”成为可能。没有并行计算，千亿参数的模型训练时间可能不是几个月，而是几十年。

Q: Encoder 和 Decoder 到底什么关系？为什么现在很多模型只用一边？

A: Encoder 负责”读懂”输入（双向注意力，看全文），Decoder 负责”生成”输出（单向注意力，逐个词蹦）。BERT 只要理解能力，所以纯 Encoder；GPT 只要生成能力，所以纯 Decoder；翻译模型既要读懂原文又要生成译文，所以两个都要。现在的大语言模型走 GPT 路线，基本都是 Decoder-only。

第 28 章 分词器与 BPE：模型的“识字课本”

观光路线：理解模型怎么看“文字”——不是按字、不是按词，而是一种聪明的折中方案，让你用有限的词汇表表达无限的语言 **登山路线：**BPE 五步法详解（统计、找最高频、合并、循环），从零实现 BPETokenizer，中文分词的额外挑战

你有一盒乐高积木。标准块、长条块、斜面块——种类不多，大概五万种。但用这五万种积木，你可以拼出任何东西。房子、飞机、恐龙、埃菲尔铁塔。

分词器就是模型的乐高积木盒子。BPE（Byte Pair Encoding，字节对编码）就是那个决定“哪些积木该放进盒子”的算法。

翻译成成人话就是：模型不认识“苹果”，它只认识编号。你得把所有可能的文本切分成一个个小碎片（token），给每个碎片一个唯一编号，模型才知道怎么处理。分词器就是负责这个“切分”和“编号”的。

按字切 vs 按词切：两难的困境

最简单的方法是按字切。“苹果好吃”切成[“苹”，“果”，“好”，“吃”]。好处是词表极小——常用汉字也就六七千个。但坏处很明显：序列变得很长。一篇 1000 字的文章变成 1000 个 token。而且每个 token 的信息量很小——“苹”这个字本身没什么明确语义，它和“果”组合在一起才有力。

另一种方法是按词切。“苹果好吃”切成[“苹果”，“好吃”]。序列短了，每个 token 的语义也清晰了。但坏处是词表爆炸。中文有几万个常用词，加上专有名词、网络新词、中英混杂——词表可能上百万。模型的嵌入层大小 = 词表大小 * d_{model} 。词表大，参数量就爆炸。而且你永远会有没见过的词（OOV，Out of Vocabulary）——训练时没见过“栓 Q”，推理时来了，怎么办？

BPE 给出的答案很巧妙：**两边的好处都要**。它从字符级别开始，然后不断把最高频的“字符对”合并成一个新的 token。合并到约定的词汇表大小为止。

BPE 五步法

假设我们只需要把 BPE 过程讲给一个完全没编程的人听。五步：

第一步：初始化为字符。把训练语料里所有的文本拆成单个字符，末尾加一个特殊符号标记词边界（通常用 `</w>`）。

比如“苹果很好吃苹果很甜”（假设我们只有这一句训练数据，且已经分好词），初始状态是：

```
苹 果 </w> 很 好 吃 </w> 苹 果 </w> 很 甜 </w>
```

每个单元（包括 `</w>`）都是一个初始 token。

第二步：统计频率。数一数相邻的两个 token 一起出现了多少次。

“苹果”出现了2次（“苹果很好吃”里一次，“苹果很甜”里一次）。“很好”出现了1次。“吃”出现了1次。以此类推。

第三步：找最高频的相邻对。“苹果”出现了2次，是目前最高频的。合并它。

合并之后，我们的序列变成：

```
苹果 </w> 很 好 吃 </w> 苹果 </w> 很 甜 </w>
```

“苹果”现在变成了一个新 token。

第四步：循环。重新统计频率，再找最高频的相邻对，再合并。重复。

第二轮，统计后发现”苹果“出现了2次（两个”苹果“后面都跟着</w>，表示词尾）。合并。得到：

```
苹果</w> 很 好 吃 </w> 苹果</w> 很 甜 </w>
```

第三轮，发现”很“出现了2次，但它的邻居不一样（一次是”好“，一次是”甜“），没有超过1的高频对。如果设的合并次数到了，就停止。

第五步：生成词汇表。所有在合并过程中出现过的 token（包括原始字符和合并产物），就是最终的词汇表。

翻译成成人话就是：BPE 是在做一件很朴素的事——把那些经常在一起出现的字符，焊成一个整体。焊的次数越多，词汇表越大，每个 token 的平均长度越长。

从零实现 BPETokenizer

以下代码从头实现了一个微型 BPE 分词器。它不够快、不够工程化，但它清楚地展示了 BPE 的每一个步骤。

```
from collections import Counter, defaultdict
import re

class SimpleBPETokenizer:
    """
    简易 BPE 分词器：从字符级别开始，反复合并最高频的相邻 token 对
    训练完成后可以 encode（文本→token_id 列表）和 decode（token_id 列表→文本）
    """
    def __init__(self, vocab_size=10000):
        self.vocab_size = vocab_size          # 目标词汇表大小
        self.token_to_id = {}                # token → id
        self.id_to_token = {}                # id → token
        self.merges = {}                     # 记录合并顺序: (a, b) → merged

    def _init_chars(self, text):
        """第一步：把文本拆成字符序列，每个词后面加</w>标记"""
        words = text.split()
        # 每个字符单独作为一个 token，词尾加</w>
        tokens = []
        for word in words:
            for i, char in enumerate(word):
                tokens.append(char)
```

```

        tokens.append('</w>') # 词尾标记
    return tokens

def _get_pair_stats(self, token_list):
    """第二步: 统计所有相邻 token 对的频率"""
    pairs = Counter()
    for i in range(len(token_list) - 1):
        pair = (token_list[i], token_list[i + 1])
        pairs[pair] += 1
    return pairs

def _merge_tokens(self, token_list, pair_to_merge):
    """第三步: 遍历 token 列表, 合并所有匹配的 token 对"""
    merged = []
    i = 0
    while i < len(token_list):
        # 如果当前位置和下一个位置匹配要合并的 pair
        if (i < len(token_list) - 1 and
            token_list[i] == pair_to_merge[0] and
            token_list[i + 1] == pair_to_merge[1]):
            merged.append(pair_to_merge[0] + pair_to_merge[1]) # 合并
            i += 2 # 跳过两个
        else:
            merged.append(token_list[i])
            i += 1
    return merged

def train(self, corpus_text, verbose=True):
    """第四步: 迭代合并, 直到达到目标词表大小"""
    # 初始化: 拆分为字符
    tokens = self._init_chars(corpus_text)

    # 初始词汇表: 所有出现过的单字符
    base_vocab = set(tokens)
    self.token_to_id = {t: i for i, t in enumerate(sorted(base_vocab))}

    num_merges = self.vocab_size - len(base_vocab)
    if verbose:
        print(f"初始词表大小: {len(base_vocab)}")
        print(f"计划合并次数: {num_merges}")

    for merge_step in range(num_merges):
        # 统计当前所有相邻 token 对的频率
        pair_stats = self._get_pair_stats(tokens)
        if not pair_stats:
            break # 没有可合并的了

        # 找最高频的 pair (频率相同时, 选字典序最小的)
        best_pair = max(pair_stats, key=lambda p: (pair_stats[p], -ord(p[0][0]) if
p[0] else 0))

        # 记录这次合并
        merged_token = best_pair[0] + best_pair[1]
        self.merges[best_pair] = merged_token

        # 更新词汇表
        self.token_to_id[merged_token] = len(self.token_to_id)

```

```

# 执行合并
tokens = self._merge_tokens(tokens, best_pair)

if verbose and (merge_step + 1) % 50 == 0:
    print(f" 合并 {merge_step + 1}/{num_merges}: "
          f"'{best_pair[0]}' + '{best_pair[1]}' -> '{merged_token}' "
          f"(出现{pair_stats[best_pair]}次)")

# 构建反向映射
self.id_to_token = {v: k for k, v in self.token_to_id.items()}
if verbose:
    print(f"训练完成! 最终词表大小: {len(self.token_to_id)}")

def encode(self, text):
    """把文本编码为token ID 列表"""
    words = text.split()
    token_ids = []
    for word in words:
        # 拆成字符
        chars = list(word) + ['</w>']
        # 反复应用合并规则 (按合并顺序)
        while len(chars) > 1:
            # 找当前可以合并的pair 中最早被训练时发现的
            min_merge_order = float('inf')
            best_pair = None
            best_idx = -1

            for i in range(len(chars) - 1):
                pair = (chars[i], chars[i + 1])
                if pair in self.merges:
                    # 简单策略: 取第一个可合并的pair
                    best_pair = pair
                    best_idx = i
                    break

            if best_pair is None:
                break # 没有可合并的了

        # 执行合并
        merged = self.merges[best_pair]
        chars = chars[:best_idx] + [merged] + chars[best_idx + 2:]

    # 转成 ID
    for token in chars:
        token_ids.append(self.token_to_id.get(token, 0)) # 0=UNK

    return token_ids

def decode(self, token_ids):
    """把token ID 列表解码回文本"""
    tokens = [self.id_to_token.get(tid, '<UNK>') for tid in token_ids]
    # 拼接, 遇到</w>换空格
    text = ''
    for token in tokens:
        if token == '</w>':
            text += ' '
        else:
            text += token

```

```

    return text.strip()

# ===== 演示 =====
if __name__ == "__main__":
    # 训练语料（实际训练需要海量文本，这里只用几句做演示）
    corpus = (
        "深度学习 是 机器学习 的 一个 分支 深度学习 在 图像 识别 方面 表现 出色 "
        "机器学习 让 计算机 从 数据 中 学习 深度学习 使用 深层 神经网络 "
        "学习 算法 是 人工智能 的 核心 学习 如何 学习 是 元学习"
    )

    tokenizer = SimpleBPETokenizer(vocab_size=100)
    tokenizer.train(corpus, verbose=True)

# 测试编码和解码
test_texts = ["深度学习 很 强大", "机器学习 是 基础"]
for text in test_texts:
    ids = tokenizer.encode(text)
    decoded = tokenizer.decode(ids)
    print(f"\n 原文: {text}")
    print(f"编码: {ids}")
    print(f"解码: {decoded}")

```

运行这个代码，你会看到 BPE 如何一步一步地把”深”和”度”合并成”深度”，再把”深度”和”学习”合并成”深度学习”。这是从无到有”学会”词汇的过程——不是查字典，而是从数据中自然涌现。

中文分词的特殊挑战

以上演示用的是已分词的文本。但中文天然是不分词的——你写”苹果很好吃”没有空格。这对 BPE 提出了额外挑战。

现实中处理中文有两种思路。一种是先分词再 BPE——效果依赖分词器质量，容易把分词错误传导下去。另一种是直接在字符级别做 BPE，让模型自己学合并——这是大多数中文大模型的做法。

直接在字符级别做 BPE 有一个很有意思的现象：模型会学会一些”不成词”但出现频率极高的组合。比如”的一”、”了我”、”是不”——这些不是真正的词，但在中文文本里它们相邻的概率极高。BPE 会把它们焊在一起。

翻译成成人话就是：中文字符级别的 BPE，学出来的并不一定对应语言学上的”词”，而是出现频率高的”字串”。有时候它把”的”和后面的字合并了，有时候又没合并。这不是 bug，这是 feature——模型自己找到了最优的压缩方案。

另一个挑战是数字和英文的处理。中文文本里经常夹杂英文单词和数字。“iPhone15 售价 8999 元”——怎么分？好的分词器会把”iPhone”和”15”分开，把”8999”作为一个整体。这需要额外的预处理规则。

词汇表大小：一个影响深远的超参数

词汇表大小通常在 32000 到 100000 之间。GPT-2 用了 50257，LLaMA 用了 32000，GPT-4 用了 100000。

词汇表太小的后果：每个 token 覆盖的文字多，序列变短了（好事），但很多词表达不出来（坏事）。词汇表太大的后果：嵌入层的参数量爆炸（ $d_model * vocab_size$ ），而且很多罕见的 token 几乎没有被训练充分。

这个超参数的选择直接影响模型的知识密度。你的词汇表里有没有”牛顿”这个 token？如果有，模型可以直接用一个 token 表达这个人物。如果没有，它得用”牛”+”顿”两个 token 来表达，有时候还得靠上下文才能推断出这是个人名。

分词器是模型的第一道关卡。你喂进去的是文本，出来的是一个整数序列。然后这些整数被映射为嵌入向量，加上位置编码，进入 Transformer 的层叠结构。

但到目前为止我们只讲了”结构”，还没讲”训练”。把 Transformer 搭好了，分词器配好了，接下来怎么训练出一个会说话的模型？下一章，我们进国家图书馆。

人话总结

这章说到底就一件事——分词器就是模型的”识字课本”，BPE 算法决定了哪些字串配拥有自己的编号。你带走的直觉是：按字切序列太长，按词切词表爆炸，BPE 找到了第三条路——从字符出发，反复把最高频的相邻对焊在一起，直到词表大小刚好。中文分词的额外挑战在于我们天然不空格，模型会学会”的一””了我”这种语言学上不成立但统计上高频的组合——这不是 bug，是 feature。词汇表大小（通常 32000 到 100000）这个看似枯燥的数字，直接影响模型的知识密度。

常见疑问

Q: 为什么中文分词比英文难？BPE 在中文上有什么特殊问题？

A: 英文单词天然有空格分隔，BPE 只需要在单词内部做子词合并。中文没有空格，BPE 要在整个字符序列上做合并，容易产生”的一””是不”这种跨词边界的合并结果。这不是坏事——模型自己学会了最优的压缩方式。但这也意味着同样的词汇表大小，中文模型每个 token 覆盖的语义范围可能比英文模型更碎，序列长度天然更长，推理成本更高。

Q: vocab_size 设多大合适？为什么不能直接用 100 万词汇的词典？

A: 词表大小=嵌入层参数量= $d_model * vocab_size$ 。如果 $d_model=4096$ 、 $vocab_size=100$ 万，光嵌入层就要 40 亿参数，而其中大部分低频 token 的嵌入向量训练不充分，纯属浪费。32000 到 100000 是权衡后的甜点区间——既能覆盖绝大多数常见语义单元，又不至于让嵌入层吃掉太多参数和显存。

生成模型篇

公众号：AI技术圈

第 29 章 GAN：生成对抗网络

观光路线：本章带你走进 GAN 的世界，用”师生互促”的视角理解生成器和判别器的共生关系。你会看到 GAN 如何通过一场精妙的博弈让机器学会”创造”。有 Python 代码，有数学直觉，还有训练中会遇到的坑。读完你会明白为什么 GAN 是深度学习近十年最漂亮的想法之一。

登山路线：极小极大博弈的数学形式化、纳什均衡的直觉理解、交替训练的梯度流分析、模式坍塌（mode collapse）的成因与缓解策略、DCGAN 架构设计细节（转置卷积、批归一化、激活函数选择的工程原因）、完整可运行的 MNIST 手写数字生成代码。难度 ★★★★★☆

2014 年夏天，Ian Goodfellow 在蒙特利尔的一个酒吧里和朋友们喝酒。聊着聊着，他突然想到一个主意——让两个神经网络互相比较劲，一个负责画画，一个负责鉴画，两个都越来越强。他当场跑回家写代码，凌晨时分，第一个 GAN 就跑出了结果。

这个故事在 AI 圈子里传得很广，有几分传奇色彩。但说实话，GAN 的价值不在于这个戏剧性的诞生场景，而在于它提出了一种全新的思路：**创造不一定来自模仿，也可以来自博弈。**

之前的生成模型，比如自编码器，思路是”先学会压缩，再学会还原”。本质上还是在模仿训练数据。但 GAN 完全换了一个角度——我不去学数据的统计规律，我让另一个网络来告诉我”你画得像不像”。这听起来很绕，但它打开了通往高质量生成的大门。

老师和学生的共生游戏

几乎所有讲 GAN 的文章都会用”造假钞和验钞员”这个比喻。我不喜欢这个比喻。它暗示了一种对立关系：一边是骗子，一边是警察，你死我活。

真相恰恰相反。GAN 的巧妙之处在于，两个网络是**共生的**，就像一位严师和一个努力的学生。

想象这样一个场景：美术老师班上来了一位插班生。这个学生从来没学过画画，第一天交上来的作业是一团乱七八糟的线条。老师看了一眼，直接打回去：“重画。”

学生不知道该怎么画，只能瞎猜。他看到老师桌子上放着几张梵高的画，就照着那个方向去。第二天，他又交了一幅。这次线条没那么乱了，但色彩还是不对。老师还是不满意。

日复一日，学生越画越好。但这里有一个微妙的点：老师自己也在变。每看一幅学生的画，老师对于”什么是好画”的判断标准就更新一点点。学生的画逼着老师练出了更敏锐的眼力，老师的挑剔又倒逼学生往更高的水平走。

这不是对抗。这是**相互成就**。

用数学语言来讲，这里有两个函数。生成器 G （学生）拿到一个随机噪声向量 z ，输出一张假图片 $G(z)$ 。判别器 D （老师）拿到一张图片 x ，输出一个 0 到 1 之间的分数 $D(x)$ ，越接近 1 代表越像真图。

训练的目标是一个极小极大博弈：

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

翻译成成人话：判别器 **D** 想尽量给真图打高分、给假图打低分（所以它要最大化这个值）；生成器 **G** 想尽量让判别器把自己的假图也打高分（所以它要最小化这个值）。

注意这两个“想”是同时进行的，互相嵌套。这就是 GAN 的灵魂——**对抗训练**。

训练是怎么进行的

你可能会问：两个网络同时训练，不会互相拖累吗？

好问题。实际操作中是交替训练的。一步用来更新判别器，一步用来更新生成器，像两个人下棋轮流走子。

具体来说，每一轮训练长这样：

1. 从真实数据里抽一撮真图
2. 随机生成一撮噪声向量，扔给生成器，得到一撮假图
3. 拿真图和假图一起训练判别器，让 **D** 学会区分
4. 固定判别器不动，再生成一撮假图，训练生成器，让 **G** 学会骗过 **D**

第四步是关键。生成器的梯度是从判别器反向传过来的——判别器说“你这里画得不对”，梯度就顺着这句话流回生成器，告诉它该往哪个方向改。

这个过程跑上几千轮，最终生成器产出的图会以假乱真。理论上，博弈会收敛到一个纳什均衡点——生成器完美复刻了真实数据分布，判别器再也分不出真假，只能永远猜 0.5。

当然，理论归理论，现实中 GAN 的训练非常“娇气”。

模式坍塌：最头疼的问题

如果你亲手训练过 GAN，大概率遇到过这样一个场景：生成器突然只学会了画一种数字。

比如训练一个生成手写数字的 GAN。前几百轮一切正常，数字从模糊变清晰，像显影液里的照片一点一点浮现。但突然之间，生成器开始疯狂输出“1”。不管输入什么噪声，出来的都是 1，各式各样的 1，但全都是 1。

这就是**模式坍塌**（Mode Collapse）。

怎么回事呢？生成器发现了一个“作弊”策略：数字 1 的结构简单，最容易骗过判别器。既然判别器对 1 的辨别力最弱，那我就只画 1 好了。于是生成器把所有的噪声输入都映射到了数据分布中一个非常狭窄的区域。

这就像那个插班生发现老师对素描特别宽容——“那我以后只画素描，永远不碰水彩。”虽然每一幅素描都很逼真，但他永远学不会画水彩。

模式坍塌是 GAN 最经典的失败模式。后来研究者们想出了各种办法：WGAN 换了损失函数让梯度更平滑，minibatch discrimination 让判别器能比较一组样本之间的差异，还有谱归一化、梯度惩罚等等。这些改进让 GAN 的训练稳定性大幅提升，但完全消除模式坍塌仍然是一个开放问题。

DCGAN: 让 GAN 真正能用的架构

2016 年, Radford 等人提出了 DCGAN (Deep Convolutional GAN)。这个工作的贡献不在于理论创新, 而在于工程实践——它用一组行之有效的架构设计, 让 GAN 的训练变得相对稳定。

DCGAN 的核心设计原则: - 判别器里用跨步卷积代替池化层, 让网络自己学会下采样 - 生成器里用转置卷积做上采样, 从噪声向量一步步放大到图片尺寸 - 生成器和判别器都用批归一化 (BatchNorm), 稳定每一层的激活值分布 - 生成器的输出层用 tanh 激活, 把像素值约束在 $[-1, 1]$ 区间 - 隐层用 ReLU (生成器) 和 LeakyReLU (判别器), 避免梯度消失

下面这段代码演示了用一个简化的 DCGAN 在 MNIST 上生成手写数字:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# ===== 超参数 =====
latent_dim = 100      # 噪声向量的维度
img_size = 28         # MNIST 图片大小
lr = 0.0002           # 学习率
batch_size = 64
epochs = 50

# ===== 生成器: 从噪声到图片 =====
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        # 用转置卷积一步步把 100 维噪声放大成 28x28 的图片
        self.model = nn.Sequential(
            # 输入: (100,) → 重塑为 (256, 7, 7)
            nn.Linear(latent_dim, 256 * 7 * 7),
            nn.BatchNorm1d(256 * 7 * 7),
            nn.ReLU(True),
            # 重塑: (256, 7, 7)
            nn.Unflatten(1, (256, 7, 7)),
            # 转置卷积: (256, 7, 7) → (128, 14, 14)
            nn.ConvTranspose2d(256, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            # 转置卷积: (128, 14, 14) → (1, 28, 28)
            nn.ConvTranspose2d(128, 1, 4, 2, 1),
            nn.Tanh() # 输出范围 [-1, 1]
        )

    def forward(self, z):
        return self.model(z)

# ===== 判别器: 给图片打分 =====
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            # 跨步卷积: (1, 28, 28) → (64, 14, 14)
            nn.Conv2d(1, 64, 4, 2, 1),
```

```

        nn.LeakyReLU(0.2, True),
        # 跨步卷积: (64, 14, 14) → (128, 7, 7)
        nn.Conv2d(64, 128, 4, 2, 1),
        nn.BatchNorm2d(128),
        nn.LeakyReLU(0.2, True),
        nn.Flatten(),
        nn.Linear(128 * 7 * 7, 1), # 输出一个分数
        nn.Sigmoid()             # 压缩到 [0, 1]
    )

    def forward(self, img):
        return self.model(img)

# ===== 准备数据 =====
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5]) # 归一化到 [-1, 1]
])
dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# ===== 初始化 =====
G = Generator()
D = Discriminator()
loss_fn = nn.BCELoss() # 二分类交叉熵
opt_G = optim.Adam(G.parameters(), lr=lr, betas=(0.5, 0.999))
opt_D = optim.Adam(D.parameters(), lr=lr, betas=(0.5, 0.999))

# ===== 训练循环 =====
for epoch in range(epochs):
    for real_imgs, _ in dataloader:
        current_batch = real_imgs.size(0)

        # --- 第1步: 训练判别器 ---
        # 真图标签=1, 假图标签=0
        real_labels = torch.ones(current_batch, 1)
        fake_labels = torch.zeros(current_batch, 1)

        # 真图损失
        outputs_real = D(real_imgs)
        loss_D_real = loss_fn(outputs_real, real_labels)

        # 假图损失
        z = torch.randn(current_batch, latent_dim) # 随机噪声
        fake_imgs = G(z)
        outputs_fake = D(fake_imgs.detach()) # detach 断开梯度, 不更新G
        loss_D_fake = loss_fn(outputs_fake, fake_labels)

        loss_D = loss_D_real + loss_D_fake
        opt_D.zero_grad()
        loss_D.backward()
        opt_D.step()

        # --- 第2步: 训练生成器 ---
        # 生成器希望假图被判别器判为真 (标签=1)
        z = torch.randn(current_batch, latent_dim)
        fake_imgs = G(z)
        outputs = D(fake_imgs)

```

```

    loss_G = loss_fn(outputs, real_labels) # 注意！标签用1

    opt_G.zero_grad()
    loss_G.backward()
    opt_D.zero_grad() # 确保不误更新D
    opt_G.step()

    print(f'Epoch {epoch+1}/{epochs} | D_loss: {loss_D.item():.4f} | G_loss: {loss_G.item():.4f}')

print('训练完成！生成器已经学会画手写数字了。')
```

跑完这段代码，你就能从随机噪声中生成像模像样的手写数字了。每一张都是生成器”凭空想象”出来的，但在人类眼里和真实数字几乎分不出区别。

GAN 的遗产

GAN 的出现是一个分水岭。在此之前，“生成模型”更多是一个学术概念——做出来的图模糊、扭曲、缺乏细节。GAN 证明了一件事：通过博弈机制，神经网络可以学会创造高度逼真的内容。

后来的 StyleGAN 能生成以假乱真的人脸——那些”这个人不存在”网站上的照片，全是 GAN 画出来的。CycleGAN 能在没有配对数据的情况下完成风格迁移，比如把照片变成梵高风格的油画。Pix2Pix 则做到了图像到图像的翻译，素描上色、航拍转地图、白天变黑夜。

GAN 的思路也启发了后来的很多工作。某种程度上，ChatGPT 背后的 RLHF（人类反馈强化学习）也有 GAN 的影子——让一个奖励模型来评判生成结果的好坏，然后反馈给生成模型。

不过 GAN 最终没有成为生成模型的主流。它不是被”打败”的——毕竟 StyleGAN 的生成质量至今仍是天花板级别的——而是被”绕过去”的。扩散模型的出现改变了游戏规则：训练更稳定、生成更多样、不需要精巧的博弈平衡。

但回头来看，GAN 的核心思想——两个网络在对抗中共生进化——仍然是深度学习历史上最优雅的构想之一。2014 年 Goodfellow 在酒吧里那个灵光一闪，改变了整个生成模型领域的走向。这才是 GAN 真正的遗产。

下一章我们会聊变分自编码器（VAE）。VAE 走的是另一条路——不靠博弈，而是靠概率推断来生成。两条路各有各的美，读完你会有自己的判断。

人话总结

这章讲的是深度学习里最优雅的对抗游戏。GAN 有两个玩家：生成器 G（想画假画以假乱真）和判别器 D（想练出火眼金睛分辨真假）。训练过程就是一攻一防的博弈——D 努力把真图和假图分开，G 努力把假图画得更像真的。这个博弈如果打到纳什均衡，两个玩家谁也赢不了谁，生成器就成了”造假大师”。GAN 的代码逻辑很直观：训 D 的时候，真图标签标 1、假图标签标 0；训 G 的时候反过来——G 希望自己的假图被 D 判为 1（真），所以损失函数里故意把假图的标签设成 1。这个”反向标签”的 trick 是整个训练循环的精髓。GAN 的遗产不

在于它现在还排第几，而在于它开创了”对抗共生”这种训练范式——后来 RLHF 里奖励模型和生成模型的互动、甚至现在很多自博弈的强化学习方法，都能看到 GAN 的影子。

常见疑问

Q: GAN 训练不稳定具体是什么表现？为什么会这样？ A: 最常见的翻车场面叫”模式坍塌”（mode collapse）：生成器发现了一种骗过判别器的捷径，然后它就一直画那一种图——比如生成手写数字时永远画”1”，不画其他数字。原因是博弈失衡：G 太快变强或者 D 太快变弱，纳什均衡没达成。另一个表现是 loss 不收敛，上下狂跳——D 的 loss 越来越低（区分能力太强），导致 G 的梯度消失，没法进步。这也是为什么 GAN 训练需要精细调参：学习率、优化器、网络容量、训 D 和训 G 的频率比，差一点都不行。

Q: 现在 GAN 还有人在用吗？被 Diffusion 完全取代了吗？ A: 在某些特定场景 GAN 依然是王者。最典型的是图像的忠实编辑——你想把一个人的发型换了但脸完全不变，用 StyleGAN 的潜在空间插值就可以精确控制。扩散模型生成多样性好但可控性弱一些。另一个场景是实时生成——GAN 一步生成，扩散模型需要几十步。在需要毫秒级响应的应用里（比如视频特效实时预览），GAN 仍然有自己的位置。但纯粹”从文本生成一张好看的图”这个赛道，扩散模型确实已经完胜了。

第 30 章 VAE：变分自编码器

观光路线：从普通的自编码器出发，看看它为什么不适合做生成。然后引入 VAE 的核心洞察——不学习一个确定的编码，而是学习一个概率分布。你会看到猫和狗在隐含空间里如何优雅地过渡，理解“重参数化技巧”这件聪明的小发明。读完你会发现，VAE 和 GAN 虽然都做生成，但哲学完全不同。

登山路线：普通 AE 到 VAE 的数学跃迁、变分推断的直觉解释（ELBO 推导不做形式化展开）、编码器输出 μ 和 $\log(\sigma^2)$ 而非 σ 的原因、重参数化技巧 $z = \mu + \sigma \cdot \epsilon$ 的梯度流分析、KL 散度项的解析解、 β -VAE 的解纠缠扩展、完整可运行的 VAE 代码（含重构+采样+潜在空间插值）、在 MNIST 上展示 2D 潜在空间的平滑语义过渡。难度 ★★★★★☆

2013 年底，Diederik Kingma 和 Max Welling 发表了一篇论文，叫《Auto-Encoding Variational Bayes》。光看标题你可能不会多看一眼——又是变分推断又是贝叶斯，听起来像纯理论文章。但这篇论文所提出的 VAE（变分自编码器），后来成了生成模型领域最重要的基石之一。

如果说 GAN 是一场博弈游戏，那 VAE 就是一道数学推理题。它的思路优雅而严谨：如果能用概率分布来描述数据的隐含特征，那生成新数据就变成了从分布中采样。

普通自编码器为什么不行

在你做图像处理的时候，可能遇到过这样一个场景：一张高清图片太大了，传不动，于是你用压缩软件把它压成一个小文件，对方收到后再解压还原。

自编码器（Autoencoder）干的差不多就是这个事。它由两部分组成：编码器把输入压缩成一个低维向量（叫“瓶颈”或“隐含表示”），解码器再从这个向量把图片重建出来。训练的目标很简单——让重建的图和原图尽量接近。

这个想法直观且有效。但如果你想用它来生成新图片，问题就来了。

想象你把世界上所有的猫图片喂给一个自编码器。训练完成后，编码器学会了把任何一只猫压缩成一个隐含向量。现在你想生成一只新猫——你拿一个随机的隐含向量扔给解码器，出来的大概率是一团模糊。

为什么？因为普通自编码器学的隐含空间是“散点式”的。每只训练过的猫对应空间里的一个孤立的点，点与点之间是“无人区”。你在无人区随便抓一个位置，解码器从来没见过这种输入，它只能瞎猜。

用一个更形象的比喻：普通自编码器的隐含空间像一份稀疏的通讯录，上面只记录了训练时见过的每只猫。你想找一个“介于波斯猫和布偶猫之间”的猫，通讯录翻烂了也找不到。

VAE 的改进直击要害：不要记录一个点，记录一个范围。

每张图片的编码不是一个确定的向量，而是一个概率分布——通常是高斯分布，由均值 μ 和方差 σ^2 两个参数来描述。这只猫“大概”在这个区域，“大概”在这个范围内。整张地图上相邻的分布彼此重叠，形成一片连续的大陆。你从任何位置采样，解码器都见过类似的输入。

猫狗的编码范围

拿猫和狗来举例会比较直观。

在普通 AE 里，一只暹罗猫被编码成向量[0.8, -0.3]，一只柯基犬被编码成[0.2, 0.9]。这两个点之间没有任何东西。你想生成一个”像猫又像狗”的动物——解码器在 0.5, 0.3 这个位置看到的输入它从来没见过，出来的结果惨不忍睹。

VAE 不一样。编码器不直接输出一个点，而是输出一个分布的参数：均值向量 μ 和对数方差 $\log(\sigma^2)$ 。（为什么输出 $\log(\sigma^2)$ 而不是 σ ？因为 σ 必须是正数，而 $\log(\sigma^2)$ 可以是任意实数，让网络输出没有约束会更稳定。）

暹罗猫被编码成： $\mu=[0.8, -0.3]$ ， $\sigma^2=[0.05, 0.04]$ 。意思是这只猫”大概率在以(0.8, -0.3)为中心、半径 0.2 左右的一个小圆内”。

柯基犬被编码成： $\mu=[0.2, 0.9]$ ， $\sigma^2=[0.06, 0.05]$ 。

这两片”概率云”是连续的。你在两片云之间的过渡地带采样，解码器能生成一个看起来既像猫又像狗的”中间态”——可能是耳朵像猫但体型像狗的小动物。

这就是 VAE 作为生成模型的魔力。潜在空间是平滑的、连续的、有结构的。你沿着一个方向走，数字”1”会逐渐变成”7”，猫脸的笑容会逐渐变成悲伤的表情。这种”语义插值”能力，是做创意工具时最有价值的东西。

训练的秘密：两个损失打架

VAE 的损失函数由两项组成，它们互相制约：

$$\mathcal{L} = \mathcal{L}_{recon} + \beta \cdot D_{KL}(q(z|x) \parallel p(z))$$

第一项是重构损失——生成的图片和原图像不像？越像越好。它逼着编码器把信息尽量保留下来。

第二项是 KL 散度——编码出来的分布和标准正态分布像不像？越像越好。它逼着潜在空间规整、连续、没有空洞。

这两项天然打架。重构项说：“每个样本给我独特的编码，把细节全记住！”KL 项说：“不行，所有编码都得聚拢在原点附近，分布要平滑！”

这个张力正好造就了 VAE 的性质：它不能完美重建每一张图片（这是 GAN 的强项），但它有一个结构化、可导航的潜在空间（这是 VAE 的强项）。GAN 生成的图更清晰锐利，VAE 的潜在空间更优美规整。没有谁更好，看你要什么。

重参数化：最聪明的一招

读 VAE 的论文时，你可能会问：编码器输出的是 μ 和 σ ，不是直接的编码。那解码器的输入从哪来？

答案是**采样**：从 $N(\mu, \sigma^2)$ 这个分布里随机抽一个值： $z = \mu + \sigma \cdot \epsilon$ ，其中 $\epsilon \sim N(0, 1)$ 。

等一下。采样操作是不可导的。梯度没法穿过一个”随机抽取”的步骤传到编码器。VAE 不就训不了了吗？

这里有一个绝妙的技巧：**重参数化**（reparameterization trick）。

把”从 $N(\mu, \sigma^2)$ 中采样”拆成两步：1. 从一个固定的标准正态分布 $N(0,1)$ 中采样 ϵ 2. 用 $z = \mu + \sigma \cdot \epsilon$ 得到最终编码

这样一来， μ 和 σ 就参与了确定性计算（ $\mu + \sigma \cdot \epsilon$ ），梯度可以畅通无阻地流过去。 ϵ 本身不依赖任何参数，不需要梯度。

这个 trick 小得几乎不值一提——就一行代码的事——但它让整个 VAE 的训练变成了可能。很多时候，一个领域的突破不在于宏大的理论，而在于这种”啊原来可以这样”的小聪明。

完整代码：从零实现 VAE

下面用 PyTorch 实现一个完整的 VAE，在 MNIST 上训练，然后展示潜在空间的平滑插值。

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import numpy as np

# ===== 超参数 =====
latent_dim = 2          # 2 维潜在空间，方便可视化
hidden_dim = 400        # 隐层维度
batch_size = 128
epochs = 30
lr = 1e-3

# ===== VAE 模型 =====
class VAE(nn.Module):
    def __init__(self):
        super().__init__()
        # --- 编码器: 图片 →  $\mu$ ,  $\log_{var}$  ---
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
        )
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)      # 均值
        self.fc_logvar = nn.Linear(hidden_dim, latent_dim)  # 对数方差

        # --- 解码器:  $z \rightarrow$  重建图片 ---
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 28 * 28),
            nn.Sigmoid()  # 输出像素值在 [0, 1]
        )

    def encode(self, x):
        """编码: 输入图片, 输出分布参数  $\mu$  和  $\log_{var}$ """
```

```

        h = self.encoder(x)
        return self.fc_mu(h), self.fc_logvar(h)

def reparameterize(self, mu, log_var):
    """重参数化技巧:  $z = \mu + \sigma \cdot \epsilon$ """
    std = torch.exp(0.5 * log_var)          # log_var → std
    eps = torch.randn_like(std)             # 标准正态噪声
    return mu + eps * std

def decode(self, z):
    """解码: 从潜在向量重建图片"""
    return self.decoder(z).view(-1, 1, 28, 28)

def forward(self, x):
    mu, log_var = self.encode(x)
    z = self.reparameterize(mu, log_var)
    recon = self.decode(z)
    return recon, mu, log_var

# ===== 损失函数 =====
def vae_loss(recon, x, mu, log_var):
    """
    两项损失:
    1. 重构损失: 用 BCE 衡量重建图和原图的差异
    2. KL 散度: 让编码分布逼近标准正态  $N(0, 1)$ 
    """
    KL = -0.5 * sum(1 + log_var - mu^2 - exp(log_var))

    # 重构损失 (二值交叉熵, 逐像素求和)
    recon_loss = F.binary_cross_entropy(
        recon, x, reduction='sum'
    ) / x.size(0)

    # KL 散度 (解析解, 每一项独立计算然后求和)
    kl_div = -0.5 * torch.sum(
        1 + log_var - mu.pow(2) - log_var.exp()
    ) / x.size(0)

    return recon_loss + kl_div, recon_loss, kl_div

# ===== 数据准备 =====
transform = transforms.ToTensor() # 像素已经归一化到 [0, 1]
dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# ===== 训练 =====
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = VAE().to(device)
optimizer = optim.Adam(model.parameters(), lr=lr)

model.train()
for epoch in range(epochs):
    total_loss = 0
    for imgs, _ in dataloader:
        imgs = imgs.to(device)
        optimizer.zero_grad()

        recon, mu, log_var = model(imgs)
        loss, recon_l, kl_l = vae_loss(recon, imgs, mu, log_var)

```

```

        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(
        f'Epoch {epoch+1:2d}/{epochs} | '
        f'Total: {total_loss/len(dataloader):.2f} | '
        f'Recon: {recon_l:.2f} | KL: {kl_l:.4f}'
    )

print('训练完成! VAE 的潜在空间已经结构化了。')

# ===== 潜在空间的平滑插值演示 =====
model.eval()
with torch.no_grad():
    # 生成一个 15x15 的网格, 覆盖潜在空间中 [-3, 3] x [-3, 3] 的范围
    n = 15
    grid_x = np.linspace(-3, 3, n)
    grid_y = np.linspace(-3, 3, n)

    # 逐点采样并解码
    canvas = np.zeros((n * 28, n * 28))
    for i, yi in enumerate(grid_y):
        for j, xi in enumerate(grid_x):
            z = torch.tensor([[xi, yi]], dtype=torch.float32).to(device)
            img = model.decode(z).cpu().squeeze().numpy()
            canvas[i*28:(i+1)*28, j*28:(j+1)*28] = img

    print(
        f'潜在空间网格已生成。'
        f'左上角(-3,-3)到右下角(3,3), '
        f'你会看到数字平滑地从一种形态过渡到另一种。'
    )
    # 实际使用中可以把 canvas 保存为图片查看

```

跑完这段代码，你会看到一个 15x15 的”数字地图”。左上方可能是歪歪扭扭的”1”，往右走过几格变成了”7”，再往下走变成了”9”。每个数字都不是突兀地跳到另一个，而是有一个连续变形的过程——笔画的弧度在增加，倾斜的角度在变化，圆圈在闭合或打开。

这种”渐变动画”的效果正是 VAE 的招牌。潜在空间被 KL 散度约束成了一个平滑的流形，相邻的点生成相似的图，任何位置的采样都有意义。

VAE 的意义

VAE 的意义远超”生成图片”这件事本身。

它给出了一套用神经网络做概率建模的通用框架。编码器学会推断隐含变量，解码器学会从隐含变量重构数据。这个”编码-采样-解码”的范式后来被很多工作继承：从音乐生成到分子设计，从推荐系统到强化学习中的世界模型。

还有一个重要的扩展方向：**解纠缠**（disentanglement）。 β -VAE 在 KL 散度上加大权重，迫使每个潜在维度只控制一个独立的语义属性——比如一个维度管数字的形状，另一个维度管笔画的粗细。这种对隐含因素的分离能力，让我们离”可解释的 AI”更近了一步。

在今天的 AI 版图中，VAE 可能不像 GAN 和 Diffusion 那样夺目。但它对后来者的影响是深远的。Stable Diffusion 的第一个组件就是 VAE——它用 VAE 把图片压缩到一个高效的隐含空间，让扩散过程不用在像素级别费力地算。没有 VAE 奠定的基础，SD 的高效生成就不会那么自然。

VAE 代表的是一种**概率世界观**：世界是不确定的，最好的描述不是说“这个东西是什么”，而是说“这个东西大概是什么样，可能有哪几种变体”。这种思维方式，比任何具体的模型都重要。

下一章我们进入扩散模型的世界。如果说 VAE 是“从分布中采样”，那扩散模型就是“从噪声中还原”——一种物理直觉极强的生成范式。你会看到墨水如何在清水中扩散，又如何从扩散的痕迹一步一步倒推回最初那一滴。

人话总结

VAE 和 GAN 走的是两条完全不同的路。GAN 靠对抗博弈，VAE 靠概率推断。VAE 的核心设计是：编码器不直接输出一个确定的隐含向量 z ，而是输出一个“范围”——均值 μ 和方差 $\log_var$ ，然后用重参数化技巧从这个范围内随机采一个点。这样做的好处是，潜在空间被 KL 散度约束成了一个平滑的连续流形——你在空间里随便走一步，解码出来的图只会平滑变形，不会突然变成完全不同的东西。重建损失（让你生成的图和输入差不多）和 KL 损失（让潜在分布接近标准正态）是一对拉扯的力量——重建权重太大，VAE 退化成普通自编码器；KL 权重太大，生成质量下降。跑一遍 MNIST 代码你会看到那个 15x15 的“数字渐变地图”，相邻位置解码出来的数字之间平滑过渡——这就是 VAE 最迷人的招牌特性。这章的真正遗产是“潜在空间”这个思想：把复杂数据压缩到一个结构化的低维空间里，SD 后来把这条路走到了极致。

常见疑问

Q: VAE 生成出来的图为什么一般比 GAN 模糊？ A: 根本原因是训练目标不同。VAE 优化的逐像素重建误差（MSE）天然倾向于输出“安全”的模糊平均。比如在手写数字里，同一个位置的像素有时是黑的有时是白的，VAE 学了均值就给你一个灰色——不会犯错，但也不出彩。GAN 没有像素级重建的要求，判别器只关心整体是否“像真的”，所以 GAN 敢冒险画出清晰的笔触。后来有一些改进比如 VQ-VAE（向量量化 VAE）和 VD-VAE（深层 VAE），在保真度上追了回来。SD 其实也用了 VAE 做压缩，但它把生成任务甩给了扩散模型，VAE 只管编码解码，避开了这个模糊的问题。

Q: 重参数化技巧到底是怎么让 VAE 可训练的？不做重参数化会怎样？ A: 如果不做重参数化， $z = \mu + \epsilon \cdot \sigma$ 里， ϵ 是从 $N(0,1)$ 随机采样的，这个“采样”操作本身不可导——梯度没法通过随机节点往回传。重参数化把随机性“外化”了： z 写成 $\mu + \sigma \cdot \epsilon$ 的形式， μ 和 σ 来自编码器的输出（可导）， ϵ 来自一个跟网络参数无关的标准正态分布。这样梯度可以绕过 ϵ ，直接通过 μ 和 σ 往回传。一句话：重参数化把“不可导的采样”变成了“可导的确定性变换加上外部随机噪声”。

第 31 章 Diffusion: 扩散模型

观光路线：用一杯滴了墨水的清水，理解扩散模型的核心直觉——正向加噪、反向去噪。你会看到为什么扩散模型的训练如此稳定（比 GAN 容易一百倍），以及 U-Net 这个架构为什么特别适合做去噪。读完你会理解，Midjourney 和 DALL·E 背后那个“从噪声中浮现图像”的魔法是怎么运作的。

登山路线：正向扩散的马尔可夫链形式化（从 x_0 到 x_T 的逐步加噪过程）、反向去噪的参数化选择（预测噪声而非预测图像）、DDPM 的训练目标推导（简化为加权 MSE 回归）、U-Net 的跳跃连接和时间步嵌入设计、简化版 DDPM 训练 + DDIM 采样的完整代码、去噪过程的多步可视化（用 SVG 或文字描述渐进式还原过程）。难度 ★★★★★

2015 年，一篇关于“深度无监督学习”的论文发表在了 ICML 上。论文的 idea 很漂亮，但实验结果说实话不怎么样——生成的图像模糊，采样慢得要命。作者之一的 Jascha Sohl-Dickstein 后来回忆说，当时没几个人觉得这条路能走通。

五年后，2020 年，Jonathan Ho 和他的合作者重新捡起了这个想法，发表了一篇叫《Denoising Diffusion Probabilistic Models》的论文。这一次，他们在关键细节上做了几个改进——结果惊艳了整个 CV 圈子。

又过了一年，OpenAI 的 DALL·E 2、Google 的 Imagen、Stability AI 的 Stable Diffusion 相继问世。扩散模型从一个无人问津的学术怪想法，变成了 AI 生成图像的事实标准。Midjourney v5 生成的图片让摄影师开始怀疑自己的职业前景。

这个沉寂了五年的 idea 到底是什么？

一滴墨水与一杯水

在讲数学之前，先做一个思想实验。

你面前有一杯清水。你拿起一支吸满墨水的滴管，往水杯正中央滴了一滴墨。这一刻，墨水的分布是一个极其精致的结构——漆黑的液滴，边缘带着丝状的纹路，像一个微型的烟花。

然后你等着。

墨滴开始扩散。先是一团浓黑，然后边缘变灰，灰色区域不断扩大。几秒钟后，整杯水变成了均匀的灰色。你看不出这滴水是从哪个位置滴入的，也看不出它最初是什么形状。所有的信息都被扩散过程抹平了。

现在，把时间倒放。

一杯均匀的灰水，墨水分子像被无形的力量牵引着，从四面八方聚拢回来。灰色越来越集中，中心越来越黑，最后，那滴形状精致的墨滴重新浮现。

这个“倒放”的过程，就是扩散模型的核心直觉。

正向扩散：有序变混沌

用数学来描述这个过程，就是一条马尔可夫链。

我们从一张真实的图片 x_0 开始。每过一步，往图片里加一点高斯噪声，得到 x_1 。然后再加一点，得到 x_2 。这样重复 T 步（通常 $T=1000$ ），最终 x_T 变成了一张纯粹的噪声图——每个像素都是从 $N(0, 1)$ 里独立采样出来的。

这个过程叫**正向扩散**（forward diffusion）。每一步都是一个确定性的退化：

$$x_t = \sqrt{1 - \beta_t} \cdot x_{t-1} + \sqrt{\beta_t} \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

这里的 β_t 是一个很小的数——比如从 0.0001 逐渐增加到 0.02。它控制每一步加多少噪声。一开始噪声很少，图片几乎看不出变化；越往后噪声越重。

正向扩散不需要学习任何东西。它就是一个纯粹的数据破坏过程。一张猫的照片经过这个过程的 1000 步折腾，最终变成了一团高斯分布的噪声。你甚至可以直接写出从 x_0 到任意一步 x_t 的公式，不需要一步步地算。

反向去噪：混沌变有序

聪明的你肯定已经想到了：如果我们能学会倒放这个过程，就能从一团噪声中生成图片。

反向去噪（reverse denoising）要学的是：给定一张带噪声的图 x_t ，预测它在上一时刻 x_{t-1} 应该是什么样。本质上就是去噪——只不过不是一步到位地去，而是一点一点地去。

这里有一个关键的设计选择，是 DDPM 成功的重要原因。

你可能会想，直接让模型预测 x_{t-1} （上一时刻的图片）不就行了？但这样做效果不好。DDPM 的选择更聪明：**让模型预测当前时刻图片中的噪声分量**。

为什么预测噪声比预测图片好？因为噪声的结构很简单——就是独立的高斯噪声，每个像素之间没有相关性。这是一个“小学生也能理解的任务”：给定一张被污染的图，猜猜噪声长什么样。模型不需要理解图片的全局语义结构，只需要在每个局部区域找出“哪些像素看起来像是噪声”。

等你从 x_t 中减掉预测出的噪声，再做一些缩放调整，就能得到去噪后的 x_{t-1} 。

DDPM 的训练目标简化到极致，就是一个**均方误差回归**（MSE）：

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

翻译：随机选一张训练图片 x_0 和一个时间步 t ，加入对应强度的随机噪声 ϵ ，然后让模型 ϵ_θ 猜“我加了什么噪声”。猜得越接近越好。

这个目标函数干净得像教科书上的例题。没有对抗、没有博弈、没有精巧的平衡。你只需要让模型做好一件事：**猜噪声**。

这也解释了为什么扩散模型的训练比 GAN 稳定得多——你不需要费尽心思平衡两个网络，不会遇到模式坍塌，不会梯度爆炸。你只是在做一个监督回归任务。批一批数据过一轮，loss 稳步下降，一切都可控。

U-Net：天生的去噪器

扩散模型的主干网络几乎毫无例外地选择了 U-Net。

U-Net 是一个对称的编码器-解码器架构，中间有跳跃连接把编码器每一层的特征直接送到解码器对应的层。这种设计最初是为医学图像分割发明的——你要从整张 CT 图中精确地勾画出肿瘤的轮廓，低层的位置信息和高层的语义信息缺一不可。

去噪任务天然需要同样的能力。要猜出图中哪个像素是噪声、哪个是信号，你既需要看到全局（这片区域是草地还是天空），也需要看到细节（这个像素是不是比周围亮得不自然）。U-Net 的跳跃连接完美地满足了这两个需求。

DDPM 还加了一个小改进：**时间步嵌入**。把当前是第几步 t 编码成一个向量，注入到 U-Net 的每一层。这样一来，网络知道“我正在处理的是第 500 步的噪声（非常重）还是第 50 步的噪声（比较轻）”，从而调整去噪策略。

把时间信息注入网络的方式很巧妙——不是拼在输入后面，而是通过一层简单的 MLP 把时间步映射到一个向量，然后在每一层的特征图上做加法或乘法，像一个“旋钮”调节该层的处理强度。

简化版 DDPM 代码

下面这段代码实现了一个极简的扩散模型训练和采样流程，在 MNIST 上演示。不用 1000 步，我们用 200 步就够了。

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import numpy as np

# ===== 超参数 =====
T = 200                      # 扩散总步数
beta_start = 1e-4           # 起始噪声强度
beta_end = 0.02             # 终止噪声强度
batch_size = 128
epochs = 20
lr = 2e-4
img_size = 28

# ===== 噪声调度：从  $\beta_1$  到  $\beta_T$  线性增长 =====
betas = torch.linspace(beta_start, beta_end, T)

# 预计算一些有用的量，避免反复算
alphas = 1.0 - betas                #  $\alpha_t = 1 - \beta_t$ 
alpha_bars = torch.cumprod(alphas, dim=0)  #  $\bar{\alpha}_t = \prod \alpha_s$  (累积乘积)

# 正向扩散：一步从  $x_0$  到  $x_t$ 
#  $x_t = \sqrt{\bar{\alpha}_t} * x_0 + \sqrt{1 - \bar{\alpha}_t} * \epsilon$ 
sqrt_alpha_bars = torch.sqrt(alpha_bars)
sqrt_one_minus_alpha_bars = torch.sqrt(1.0 - alpha_bars)

# ===== U-Net 风格的简易去噪网络 =====
class SimpleUNet(nn.Module):
    def __init__(self):
        super().__init__()
        # 编码器
```

```

self.enc1 = nn.Conv2d(1, 64, 3, padding=1)    # 28x28
self.enc2 = nn.Conv2d(64, 128, 3, padding=1) # 14x14 (池化后)
self.enc3 = nn.Conv2d(128, 256, 3, padding=1) # 7x7 (池化后)

# 时间嵌入: 把一个整数t 变成一个特征向量
self.time_embed = nn.Sequential(
    nn.Linear(1, 128),
    nn.ReLU(),
    nn.Linear(128, 256),
)

# 解码器 (对称结构, 跳跃连接拼回去)
self.dec1 = nn.Conv2d(256 + 128, 128, 3, padding=1) # 拼了e2
self.dec2 = nn.Conv2d(128 + 64, 64, 3, padding=1) # 拼了e1
self.dec3 = nn.Conv2d(64 + 1, 32, 3, padding=1)    # 拼了原始输入x
self.out = nn.Conv2d(32, 1, 1)                    # 输出预测的噪声

self.pool = nn.MaxPool2d(2)

def forward(self, x, t):
    # t 是 batch 中每个样本的时间步 (标量), 扩展成嵌入向量
    t = t.float().view(-1, 1)                    # (B,) → (B, 1)
    t_emb = self.time_embed(t)                    # (B, 256)

    # --- 编码 ---
    e1 = F.relu(self.enc1(x))                      # (B, 64, 28, 28)
    e2 = F.relu(self.enc2(self.pool(e1)))          # (B, 128, 14, 14)
    e3 = F.relu(self.enc3(self.pool(e2)))          # (B, 256, 7, 7)

    # 把时间嵌入注入到瓶颈层 (广播加法)
    e3 = e3 + t_emb.view(-1, 256, 1, 1)

    # --- 解码 ---
    d1 = F.interpolate(e3, scale_factor=2)         # (B, 256, 14, 14)
    d1 = F.relu(self.dec1(torch.cat([d1, e2], dim=1))) # 拼e2
    d2 = F.interpolate(d1, scale_factor=2)         # (B, 128, 28, 28)
    d2 = F.relu(self.dec2(torch.cat([d2, e1], dim=1))) # 拼e1
    d3 = F.relu(self.dec3(torch.cat([
        d2, x # 也把原始输入拼进来, 方便网络参考
    ], dim=1)))

    return self.out(d3) # 预测的是噪声ε

# ===== 训练循环 =====
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = SimpleUNet().to(device)
optimizer = optim.Adam(model.parameters(), lr=lr)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5])
])
dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# 把预计算的量搬到 GPU
sqrt_alpha_bars = sqrt_alpha_bars.to(device)

```

```

sqrt_one_minus_alpha_bars = sqrt_one_minus_alpha_bars.to(device)
alphas = alphas.to(device)
betas = betas.to(device)

model.train()
for epoch in range(epochs):
    total_loss = 0
    for imgs, _ in dataloader:
        imgs = imgs.to(device)
        B = imgs.size(0)

        # 随机为 batch 中每个样本选一个时间步
        t = torch.randint(0, T, (B,), device=device)

        # 生成噪声
        noise = torch.randn_like(imgs)

        # 正向扩散: 从  $x_0$  一步跳到  $x_t$ 
        a_bar_t = sqrt_alpha_bars[t].view(B, 1, 1, 1)
        b_bar_t = sqrt_one_minus_alpha_bars[t].view(B, 1, 1, 1)
        x_t = a_bar_t * imgs + b_bar_t * noise

        # 模型预测噪声
        noise_pred = model(x_t, t)

        # 损失: 预测的噪声和真实噪声的 MSE
        loss = F.mse_loss(noise_pred, noise)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f'Epoch {epoch+1}/{epochs} | Loss: {total_loss/len(dataloader):.6f}')

print('训练完成! 模型已经学会猜噪声了。')

# ===== 采样 (反向去噪): 从纯噪声生成图片 =====
@torch.no_grad()
def sample(model, n_samples=16):
    """从纯噪声开始, 逐步去噪 T 步, 生成图片"""
    model.eval()
    x = torch.randn(n_samples, 1, img_size, img_size).to(device)

    # 从  $x_T$  开始, 一步步退到  $x_0$ 
    for t in reversed(range(T)):
        t_batch = torch.full((n_samples,), t, device=device, dtype=torch.long)

        # 模型预测当前步的噪声
        noise_pred = model(x, t_batch)

        # DDPM 的逆向一步公式 (简化版):
        alpha_t = alphas[t]
        beta_t = betas[t]
        alpha_bar_t = alpha_bars[t]

        # 从噪声预测中恢复  $x_{t-1}$  的均值
        coeff = beta_t / torch.sqrt(1.0 - alpha_bar_t)

```

```

x_mean = (x - coeff * noise_pred) / torch.sqrt(alpha_t)

# 加上一点点噪声（除非是最后一步）
if t > 0:
    noise = torch.randn_like(x)
    x = x_mean + torch.sqrt(beta_t) * noise
else:
    x = x_mean

# 反归一化：从[-1, 1]回到[0, 1]
x = (x + 1) / 2
x = torch.clamp(x, 0, 1)
return x

# 生成 16 张手写数字
generated = sample(model, n_samples=16)
print('采样完成！生成了 16 张全新的手写数字。')

```

跑完这段代码，你会看到扩散模型生成的 16 张手写数字。每一张都是从纯噪声出发，经过 200 步渐进式去噪，最后浮现出来的。

去噪过程的视觉描述

如果把采样过程中间步的画面截取下来，你会看到这样的演变：

- **第 200 步（起点）：**一片均匀的雪花噪点，像电视机没有信号时的画面。什么也看不出。
- **第 150 步：**噪声的粒度开始变粗。隐隐约约能看到一些明暗区域，但还谈不上是什么形状。
- **第 100 步：**一个大致的轮廓浮现出来。可能是一个椭圆的形状，周围有一些模糊的笔画延伸。你能猜到这是一个数字，但还看不清是几。
- **第 50 步：**数字的笔画结构清晰了很多。如果这是“8”，两个圆圈的轮廓基本成型，只是边缘还很毛糙。
- **第 20 步：**细节开始锐化。笔画的粗细、弧度的平滑程度都出来了。和最终结果已经非常接近。
- **第 1 步（终点）：**干净清晰的手写数字，像从打印机里刚出来的。

这种“从混沌到有序”的渐进过程，是扩散模型最迷人的视觉特征。它不像 GAN 那样一步生成，也不像 VAE 那样从分布中采样——它是一场精心编排的“去噪表演”。

为什么扩散模型赢了

现在来回答本章开头的问题：为什么扩散模型能从 GAN 手中接过“生成模型之王”的权杖？

三个原因。

第一，训练极其稳定。不需要对抗博弈，不需要精巧的平衡术。目标函数就是一个干净 MSE 回归。梯度平稳，loss 稳定下降。你甚至可以放心地跑过夜，不用担心半夜模式坍塌。

第二，生成多样性好。 GAN 有一个天生的矛盾：判别器越好，生成器越难训练（梯度消失）。扩散模型没有这个矛盾——每一步的去噪都是一个有良好梯度的监督任务。它能覆盖数据分布中更广的模式。

第三，架构简单可扩展。 U-Net 是一个被充分验证过的架构，在无数任务上证明了自己。你可以在 U-Net 里加注意力层、换更大的编码器、引入文本条件——一切都是模块化的。

当然，扩散模型也有代价：采样慢。从纯噪声到生成图片需要走几百步甚至上千步。虽然后来 DDIM 等改进把步数压到了几十步，但比 GAN 的一步生成还是要慢不少。

但话说回来，用户在乎的是最终图片的质量，而不是生成花费了几秒。当 Midjourney 能画出比摄影师还好的照片时，多等几秒钟又算什么呢？

下一章，我们终于要讲到 Stable Diffusion 了。如果说扩散模型是发动机，那 SD 就是一台完整的汽车——VAE 做底盘，CLIP 做导航，潜在空间的 U-Net 做引擎。准备好，这是目前 AI 图像生成领域最强大的架构之一。

人话总结

这章带你理解了为什么扩散模型能从 GAN 手里抢走“生成之王”的权杖。核心思路朴素到像一个物理实验：往一张图片上逐步加噪声，加了足够多步之后图片变成完全的雪花噪点；然后训练一个神经网络，让它学会“猜噪声”——给定一张带噪声的图和当前的噪声步数，预测出噪声长什么样。推理时反过来：从纯噪声出发，每一步用网络猜出当前噪声然后减去，200 步之后一张清晰的图片就浮现出来了。整个过程是从混沌到有序的渐进表演，每一步都是一个小小的净化。扩散模型打赢 GAN 靠的是三个优势：训练极其稳定（MSE 回归，没对抗博弈那档子事）、生成多样性好（每个像素都在被细致地优化，不容易模式坍塌）、架构模块化好扩展（U-Net 里随便加注意力、加文本条件）。代价是采样慢——但当你看到 Midjourney 画出来的图时，多等几秒真不算什么。

常见疑问

Q: 为什么扩散模型加噪声和去噪声的步骤数一样多？不能加速吗？ A: DDPM 原论文确实需要 1000 步，每一步都是对上一步的微小修正。但后来 DDIM 发现了一个关键事实：去噪过程不一定是严格马氏链——你可以跳步！DDIM 用了一种非马尔可夫的采样策略，把 1000 步压缩到 50 甚至 20 步，画质几乎不降。现在主流生成工具（SD、Midjourney）用的都是这类加速采样器。还有一个方向是“蒸馏”——把多步去噪过程蒸馏成一个只需要 4-8 步的小模型（比如 LCM、SD Turbo），生成的图略糙一点但快到极致。

Q: 扩散模型和之前学的那些分类网络（CNN、ResNet）有什么关系？能用同一个架构吗？ A: 关系很紧密。扩散模型里的去噪网络最常用的架构就是 U-Net——一个带跳跃连接的 CNN 变种。U-Net 的编码器部分就是标准的 CNN 下采样（卷积+池化），解码器部分是 CNN 上采样，中间用跳跃连接把编码器各层的特征直接传给解码器的对应层。所以你在前几章学的卷积、池化、残差连接，在扩散模型里全都用得上。不同之处在于 U-Net 里还嵌了自注意力层和时间步嵌入——时间步 t 告诉网络“你现在处于去噪的第几步”，以便网络知道该去掉多大强度的噪声。

第 32 章 Stable Diffusion 大揭秘

观光路线：从一个读者在家用 SD 生成图片的小故事开始，走进 Stable Diffusion 的内部。你会认识它的三大核心组件——VAE 压缩空间、CLIP 文本编码、潜在空间 U-Net 去噪。理解 CFG（无分类器引导）这个让 prompt 得以精确生效的关键机制。最后你会掌握写 prompt 的艺术：从“一只猫”到“夕阳下的赛博朋克城市，霓虹灯光映在潮湿的街道上”。

登山路线：SD 三组件（VAE、CLIP Text Encoder、Latent U-Net）的协同工作原理、潜在空间压缩比（8x 下采样）的计算和意义、CFG 引导强度的数学形式（score interpolation）与超参数调节、SD 推理流程的伪代码实现（文本编码→潜在空间去噪→VAE 解码）、文生图 prompt 的构成要素体系（主体描述、风格锚定、光照氛围、视角构图、情感语调、负向词）。难度 ★★★☆☆

故事从一位读者的周末开始。

周六下午，他刷到了小红书上的一组图——赛博朋克风格的猫咪武士，毛发在霓虹灯下闪着蓝紫色的光。评论里有人问“这是哪个画师的画？”，楼主回复说：“不是画的，Stable Diffusion 生成的。”

他有点不信。AI 生成？能画成这样？

他装了个 SD WebUI，打开界面，在 prompt 框里敲下几个字：“a cat wearing samurai armor, cyberpunk style, neon lights, highly detailed”。点下生成，等了大概十秒——屏幕上出现了一只穿着武士铠甲的猫，背景是霓虹闪烁的未来都市。猫的眼神凌厉，铠甲的金属反光熠熠生辉。

他被彻底击中了。那天下午，他生成了两百多张图，从蒸汽朋克猫头鹰到水墨山水里的宇航员。每张图都像从另一个平行世界传来的明信片。

这个周末故事在 AI 圈子里每天都在上演。但这一幕背后的技术——Stable Diffusion——到底是怎么运作的？

三大组件，各司其职

SD 不是一个网络，而是三个网络组成的流水线。

组件一：VAE（变分自编码器），负责压缩和解压。

上一章我们讲了扩散模型怎么做生成——在像素空间里一步步去噪。但像素空间太大了。一张 512x512 的 RGB 图片有 786,432 个值。在这上面跑扩散过程，每一步都要算一个巨大的张量。1000 步下来，计算量吓人。

所以 SD 的第一招是：不在像素空间做扩散，在压缩后的隐含空间做。

VAE 的编码器把一张 512x512x3 的图片压缩成 64x64x4 的潜在表示。下采样因子是 8，数据量从 78 万降到了 1.6 万——压缩了近 50 倍。扩散过程在这个 64x64 的小空间里运行，每一步的计算量骤降，跑完 1000 步也不再是天方夜谭。

生成完成后，VAE 的解码器再把这个 64x64x4 的潜在表示放大回 512x512x3 的图片。因为 VAE 的学习保证，放大回来的图片保留了丰富的细节。

组件二：CLIP 文本编码器，负责理解你的 prompt。

你想生成” 夕阳下的赛博朋克城市”。SD 怎么知道这八个字对应什么样的画面？

答案是 CLIP (Contrastive Language-Image Pre-training)。CLIP 是 OpenAI 训练的一个模型，看过 4 亿对图文数据，学会了把文字和图像映射到同一个语义空间。

你输入一句 prompt，CLIP 文本编码器把它变成一个 768 维（或更大的）向量。这个向量不是随便的编码——“夕阳”对应日落的光谱色调和低角度光照，“赛博朋克”对应霓虹色、雨夜街道、巨大的建筑立面，“城市”对应高密度楼群、街道、灯光。所有这些语义信息都被编码进了那个向量里。

组件三：潜在空间 U-Net，负责实际去噪。

这个 U-Net 和上一章讲的结构差不多，但有两个关键区别。

第一，它工作在 VAE 压缩后的潜在空间里，不是像素空间。输入输出都是 64x64x4 的张量。

第二，它接收文本条件的引导。CLIP 生成的文本向量通过交叉注意力 (cross-attention) 注入到 U-Net 的每一层。在编码和解码的过程中，U-Net 的每一层都会”看一眼”文本向量，确认自己正在生成的局部区域应该对应什么语义内容。

比如 U-Net 在处理图像左上角一个高亮的区域时，交叉注意力机制让这个区域和文本向量中”霓虹灯”的部分建立强关联——“对，这个亮斑应该是一盏霓虹灯，不是一盏路灯”。

三个组件协同工作时，流程是这样的：

1. 你输入 prompt，CLIP 把它编码成一个文本向量
2. 随机初始化一个噪声潜在表示 (64x64x4 的纯噪声)
3. U-Net 在文本向量的引导下，对这个潜在表示逐步去噪 (通常 20-50 步)
4. VAE 解码器把去噪后的潜在表示放大成最终图片

CFG：让 prompt 真正生效的魔法

上面的描述漏掉了一个关键环节。如果直接把文本条件输入 U-Net，生成的图通常和你想要的差很远——看起来像是”勉强相关但不是你要求的那个东西”。

这个问题的解决方法是 CFG (Classifier-Free Guidance，无分类器引导)。

CFG 的思路有点反直觉。每一步去噪时，U-Net 实际上跑了两遍：- 一遍带着文本条件，输出”有引导的预测” - 一遍不带文本条件 (null prompt)，输出”无引导的预测”

然后，把无引导的预测作为”基准线”，往有引导的方向多推一点：

$$\epsilon_{guided} = \epsilon_{uncond} + w \cdot (\epsilon_{cond} - \epsilon_{uncond})$$

w 是引导强度 (guidance scale)，通常取 7 到 9。

翻译成成人话：无引导预测告诉你”如果没有 prompt 的约束，U-Net 会自然而然往哪个方向去噪”，有引导预测告诉你” prompt 要你往哪个方向走”。CFG 做的就是——沿着” prompt 要求的”减去”自然趋势”这个方向，多走几步。

w 越大，图片越符合你的 prompt，但代价是可能变得不自然、过度饱和。w 越小，图片越自然多样，但可能偏离你的指令。这就是 prompt 工程中需要权衡的东西。

从另一个角度看，CFG 其实是”提高信噪比”——把文本条件带来的真信号从无条件的背景噪声中放大。这和统计学里的”对照实验”思路如出一辙：想知道一个因素有没有效果，做一组有干预、一组没干预，比较两者的差异。

SD 推理流程的简化代码

虽然完整的 SD 代码极其复杂（涉及 VAE 训练、CLIP 加载、U-Net 的交叉注意力等），但推理流程的核心逻辑可以浓缩为几十行代码。下面是一个概念演示：

```
# ===== SD 推理流程（概念演示，非可运行代码） =====

import torch

def stable_diffusion_inference(prompt, guidance_scale=7.5, steps=50):
    """
    输入：一句 prompt 文本
    输出：一张 512x512 的 RGB 图片
    """

    # 步骤1: CLIP 文本编码 — prompt 变成语义向量
    text_emb = clip_encode_text(prompt)
    # text_emb 的形状: (1, 77, 768)
    # 77 是最大 token 数, 768 是 CLIP 的嵌入维度

    # 同时编码空文本, 用于 CFG
    null_emb = clip_encode_text("")

    # 步骤2: 初始化潜在空间 — 纯噪声
    # 潜在空间尺寸 = 图片尺寸 / 8
    # 512/8=64, 通道数=4
    latent = torch.randn(1, 4, 64, 64) # 一张纯噪声的潜在表示

    # 步骤3: 逐步去噪（通常用 DDIM 采样器, 20-50 步就够了）
    for t in timesteps:
        # 3a. 有文本条件的预测
        noise_cond = unet(latent, t, text_emb)

        # 3b. 无文本条件的预测 (null prompt)
        noise_uncond = unet(latent, t, null_emb)

        # 3c. CFG 引导: 往有条件的方向多推一点
        noise_pred = noise_uncond + guidance_scale * (
            noise_cond - noise_uncond
        )

        # 3d. 从 Latent 中减去预测的噪声, 得到下一步的 Latent
        latent = scheduler_step(latent, noise_pred, t)
```

```

# 步骤4: VAE 解码 — 把潜在表示放大回像素空间
image = vae_decode(latent)
# image 的形状: (1, 3, 512, 512)

return image

# ===== Prompt 示例 =====
prompt_simple = "a cat sitting on a sofa"
prompt_rich = (
    "a fluffy orange tabby cat sitting on a vintage velvet sofa, "
    "warm afternoon sunlight streaming through a window, "
    "cozy living room atmosphere, soft bokeh background, "
    "photorealistic, 85mm lens, f/1.8, golden hour lighting"
)

# guidance_scale=7.5 是常用默认值, 追求创意可以降到5,
# 追求精确度可以升到10-12
image = stable_diffusion_inference(prompt_rich, guidance_scale=7.5, steps=30)

```

Prompt 的艺术：主体+风格+光照+视角+情绪

SD 的普及让“写 prompt”变成了一种新的技能。好的 prompt 和差的 prompt 之间的差距，比不同的模型版本之间的差距还大。

怎么写出好 prompt？一个实用的框架是把 prompt 拆成五个维度：

- 1. 主体 (Subject)：**画面里有什么？越具体越好。不是“a person”，而是“an elderly Japanese woman in an indigo kimono”。不是“a building”，而是“a weathered stone cathedral with flying buttresses”。形容词很重要——“weathered”（风化的）、“fluffy”（毛茸茸的）、“rusty”（生锈的）——这些词给了模型具体的视觉线索。
- 2. 风格 (Style)：**什么画风？SD 能理解大量风格词。“oil painting by Monet”、“ink wash painting”（水墨画）、“isometric pixel art”（等角像素风）、“Unreal Engine 5 render”（虚幻引擎渲染）、“1980s anime cel shading”（80 年代赛璐璐动画）。

风格词还能叠加型组合。比如“watercolor and ink sketch, loose brushstrokes, artistic”——水彩加钢笔素描，松散的笔触，有艺术感。

- 3. 光照 (Lighting)：**光决定了画面的氛围。“golden hour sunlight”是温暖的斜阳，“neon lights reflecting on wet pavement”是赛博朋克的标准夜景，“soft diffused studio lighting”是干净的商品摄影光，“cinematic rim lighting”让人物轮廓发光。

很多新手 prompt 出来的图看起来“塑料感”或“AI 感”，其实就是缺了光照描述。SD 的默认“光”是漫反射均匀光——你在现实中几乎见不到的光照条件。

- 4. 视角 (Composition)：**从哪个角度看？“close-up portrait”（面部特写）、“birds-eye view”（鸟瞰）、“dutch angle”（倾斜构图）、“over-the-shoulder shot”（过肩镜头）、“wide establishing shot”（广角定场镜头）。

视角决定构图。SD 不会“构图”，它只是根据你给的视角词去模仿类似的构图。

5. 情绪 (Mood)：这幅图给人什么感觉？“melancholic”（忧郁的）、“serene and peaceful”（宁静的）、“chaotic and energetic”（混乱而有能量的）、“mysterious and haunting”（神秘而令人难忘的）。

情绪词会影响色彩、对比度、甚至画面元素的组织方式。试一下” a forest” 和” a mysterious haunted forest, fog creeping between twisted trees”，出来的结果是两个世界。

另外别忘了**负向词 (negative prompt)**。告诉 SD 不要画什么和告诉它要画什么同样重要。通用的负向词包括：“blurry, distorted, low quality, watermark, text, ugly, bad anatomy, extra fingers”。

把这些维度都用上，一个完整的 prompt 大概长这样：

“A weathered stone cathedral in the rain, flying buttresses and gargoyles, cinematic lighting with dramatic shadows, wide-angle establishing shot from street level, moody and atmospheric, photorealistic, 35mm film grain”

写 prompt 是一种”翻译”——把脑海中的画面翻译成机器能理解的关键词。越具体越好，但不是越啰嗦越好。每个词都应该携带信息量。如果你发现自己在写” very very beautiful amazing wonderful”，那就说明你还不知道该要什么。

SD 的意义：生成不再是少数人的特权

SD 最大的贡献不在于技术的先进性（当然它非常先进），而在于把图像生成变成了人人可及的事。

在 SD 之前，要生成高质量图像，你需要跑 DALL·E 2 的 API（花钱，排队），或者在 Google Colab 上跑扩散模型（慢，还要折腾环境）。SD 把这一切压缩到了一个消费级显卡能跑的模式——一张 RTX 3060（12GB 显存）就能流畅运行。

这直接引爆了开源社区的创造力。ControlNet 让你用线条稿、深度图、人体姿态来控制构图。LoRA 让你用几张照片就教会模型一个新角色或新风格。SDXL 把分辨率推到了 1024x1024。AnimateDiff 让静态图动了起来。

一个完整的生态在两年内从无到有，以开源社区从未见过的速度成长。这在 AI 历史上是罕见的——通常这种级别的模型发布后，社区要做大量适配工作才能”跑起来”，而 SD 的社区在一周内就做出了比官方更好的 UI。

现在回头看，2022 年 8 月 SD 的开源发布，可能是生成式 AI 民主化进程中最重要的单一事件之一。

下一章我们要进入新的维度——时间。如果说 Stable Diffusion 教会了 AI 画图，那这些视频生成模型正在教会 AI 拍电影。从 Sora 到可灵，从 Runway 到 Pika，视频生成的战争已经打响。而这场战争的核心，是一个叫 DiT 的架构。

人话总结

这章把 SD 拆成了四个组件，你可以把它理解成一台完整的汽车。VAE 是”压缩引擎”——一把 512x512 的大图压成 64x64 的潜在表示，让扩散过程不用在像素级费力；CLIP 文本编码

器是”导航系统”——把”一只坐在沙发上的橘猫”这种文字变成语义向量，告诉 U-Net 该往哪个方向去噪；U-Net 是”主引擎”——在潜在空间里一步步去掉噪声，每次都会看文本条件来引导生成方向；CFG 无分类器引导是”方向盘”——让模型把”有文本条件”和”无文本条件”的预测结果做对比，然后使劲往有条件的那个方向推。这四件套配合起来，让一张消费级显卡就能跑文生图。这章还给了你一套实用的 prompt 五维框架——主体、风格、光照、视角、情绪——写 prompt 不是写作文，是把脑海中的画面翻译成模型能理解的关键词。SD 最大的贡献不是技术，而是把图像生成从少数人的特权变成了人人可及的工具。

常见疑问

Q: 为什么 SD 要在潜在空间里做扩散，而不直接在像素空间做？有什么代价吗？ A: 纯粹是为了省计算。一张 512x512 的 RGB 图有 786432 个值（512x512x3），4090 显卡在全像素空间上做扩散，效果不错但慢得离谱。VAE 把图压到 64x64x4=16384 个值——压缩了近 50 倍，扩散过程在这么小的空间里跑，速度快了一个数量级。代价也有：VAE 的压缩是有损的，极细的纹理和文字有时候在解码后会失真（这就是为什么 SD 生成的中文文字经常像鬼画符）。不过对于大部分”画一张好看的图”的需求，这点代价完全可以接受。

Q: CFG 的 guidance_scale 设多少合适？太高或太低会有什么后果？ A: 7.5 是社区公认的”甜点位”。低于 5，模型太自由，可能不按你的 prompt 画；高于 12，画面会变得过度锐化、对比度过高，看起来像 P 过了头——因为 CFG 本质上在放大”有条件”和”无条件”的差异，放太大之后连噪声都被放大了。追求创意可以降到 5-6，追求精确可以升到 10-12，但超过 15 基本上画面就崩了。如果你的 prompt 特别详细（像书里那个 5 句话的豪华版），甚至可以用更低的 CFG，因为 prompt 本身已经提供了足够强的引导。

第 33 章 文生视频：让画面流动起来

观光路线：从“AI 只能生成静态图”到“AI 能生成一分钟的高清视频”，这中间发生了什么？本章带你理解视频生成的核心挑战——帧间一致性和运动合理性。你会认识 DiT（Diffusion Transformer）这个 Sora 背后的核心架构，了解 Sora、可灵、Runway 各自的路数。读完你会明白，视频生成不是“把图片连起来”那么简单。

登山路线：视频生成相比图像生成的额外挑战（时间维度带来的计算爆炸、帧间闪烁与抖动、长程一致性）、潜在空间时空压缩（3D VAE 将视频压缩为时空 patch 序列）、DiT 架构的本质（用 Transformer 替代 U-Net 做扩散去噪，patchify 操作将空间维度转为序列）、Sora/可灵（Kling）/Runway Gen-3 的架构差异与定位对比、简化的 DiT 概念演示代码（将 U-Net 的去噪核心换为 Transformer 块）、文生视频的应用边界与未来展望。难度 ★★★☆☆

2024 年 2 月 15 日，OpenAI 发布了一段一分钟长的视频。视频里，一位穿着红色连衣裙的女性走在东京街头，霓虹灯光映在她墨镜上，行人、车流、被风扬起的樱花——一切都那么自然。唯一的问题是：这段视频是完全由 AI 生成的。没有人、没有街道、没有摄像机——只有一句 prompt。

那一天，Sora 刷爆了朋友圈。所有做视频的人都在问同一个问题：我的工作还能保住吗？

几个月后，可灵 AI 上线。和 Sora 不同，可灵不需要申请 waitlist——你注册就能用。中国用户第一次在自己手机里体验到了“输入一句话，生成一段短视频”。朋友圈又被刷了一次。

从那以后，“文生视频”从一个实验室里的 demo，变成了一个正在跑步进入普通人生活的技术。

但说实话，大多数人对视频生成的理解是模糊的。“不就是把图片扩散模型加个时间维度吗？”——如果真这么简单，OpenAI 不会花了那么久才做出 Sora。

为什么视频比图片难一个数量级

图片生成是 2D 问题——宽和高两个维度。视频生成是 3D 问题——宽、高，再加一个时间轴。这不只是多了一个维度，而是带来了两个全新的、非常棘手的挑战。

挑战一：帧间一致性。

你生成一张图片，只要这张图本身看起来合理就够了。但视频不是——你生成 60 张连续的图，每一张都不光要合理，还要和前后帧严丝合缝地对齐。

如果 AI 只是把图片扩散模型在 60 帧上各跑一遍，会发生什么？第 1 帧的猫毛是往左飘的，第 2 帧突然往右飘。第 5 帧背景里那辆车是红色的，第 8 帧变成了蓝色。角色脸上的光影每一帧都在微微闪烁——这就是经典的“AI 视频闪烁”。

这种帧间的不一致对人的眼睛来说非常刺眼。人类视觉系统对运动中的不连续极其敏感——这是进化的结果，在野外你需要在草丛晃动中识别出猎物的运动轨迹。

所以视频生成模型必须建立跨帧的关联，让模型在生成第 n 帧的时候知道第 $n-1$ 帧和第 $n+1$ 帧是什么。这需要的计算量和模型容量远超图片生成。

挑战二：运动合理性。

比帧间一致更难的是——物体到底该怎么动？

一个球从台阶上滚下来。它应该怎么弹？往哪个方向？弹多高？每个物理碰撞参数都是确定的，但模型不知道物理定律，它只能从训练数据中模仿。

一个人转身。袖子应该怎么飘？头发应该怎么甩？人的关节角度变化应该是连续的吗？肌肉张力在转身的哪个瞬间达到最大？

这些问题的答案藏在无数小时的视频数据中，但把它们从数据中学出来，需要模型对物理世界有深层理解——不是表面上的像素匹配，而是对“物体如何存在、如何运动”的认知。

把这两个挑战叠在一起，你就明白视频生成为什么难了。它不是在“画图”的基础上多了“时间轴”——它是要求 AI 理解世界是怎么运转的。

DiT：用 Transformer 替代 U-Net

上一章我们讲了 SD 用 U-Net 在潜在空间里做扩散去噪。到了视频生成，U-Net 开始显现出短板。

U-Net 擅长处理规则空间网格——2D 图片的像素网格，或者 3D 视频的时空网格。但它的归纳偏置（inductive bias）太强了——卷积核的局部感受野天然适合捕捉局部特征，但对于“第 1 帧角落里那只鸟和第 50 帧画面中央那个人的视线方向是否有因果关系”这种全局的、跨时空的关联，U-Net 力不从心。

DiT（Diffusion Transformer）的解决思路很直接：**把 U-Net 换成 Transformer。**

Transformer 不需要局部感受野——它的自注意力机制让每一帧的每一个 patch 都能看到所有其他帧的所有其他 patch。第 1 帧的鸟和第 50 帧的人可以在注意力矩阵中建立直接的关联。

具体是怎么做的呢？

用 VAE 把视频的每一帧都压缩到潜在空间，变成一个时空块（spacetime latent）。然后把这个时空块切分成小的 patch——和 ViT 切图片的方式一模一样，只不过多了时间维度。每个 patch 加上位置编码（告诉模型“你在空间坐标(x,y)和时间坐标 t”），然后送入标准的 Transformer 层。

Transformer 的自注意力在所有这些 patch 之间运算，学会哪些 patch 在时间和空间上是相关的。比如一个球的 patch 在第 t 帧的位置和它在第 t+1 帧的位置——模型学会这两者之间的位移就是球的运动向量。

DiT 的训练目标和 DDPM 相同：预测噪声。只是预测的对象从 2D 图片噪声变成了时空块的噪声。

下面是一个 DiT 核心概念的精简演示代码，展示了“用 Transformer 块替代 U-Net 的卷积层”这个关键思路：

```
import torch
import torch.nn as nn
```

```
# ===== DiT Block: Transformer 替代 U-Net 的核心单元 =====
class DiTBlock(nn.Module):
```

```

"""一个DiT的基本构建块，对应Transformer的一层"""
def __init__(self, dim=512, num_heads=8):
    super().__init__()
    self.norm1 = nn.LayerNorm(dim)

    # 多头自注意力 — 让所有时空patch互相看到彼此
    self.attn = nn.MultiheadAttention(
        dim, num_heads, batch_first=True
    )

    self.norm2 = nn.LayerNorm(dim)

    # 前馈网络
    self.ffn = nn.Sequential(
        nn.Linear(dim, dim * 4),
        nn.GELU(),
        nn.Linear(dim * 4, dim),
    )

    # 自适应层归一化：让时间步t和文本条件影响每一层
    self.adaLN = nn.Sequential(
        nn.SiLU(),
        nn.Linear(dim, dim * 6) # 6 = 2(γ, θ) × 3(attn, ffn, 最终)
    )

def forward(self, x, cond):
    """
    x:      (B, N, dim) — N个时空patch的序列
    cond:   (B, dim)   — 时间步+文本条件的综合嵌入
    """
    # 自适应LN的参数由条件生成
    gamma1, beta1, gamma2, beta2, gamma3, beta3 = \
        self.adaLN(cond).chunk(6, dim=-1)

    # 自注意力 + 残差
    x = x + gamma1.unsqueeze(1) * self.attn(
        self.norm1(x) * (1 + gamma2.unsqueeze(1)) + beta2.unsqueeze(1),
        self.norm1(x) * (1 + gamma2.unsqueeze(1)) + beta2.unsqueeze(1),
        self.norm1(x) * (1 + gamma2.unsqueeze(1)) + beta2.unsqueeze(1),
    )[0]

    # 前馈网络 + 残差
    x = x + gamma3.unsqueeze(1) * self.ffn(
        self.norm2(x) * (1 + gamma1.unsqueeze(1)) + beta1.unsqueeze(1)
    )

    return x

# ===== 简化的DiT模型 =====
class SimpleDiT(nn.Module):
    """
    概念演示：把视频的时空潜在表示变成patch序列，
    用Transformer做扩散去噪
    """
    def __init__(self, latent_channels=4, patch_size=2, dim=512, depth=12):
        super().__init__()
        self.patch_size = patch_size

```

```

# 输入投影: 把每个patch 展平成一个dim 维的向量
# 假设潜在空间是 (B, 4, T, H, W), 要分patch
patch_dim = latent_channels * patch_size * patch_size * patch_size
self.patch_embed = nn.Linear(patch_dim, dim)

# 位置编码 (可学习): 空间+时间的位置信息
# 实际使用时根据序列长度动态生成
self.pos_embed = nn.Parameter(torch.randn(1, 1024, dim) * 0.02)

# N 层DiTBlock
self.blocks = nn.ModuleList([
    DiTBlock(dim) for _ in range(depth)
])

# 条件嵌入: 把时间步t 和文本条件统一编码
self.cond_embed = nn.Sequential(
    nn.Linear(dim, dim),
    nn.SiLU(),
    nn.Linear(dim, dim),
)

self.final_norm = nn.LayerNorm(dim)

# 输出投影: 预测的噪声, 和输入同形状
self.output_proj = nn.Linear(dim, patch_dim)

def forward(self, latent, t_emb, text_emb):
    """
    Latent: (B, C, T, H, W) — 视频的时空潜在表示
    t_emb: (B, dim) — 时间步嵌入
    text_emb: (B, dim) — 文本条件嵌入
    """
    B = latent.size(0)

    # 把潜在表示切成时空patch 并展平为序列
    # 这一步在概念上和ViT 的patchify 完全一样,
    # 只是多了一个时间维度
    patches = self._patchify(latent) # (B, N, patch_dim)
    x = self.patch_embed(patches) # (B, N, dim)
    x = x + self.pos_embed[:, :x.size(1), :] # 加上位置信息

    # 综合条件: 时间+文本
    cond = self.cond_embed(t_emb + text_emb) # (B, dim)

    # 通过多层DiTBlock, 每层都接收条件引导
    for block in self.blocks:
        x = block(x, cond)

    x = self.final_norm(x)

    # 预测的噪声, 形状恢复为和patches 输入一致
    noise_patches = self.output_proj(x) # (B, N, patch_dim)
    noise = self._unpatchify(noise_patches, latent.shape)

    return noise # 预测的是加到Latent 上的噪声

```

```

def _patchify(self, latent):
    """把(B,C,T,H,W)切成时空 patch 序列(B,N,patch_dim)"""
    # 简化实现，实际中会更复杂
    B, C, T, H, W = latent.shape
    p = self.patch_size
    # 假设维度都能被 patch_size 整除
    patches = latent.reshape(
        B, C, T//p, p, H//p, p, W//p, p
    )
    patches = patches.permute(0, 2, 4, 6, 3, 5, 7, 1)
    patches = patches.reshape(B, -1, C * p * p * p)
    return patches

def _unpatchify(self, patches, latent_shape):
    """_patchify 的逆操作"""
    # 实际实现中根据 latent_shape 恢复维度
    B, C, T, H, W = latent_shape
    p = self.patch_size
    N = (T//p) * (H//p) * (W//p)
    patches = patches.reshape(B, T//p, H//p, W//p, p, p, p, C)
    patches = patches.permute(0, 7, 1, 4, 2, 5, 3, 6)
    latent = patches.reshape(B, C, T, H, W)
    return latent

```

这段代码当然不能直接跑（它缺了训练循环、数据加载、VAE 编解码等大量基础设施），但它展示了 DiT 最核心的设计思想：放弃卷积，拥抱注意力，让模型自己去发现时空中的关联。

三足鼎立：Sora vs 可灵 vs Runway

2024 年到 2025 年，文生视频的格局大致三足鼎立。

Sora (OpenAI)——架构探索者的路线。Sora 不是“升级版的 SD”，它是从底层架构就开始和 SD 走完全不同的路。DiT 替代 U-Net，3D VAE 做时空压缩，大量使用合成数据和模拟数据进行训练来让模型理解物理规律。Sora 的 demo 展示了令人印象深刻的物理一致性——衣服随走路摆动的节奏、水花溅起后落下的轨迹、光影在移动物体上的变化。但 Sora 的推理成本极高，至今未对大众开放。

可灵 (Kling, 快手)——实用主义的路线。可灵同样基于 DiT 架构，但在隐空间编解码和时序建模上做了自己的升级优化。它的策略更务实：用大量高质量视频数据训练，在 prompt 理解和画面质量上下功夫，同时把推理速度做到能用。可灵对中国文化的理解特别强——你输入“一位老人在胡同口下棋”，出来的画面真有北京胡同的质感，不是那种“洋人想象中的中国”。

Runway Gen-3——创作者工具的路线。Runway 从一开始就定位为“视频创作者的 AI 助手”，它的产品化做得最好。时间线编辑、局部重绘、运动画笔——这些都是 Runway 先做出来的。Gen-3 在画面质量和风格控制上很强，但长视频的一致性不如 Sora。

三家各有优势。Sora 在物理一致性上领先，可灵在中文理解和实用速度上突出，Runway 在创作者工具链上最完善。这种竞争对用户是好事——每家公司都在拼命往前跑。

视频只是开始

写到这一章的结尾，我想跳出技术细节，看一个更大的图景。

2012 年，AlexNet 让计算机学会了对图片”分类”。2015 年，GAN 让计算机学会了”生成”图片。2022 年，Stable Diffusion 让”生成”变得大众化。2024 年，Sora 让计算机学会了”生成”视频。

这条轨迹指向一个清晰的方向：AI 正在逐步学会理解和生成这个世界的每一种模态——文字、图片、声音、视频、3D。每种模态的”解锁”，用时越来越短。

视频不会是终点。视频之后是 3D 世界（NeRF、3D Gaussian Splatting 已经把这条路照亮了），3D 之后是交互式世界——不是生成一段你被动观看的视频，而是生成一个你可以走进去的空间，一个 AI 模拟器里的世界。

有人把 Sora 称为”世界模拟器”的雏形。这个说法可能有点过早，但方向没错。当一个模型看到足够多的视频数据，学到的不只是”像素怎么排列”，而是”世界怎么运转”——因果关系、物理规律、人类行为模式。这种隐含知识是模型中最有价值的部分。

视频只是开始。AI 正在学会理解和生成整个世界。而你读到的这一章，就是这条道路上的一小块路标。

第五部分到这里就结束了。我们从 GAN 走到了 VAE，经历了扩散模型的崛起，拆解了 Stable Diffusion 的引擎，最后把目光投向了视频这个正在被 AI 征服的新疆域。五章下来，你应该对”AI 怎么创造内容”有了一个完整的认知地图。

下一部分，我们要进入另一个维度——语言。如果说图像是空间的艺术，那语言就是思想的载体。准备好，第六部分会让你理解，为什么一个只学会了”预测下一个词”的模型，最终能写诗、编程、和你聊哲学。

人话总结

这章是第五部分的收官之作，从”AI 画一张图”升级到”AI 拍一段视频”。核心挑战一目了然：图片只需要处理空间上的像素布局，视频还多了一个时间维度——帧与帧之间必须连贯，猫咪眨眼的下一帧眼皮位置不能瞬移。DiT 是这个领域的架构新星：它把扩散模型里的 U-Net 换成了 Transformer，把视频的潜在表示切成时空 patches（就像 ViT 切图片 patches 一样，只不过多切了一个时间方向），然后让自注意力同时看空间和时间两个维度的关联。Sora 走的是架构探索路线，物理一致性惊艳但推理巨贵；可灵走实用主义路线，又快又懂中文场景；Runway 走创作者工具路线，产品化做得最好。三足鼎立只是开始——视频之后还有 3D 世界、交互式世界，AI 正在逐步”解锁”每一种模态。这章想让你带走的不只是 DiT 的技术细节，更是一种视野：从 2012 年 AlexNet 学会分类图片，到 2024 年 Sora 学会生成视频，仅仅 12 年。下一个 12 年会发生什么？没人知道，但一定更精彩。

常见疑问

Q: 文生视频为什么比文生图难那么多？不就是多了一个时间维度吗？ A: 多了个时间维度是”一句话轻描淡写，工程上痛不欲生”。首先是数据量爆炸——一秒钟 30 帧的视频，哪怕只生成 4 秒，那也是 120 帧要全部彼此连贯。模型中任何一帧出现不自然的跳跃，整段视频就废了。其次是计算量——一个 1024x1024 的图片扩散就已经要吃大片显存了，你现在要处理的是(B, C, T, H, W)五维张量，显存压力是图片的几十倍。最后是物理一致性——图片不需要考

虑”重力”、“惯性”、“遮挡关系随时间的变化”，但视频生成必须让模型隐式地学会这些物理规律，否则人一眼就能看出违和感。

Q: DiT 和普通 ViT 有什么本质区别？为什么说 DiT 是视频生成的未来方向？ A: DiT 本质上就是一个条件化的 ViT，最大的区别在于”条件注入”的方式。ViT 纯粹做分类，输入就是一张图。DiT 的输入除了视频的潜在表示，还注入了两个关键条件：时间步 t 的嵌入（告诉模型当前处于去噪的第几步）和文本嵌入（告诉模型要生成什么内容）。这两种条件不是简单地拼到输入里，而是通过 adaLN（自适应层归一化）注入到每个 Transformer block 的归一化层里——相当于每个 block 都在实时根据”去噪进度”和”文本要求”调整自己的行为。这种深度条件注入让 DiT 比 U-Net 更灵活，也更容易 scale up——这也是 Sora 团队选择它的关键原因。

大模型核心篇

公众号：AI技术圈

第 34 章 自监督学习：当完形填空遇上 AI

观光路线：理解为什么“自己教自己”是 AI 最具革命性的学习方式，看一个 BERT 如何通过做填空题学会语言 **登山路线：**深入 MLM 的 80/10/10 掩码策略，对比因果语言建模与掩码语言建模的数学本质，手写 BERT 预测代码

你有没有做过那种英语考试的完形填空？一篇文章，每隔几个词挖一个空，让你根据上下文猜出被遮住的词是什么。你做得时候得调动全部语言知识——语法、搭配、常识、逻辑推理。

现在想象一下：我让你做 10 亿篇完形填空。你做完第 10 亿篇的时候，语言能力会发生什么变化？

这就是自监督学习。

它没有标注员在背后吭哧吭哧地标注数据。不需要人手写“这句话主语是张三，宾语是李四，情感倾向为正向”。它自己从数据里制造问题，自己找答案。所以叫“自”监督——老师就是数据本身。

“AI 科普系列”丛书和本书剩余章节
请关注公众号获取。

关注后发送 “AI 科普书”

AI科普系列

拆解AI大模型

人人都看得懂AI大脑

“每天被AI刷屏，却始终说不清它到底是什么？”

这本书带你亲手拆开AI的大脑：

不堆公式、不讲黑话，用你能懂的比喻，把大模型从诞生到爆发，掰开揉碎讲明白，零基础也能读懂。

这是一本你在咖啡馆读得进去的技术书——

合上时你会发现，你已经建立了清晰的知识地图，AI没你想的那么神秘。

同系列丛书

- 《拆解AI大模型》
- 《开发AI智能助手》
- 《企业AI落地实践》



看懂 AI 原理 → 制作 AI 智能体 → 实现 AI 在企业落地